Machine Learning: The Basics

Alexander Jung, October 9, 2025

please cite as:

| A. Jung, "Machine Learning: The Basics," Springer, Singapore, 2022

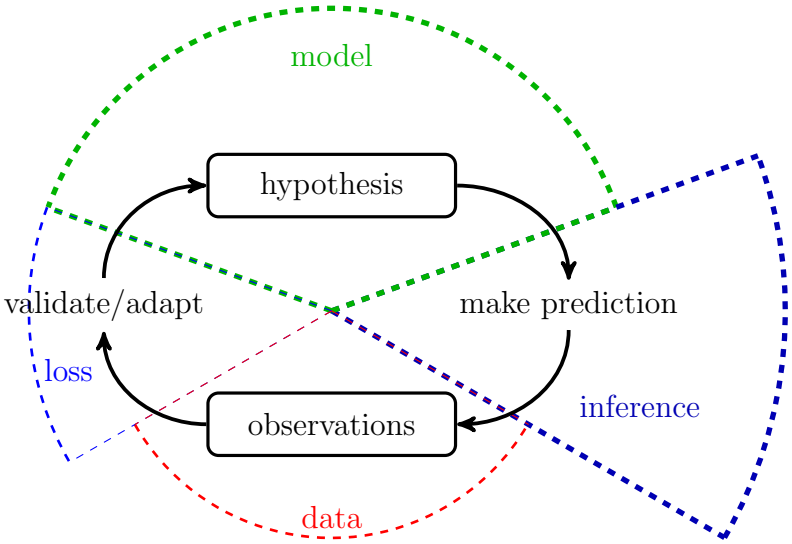


Figure 1: Machine learning combines three main components: model, data and loss. Machine learning methods implement the scientific principle of "trial and error". These methods continuously validate and refine a model based on the loss incurred by its predictions about a phenomenon that generates data.

Preface

Machine learning (ML) influences our daily lives in several aspects. We routinely ask ML empowered smartphones to suggest lovely restaurants or to guide us through a strange place. ML methods have also become standard tools in many fields of science and engineering. ML applications transform human lives at unprecedented pace and scale.

This book portrays ML as the combination of three basic components: data, model and loss. ML methods combine these three components within computationally efficient implementations of the basic scientific principle "trial and error". This principle consists of the continuous adaptation of a hypothesis about a phenomenon that generates data.

ML methods use a hypothesis map to compute a predictions for some quantity of interest (or higher level fact) that is referred to as the label of a data point. A hypothesis reads in low-level properties (referred to as features) of a data point and delivers a prediction for the label of that data point. machine learning (ML) methods choose (or learn) a hypothesis map from a (typically very) large set of candidate maps. We refer to this set as of candidate maps as the hypothesis space or model underlying an ML method.

The adaptation or improvement of the hypothesis is based on the discrepancy between predictions and observed data. ML methods use a loss function to quantify this discrepancy. A plethora of different ML methods is obtained by different design choices for the data (representation), model and loss. ML methods also differ vastly in their practical implementations which might obscure their unifying basic principles. Two examples for such ML methods are deep learning and linear regression.

Deep learning methods use cloud computing frameworks to train large models on large datasets. Operating on a much finer granularity for data and computation, linear regression can typically be implemented on small embedded systems. Nevertheless, deep learning methods and linear regression use the very same principle of iteratively updating a model based on the discrepancy between model predictions and actual observed data. This discrepancy is measured via a loss function.

We believe that thinking about ML as combinations of three components given by data, model and loss function helps to navigate the steadily growing offer for ready-to-use ML methods. Our three-component picture allows a unified treatment of ML techniques, such as "early stopping", "privacy-preserving" ML and "XML", that might seem unrelated at first sight. The regularization effect of early stopping in gradient-based methods arises from the shrinking of the effective hypothesis space. Privacy-preserving ML methods can be obtained by particular choices for the features used to characterize data points (see Section 9.5). XML methods can be obtained by particular choices for the hypothesis space and loss function (see Chapter 10).

To make good use of ML tools it is instrumental to understand its underlying principles at the appropriate level of detail. It is typically not necessary to understand the mathematical details of advanced optimization methods to successfully apply deep learning methods. On a lower level, this tutorial helps ML engineers choose suitable methods for the application at hand. The book also offers a higher level view on the implementation of ML methods which is typically required to manage a team of ML engineers and data scientists.

Acknowledgement

This book grew from lecture notes prepared for - and the student feedback received from - the courses CS-E3210 "Machine Learning: Basic Principles", CS-E4800 "Artificial Intelligence", CS-EJ3211 "Machine Learning with Python", CS-EJ3311 "Deep Learning with Python" and CS-C3240 "Machine Learning" offered at Aalto University and within the Finnish University network `fitech.io`. The author is indebted to Shamsiiat Abdurakhmanova, Tomi Janhunen, Yu Tian, Natalia Vesselinova, Linli Zhang, Ekaterina Voskoboinik, Buse Atli, Stefan Mojsilovic for carefully reviewing early drafts of this book. Some of the figures have been generated with the help of Linli Zhang. The author is also grateful for the feedback received from Luca Vassio, Lasse Leskelä, Jukka Suomela, Ekkehard Schnoor, Homayun Afrabandpey, Linli Zhang, Väinö Mehtola, Anselmi Jokinen, Oleg Vlasovetc, Anni Niskanen, Dominik Hodan, Georgios Karakasidis, Yuvrajsinh Chudasama, Joni Pääkkö, Harri Wallenius, Nuutti Sten, ST John, Antti Ainamo, Arastoo Ajorian, Sorin Patrasoiu and Satu Korhonen.

This work has been partially supported by the Research Council of Finland (under grants 331197, 349966).

Lists of Symbols

TBD

Contents

1 Introduction **9**

1.1 Relation to Other Fields . . . 13

1.1.1 Linear Algebra . . . 13

1.1.2 Optimization . . . 14

1.1.3 Theoretical Computer Science . . . 15

1.1.4 Information Theory . . . 16

1.1.5 Probability Theory and Statistics . . . 18

1.1.6 Artificial Intelligence . . . 19

1.2 Flavours of Machine Learning . . . 22

1.2.1 Supervised Learning . . . 23

1.2.2 Unsupervised Learning . . . 23

1.2.3 Reinforcement Learning . . . 24

1.3 Organization of this Book . . . 26

2 Three Components of ML **28**

2.1 The Data . . . 29

2.1.1 Features . . . 31

2.1.2 Labels . . . 36

2.1.3 Scatterplot . . . 39

2.1.4 Probabilistic Models for Data . . . 40

2.2 The Model . . . 41

2.2.1 Parametrized Hypothesis spaces . . . 44

2.2.2 The Size of a Hypothesis Space . . . 48

2.3 The Loss . . . 50

2.3.1 Loss Functions for Numeric Labels . . . 52

2.3.2 Loss Functions for Categorical Labels . . . 54

2.3.3 Loss Functions for Ordinal Label Values . . . 58
2.3.4 Empirical Risk . . . 59
2.3.5 Regret . . . 62
2.3.6 Rewards as Partial Feedback . . . 63
2.4 Putting Together the Pieces . . . 63
2.5 Exercises . . . 66

3 The Landscape of ML **74**
3.1 Linear Regression . . . 75
3.2 Polynomial Regression . . . 76
3.3 Least Absolute Deviation Regression . . . 78
3.4 The Lasso . . . 79
3.5 Gaussian Basis Regression . . . 81
3.6 Logistic Regression . . . 82
3.7 Support Vector Machines . . . 85
3.8 Bayes Classifier . . . 88
3.9 Kernel Methods . . . 89
3.10 Decision Trees . . . 90
3.11 Deep Learning . . . 93
3.12 Maximum Likelihood . . . 95
3.13 Nearest Neighbour Methods . . . 96
3.14 Deep Reinforcement Learning . . . 97
3.15 LinUCB . . . 99
3.16 Exercises . . . 100

4 Empirical Risk Minimization **103**
4.1 Approximating Risk by Empirical Risk . . . 105
4.2 Computational and Statistical Aspects of ERM . . . 107
4.3 ERM for Linear Regression . . . 109
4.4 ERM for Decision Trees . . . 113
4.5 ERM for Bayes Classifiers . . . 115
4.6 Training and Inference Periods . . . 118
4.7 Online Learning . . . 118
4.8 Weighted ERM . . . 120
4.9 Exercise . . . 122

5 Gradient-Based Learning **124**
5.1 The Basic Gradient Step . . . 125
5.2 Choosing the Learning Rate . . . 128
5.3 When To Stop? . . . 130
5.4 GD for Linear Regression . . . 131
5.5 GD for Logistic Regression . . . 134
5.6 Data Normalization . . . 136
5.7 Stochastic GD . . . 137
5.8 Advanced Gradient-Based Methods . . . 139
5.9 Exercises . . . 141

6 Model Validation and Selection **144**
6.1 Overfitting . . . 146
6.2 Validation . . . 150
6.2.1 The Size of the Validation Set . . . 150
6.2.2 k-Fold Cross Validation . . . 152
6.2.3 Imbalanced Data . . . 153
6.3 Model Selection . . . 155
6.4 A Probabilistic Analysis of Generalization . . . 160
6.5 The Bootstrap . . . 167
6.6 Diagnosing ML . . . 168
6.7 Exercises . . . 171

7 Regularization **173**
7.1 Structural Risk Minimization . . . 176
7.2 Robustness . . . 179
7.3 Data Augmentation . . . 181
7.4 Statistical and Computational Aspects of Regularization . . . 184
7.5 Semi-Supervised Learning . . . 187
7.6 Multitask Learning . . . 188
7.7 Transfer Learning . . . 190
7.8 Exercises . . . 191

8 Clustering **193**
8.1 Hard Clustering with k-means . . . 196

8.2 Soft Clustering with Gaussian Mixture Models 205
8.3 Connectivity-based Clustering 211
8.4 Clustering as Preprocessing 214
8.5 Exercises 215

9 Feature Learning **216**
9.1 Basic Principle of Dimensionality Reduction 218
9.2 Principal Component Analysis 219
9.2.1 Combining principal component analysis (PCA) with linear regression 222
9.2.2 How To Choose Number of PC? 223
9.2.3 Data Visualisation 223
9.2.4 Extensions of PCA 223
9.3 Feature Learning for Non-Numeric Data 226
9.4 Feature Learning for Labeled Data 228
9.5 Privacy-Preserving Feature Learning 230
9.6 Random Projections 232
9.7 Dimensionality Increase 233
9.8 Exercises 233

10 Transparent and Explainable ML **235**
10.1 Personalized Explanations for ML Methods 237
10.1.1 Probabilistic Data Model for XML 239
10.1.2 Computing Optimal Explanations 240
10.2 Explainable Empirical Risk Minimization 243
10.3 Exercises 245

Chapter 1

Introduction

Consider waking up one winter morning in Finland and looking outside the window (see Figure 1.1). It seems to become a nice sunny day which is ideal for a ski trip. To choose the right gear (clothing, wax) it is vital to have some idea for the maximum daytime temperature which is typically reached around early afternoon. If we expect a maximum daytime temperature of around plus 5 degrees, we might not put on the extra warm jacket but rather take only some extra shirt for change.

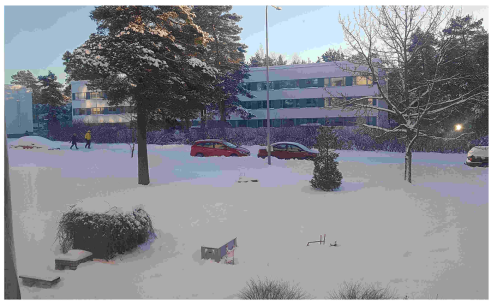


Figure 1.1: Looking outside the window during the morning of a winter day in Finland.

We can use ML to learn a predictor for the maximum daytime temperature for the specific day depicted in Figure 1.1. The prediction shall be based solely on the minimum temperature observed in the morning of that day. ML methods can learn a predictor in a data-driven fashion using historic weather observations provided by the Finnish Meteorological Institute (FMI). We can download the recordings of minimum and maximum daytime temperature for the most recent days and denote the resulting dataset by

$$\mathcal{D} = \big\{\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(m)}\big\}. \tag{1.1}$$

Each data point $\mathbf{z}^{(i)} = \big(x^{(i)}, y^{(i)}\big)$, for $i = 1, \ldots, m$, represents some previous day for which the minimum and maximum daytime temperature $x^{(i)}$ and $y^{(i)}$ has been recorded. We depict the data (1.1) in Figure 1.2. Each dot in Figure 1.2 represents a specific day with minimum temperature x and maximum temperature y.

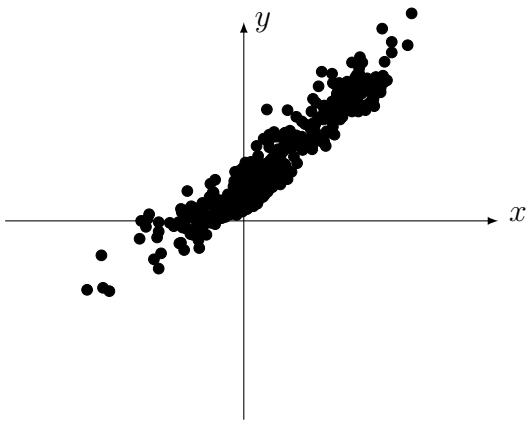


Figure 1.2: Each dot represents a specific day that is characterized by its minimum daytime temperature x as feature and its maximum daytime temperature y as label. These temperatures are measured at some FMI weather station.

ML methods learn a hypothesis $h(x)$, that reads in the minimum temperature x and delivers a prediction (forecast or approximation) $\hat{y} = h(x)$ for the maximum daytime temperature y. Every practical ML method uses a particular hypothesis space out of which the hypothesis h is chosen. This hypothesis space of candidates for the hypothesis map is an important design choice and might be based on domain knowledge.

In what follows, we illustrate how to use domain knowledge to motivate a choice for the hypothesis space. Let us assume that the minimum and maximum daytime temperature of an arbitrary day are approximately related via

$$y \approx w_1 x + w_0 \text{ with some weights } w_1 \in \mathbb{R}_+, w_0 \in \mathbb{R}. \tag{1.2}$$

The assumption (1.2) reflects the intuition (domain knowledge) that the maximum daytime temperature y should be higher for days with a higher minimum daytime temperature x. The assumption (1.2) contains two weights w_1 and w_0. These weights are tuning parameters that allow for some flexibility in our assumption. We require the weight w_1 to be non-negative

but otherwise leave these weights unspecified for the time being. The main subject of this book are ML methods that can be used to learn suitable values for the weights w_1 and w_0 in a data-driven fashion.

Before we detail how ML can be used to find or learn good values for the weights w_0 in w_1 in (1.2) let us interpret them. The weight w_1 in (1.2) can be interpreted as the relative increase in the maximum daytime temperature for an increased minimum daytime temperature. Consider an earlier day with recorded maximum daytime temperature of 10 degrees and minimum daytime temperature of 0 degrees. The assumption (1.2) then means that the maximum daytime temperature for another day with minimum temperature of +1 degrees would be $10 + w_1$ degrees. The second weight w_0 in our assumption (1.2) can be interpreted as the maximum daytime temperature that we anticipate for a day with minimum daytime temperature equal to 0.

Given the assumption (1.2), it seems reasonable to restrict the ML method to only consider linear maps

$$h(x) := w_1 x + w_0 \text{ with some weights } w_1 \in \mathbb{R}_+, w_0 \in \mathbb{R}. \tag{1.3}$$

Since we require $w_1 \geq 0$, the map (1.3) is monotonically increasing with respect to the argument x. Therefore, the prediction $h(x)$ for the maximum daytime temperature becomes higher with higher minimum daytime temperature x.

The expression (1.3) defines a whole ensemble of hypothesis maps. Each individual map corresponding to a particular choice for $w_1 \geq 0$ and w_0. We refer to such an ensemble of potential predictor maps as the model or hypothesis space that is used by a ML method.

We say that the map (1.3) is parametrized by the vector $\mathbf{w} = \big(w_1, w_0\big)^T$ and indicate this by writing $h^{(\mathbf{w})}$. For a given parameter vector $\mathbf{w} = \big(w_1, w_0\big)^T$, we obtain the map $h^{(\mathbf{w})}(x) = w_1 x + w_0$. Figure 1.3 depicts three maps $h^{(\mathbf{w})}$ obtained for three different choices for the parameters $\mathbf{w}$.

ML would be trivial if there is only one single hypothesis. Having only a single hypothesis means that there is no need to try out different hypotheses to find the best one. To enable ML, we need to choose between a whole space of different hypotheses. ML methods are computationally efficient methods to choose (learn) a good hypothesis out of a (typically very large) hypothesis spaces. Note that the hypothesis space constituted by linear maps (1.3) with different weights is already uncountably infinite.

To find, or **learn**, a good hypothesis out of the set (1.3), we need to somehow assess the quality of a particular hypothesis map. ML methods use a loss function for this purpose. A

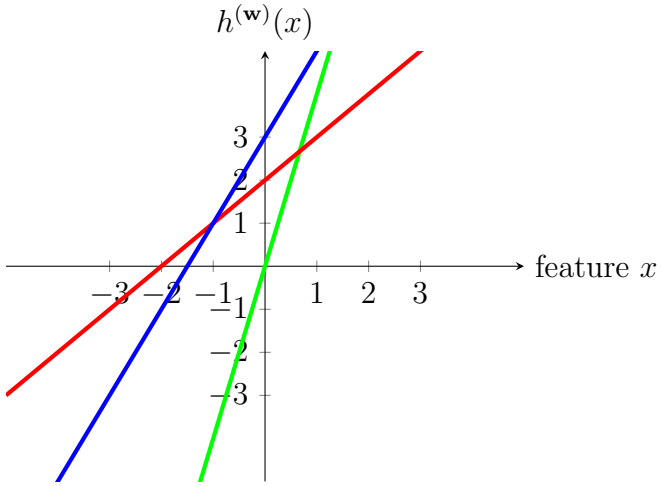


Figure 1.3: Three hypothesis maps of the form (1.3).

loss function is used to quantify the difference between the actual data and the predictions obtained from a hypothesis map (see Figure 1.4). We obtain different ML methods from using different loss functions. One widely-used example of a loss function is the squared error loss $(y - h(x))^2$. The resulting ML method learns a hypothesis from the model (1.3) by tuning w_1, w_0 to minimize the average loss

$$(1/m) \sum_{i=1}^{m} \big(y^{(i)} - h\big(x^{(i)}\big)\big)^2.$$

The above weather prediction is prototypical for many other ML applications. Figure 1 illustrates the typical workflow of a ML method. Starting from some initial guess, ML methods repeatedly improve their current hypothesis based on (new) observed data.

At any given point in time, ML methods use the current hypothesis to predict or (forecast) properties of future observations. The discrepancy between the predictions and the actual observations, as measured using some loss function, is used to improve the current hypothesis. Learning happens by improving the current hypothesis based on the discrepancy between its predictions and the actual observations.

ML methods must start with some initial guess or choice for a good hypothesis. This initial guess can be based on some prior knowledge or domain expertise [96]. While the initial guess for a hypothesis might not be made explicit in some ML methods, each method must use such an initial guess. In our weather prediction application discussed above, we used the linear model (1.2) as the initial hypothesis.

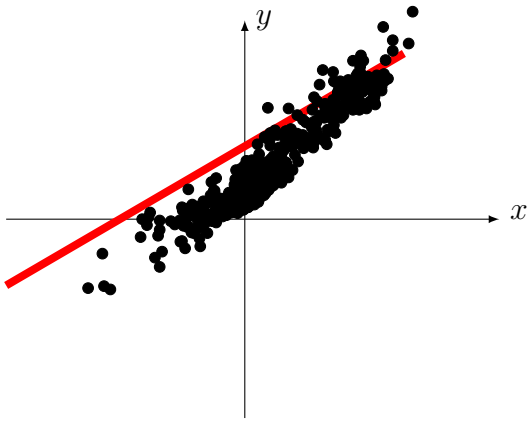


Figure 1.4: Each dot represents a specific days that is characterized by its minimum daytime temperature x and its maximum daytime temperature y. We also depict a straight line representing a linear predictor map. A main principle of ML methods is to learn a predictor (or hypothesis) map with minimum discrepancy between predictor map and data points. Different ML methods use different types of predictor maps (hypothesis space) and loss functions to quantify the discrepancy between hypothesis and actual data points.

1.1 Relation to Other Fields

ML builds on concepts from several other scientific fields. Conversely, ML provides important tools for many other scientific fields.

1.1.1 Linear Algebra

Modern ML methods are computationally efficient methods to fit high-dimensional models to large amounts of data. The models underlying state-of-the-art ML methods can contain billions of tunable or learnable parameters. To make ML methods computationally efficient we need to use suitable representations for data and models.

Maybe the most widely used mathematical structure to represent data is the Euclidean space $\mathbb{R}^n$ with some dimension $n \in \mathbb{N}$ [119]. The rich algebraic and geometric structure of $\mathbb{R}^n$ allows us to design of ML algorithms that can process vast amounts of data to quickly update a model (parameters). Figure 1.5 depicts the Euclidean space $\mathbb{R}^n$ for $n = 2$, which is used to construct scatterplots.

The scatterplot in Figure 1.2 depicts data points (representing individual days) as vectors

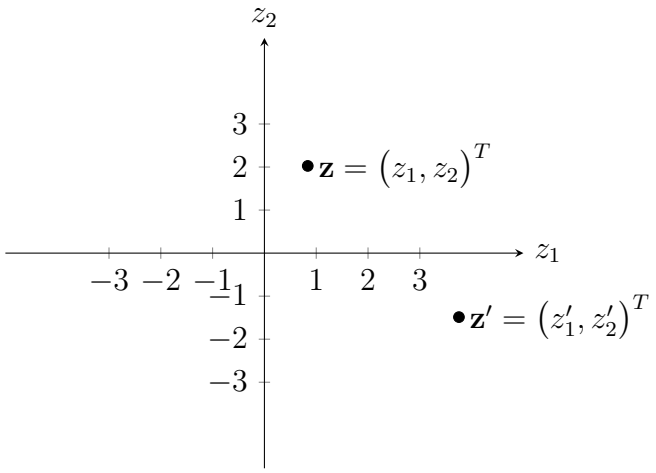


Figure 1.5: The Euclidean space $\mathbb{R}^2$ is constituted by all vectors (or points) $\mathbf{z} = \left(z_1, z_2\right)^T$ (with $z_1, z_2 \in \mathbb{R}$) together with the inner product $\mathbf{z}^T\mathbf{z}' = z_1 z_1' + z_2 z_2'$.

in the Euclidean space $\mathbb{R}^2$. For a given data point, we obtain its associated vector $\mathbf{z} = (x, y)^T$ in $\mathbb{R}^2$ by stacking the minimum daytime temperature x and the maximum daytime temperature y into the vector $\mathbf{z}$ of length two.

We can use the Euclidean space $\mathbb{R}^n$ not only to represent data points but also to represent models for these data points. One such class of models is obtained by linear maps on $\mathbb{R}^n$. Figure 1.3 depicts some examples for such linear maps. We can then use the geometric structure of $\mathbb{R}^n$, defined by the Euclidean norm, to search for the best model. As an example, we could search for the linear model, represented by a straight line, such that the average (Euclidean) distance to the data points in Figure 1.2 is as small as possible (see Figure 1.4). The properties of linear structures are studied within linear algebra [136]. Some important ML methods, such as linear classifier (see Section 3.1) or PCA (see Section 9.2) are direct applications of methods from linear algebra.

1.1.2 Optimization

A main design principle for ML methods is the formulation of ML problems as optimization problems [133]. The weather prediction problem above can be formulated as the problem of optimizing (minimizing) the prediction error for the maximum daytime temperature. Many ML methods are obtained by straightforward applications of optimization methods to the optimization problem arising from a ML problem (or application).

The statistical and computational properties of such ML methods can be studied using

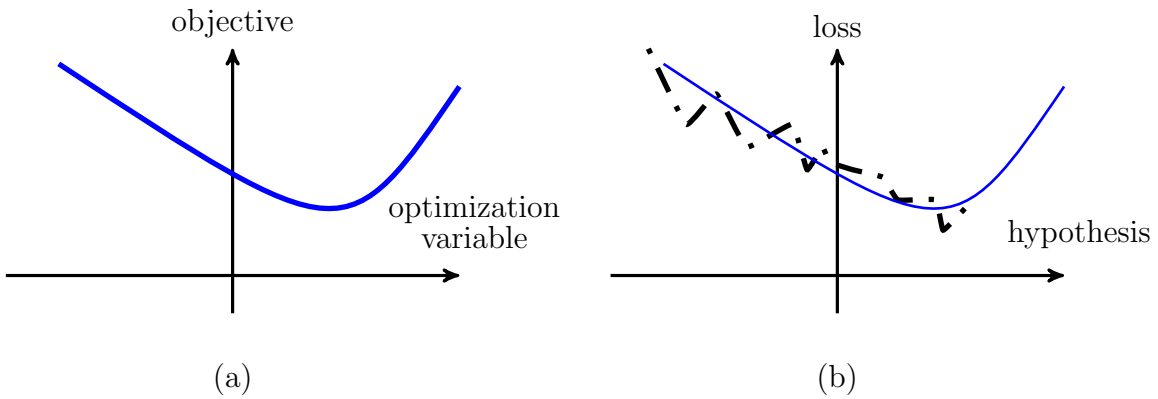


Figure 1.6: (a) A simple optimization problem consists of finding the values of an optimization variable that results in the minimum objective value. (b) ML methods learn (find) a hypothesis by minimizing a (average) loss. This average loss is a noisy version of the ultimate objective. This ultimate objective function is often defined as an expectation whose underlying probability distribution is unknown (see Section 2.3.4).

tools from the theory of optimization. What sets the optimization problems in ML apart from "plain vanilla" optimization problems (see Figure 1.6-(a)) is that we rarely have perfect access to the objective function to be minimized. ML methods learn a hypothesis by minimizing a noisy (or even incomplete) version (see Figure 1.6-(b)) of the ultimate objective function. The ultimate objective function for ML methods is often defined using an expectation over an unknown probability distribution for data points. Chapter 4 discusses methods that are based on estimating the objective function by empirical averages that are computed over a set of data points (which serve as a training set).

1.1.3 Theoretical Computer Science

Practical ML methods form a specific subclass of computing systems. Indeed, ML methods apply a sequence of computational operations to input data. The result of these computational operations are the predictions delivered to the user of the ML method. The interpretation of ML as computational systems allows to use tools from theoretical computer science to study the feasibility and intrinsic difficulty of ML problems. Even if a ML problem can be solved in theoretical sense, every practical ML method must fit the available computational infrastructure [113, 143].

The available computational resources, such as processor time, memory and communication bandwidth, can vary significantly between different infrastructures. One example for such a computational infrastructure is a single desktop computer. Another example

for a computational infrastructure is a cloud computing service which distributes data and computation over large networks of physical computers [95].

The focus of this book is on ML methods that can be understood as numerical optimization algorithms (see Chapter 4 and 5). Most of these ML methods amount to (a large number of) matrix operations such as matrix multiplication or matrix inversion [46]. Numerical linear algebra provides a vast algorithmic toolbox for the design of such ML methods [136, 135]. The recent success of ML methods in several application domains might be attributed to their efficient use of matrices to represent data and models. Using this representation allows us to implement the resulting ML methods using highly efficient hard- and software implementations for numerical linear algebra [48].

1.1.4 Information Theory

Information theory studies the problem of communication via noisy channels [140, 127, 27, 42]. Figure 1.7 depicts the most simple communication problem that consists of an information source that wishes communicate a message m over an imperfect (or noisy) channel to a receiver. The receiver tries to reconstruct (or learn) the original message based solely on the noisy channel output. Two main goals of information theory are (i) the characterization of conditions that allow reliable, i.e., nearly error-free, communication and (ii) the design of efficient transmitter (coding and modulation) and receiver (demodulation and decoding) methods.

It turns out that many concepts from information theory are very useful for the analysis and design of ML methods. As a point in case, Chapter 10 discusses the application of information-theoretic concepts to the design of XML methods. On a more fundamental level, we can identify two core communication problems that arise within ML. These communication problems correspond, respectively, to the inference (making a prediction) and the learning (adjusting or improving the current hypothesis) step of a ML method (see Figure 1).

We can an interpret the inference step of ML as the problem of decoding the true label of a data point for which we only know its features. This communication problem is depicted in Figure 1.7-(b). Here the message to be communicated is the true label of a random data point. This data point is "communicated" over a channel that only passes through its features. The inference step within a ML method then tries to decode the original message (true label) from the channel output (features) resulting in the predicted label. A recent line of work used this communication problem to study deep learning methods [140].

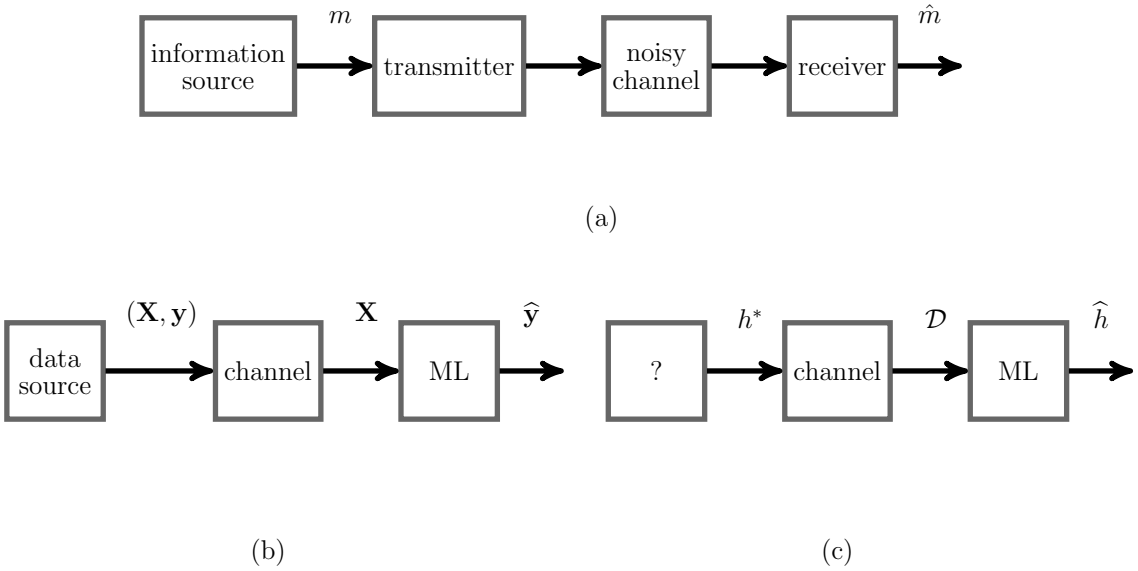


Figure 1.7: (a) A basic communication system involves an information source that emits a message m. The message is processed by some transmitter and passed through a noisy channel. The receiver tries to recover the original message m by computing the decoded message $\hat{m}$. (b) The inference step of ML (see Figure 1) corresponds to a communication problem with an information source emitting a data point with features $\mathbf{x}$ and label y. The ML method receives the features $\mathbf{x}$ and, in an effort to recover the true label y, computes the predicted label $\hat{y}$. (c) The learning or adaptation step of ML (see Figure 1) solves a communication problem with some source that selects a true (but unknown) hypothesis h^* as the message. The message is passed through an abstract channel that outputs a set $\mathcal{D}$ of labeled data points which are used as the training set by an ML method. The ML method tries to decode the true hypothesis resulting in the learnt the hypothesis $\widehat{h}$.

A second core communication problem of ML corresponds to the problem of learning (or adjusting) a hypothesis (see Figure 1.7-(c)). In this problem, the source selects some "true" hypothesis as message. This message is then communicated to an abstract channel that models the data generation process. The output of this abstract channel are data points in a training set $\mathcal{D}$ (see Chapter 4). The learning step of a ML method, such as empirical risk minimization (ERM) of Chapter 4, then amounts to the decoding of the message (true hypothesis) based on the channel output (training set). There is significant line or research that uses the communication problem in Figure 1.7-(c) to characterize the fundamental limits of ML problems and methods such as the minimum required number of training data points that makes learning feasible [155, 152, 122, 141, 71].

The relevance of information theoretic concepts and methods for ML is boosted by the recent trend towards distributed or federated ML [93, 130, 124, 123]. We can interpret federated learning (FL) applications as a specific type of network communication problems [42]. In particular, we can apply network coding techniques to the design and analysis of FL methods [42].

1.1.5 Probability Theory and Statistics

Consider the data points $\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(m)}$ depicted in Figure 1.2. Each data point represents some previous day that is characterized by its minimum and maximum daytime temperature as measured at a specific FMI weather observation station. It might be useful to interpret these data points as realizations of independent and identically distributed (i.i.d.) random variable (RV)s with common (but typically unknown) probability distribution $p(\mathbf{z})$. Figure 1.8 extends the scatterplot in Figure 1.2 by adding a contour line of the underlying probability distribution $p(\mathbf{z})$ [9].

Probability theory offers principled methods for estimating a probability distribution from a set of data points (see Section 3.12). Let us assume we know (an estimate of) the (joint) probability distribution $p(\mathbf{z})$ of features and label of a data point $\mathbf{z} = \big(\mathbf{x}, y\big)$. A principled approach to predict the label value of a data point with features $\mathbf{x}$ is based on evaluating the conditional probability distribution $p(y = \hat{y}|\mathbf{x})$. The conditional probability distribution $p(\hat{y} = y|\mathbf{x})$ quantifies how likely it is that $\hat{y}$ is the actual label value of a data point. We can evaluate the quantity $p(\hat{y} = y|\mathbf{x})$ for any candidate value $\hat{y}$ as soon as we know the features $\mathbf{x}$ of this data point.

Depending on the performance criterion or loss function, the optimal prediction $\hat{y}$ is either given by the mode of $p(\hat{y} = y|\mathbf{x})$ its mean or some other characteristic value. It is important to

note that this probabilistic approach not only provides a specific prediction (point-estimate) but an entire distribution $p(\hat{y} = y|\mathbf{x})$ over possible predictions. This distribution allows to construct confidence measures, such as the variance, that can be provided along with the prediction.

Having a probabilistic model, in the form of a probability distribution $p(\mathbf{z})$, for the data arising in an ML application not only allows us to compute predictions for labels of data points. We can also use $p(\mathbf{z})$ to augment the available dataset by randomly drawing (generating) new data points from $p(\mathbf{z})$ (see Section 7.3). ML methods that rely on a probabilistic model for data are sometimes referred to as generative methods. A recently popularized class of generative methods, that uses models obtained from artificial neural network (ANN), is known as generative adversarial networks [49].

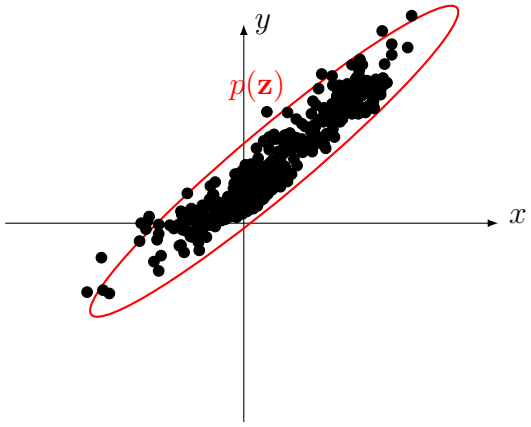


Figure 1.8: Each dot represents a data point $\mathbf{z} = \big(x, y\big)$ that is characterized by a numeric feature x and a numeric label y. We also indicate a contour-line of a probability distribution $p(\mathbf{z})$ that could be used to interpret data points as realizations of i.i.d. RVs with common probability distribution $p(\mathbf{z})$.

1.1.6 Artificial Intelligence

ML theory and methods are instrumental for the analysis and design of AI [121]. An AI system, typically referred to as an agent, interacts with its environment by executing (choosing between different) actions. These actions influence the environment as well as the state of the AI agent. The behaviour of an AI system is determined by how the perceptions made about the environment are used to form the next action.

From an engineering point of view, AI aims at optimizing behaviour to maximize a long-term return. The optimization of behaviour is based solely on the perceptions made by the agent. Let us consider some application domains where AI systems can be used:

- a forest fire management system: perceptions given by satellite images and local observations using sensors or "crowd sensing" via some mobile application which allows humans to notify about relevant events; actions amount to issuing warnings and bans of open fire; return is the reduction of number of forest fires.

- a control unit for combustion engines: perceptions given by various measurements such as temperature, fuel consistency; actions amount to varying fuel feed and timing and the amount of recycled exhaust gas; return is measured in reduction of emissions.

- a severe weather warning service: perceptions given by weather radar; actions are preventive measures taken by farmers or power grid operators; return is measured by savings in damage costs (see `https://www.munichre.com/`)

- an automated benefit application system for the Finnish social insurance institute ("Kela"): perceptions given by information about application and applicant; actions are either to accept or to reject the application along with a justification for the decision; return is measured in reduction of processing time (applicants tend to prefer getting decisions quickly)

- a personal diet assistant: perceived environment is the food preferences of the app user and their health condition; actions amount to personalized suggestions for healthy and tasty food; return is the increase in well-being or the reduction in public spending for health-care.

- the cleaning robot Rumba (see Figure 1.9) perceives its environment using different sensors (distance sensors, on-board camera); actions amount to choosing different moving directions ("north", "south", "east", "west"); return might be the amount of cleaned floor area within a particular time period.

- personal health assistant: perceptions given by current health condition (blood values, weight,...), lifestyle (preferred food, exercise plan); actions amount to personalized suggestions for changing lifestyle habits (less meat, more walking,...); return is measured via the level of well-being (or the reduction in public spending for health-care).

- a government-system for a country: perceived environment is constituted by current economic and demographic indicators such as unemployment rate, budget deficit, age distribution,...; actions involve the design of tax and employment laws, public investment in infrastructure, organization of health-care system; return might be determined by the gross domestic product, the budget deficit or the gross national happiness (cf. `https://en.wikipedia.org/wiki/Gross_National_Happiness`).

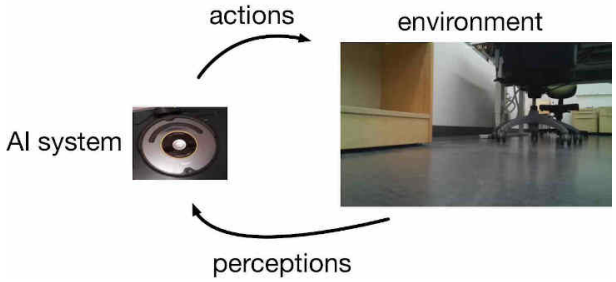


Figure 1.9: A cleaning robot chooses actions (moving directions) to maximize a long-term reward measured by the amount of cleaned floor area per day.

ML methods are used on different levels by an AI agent. On a lower level, ML methods help to extract the relevant information from raw data. ML methods are used to classify images into different categories which are then used an input for higher level functions of the AI agent.

ML methods are also used for higher level tasks of an AI agent. To behave optimally, an agent is required to learn a good hypothesis for how its behaviour affects its environment. We can think of optimal behaviour as a consequent choice of actions that might be predicted by ML methods.

What sets AI applications apart from more traditional ML application is that there is an strong interaction between ML method and the data generation process. Indeed, AI agents use the predictions of an ML method to select its next action which, in turn, influences the environment which generates new data points. The ML subfield of active learning studies methods that can influence the data generation [25].

Another characteristic of AI applications is that they typically allow ML methods to evaluate the quality of a hypothesis only in hindsight. Within a basic (supervised) ML application it is possible for a ML method to try out many different hypotheses on the same data point. These different hypotheses are then scored by their discrepancies with a known

correct predictions. In contrast to such passive ML applications, AI applications involve data points for which it is infeasible to determine the correct predictions.

Let us illustrate the above differences between ML and AI applications with the help of a self-driving toy car. The toy-car is equipped with a small onboard computer, camera, sensors and actors that allow to define the steering direction. Our goal is to program the onboard computer such that it implements an AI agent that optimally steers the toy-car. This AI application involves data points that represent the different (temporal) states of the toy car during its ride. We use a ML method to predict the optimal steering direction for the current state. The prediction for the optimal steering angle is obtained by a hypothesis map that reads a snapshot form an on-board camera. Since these predictions are used to actually steer the car, they influence the future data points (states) that will be obtained.

Note that we typically do not know the actual optimal steering direction for each possible state of the car. It is infeasible to let the toy car roam around along any possible path and then manually label each on-board camera snapshot with the optimal steering direction (see Figure 1.12). The usefulness of a prediction can be measured only in an indirect fashion by using some form of reward signal. Such a reward signal could be obtained from a distance sensor that allows to determine if the toy car reduced the distance to a target location.

1.2 Flavours of Machine Learning

ML methods read in data points which are generated within some application domain. An individual data point is characterized by various properties. We find it convenient to divide the properties of data points into two groups: features and labels (see Chapter 2.1). Features are properties that we measure or compute easily in an automated fashion. Labels are properties that cannot be measured easily and often represent some higher level fact (or quantity of interest) whose discovery often requires human experts.

Roughly speaking, ML aims at learning to predict (approximating or guessing) the label of a data point based solely on the features of this data point. Formally, the prediction is obtained as the function value of a hypothesis map whose input argument are the features of a data point. Since any ML method must be implemented with finite computational resources, it can only consider a subset of all possible hypothesis maps. This subset is referred to as the hypothesis space or model underlying a ML method. Based on how ML methods assess the quality of different hypothesis maps we distinguish three main flavours of ML: supervised, unsupervised and reinforcement learning.

1.2.1 Supervised Learning

The main focus of this book is on supervised ML methods. These methods use a training set that consists of labeled data points (for which we know the correct label values). We refer to a data point as labeled if its label value is known. Labeled data points might be obtained from human experts that annotate ("label") data points with their label values. There are marketplaces for renting human labelling workforce [132]. Supervised ML searches for a hypothesis that can imitate the human annotator and allows to predict the label solely from the features of a data point.

Figure 1.10 illustrates the basic principle of supervised ML methods. These methods learn a hypothesis with minimum discrepancy between its predictions and the true labels of the data points in the training set. Loosely speaking, supervised ML fits a curve (the graph of the predictor map) to labeled data points in a training set. For the actual implementing of this curve fitting we need a loss function that quantifies the fitting error. Supervised ML method differ in their choice for a loss function to measure the discrepancy between predicted label and true label of a data point.

While the principle behind supervised ML sounds trivial, the challenge of modern ML applications is the sheer amount of data points and their complexity. ML methods must process billions of data points with each single data points characterized by a potentially vast number of features. Consider data points representing social network users, whose features include all media that has been posted (videos, images, text). Besides the size and complexity of datasets, another challenge for modern ML methods is that they must be able to fit highly non-linear predictor maps. Deep learning methods address this challenge by using a computationally convenient representation of non-linear maps via artificial neural networks [48].

1.2.2 Unsupervised Learning

Some ML methods do not require knowing the label value of any data point and are therefore referred to as unsupervised ML methods. Unsupervised methods must rely solely on the intrinsic structure of data points to learn a good hypothesis. Thus, unsupervised methods do not need a teacher or domain expert who provides labels for data points (used to form a training set). Chapters 8 and 9 discuss two large families of unsupervised methods, referred to as clustering and feature learning methods.

Clustering methods group data points into few subsets or cluster. The data points within

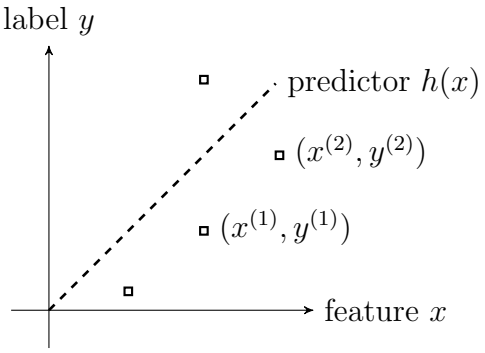


Figure 1.10: Supervised ML methods fit a curve to a set of data points (which serve as a training set). The curve represents a hypothesis out of some hypothesis space or model. The fitting error (or training error) is measured using a loss function. Different ML methods use different combinations of model and loss function.

the same cluster should be more similar with each other than with data points outside the cluster (see Figure 1.11). Feature learning methods determine numeric features such that data points can be processed efficiently using these features. Two important applications of feature learning are dimensionality reduction and data visualization.

1.2.3 Reinforcement Learning

In general, ML methods use a loss function to evaluate and compare different hypotheses. The loss function assigns a (typically non-negative) loss value to a pair of a data point and a hypothesis. ML methods search for a hypothesis, out of (typically large) hypothesis space, that incurs minimum loss for any data point. Reinforcement learning (RL) studies applications where the predictions obtained by a hypothesis influences the generation of future data points. RL applications involve data points that represent the states of a programmable system (an AI agent) at different time instants. The label of such a data point has the meaning of an optimal action that the agent should take in a given state. Similar to unsupervised ML, RL methods often must learn a hypothesis without having access to any labeled data point.

In stark contrast to supervised and unsupervised ML methods, RL methods cannot evaluate the loss function for different choices of a hypothesis. Consider a RL method that has to predict the optimal steering angle of a car. Naturally, we can only evaluate the usefulness specific combination of predicted label (steering angle) and the current state of the car. It is impossible to try out two different hypotheses at the same time as the car cannot follow

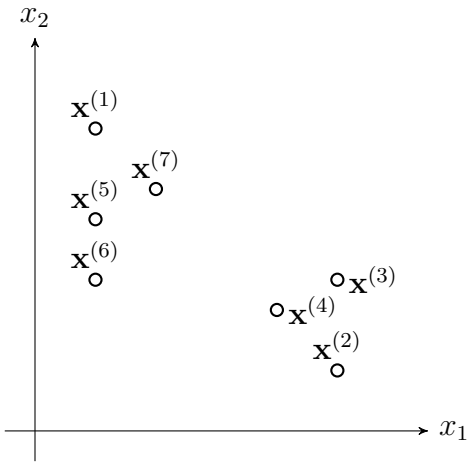


Figure 1.11: Clustering methods learn to predict the cluster (or group) assignments of data points based solely on their features. Chapter 8 discusses clustering methods that are unsupervised in the sense of not requiring the knowledge of the true cluster assignment of any data point.

two different steering angles (obtained by the two hypotheses) at the same time.

Mathematically speaking, RL methods can evaluate the loss function only point-wise for the current hypothesis that has been used to obtain the most recent prediction. These point-wise evaluations of the loss function are typically implemented by using some reward signal [137]. Such a reward signal might be obtained from a sensing device and allows to quantify the usefulness of the current hypothesis.

One important application domain for RL methods is autonomous driving (see Figure 1.12). Consider data points that represent individual time instants $t = 0, 1, \ldots$ during a car ride. The features of the tth data point are the pixel intensities of an on-board camera snapshot taken at time t. The label of this data point is the optimal steering direction at time t to maximize the distance between the car and any obstacle. We could use a ML method to learn hypothesis for predicting the optimal steering direction solely from the pixel intensities in the on-board camera snapshot. The loss incurred by a particular hypothesis is determined from the measurement of a distance sensor after the car moved along the predicted direction. We can evaluate the loss only for the hypothesis that has actually been used to predict the optimal steering direction. It is impossible to evaluate the loss for other predictions of the optimal steering direction since the car already moved on.

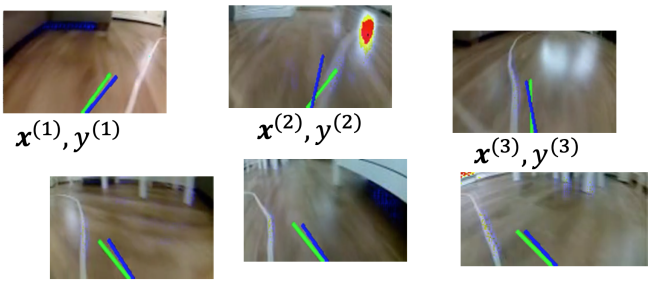


Figure 1.12: Autonomous driving requires to predict the optimal steering direction (label) based on an on-board camera snapshot (features) in each time instant. RL methods sequentially adjust a hypothesis for predicting the steering direction from the snapshot. The quality of the current hypothesis is evaluated by the measurement of a distance sensor (to avoid collisions with obstacles).

1.3 Organization of this Book

Chapter 2 introduces the notions of data, a model and a loss function as the three main components of ML. We will also highlight some of the computational and statistical aspects that might guide the design choices arising for these three components. A guiding theme of this book is the depiction of ML methods as combinations of specific design choices for data representation, the model and the loss function. Put differently, we aim at mapping out the vast landscape of ML methods in an abstract three-dimensional space spanned by the three dimensions: data, model and loss.

Chapter 3 details how several well-known ML methods are obtained by specific design choices for data (representation), model and loss function. Examples range from basic linear regression (see Section 3.1) via support vector machine (SVM) (see Section 3.7) to deep reinforcement learning (see Section 3.14).

Chapter 4 discusses a principle approach to combine the three components within a practical ML method. In particular, Chapter 4 explains how a simple probabilistic model for data lends naturally to the principle of ERM. This principle translates the problem of learning into an optimization problem. ML methods based on the ERM are therefore a special class of optimization methods. The ERM principle can be interpreted as a precise mathematical formulation of the "learning by trial and error" paradigm.

Chapter 5 discusses a family of iterative methods for solving the ERM problem introduced in Chapter 4. These methods use the concept of a gradient to locally approximate the objective function used in ERM. Gradient-based methods are widely used within deep

learning methods to learn useful weights for large ANN (see Section 3.11 and [48]).

The ERM principle of Chapter 4 requires a hypothesis to accurately predict the labels of data points in a training set. However, the ultimate goal of ML is to learn a hypothesis that delivers accurate predications for any data point and not only the training set. Chapter 6 discusses some basic validation techniques that allow to probe a hypothesis outside the training set that has been used to learn (optimize) this hypothesis via ERM. Validation techniques are instrumental for model selection, i.e., to choose the best model from a given set of candidate models. Chapter 7 presents regularization techniques that aim at replacing the training error of a candidate hypothesis with an estimate (or approximation) of its average loss incurred for data points outside the training set.

The focus of Chapters 3 - 7 is on supervised ML methods that require a training set of labeled data points. Chapters 8 and 9 are devoted to unsupervised ML methods which do not require any labeled data. Chapter 8 discusses clustering methods that partition data points into coherent groups which are referred to as clusters. Chapter 9 discusses methods that automatically determine the most relevant characteristics (or features) of a data point. This chapter also highlights the importance of using only the most relevant features of a data point, and to avoid irrelevant features, to reduce computational complexity and improve the accuracy of ML methods (such as those discussed in Chapter 3).

The success of ML methods becomes increasingly dependent on their explainability or transparency for the user of the ML method [54, 92]. The explainability of a ML method and its predictions typically depends on the background knowledge of the user which might vary significantly. Chapter 10 discusses two different approaches to obtain personalized XML. These techniques use a feedback signal, which is provided for the data points in a training set, to either compute personalized explanations for a given ML method or to choose models that are intrinsically explainable to a specific user.

Prerequisites. We assume familiarity with basic notions and concepts of linear algebra, real analysis, and probability theory [136, 119]. For a brief review of those concepts, we recommend [48, Chapter 2-4] and the references therein. A main goal of this book is to develop the basic ideas and principles behind ML methods using a minimum of probability theory. However, some rudimentary knowledge about the concept of expectations, probability density function of a continuous (real-valued) RV and probability mass function of a discrete RV is helpful [9, 50].

Chapter 2

Three Components of ML

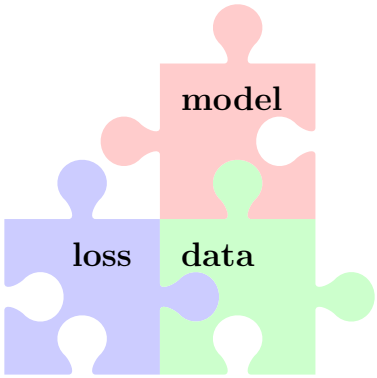


Figure 2.1: ML methods fit a model to data via minimizing a loss function (see Figure 1). We obtain a variety of ML methods from different design choices for the model, loss function and data representation (see Chapter 3). A principled approach to combine these three components is ERM (see Chapter 4).

This book portrays ML as combinations of three components as depicted in Figure 2.1. These components are

- data as collections of individual data points that are characterized by features (see Section 2.1.1) and labels (see Section 2.1.2)
- a model or hypothesis space that consists of computationally feasible hypothesis maps from feature space to label space (see Section 2.2)
- a loss function (see Section 2.3) to measure the quality of a hypothesis map.

A ML problem involves specific design choices for data points, its features and labels, the hypothesis space and the loss function to measure the quality of a particular hypothesis. Similar to ML problems (or applications), we can also characterize ML methods as combinations of these three components. This chapter discusses in some depth each of the above ML components and their combination within some widely-used ML methods [112].

We detail in Chapter 3 how some of the most popular ML methods, including linear regression (see Section 3.1) as well as deep learning methods (see Section 3.11), are obtained by specific design choices for the three components. Chapter 4 will then introduce ERM as a main principle for how to operationally combine the individual ML components. Within the ERM principle, ML problems become optimization problems and ML methods become optimization methods.

2.1 The Data

Data as Collections of Data points. Maybe the most important component of any ML problem (and method) is data. We consider data as collections of individual data points which are atomic units of "information containers". Data points can represent text documents, signal samples of time series generated by sensors, entire time series generated by collections of sensors, frames within a single video, random variables, videos within a movie database, cows within a herd, trees within a forest, or forests within a collection of forests. Mountain hikers might be interested in data points that represent different hiking tours (see Figure 2.2).



Figure 2.2: A snapshot taken at the beginning of a mountain hike.

We use the concept of data points in a very abstract and therefore highly flexible manner.

Data points can represent very different types of objects that arise in fundamentally different application domains. For an image processing application it might be useful to define data points as images. When developing a recommendation system we might define data points to represent customers. In the development of new drugs we might use data points to represent different diseases. Ultimately, the choice or definition of data points is a design choice. We might refer to the task of finding a useful definition of data points as "data point engineering".

One practical requirement for a useful definition of data points is that we should have access to many of them. Many ML methods construct estimates for a quantity of interest (such as a prediction or forecast) by averaging over a set of reference (or training) data points. These estimates become more accurate for an increasing number of data points used for computing the average. A key property of a features is the number m of individual data points it contains. The number of data points within a features is also referred to as the sample size. From a statistical point of view, the larger the sample size m the better. However, there might be restrictions on computational resources (such as memory size) that limit the maximum sample size m that can be processed.

For most applications, it is impossible to have full access to every single microscopic property of a data point. Consider a data point that represents a vaccine. A full characterization of such a data point would require to specify its chemical composition down to level of molecules and atoms. Moreover, there are properties of a vaccine that depend on the patient who received the vaccine.

We find it useful to distinguish between two different groups of properties of a data point. The first group of properties is referred to as features and the second group of properties is referred to as labels. Roughly speaking, features are low-level properties of a data point that can be measured or computed easily in an automated fashion. In contract, labels are high-level properties of a data points that represent some quantity of interest. Determining the label value of a data point often requires human labour, e.g., a domain expert who has to examine the data point. Some widely used synonyms for features are "covariate","explanatory variable", "independent variable", "input (variable)", "predictor (variable)" or "regressor" [52, 30, 38]. Some widely used synonyms for the label of a data point are "response variable", "output variable" or "target" [52, 30, 38].

We will discuss the concepts of features and labels in somewhat more detail in Sections 2.1.1 and 2.1.2. However, we would like to point out that the distinction between features and labels is blurry. The same property of a data point might be used as a feature in one

application, while it might be used as a label in another application. Let us illustrate this blurry distinction between features and labels using the problem of missing data.

Assume we have a list of data points each of which is characterized by several properties that could be measured easily in principles (by sensors). These properties would be first candidates for being used as features of the data points. However, some of these properties are unknown (missing) for a small set of data points (e.g., due to broken sensors). We could then define the properties which are missing for some data points as labels and try to predict these labels using the remaining properties (which are known for all data points) as features. The task of determining missing values of properties that could be measured easily in principle is referred to as imputation [1].

Missing data might also arise in image processing applications. Consider data points being images (snapshots) generated by a smartphone camera. Maybe the most intuitive choice for the features of a (bitmap) image are the colour intensities for each pixel (see Figure 2.5). Due to hardware failures some of the image pixels might be corrupted or (their colour intensities) even completely missing. We could then try to use to learn to predict the colour intensities of a pixel based on the colour intensities of the neighbouring pixels. To this end, we might define new data points as small rectangular regions (patches) of the image and use the colour intensity of the centre pixel ("target pixel") as the label of such a patch.

Figure 2.3 illustrates two key properties of a features. The first property is the sample size m, i.e., the number of individual data points that constitute the features. The second key property of is the number n of features that are used to characterize an individual data point. The behaviour of ML methods often depends crucially on the ratio m/n. The performance of ML methods typically improves with increasing m/n. As a rule of thumb, we should use datasets for which $m/n \gg 1$. We will make the informal condition $m/n \gg 1$ more precise in Chapter 6.

2.1.1 Features

Similar to the choice (or definition) of data points with an ML application, also the choice of which properties to be used as their features is a design choice. In general, features are (low-level) properties of a data point that can be computed or measured easily. This is obviously a highly informal characterization since there is no universal criterion for the difficulty of computing of measuring a specific property of data points. The task of choosing which properties to use as features of data points might be the most challenging part in the

n

m

Year	m	d	Time	precp	snow	airtmp	mintmp	maxtmp
2020	1	2	00:00	0,4	55	2,5	-2	4,5
2020	1	3	00:00	1,6	53	0,8	-0,8	4,6
2020	1	4	00:00	0,1	51	-5,8	-11,1	-0,7
2020	1	5	00:00	1,9	52	-13,5	-19,1	-4,6
2020	1	6	00:00	0,6	52	-2,4	-11,4	-1
2020	1	7	00:00	4,1	52	0,4	-2	1,3

Figure 2.3: Two key properties of a features are the number (sample size) m of individual data points that constitute the features and the number n of features used to characterize individual data points. The behaviour of ML methods typically depends crucially on the ratio m/n.

application of ML methods. Chapter 9 discusses feature learning methods that automate (to some extend) the construction of good features.

In some application domains there is a rather natural choice for the features of a data point. For data points representing audio recording (of a given duration) we might use the signal amplitudes at regular sampling instants (e.g., using sampling frequency 44 kHz) as features. For data points representing images it seems natural to use the colour (red, green and blue) intensity levels of each pixel as a feature (see Figure 2.5).

The feature construction for images depicted in Figure 2.5 can be extended to other types of data points as long as they can be visualized efficiently [40]. Audio recordings are typically available a sequence of signal amplitudes a_t collected regularly at time instants $t = 1, \ldots, n$ with sampling frequency ≈ 44 kHz. From a signal processing perspective, it seems natural to directly use the signal amplitudes as features, $x_j = a_j$ for $j = 1, \ldots, n$. However, another choice for the features would be the pixel RGB values of some visualization of the audio recording.

Figure 2.4 depicts two possible visualizations of an audio signal. The first visualization is obtained from a line plot of the signal amplitudes as a function of time t. Another visualization of an audio recording is obtained from an intensity plot of its spectrogram[13, 90]. We can then use the pixel RGB intensities of these visualizations as the features for an audio recording. Using this trick we can transform any ML method for image data to an ML method for audio data. We can use the scatterplot of a features to use ML methods for

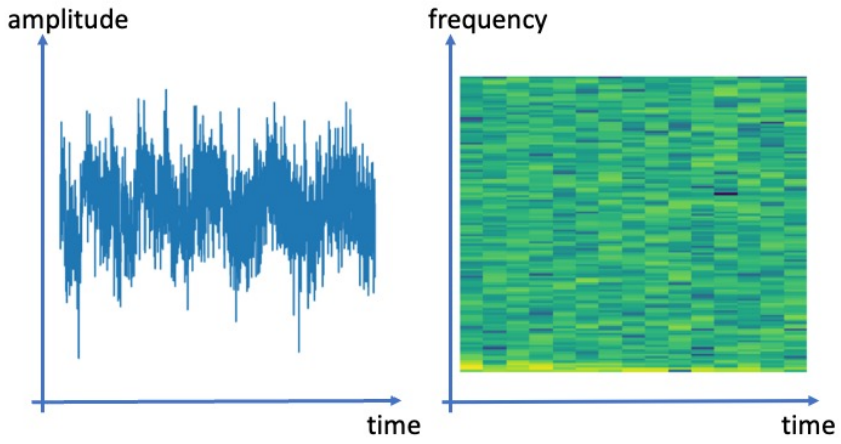


Figure 2.4: Two visualizations of a data point that represents an audio recording. The left figure shows a line plot of the audio signal amplitudes. The right figure shows a spectrogram of the audio recording.

image segmentation to cluster the features(see Chapter 8).

Many important ML application domains generate data points that are characterized by several numeric features $x_1, \ldots, x_n$. We represent numeric features by real numbers $x_1, \ldots, x_n \in \mathbb{R}$ which might seem impractical. Indeed, digital computers cannot store a real number exactly as this would require an infinite number of bits. However, numeric linear algebra soft- and hardware allows to approximate real numbers with sufficient accuracy.

The majority of ML methods discussed in this book assume that data points are characterized by real-valued features. Section 9.3 discusses methods for constructing numeric features of data points whose natural representation is non-numeric.

We assume that data points arising in a given ML application are characterized by the same number n of individual features $x_1 \ldots, x_n$. It is convenient to stack the individual features of a data point into a single feature vector

$$\mathbf{x} = \big(x_1, \ldots, x_n\big)^T.$$

Each data point is then characterized by its feature vector $\mathbf{x}$. Note that stacking the features of a data point into a column vector $\mathbf{x}$ is pure convention. We could also arrange the features as a row vector or even as a matrix, which might be even more natural for features obtained by the pixels of an image (see Figure 2.5).

We refer to the set of possible feature vectors of data points arising in some ML appli-

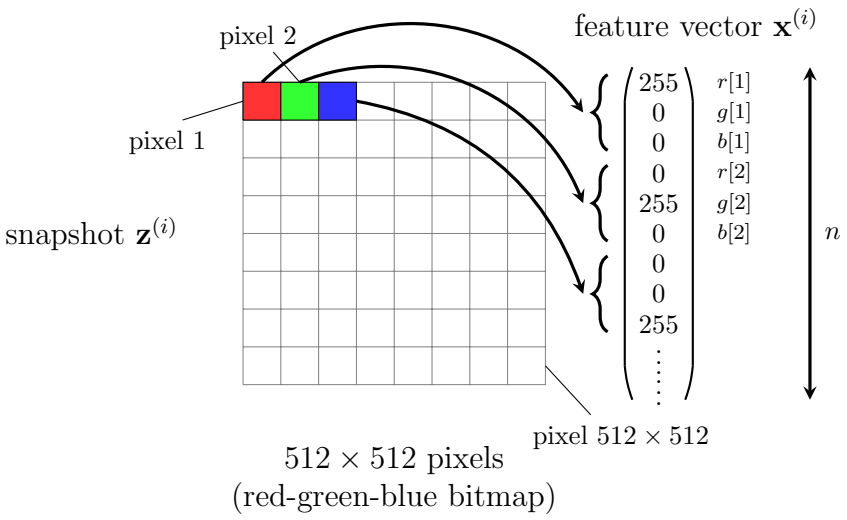


Figure 2.5: If the snapshot $\mathbf{z}^{(i)}$ is stored as a 512×512 RGB bitmap, we could use as features $\mathbf{x}^{(i)} \in \mathbb{R}^n$ the red-, green- and blue component of each pixel in the snapshot. The length of the feature vector would then be $n = 3 \times 512 \times 512 \approx 786000$.

cation as the feature space and denote it as $\mathcal{X}$. The feature space is a design choice as it depends on what properties of a data point we use as its features. This design choice should take into account the statistical properties of the data as well as the available computational infrastructure. If the computational infrastructure allows for efficient numerical linear algebra, then using $\mathcal{X} = \mathbb{R}^n$ might be a good choice.

The Euclidean space $\mathbb{R}^n$ is an example of a feature space with a rich geometric and algebraic structure [119]. The algebraic structure of $\mathbb{R}^n$ is defined by vector addition and multiplication of vectors with scalars. The geometric structure of $\mathbb{R}^n$ is obtained by the Euclidean norm as a measure for the distance between two elements of $\mathbb{R}^n$. The algebraic and geometric structure of $\mathbb{R}^n$ often enables an efficient search over $\mathbb{R}^n$ to find elements with desired properties. Chapter 4.3 discusses examples of such search problems in the context of learning an optimal hypothesis.

Modern information-technology, including smartphones or wearables, allows us to measure a huge number of properties about data points in many application domains. Consider a data point representing the book author "Alex Jung". Alex uses a smartphone to take roughly five snapshots per day (sometimes more, e.g., during a mountain hike) resulting in more than 1000 snapshots per year. Each snapshot contains around 10^6 pixels whose greyscale levels we can use as features of the data point. This results in more than 10^9

features (per year!).

As indicated above, many important ML applications involve data points represented by very long feature vectors. To process such high-dimensional data, modern ML methods rely on concepts from high-dimensional statistics [17, 151]. One such concept from high-dimensional statistics is the notion of sparsity. Section 3.4 discusses methods that exploit the tendency of high-dimensional data points, which are characterized by a large number n of features, to concentrate near low-dimensional subspaces in the feature space [147].

At first sight it might seem that "the more features the better" since using more features might convey more relevant information to achieve the overall goal. However, as we discuss in Chapter 7, it can be detrimental for the performance of ML methods to use an excessive amount of (irrelevant) features. Computationally, using too many features might result in prohibitive computational resource requirements (such as processing time). Statistically, each additional feature typically introduces an additional amount of noise (due to measurement or modelling errors) which is harmful for the accuracy of the ML method.

It is difficult to give a precise and broadly applicable characterization of the maximum number of features that should be used to characterize the data points. As a rule of thumb, the number m of (labeled) data points used to train a ML method should be much larger than the number n of numeric features (see Figure 2.3). The informal condition $m/n \gg 1$ can be ensured by either collecting a sufficiently large number m of data points or by using a sufficiently small number n of features. We next discuss implementations for each of these two complementary approaches.

The acquisition of (labeled) data points might be costly, requiring human expert labour. Instead of collecting more raw data, it might be more efficient to generate new artificial (synthetic) data via data augmentation techniques. Section 7.3 shows how intrinsic symmetries in the data can be used to augment the raw data with synthetic data. As an example for an intrinsic symmetry of data, consider data points being images. We assign each image the label $y = 1$ if it shows a cat and $y = -1$ otherwise. For each image with known label we can generate several augmented (additional) images with the same label. These additional images might be obtained by simple image transformation such as rotations or re-scaling (zoom-in or zoom-out) that do not change the depicted objects (the meaning of the image). Chapter 7 shows that some basic regularization techniques can be interpreted as an implicit form of data augmentation.

The informal condition $m/n \gg 1$ can also be ensured by reducing the number n of features used to characterize data points. In some applications, we might use some domain

knowledge to choose the most relevant features. For other applications, it might be difficult to tell which quantities are the best choice for features. Chapter 9 discusses methods that learn, based on some given features, to determine a small number of relevant features of data points.

Beside the available computational infrastructure, also the statistical properties of datasets must be taken into account for the choices of the feature space. The linear algebraic structure of $\mathbb{R}^n$ allows us to efficiently represent and approximate datasets that are well aligned along linear subspaces. Section 9.2 discusses a basic method to optimally approximate datasets by linear subspaces of a given dimension. The geometric structure of $\mathbb{R}^n$ is also used in Chapter 8 to decompose a features into few groups or clusters that consist of similar data points.

Throughout this book we will mainly use the feature space $\mathbb{R}^n$ with dimension n being the number of features of a data point. This feature space has proven useful in many ML applications due to availability of efficient soft- and hardware for numerical linear algebra. Moreover, the algebraic and geometric structure of $\mathbb{R}^n$ reflect the intrinsic structure of data arising in many important application domains. This should not be too surprising as the Euclidean space has evolved as a useful mathematical abstraction of physical phenomena [77].

In general there is no mathematically correct choice for which properties of a data point to be used as its features. Most application domains allow for some design freedom in the choice of features. Let us illustrate this design freedom with a personalized health-care applications. This application involves data points that represent audio recordings with the fixed duration of three seconds. These recordings are obtained via smartphone microphones and used to detect coughing [6].

2.1.2 Labels

Besides its features, a data point also have another type of properties. These properties represent a higher-level fact or quantity of interest that is associated with the data point. We refer to such properties of a data point as its label (or "output" or "target") and typically denote it by y (if it is a single number) or by $\mathbf{y}$ (if it is a vector of different label values, such as in multi-label classification). We refer to the set of all possible label values of data points arising in a ML application is the label space $\mathcal{Y}$. In general, determining the label of a data point is more difficult (to automate) compared to determining its features. Many ML methods revolve around finding efficient ways to predict (estimate or approximate) the label of a data point based solely on its features.

The distinction of data point properties into labels and features is blurry. Roughly speaking, labels are properties of data points that might only be determined with the help of human experts. For data points representing humans we could define its label y as an indicator if the person has flu ($y = 1$) or not ($y = 0$). This label value can typically only be determined by a physician. However, in another application we might have enough resources to determine the flu status of any person of interest and could use it as a feature that characterizes a person.

Consider a data point that represents a hike, at the start of which the snapshot in Figure 2.2 has been taken. The features of this data point could be the red, green and blue (RGB) intensities of each pixel in the snapshot in Figure 2.2. We stack these RGB values into a vector $\mathbf{x} \in \mathbb{R}^n$ whose length n is three times the number of pixels in the image. The label y associated with a data point (a hike) could be the expected hiking time to reach the mountain in the snapshot. Alternatively, we could define the label y as the water temperature of the lake that is depicted in the snapshot.

Numeric Labels - Regression. For a given ML application, the label space $\mathcal{Y}$ contains all possible label values of data points. In general, the label space is not just a set of different elements but also equipped (algebraic or geometric) structure. To obtain efficient ML methods, we should exploit such structure. Maybe the most prominent example for such a structured label space are the real numbers $\mathcal{Y} = \mathbb{R}$. This label space is useful for ML applications involving data points with numeric labels that can be modelled by real numbers. ML methods that aim at predicting a numeric label are referred to as regression methods.

Categorical Labels - Classification. Many important ML applications involve data points whose label indicate the category or class to which data points belongs to. ML methods that aim at predicting such categorical labels are referred to as classification methods. Examples for classification problems include the diagnosis of tumours as benign or malefic cent, the classification of persons into age groups or detecting the current floor conditions ("grass", "tiles" or "soil") for a mower robot.

The most simple type of a classification problems is a binary classification problem. Within binary classification, each data point belongs to exactly one out of two different classes. Thus, the label of a data point takes on values from a set that contains two different elements such as $\{0, 1\}$ or $\{-1, 1\}$ or {"shows cat", "shows no cat"}.

We speak of a multi-class classification problem if data points belong to exactly one out of more than two categories (e.g., image categories "no cat shown" vs. "one cat shown" and "more than one cat shown"). If there are K different categories we might use the label

values $\{1, 2, \ldots, K\}$.

There are also applications where data points can belong to several categories simultaneously. For example, an image can be cat image and a dog image at the same time if it contains a dog and a cat. Multi-label classification problems and methods use several labels $y_1, y_2, \ldots$, for different categories to which a data point can belong to. The label y_j represents the jth category and its value is $y_j = 1$ if the data point belongs to the j-th category and $y_j = 0$ if not.

Ordinal Labels. Ordinal label values are somewhat in between numeric and categorical labels. Similar to categorical labels, ordinal labels take on values from a finite set. Moreover, similar to numeric labels, ordinal labels take on values from an ordered set. For an example for such an ordered label space, consider data points representing rectangular areas of size 1 km by 1 km. The features $\mathbf{x}$ of such a data point can be obtained by stacking the RGB pixel values of a satellite image depicting that area (see Figure 2.5). Beside the feature vector, each rectangular area is characterized by a label $y \in \{1, 2, 3\}$ where

- $y = 1$ means that the area contains no trees.
- $y = 2$ means that the area is partially covered by trees.
- $y = 3$ means that the are is entirely covered by trees.

Thus we might say that label value $y = 2$ is "larger" than label value $y = 1$ and label value $y = 3$ is "larger" than label value $y = 2$.

The distinction between regression and classification problems and methods is somewhat blurry. Consider a binary classification problem based on data points whose label y takes on values -1 or 1. We could turn this into a regression problem by using a new label y' which is defined as the confidence in the label y being equal to 1. On the other hand, given a prediction $\hat{y'}$ for the numeric label $y' \in \mathbb{R}$ we can obtain a prediction $\hat{y}$ for the binary label $y \in \{-1, 1\}$ by setting $\hat{y} := 1$ if $\hat{y'} \geq 0$ and $\hat{y} := -1$ otherwise. A prominent example for this link between regression and classification is logistic regression which is discussed in Section 3.6. Logistic regression is a binary classification method that uses the same model as linear regression but a different loss function.

We refer to a data point as being *labeled* if, besides its features $\mathbf{x}$, the value of its label y is known. The acquisition of labeled data points typically involves human labour, such as verifying if an image shows a cat. In other applications, acquiring labels might require sending out a team of marine biologists to the Baltic sea [131], to run a particle physics

experiment at the European organization for nuclear research (CERN) [18], or to conduct animal trials in pharmacology [43].

Let us also point out online market places for human labelling workforce [99]. These market places, allow to upload data points, such as collections of images or audio recordings, and then offer an hourly rate to humans that label the data points. This labeling work might amount to marking images that show a cat.

Many applications involve data points whose features can be determined easily, but whose labels are known for few data points only. Labeled data is a scarce resource. Some of the most successful ML methods have been devised in application domains where label information can be acquired easily [55]. ML methods for speech recognition and machine translation can make use of massive labeled datasets that are freely available [78].

In the extreme case, we do not know the label of any single data point. Even in the absence of any labeled data, ML methods can be useful for extracting relevant information from features only. We refer to ML methods which do not require any labeled data points as "unsupervised" ML methods. We discuss some of the most important unsupervised ML methods in Chapter 8 and Chapter 9).

ML methods learn (or search for) a "good" predictor $h : \mathcal{X} \to \mathcal{Y}$ which takes the features $\mathbf{x} \in \mathcal{X}$ of a data point as its input and outputs a predicted label (or output, or target) $\hat{y} = h(\mathbf{x}) \in \mathcal{Y}$. A good predictor should be such that $\hat{y} \approx y$, i.e., the predicted label $\hat{y}$ is close (with small error $\hat{y} - y$) to the true underlying label y.

2.1.3 Scatterplot

Consider data points characterized by a single numeric feature x and single numeric label y. To gain more insight into the relation between the features and label of a data point, it can be instructive to generate a scatterplot as shown in Figure 1.2. A scatterplot depicts the data points $\mathbf{z}^{(i)} = (x^{(i)}, y^{(i)})$ in a two-dimensional plane with the axes representing the values of feature x and label y.

The visual inspection of a scatterplot might suggest potential relationships between feature x (minimum daytime temperature) and label y (maximum daytime temperature). From Figure 1.2, it seems that there might be a relation between feature x and label y since data points with larger x tend to have larger y. This makes sense since having a larger minimum daytime temperature typically implies also a larger maximum daytime temperature.

To construct a scatterplot for data points with more than two features we can use feature learning methods (see Chapter 9). These methods transform high-dimensional data points,

having billions of raw features, to three or two new features. These new features can then be used as the coordinates of the data points in a scatterplot.

2.1.4 Probabilistic Models for Data

A powerful idea in ML is to interpret each data points as the realization of a RV. For ease of exposition let us consider data points that are characterized by a single feature x. The following concepts can be extended easily to data points characterized by a feature vector $\mathbf{x}$ and a label y.

One of the most basic examples of a probabilistic model for data points in ML is the "i.i.d. assumption". This assumption interprets data points $x^{(1)}, \ldots, x^{(m)}$ as realizations of statistically independent RVs with the same probability distribution $p(x)$. It might not be immediately clear why it is a good idea to interpret data points as realizations of RVs with the common probability distribution $p(x)$. However, this interpretation allows us to use the properties of the probability distribution to characterize overall properties of entire datasets, i.e., large collections of data points.

The probability distribution $p(x)$ underlying the data points within the independent and identically distributed assumption (i.i.d. assumption) is either known (based on some domain expertise) or estimated from data. It is often enough to estimate only some parameters of the distribution $p(x)$. Section 3.12 discusses a principled approach to estimate the parameters of a probability distribution from a given set of data points. This approach is sometimes referred to as maximum likelihood and aims at finding (parameter of) a probability distribution $p(x)$ such that the probability (density) of observing the given data points is maximized [85, 76, 9].

Two of the most basic and widely used parameters of a probability distribution $p(x)$ are the expected value or mean [11]

$$\mu_x = \mathbb{E}\{x\} := \int_{x'} x' p(x') dx'$$

and the variance

$$\sigma_x^2 := \mathbb{E}\big\{\big(x - \mathbb{E}\{x\}\big)^2\big\}.$$

These parameters can be estimated using the sample mean (average) and sample variance,

$$\hat{\mu}_x := (1/m) \sum_{i=1}^{m} x^{(i)} \text{ , and}$$
$$\widehat{\sigma_x^2} := (1/m) \sum_{i=1}^{m} \big(x^{(i)} - \hat{\mu}_x\big)^2. \tag{2.1}$$

The sample mean and sample variance (2.1) are the ML estimators for the mean and variance of a normal (Gaussian) distribution $p(x)$ (see [12, Chapter 2.3.4]).

Most of the ML methods discussed in this book are motivated by an i.i.d. assumption. It is important to note that this i.i.d. assumption is only a modelling assumption (or hypothesis). There is no means to verify if an arbitrary set of data points are "exactly" realizations of i.i.d. RVs. There are principled statistical methods (hypothesis tests) that allow to verify if a given set of data point can be well approximated as realizations of i.i.d. RVs [88]. Alternatively, we can enforce the i.i.d. assumption if we generate synthetic data using a random number generator. Such synthetic i.i.d. data points could be obtained by sampling algorithms that incrementally build a synthetic features by adding randomly chosen raw data points [32].

2.2 The Model

Consider some ML application that generates data points, each characterized by features $\mathbf{x} \in \mathcal{X}$ and label $y \in \mathcal{Y}$. The goal of a ML method is to learn a hypothesis map $h : \mathcal{X} \to \mathcal{Y}$ such that

$$y \approx \underbrace{h(\mathbf{x})}_{\hat{y}} \text{ for any data point.} \tag{2.2}$$

The informal goal (2.2) will be made precise in several aspects throughout the rest of our book. First, we need to quantify the approximation error (2.2) incurred by a given hypothesis map h. Second, we need to make precise what we actually mean by requiring (2.2) to hold for "any" data point. We solve the first issue by the concept of a loss function in Section 2.3. The second issue is then solved in Chapter 4 by using a simple probabilistic model for data.

Let us assume for the time being that we have found a reasonable hypothesis h in the sense of (2.2). We can then use this hypothesis to predict the label of any data point for which we know its features. The prediction $\hat{y} = h(\mathbf{x})$ is obtained by evaluating the hypothesis for the features $\mathbf{x}$ of a data point (see Figure 2.6 and 2.7). We refer to a hypothesis map

also as a predictor map since it is used or compute the prediction $\hat{y}$ of the label y.

For ML problems using a finite label space $\mathcal{Y}$ (e.g, $\mathcal{Y} = \{-1, 1\}$), we refer to a hypothesis map also as a classifier. Given a finite label space $\mathcal{Y}$, we can characterize a particular classifier $h(\mathbf{x})$ using its different decision regions

$$\mathcal{R}_a := \big\{\mathbf{x} \in \mathbb{R}^n : h(\mathbf{x}) = a\big\} \subseteq \mathcal{X}. \tag{2.3}$$

Each label value $a \in \mathcal{Y}$ is associated with a specific decision region $\mathcal{R}_a$. For a given label value $a \in \mathcal{Y}$, the decision region $\mathcal{R}_a$ is constituted by all feature vectors $\mathbf{x} \in \mathcal{X}$ which are mapped to this label value, $h(\mathbf{x}) = a$.

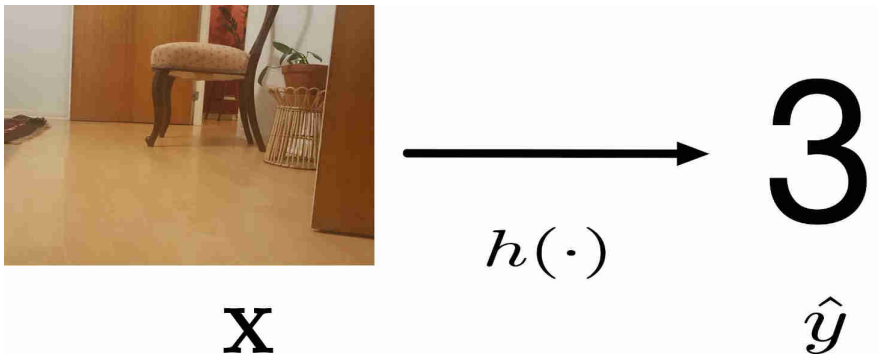


Figure 2.6: A hypothesis (or predictor) h maps features $\mathbf{x} \in \mathcal{X}$, of an on-board camera snapshot, to the prediction $\hat{y} = h(\mathbf{x}) \in \mathcal{Y}$ for the coordinate of the current location of a cleaning robot. ML methods use data to learn predictors h such that $\hat{y} \approx y$ (with true label y).

In principle, ML methods could use any possible map $h : \mathcal{X} \to \mathcal{Y}$ to predict the label $y \in \mathcal{Y}$ via computing $\hat{y} = h(\mathbf{x})$. The set of all maps from the feature space $\mathcal{X}$ to the label space is typically denoted as $\mathcal{Y}^{\mathcal{X}}$.[1] In general, the set $\mathcal{Y}^{\mathcal{X}}$ is way too large to be searched over by a practical ML methods. As a point in case, consider data points characterized by a single numeric feature $x \in \mathbb{R}$ and label $y \in \mathbb{R}$. The set of all real-valued maps $h(x)$ of a real-valued argument already contains uncountably infinite many different hypothesis maps [56].

Practical ML methods can search and evaluate only a (tiny) subset of all possible hypothesis maps. This subset of computationally feasible ("affordable") hypothesis maps is referred to as the hypothesis space or model underlying a ML method. As depicted in Figure 2.11, ML methods typically use a hypothesis space $\mathcal{H}$ that is a tiny subset of $\mathcal{Y}^{\mathcal{X}}$. Similar to the

[1]The notation $\mathcal{Y}^{\mathcal{X}}$ is to be understood as a symbolic shorthand and should not be understood literately as a power such as 4^5.

feature x	prediction $h(x)$
0	0
1/10	10
2/10	3
$\vdots$	$\vdots$
1	22.3

Table 2.1: A look-up table defines a hypothesis map h. The value $h(x)$ is given by the entry in the second column of the row whose first column entry is x. We can construct a hypothesis space $\mathcal{H}$ by using a collection of different look-up tables.

features and labels used to characterize data points, also the hypothesis space underlying a ML method is a design choice. As we will see, the choice for the hypothesis space involves a trade-off between computational complexity and statistical properties of the resulting ML methods.

The preference for a particular hypothesis space often depends on the computational infrastructure that is available to a ML method. Different computational infrastructures favour different hypothesis spaces. ML methods implemented in a small embedded system, might prefer a linear hypothesis space which results in algorithms that require a small number of arithmetic operations. Deep learning methods implemented in a cloud computing environment typically use much larger hypothesis spaces obtained from large ANN (see Section 3.11).

ML methods can also be implemented using a spreadsheet software. Here, we might use a hypothesis space consisting of maps $h : \mathcal{X} \rightarrow \mathcal{Y}$ that are represented by look up tables (see Table 2.1). If we instead use the programming language Python to implement a ML method, we can obtain a hypothesis class by collecting all possible Python subroutines with one input (scalar feature x), one output argument (predicted label $\hat{y}$) and consisting of less than 100 lines of code.

Broadly speaking, the design choice for the hypothesis space $\mathcal{H}$ of a ML method has to balance between two conflicting requirements.

- It has to be sufficiently large such that it contains at least one accurate predictor map $\widehat{h} \in \mathcal{H}$. A hypothesis space $\mathcal{H}$ that is too small might fail to include a predictor map required to reproduce the (potentially highly non-linear) relation between features and label.

Consider the task of grouping or classifying images into "cat" images and "no cat image". The classification of each image is based solely on the feature vector obtained from the pixel colour intensities. The relation between features and label ($y \in \{\text{cat}, \text{no cat}\}$) is highly non-linear. Any ML method that uses a hypothesis space consisting only of linear maps will most likely fail to learn a good predictor (classifier). We say that a ML method is underfitting if it uses a hypothesis space that does not contain any hypotheses maps that can accurately predict the label of any data points.

- It has to be sufficiently small such that its processing fits the available computational resources (memory, bandwidth, processing time). We must be able to efficiently search over the hypothesis space to find good predictors (see Section 2.3 and Chapter 4). This requirement implies also that the maps $h(\mathbf{x})$ contained in $\mathcal{H}$ can be evaluated (computed) efficiently [4]. Another important reason for using a hypothesis space $\mathcal{H}$ that is not too large is to avoid overfitting (see Chapter 7). If the hypothesis space $\mathcal{H}$ is too large, then we can easily find a hypothesis which (almost) perfectly predicts the labels of data points in a training set which is used to learn a hypothesis. However, such a hypothesis might deliver poor predictions for labels of data points outside the training set. We say that the hypothesis does not generalize well.

2.2.1 Parametrized Hypothesis spaces

A wide range of current scientific computing environments allow for efficient numerical linear algebra. This hard- and software allows to efficiently process data that is provided in the form of numeric arrays such as vectors, matrices or tensors [112]. To take advantage of such computational infrastructure, many ML methods use the hypothesis space

$$\mathcal{H}^{(n)} := \{h^{(\mathbf{w})} : \mathbb{R}^n \to \mathbb{R} : h^{(\mathbf{w})}(\mathbf{x}) := \mathbf{x}^T\mathbf{w} \text{ with some parameter vector } \mathbf{w} \in \mathbb{R}^n\}. \tag{2.4}$$

The hypothesis space (2.4) is constituted by linear maps (functions)

$$h^{(\mathbf{w})}\big(\mathbf{x}\big) : \mathbb{R}^n \to \mathbb{R} : \mathbf{x} \mapsto \mathbf{w}^T\mathbf{x}. \tag{2.5}$$

The function $h^{(\mathbf{w})}$ (2.5) maps, in a linear fashion, the feature vector $\mathbf{x} \in \mathbb{R}^n$ to the predicted label $h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{x}^T\mathbf{w} \in \mathbb{R}$. For $n = 1$ the feature vector reduces a single feature x and the hypothesis space (2.4) consists of all maps $h^{(w)}(x) = wx$ with weight $w \in \mathbb{R}$ (see Figure 2.8).

The elements of the hypothesis space $\mathcal{H}$ in (2.4) are parametrized by the parameter

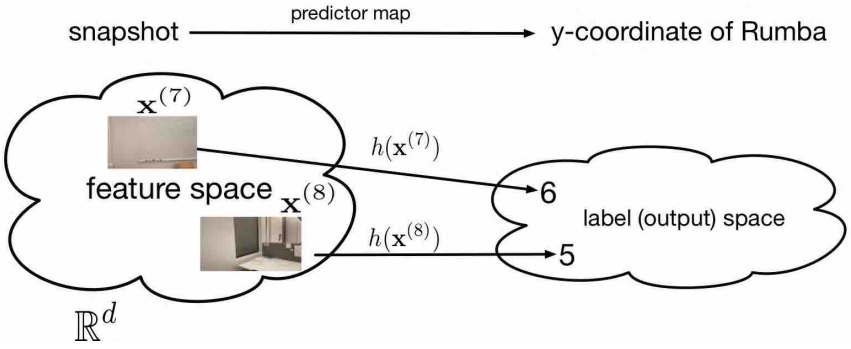


Figure 2.7: Consider a hypothesis $h : \mathcal{X} \to \mathcal{Y}$ that is used for locating a cleaning robot. The hypothesis h reads in the feature vector $\mathbf{x}^{(t)} \in \mathcal{X}$, that might be RGB pixel intensities of an on-board camera snapshot, at time t. It then outputs a prediction $\hat{y}^{(t)} = h(\mathbf{x}^{(t)})$ for the y-coordinate $y^{(t)}$ of the cleaning robot at time t. A key problem studied within ML is how to automatically learn a good (accurate) predictor h such that $y^{(t)} \approx h(\mathbf{x}^{(t)})$.

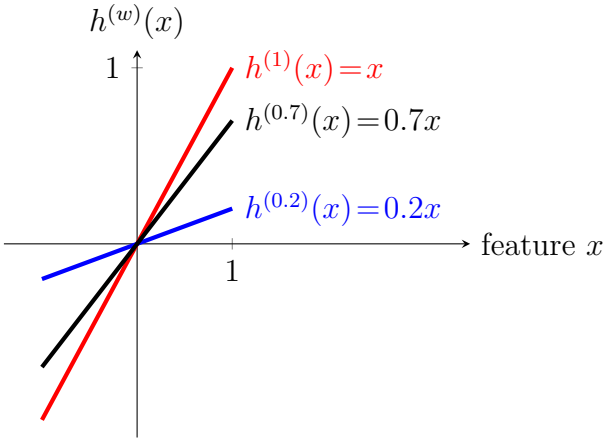


Figure 2.8: Three particular members of the hypothesis space $\mathcal{H} = \{h^{(w)} : \mathbb{R} \to \mathbb{R}, h^{(w)}(x) = wx\}$ which consists of all linear functions of the scalar feature x. We can parametrize this hypothesis space conveniently using the weight $w \in \mathbb{R}$ as $h^{(w)}(x) = wx$.

vector $\mathbf{w} \in \mathbb{R}^n$. Each map $h^{(\mathbf{w})} \in \mathcal{H}$ is fully specified by the parameter vector $\mathbf{w} \in \mathbb{R}^n$. This parametrization of the hypothesis space $\mathcal{H}$ allows to process and manipulate hypothesis maps by vector operations. In particular, instead of searching over the function space $\mathcal{H}$ (its elements are functions!) to find a good hypothesis, we can equivalently search over all possible parameter vectors $\mathbf{w} \in \mathbb{R}^n$.

The search space $\mathbb{R}^n$ is still (uncountably) infinite but it has a rich geometric and algebraic structure that allows to efficiently search over this space. Chapter 5 discusses methods that use the concept ot a gradient to implement an efficient search for useful parameter vectors $\mathbf{w} \in \mathbb{R}^n$.

The hypothesis space (2.4) is also appealing because of the broad availability of computing hardware such as graphic processing units. Another factor boosting the widespread use of (2.4) might be the offer for optimized software libraries for numerical linear algebra.

The hypothesis space (2.4) can also be used for classification problems, e.g., with label space $\mathcal{Y} = \{-1, 1\}$. Indeed, given a linear predictor map $h^{(\mathbf{w})}$ we can classify data points according to $\hat{y}=1$ for $h^{(\mathbf{w})}(\mathbf{x}) \geq 0$ and $\hat{y}=-1$ otherwise. We refer to a classifier that computes the predicted label by first applying a linear map to the features as a linear classifier.

Figure 2.9 illustrates the decision regions (2.3) of a linear classifier for binary labels. The decision regions are half-spaces and, in turn, the decision boundary is a hyperplane $\{\mathbf{x} : \mathbf{w}^T\mathbf{x} = b\}$. Note that each linear classifier corresponds to a particular linear hypothesis map from the hypothesis space (2.4). However, we can use different loss functions to measure the quality of a linear classifier.

Three widely-used examples for ML methods that learn a linear classifier are logistic regression (see Section 3.6), the SVM (see Section 3.7) and the naive Bayes classifier (see Section 3.8).

In some application domains, the relation between features $\mathbf{x}$ and label y of a data point is highly non-linear. As a case in point, consider data points that are images of animals. The map that relates the pixel intensities of an image to a label value indicating if the image shows a cat is highly non-linear. For such applications, the hypothesis space (2.4) is not suitable as it only contains linear maps.

Another popular example for a parametrized hypothesis space studied in this book, besides the space of linear maps (2.4), contains also non-linear maps. This parametrized hypothesis space is obtained from a parametrized signal flow diagram which is referred to as an ANN. Section 3.11 discusses the construction of non-linear parametrized hypothesis spaces using an ANN.

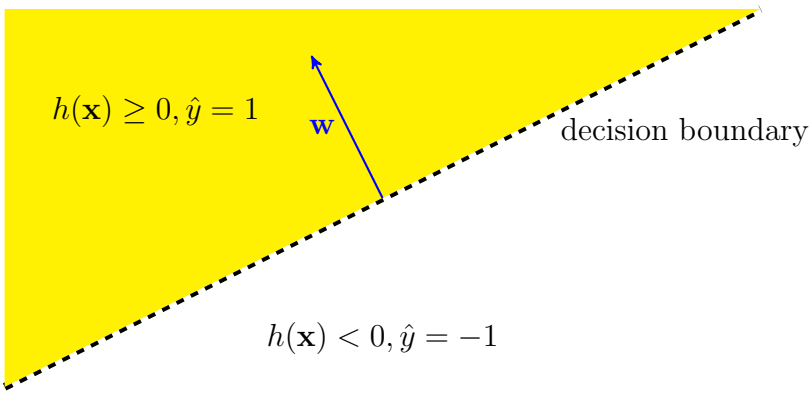


Figure 2.9: A hypothesis $h : \mathcal{X} \to \mathcal{Y}$ for a binary classification problem, with label space $\mathcal{Y} = \{-1, 1\}$ and feature space $\mathcal{X} = \mathbb{R}^2$, can be represented conveniently via the decision boundary (dashed line) which separates all feature vectors $\mathbf{x}$ with $h(\mathbf{x}) \geq 0$ from the region of feature vectors with $h(\mathbf{x}) < 0$. If the decision boundary is a hyperplane $\{\mathbf{x} : \mathbf{w}^T\mathbf{x} = b\}$ (with normal vector $\mathbf{w} \in \mathbb{R}^n$), we refer to the map h as a linear classifier.

Upgrading a Hypothesis Space via Feature Maps. Let us discuss a simple but powerful technique for enlarging ("upgrading") a given hypothesis space $\mathcal{H}$ to a larger hypothesis space $\mathcal{H}' \supseteq \mathcal{H}$ that offers a wider selection of hypothesis maps. The idea is to replace the original features $\mathbf{x}$ of a data point with new (transformed) features $\mathbf{z} = \boldsymbol{\Phi}(\mathbf{x})$. The transformed features are obtained by applying a feature map $\phi(\cdot)$ to the original features $\mathbf{x}$. This upgraded hypothesis space $\mathcal{H}'$ consists of all concatenations of the feature map ϕ and some hypothesis $h \in \mathcal{H}$,

$$\mathcal{H}' := \big\{h'(\mathbf{x}) := h\big(\boldsymbol{\Phi}(\mathbf{x})\big) : h \in \mathcal{H}\big\}. \tag{2.6}$$

The construction (2.6) allows for different combinations of a feature map $\boldsymbol{\Phi}$ and a "base" hypothesis space $\mathcal{H}$. The only requirement is that the output of the feature map can be used as (is compatible with the) input of each hypothesis $h \in \mathcal{H}$. Mathematically speaking, the range of the feature map $\boldsymbol{\Phi}$ must belong to the domain of the maps in $\mathcal{H}$.

Examples for ML methods that use a hypothesis space of the form (2.6) include polynomial regression (see Section 3.2), Gaussian basis regression (see Section 3.5) and kernel methods (see Section 3.9). The feature map in (2.6) might also be obtained from clustering or feature learning methods (see Section 8.4 and Section 9.2.1). One important family of ML methods that learn the feature map in (2.6) is deep learning (see Section 3.11). These methods use ANNs consisting of several layers with the final layer being a linear map and

the previous layers implementing a feature map of the original features (see Figure 2.10).

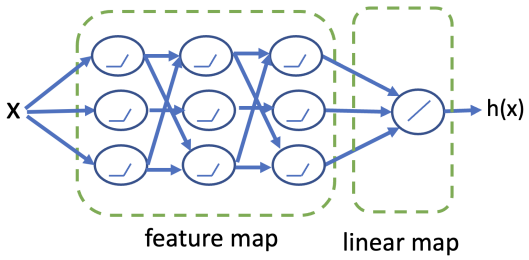


Figure 2.10: An ANN with several hidden layers that serve as a tunable (learnable) feature map which is concatenated with a linear map in the final (output) layer.

For the special case of the linear hypothesis space (2.4), the resulting enlarged hypothesis space (2.6) is given by all linear maps $\mathbf{w}^T\mathbf{z}$ of the transformed features $\boldsymbol{\Phi}(\mathbf{x})$. Combining the hypothesis space (2.4) with a non-linear feature map results in a hypothesis space that contains non-linear maps from the original features $\mathbf{x}$ to the predicted label $\hat{y}$,

$$\hat{y} = \mathbf{w}^T\mathbf{z} = \mathbf{w}^T\boldsymbol{\Phi}(\mathbf{x}). \tag{2.7}$$

Non-Numeric Features. The hypothesis space (2.4) can only be used for data points whose features are numeric vectors $\mathbf{x} = (x_1, \ldots, x_n)^T \in \mathbb{R}^n$. In some application domains, such as natural language processing, there is no obvious natural choice for numeric features. However, since ML methods based on the hypothesis space (2.4) are well developed (using numerical linear algebra), it might be useful to construct numerical features even for non-numeric data (such as text). For text data, there has been significant progress recently on methods that map a human-generated text into sequences of vectors (see [48, Chap. 12] for more details). Moreover, Section 9.3 will discuss an approach to generate numeric features for data points that have an intrinsic notion of similarity.

2.2.2 The Size of a Hypothesis Space

The notion of a hypothesis space being too small or being too large can be made precise in different ways. The size of a finite hypothesis space $\mathcal{H}$ can be defined as its cardinality $|\mathcal{H}|$ which is simply the number of its elements. For example, consider data points represented by $100 \times 10 = 1000$ black-and-white pixels and characterized by a binary label $y \in \{0, 1\}$. We can model such data points using the feature space $\mathcal{X} = \{0, 1\}^{1000}$ and label space $\mathcal{Y} = \{0, 1\}$. The largest possible hypothesis space $\mathcal{H} = \mathcal{Y}^{\mathcal{X}}$ consists of all maps from $\mathcal{X}$ to $\mathcal{Y}$. The size

or cardinality of this space is $|\mathcal{H}| = 2^{2^{1000}}$.

Many ML methods use a hypothesis space which contains infinitely many different predictor maps (see, e.g., (2.4)). For an infinite hypothesis space, we cannot use the number of its elements as a measure for its size. Indeed, for an infinite hypothesis space, the number of elements is not well-defined. Therefore, we measure the size of a hypothesis space $\mathcal{H}$ using its effective dimension $d_{\text{eff}}(\mathcal{H})$.

Consider a hypothesis space $\mathcal{H}$ consisting of maps $h : \mathcal{X} \to \mathcal{Y}$ that read in the features $\mathbf{x} \in \mathcal{X}$ and output a prediction $\hat{y} = h(\mathbf{x}) \in \mathcal{Y}$. We define the effective dimension $d_{\text{eff}}(\mathcal{H})$ of $\mathcal{H}$ as the maximum number $D \in \mathbb{N}$ such that for any set $\mathcal{D} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(D)}, y^{(D)}\big)\big\}$ of D data points with different feature vectors, we can always find a hypothesis $h \in \mathcal{H}$ that perfectly fits the labels, $y^{(i)} = h\big(\mathbf{x}^{(i)}\big)$ for $i = 1, \ldots, D$.

The effective dimension of a hypothesis space is closely related to the Vapnik–Chervonenkis dimension (VC dimension) [145]. The VC dimension is maybe the most widely used concept for measuring the size of infinite hypothesis spaces [145, 126, 12, 58]. However, the precise definition of the VC dimension are beyond the scope of this book. Moreover, the effective dimension captures most of the relevant properties of the VC dimension for our purposes. For a precise definition of the VC dimension and discussion of its applications in ML we refer to [126]. Let us illustrate the concept of effective dimension as a measure for the size of a hypothesis space with two examples: linear regression and polynomial regression.

Linear regression uses the hypothesis space

$$\mathcal{H}^{(n)} = \{h : \mathbb{R}^n \to \mathbb{R} : h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} \text{ with some vector } \mathbf{w} \in \mathbb{R}^n\}.$$

Consider a dataset $\mathcal{D}' = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}$ consisting of m data points. We refer to this number also as the sample size of the dataset $\mathcal{D}'$. Each data point is characterized by a feature vector $\mathbf{x}^{(i)} \in \mathbb{R}^n$ and a numeric label $y^{(i)} \in \mathbb{R}$.

Let us further assume that data points are realizations of i.i.d. RVs with a common probability distribution. Under this i.i.d. assumption, the feature matrix

$$\mathbf{X} = \big(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\big) \in \mathbb{R}^{n \times m},$$

which is obtained by stacking (column-wise) the feature vectors $\mathbf{x}^{(i)}$ (for $i = 1, \ldots, m$), is full (column-) rank with probability one as long as $m \leq n$. Basic linear algebra tells us that in this case we can find a linear map $h \in \mathcal{H}^{(n)}$ that perfectly fits the data points in $\mathcal{D}'$.

To summarize, as long as the number m of data points in $\mathcal{D}'$ does not exceed the number

of features characterizing each data point, i.e., as long as $m \leq n$, we can find (with probability one) a parameter vector $\widehat{\mathbf{w}}$ such that $y^{(i)} = \widehat{\mathbf{w}}^T \mathbf{x}^{(i)}$ for all $i = 1, \ldots, m$. Conversely, for $m > n$ it is easy to verify that with probability one, a dataset $\mathcal{D}'$ consisting of m randomly chosen data points cannot be perfectly fit by any linear hypothesis. Thus, the effective dimension of the linear hypothesis space $\mathcal{H}^{(n)}$ is $D = n$.

As a second example, consider the hypothesis space $\mathcal{H}^{(n)}_{\text{poly}}$ which is constituted by the set of polynomials with maximum degree n. The fundamental theorem of algebra tells us that any set of m data points with different features can be perfectly fit by a polynomial of degree n as long as $n \geq m$. Therefore, the effective dimension of the hypothesis space $\mathcal{H}^{(n)}_{\text{poly}}$ is $D = n$. Section 3.2 discusses polynomial regression in more detail.

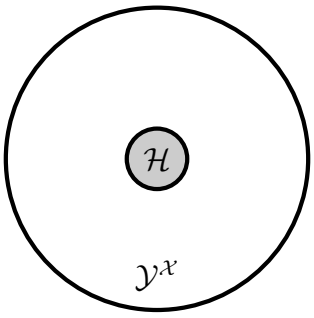


Figure 2.11: The hypothesis space $\mathcal{H}$ is a (typically very small) subset of the (typically very large) set $\mathcal{Y}^{\mathcal{X}}$ of all possible maps from feature space $\mathcal{X}$ into the label space $\mathcal{Y}$.

2.3 The Loss

Every ML method uses a (more of less explicit) hypothesis space $\mathcal{H}$ which consists of all computationally feasible predictor maps h. Which predictor map h out of all the maps in the hypothesis space $\mathcal{H}$ is the best for the ML problem at hand? To answer this questions, ML methods use the concept of a loss function. Formally, a loss function is a map

$$L : \mathcal{X} \times \mathcal{Y} \times \mathcal{H} \to \mathbb{R}_+ : \big(\big(\mathbf{x}, y\big), h\big) \mapsto L\left((\mathbf{x}, y), h\right)$$

which assigns a pair consisting of a data point, itself characterized by features $\mathbf{x}$ and label y, and a hypothesis $h \in \mathcal{H}$ the non-negative real number $L\left((\mathbf{x}, y), h\right)$.

The loss value $L\left((\mathbf{x}, y), h\right)$ quantifies the discrepancy between the true label y and the

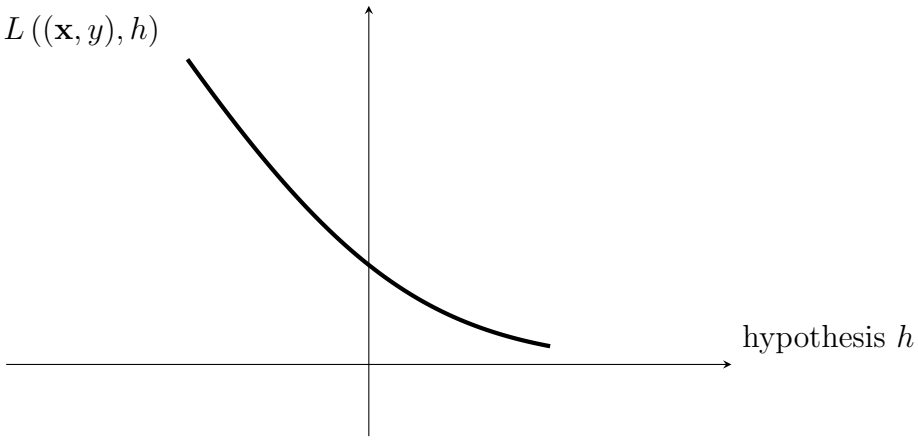


Figure 2.12: Some loss function $L\left((\mathbf{x}, y), h\right)$ for a fixed data point, with features $\mathbf{x}$ and label y, and varying hypothesis h. ML methods try to find (learn) a hypothesis that incurs minimum loss.

predicted label $h(\mathbf{x})$. A small (close to zero) value $L\left((\mathbf{x}, y), h\right)$ indicates a low discrepancy between predicted label and true label of a data point. Figure 2.12 depicts a loss function for a given data point, with features $\mathbf{x}$ and label y, as a function of the hypothesis $h \in \mathcal{H}$. The basic principle of ML methods can then be formulated as: Learn (find) a hypothesis out of a given hypothesis space $\mathcal{H}$ that incurs a minimum loss $L\left((\mathbf{x}, y), h\right)$ for any data point (see Chapter 4).

Much like the choice for the hypothesis space $\mathcal{H}$ used in a ML method, also the loss function is a design choice. We will discuss some widely used examples for loss function in Section 2.3.1 and Section 2.3.2. The choice for the loss function should take into account the computational complexity of searching the hypothesis space for a hypothesis with minimum loss. Consider a ML method that uses a parametrized hypothesis space and a loss function that is a convex and differentiable function of the parameters of a hypothesis. In this case, we can efficiently search for a hypothesis with small loss using the gradient-based methods discussed in Chapter 5. The minimization of a loss function that is either non-convex or non-differentiable tends to be computationally much more difficult. Section 4.2 discusses the computational complexities of different types of loss functions in more detail.

Beside computational aspects, the choice for the loss function should also take into account statistical aspects. For example, some loss functions result in ML methods that are more robust against outliers (see Section 3.3 and Section 3.7). The choice of loss function might also be guided by probabilistic models for the data generated in an ML application. Section 3.12 details how the ML principle of statistical inference provides an explicit construction of loss functions in terms of an (assumed) probability distribution for data points.

The choice for the loss function used to evaluate the quality of a hypothesis might also be influenced by its interpretability. Section 2.3.2 discusses loss functions for hypotheses that are used to classify data points into two categories. It seems natural to measure the quality of such a hypothesis by the average number of wrongly classified data points, which is precisely the average 0/1 loss (2.9) (see Section 2.3.2).

In contrast to its appealing interpretation as error-rate, the computational aspects of the average 0/1 loss are less pleasant. Minimizing the average 0/1 loss to learn an accurate hypothesis amounts to a non-convex and non-smooth optimization problem which is computationally challenging. Section 2.3.2 introduces the logistic loss as a computationally attractive alternative choice for the loss function in binary classification problems. Learning a hypothesis that minimizes a (average) logistic loss amounts to a smooth convex optimization problem. Chapter 5 discusses computationally cheap gradient-based methods for solving smooth convex optimization problems.

The above aspects (computation, statistic, interpretability) result typically in conflicting goals for the choice of a loss function. A loss function that has favourable statistical properties might incur a high computational complexity of the resulting ML method. Loss functions that result in computationally efficient ML methods might not allow for an easy interpretation (what does it mean intuitively if the logistic loss of a hypothesis in a binary classification problem is 10^{-1}?). It might therefore be useful to use different loss functions for the search of a good hypothesis (see Chapter 4) and for its final evaluation. Figure 2.13 depicts an example for two such loss functions, one of them used for learning a hypothesis by minimizing the loss and the other one used for the final performance evaluation.

For example, in a binary classification problem, we might use the logistic loss to search for (learn) an accurate hypothesis using the optimization methods in Chapter 4. The logistic loss is appealing for this purpose as it can be minimized via efficient gradient-based methods (see Chapter 5). After having found (learnt) an accurate hypothesis, we use the average 0/1 loss for the final performance evaluation. The 0/1 loss is appealing for this purpose as it can be interpreted as an error or misclassification rate. The loss function used for the final performance evaluation of a learnt hypothesis is sometimes referred to as metric.

2.3.1 Loss Functions for Numeric Labels

For ML problems involving data points with a numeric label $y \in \mathbb{R}$, i.e., for regression problems (see Section 2.1.2), a widely used (first) choice for the loss function can be the

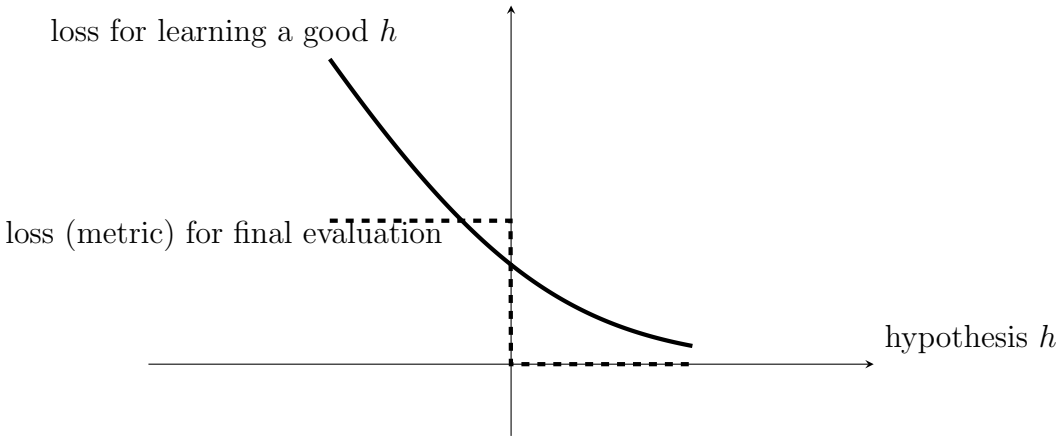


Figure 2.13: Two different loss functions for a given data point and varying hypothesis h. One of these loss functions (solid curve) is used to learn a good hypothesis by minimizing the loss. The other loss function (dashed curve) is used to evaluate the performance of the learnt hypothesis. The loss function used for this final performance evaluation is sometimes referred to as a metric.

squared error loss

$$L\left((\mathbf{x},y),h\right) := \big(y - \underbrace{h(\mathbf{x})}_{=\hat{y}}\big)^2. \tag{2.8}$$

The squared error loss (2.8) depends on the features $\mathbf{x}$ only via the predicted label value $\hat{y} = h(\mathbf{x})$. We can evaluate the squared error loss solely using the prediction $h(\mathbf{x})$ and the true label value y. Besides the prediction $h(\mathbf{x})$, no other properties of the features $\mathbf{x}$ are required to determine the squared error loss. We will slightly abuse notation and use the shorthand $L\left(y,\hat{y}\right)$ for any loss function that depends on the features $\mathbf{x}$ only via the predicted label $\hat{y} = h(\mathbf{x})$. Figure 2.14 depicts the squared error loss as a function of the prediction error $y - \hat{y}$.

The squared error loss (2.8) has appealing computational and statistical properties. For linear predictor maps $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$, the squared error loss is a convex and differentiable function of the weight vector $\mathbf{w}$. This allows, in turn, to efficiently search for the optimal linear predictor using efficient iterative optimization methods (see Chapter 5). The squared error loss also has a useful interpretation in terms of a probabilistic model for the features and labels. Minimizing the squared error loss is equivalent to maximum likelihood estimation within a linear Gaussian model [58, Sec. 2.6.3].

Another loss function used in regression problems is the absolute error loss $|\hat{y} - y|$. Using this loss function to guide the learning of a predictor results in methods that are robust against a few outliers in the training set (see Section 3.3). However, this improved

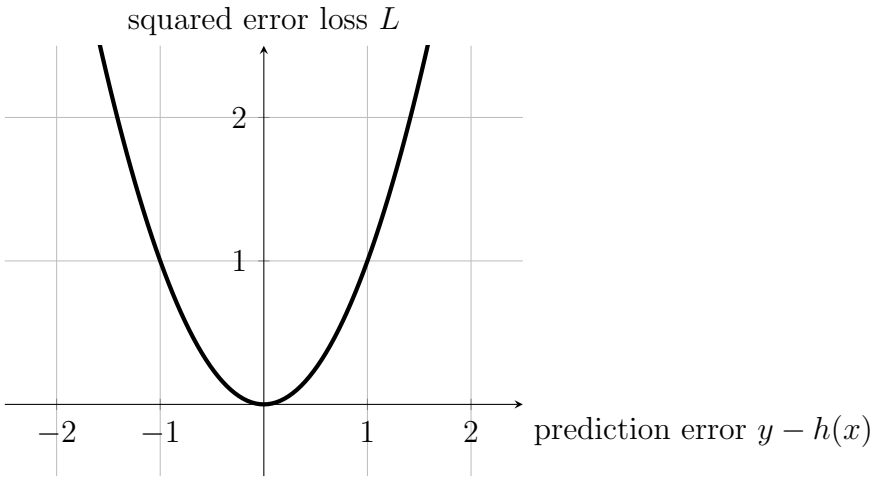


Figure 2.14: A widely used choice for the loss function in regression problems (with data points having numeric labels) is the squared error loss (2.8). Note that, for a given hypothesis h, we can evaluate the squared error loss only if we know the features $\mathbf{x}$ and the label y of the data point.

robustness comes at the expense of increased computational complexity of minimizing the (non-differentiable) absolute error loss compared to the (differentiable) squared error loss (2.8).

2.3.2 Loss Functions for Categorical Labels

Classification problems involve data points whose labels take on values from a discrete label space $\mathcal{Y}$. In what follows, unless stated otherwise, we focus on binary classification problems. Moreover, without loss of generality, we use the label space $\mathcal{Y} = \{-1, 1\}$. Classification methods aim at learning a hypothesis or classifier that maps the features $\mathbf{x}$ of a data point to a predicted label $\hat{y} \in \mathcal{Y}$.

It is often convenient to implement a classifier by thresholding $h(\mathbf{x}) \in \mathbb{R}$ of a hypothesis map that can deliver arbitrary real numbers. We then classify a data point as $\hat{y} = 1$ if $h(\mathbf{x}) > 0$ and $\hat{y} = -1$ otherwise. Thus, the ultimate prediction is obtained from the sign of the value $h(\mathbf{x})$.

While the sign of $h(\mathbf{x})$ determines the predicted label $\hat{y}$, we can use the absolute value $|h(\mathbf{x})|$ as a measure for the confidence in the classification. It is customary to abuse notation and refer to both, the final classification rule (obtained by a threshold step) $\mathbf{x} \mapsto \hat{y}$ and the hypothesis $h(\mathbf{x})$ (whose output is compared against a threshold) as a binary classifier.

In principle, we can measure the quality of a hypothesis when used to classify data points

using the squared error loss (2.8). However, the squared error is typically a poor measure for the quality of a hypothesis $h(\mathbf{x})$ that classifies a data point with binary label $y \in \{-1, 1\}$. Figure 2.15 illustrates how the squared error loss of a hypothesis can be (highly) misleading for binary classification.

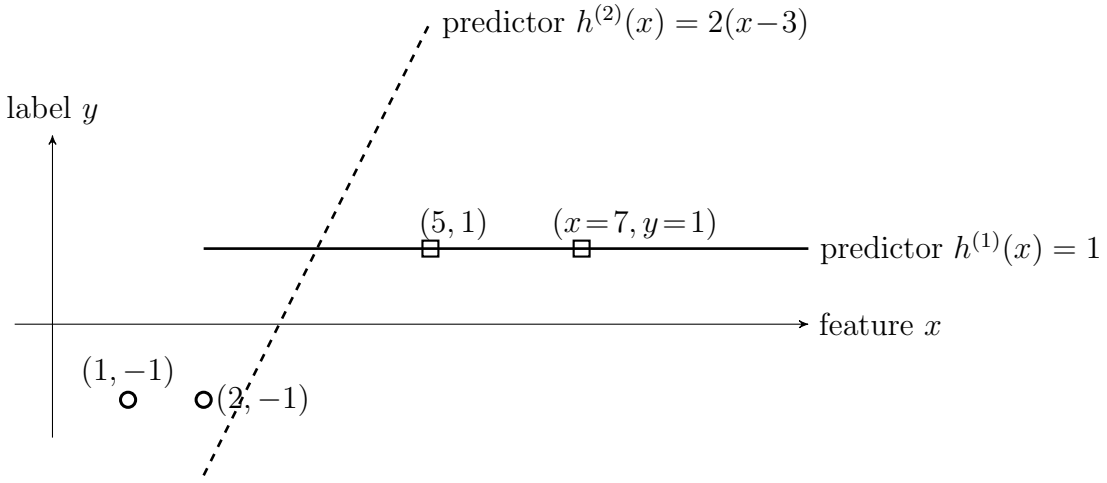


Figure 2.15: A training set consisting of four data points with binary labels $\hat{y}^{(i)} \in \{-1, 1\}$. Minimizing the squared error loss (2.8) would prefer the (poor) classifier $h^{(1)}$ over the (reasonable) classifier $h^{(2)}$.

Figure 2.15 depicts a dataset $\mathcal{D}$ consisting of $m = 4$ data points with binary labels $y^{(i)} \in \{-1, 1\}$, for $i = 1, \ldots, m$. The figure also depicts two candidate hypothesis maps $h^{(1)}(x)$ and $h^{(2)}(x)$ that can be used for classifying data points. The classifications $\hat{y}$ obtained with the hypothesis $h^{(2)}(x)$ would perfectly match the labels of the four training data points since $h^{(2)}\big(x^{(i)}\big) \geq 0$ if and if only if $y^{(i)} = 1$. In contrast, the classifications $\hat{y}^{(i)}$ obtained by thresholding $h^{(1)}(x)$ are wrong for data points with $y = -1$.

Looking at $\mathcal{D}$, it seems reasonable to prefer $h^{(2)}$ over $h^{(1)}$ for classifying data points. However, the squared error loss incurred by the (reasonable) classifier $h^{(2)}$ is much higher than the squared error loss incurred by the (poor) classifier $h^{(1)}$. The squared error loss is typically a bad choice for assessing the quality of a hypothesis that is used for classifying data points into different categories.

Generally speaking, we want the loss function to punish (deliver large values for) a hypothesis that is very confident ($|h(\mathbf{x})|$ is large) in a wrong classification ($\hat{y} \neq y$). Moreover, a useful loss function should not punish (deliver small values for) a hypothesis is very confident ($|h(\mathbf{x})|$ is large) in a correct classification ($\hat{y} = y$). However, by its very definition, the

squared error loss (2.8) yields large values if the confidence $|h(\mathbf{x})|$ is large, no matter if the resulting classification (obtained after thresholding) is correct or wrong.

We now discuss some loss functions which have proven useful for assessing the quality of a hypothesis that is used to classify data points. Unless noted otherwise, the formulas for these loss functions are valid only if the label values are the real numbers -1 and 1 (the label space is $\mathcal{Y} = \{-1, 1\}$). These formulas need to be modified accordingly if different label values are used. For example, instead of the label space $\mathcal{Y} = \{-1, 1\}$, we could equally well use the label space $\mathcal{Y} = \{0, 1\}$, or $\mathcal{Y} = \{\square, \triangle\}$ or $\mathcal{Y} = \{$ "Class 1", "Class 2"$\}$.

A natural choice for the loss function can be based on the requirement that a reasonable hypothesis should deliver a correct classifications, $\hat{y} = y$ for any data point. This suggests to learn a hypothesis $h(\mathbf{x})$ by minimizing the 0/1 loss

$$L\left((\mathbf{x}, y), h\right) := \begin{cases} 1 & \text{if } y \neq \hat{y} \\ 0 & \text{else,} \end{cases} \text{ with } \hat{y} = 1 \text{ for } h(\mathbf{x}) \geq 0, \text{ and } \hat{y} = -1 \text{ for } h(\mathbf{x}) < 0. \quad (2.9)$$

Figure 2.16 illustrates the 0/1 loss (2.9) for a data point with features $\mathbf{x}$ and label $y=1$ as a function of the hypothesis value $h(\mathbf{x})$. The 0/1 loss is equal to zero if the hypothesis yields a correct classification $\hat{y} = y$. For a wrong classification $\hat{y} \neq y$, the 0/1 loss yields the value one.

The 0/1 loss (2.9) is conceptually appealing when data points are interpreted as realizations of i.i.d. RVs with the same probability distribution $p(\mathbf{x}, y)$. Given m realizations $(\mathbf{x}^{(i)}, y^{(i)})\big\}_{i=1}^{m}$ of such i.i.d. RVs,

$$(1/m) \sum_{i=1}^{m} L\left((\mathbf{x}^{(i)}, y^{(i)}), h\right) \approx p(y \neq \hat{y}) \quad (2.10)$$

with high probability for sufficiently large sample size m. A precise formulation of the approximation (2.10) can be obtained from the law of large numbers [11, Section 1]. We can apply the law of large numbers since the loss values $L\left((\mathbf{x}^{(i)}, y^{(i)}), h\right)$ are realizations of i.i.d. RVs. It is customary to indicate the average 0/1 loss of a hypothesis indirectly via the accuracy $1 - (1/m) \sum_{i=1}^{m} L\left((\mathbf{x}^{(i)}, y^{(i)}), h\right)$.

In view of (2.10), the 0/1 loss (2.9) seems a very natural choice for assessing the quality of a classifier if our goal is to enforce correct classifications $\hat{y} = y$. This appealing statistical property of the 0/1 loss comes at the cost of a high computational complexity. Indeed, for a given data point $(\mathbf{x}, y)$, the 0/1 loss (2.9) is non-convex and non-differentiable when

viewed as a function of the hypothesis h. Thus, ML methods that use the 0/1 loss to learn a hypothesis map typically involve advanced optimization methods to solve the resulting learning problem (see Section 3.8).

To avoid the non-convexity of the 0/1 loss (2.9) we might approximate it by a convex loss function. One popular convex approximation of the 0/1 loss is the hinge loss

$$L\left((\mathbf{x}, y), h\right) := \max\{0, 1 - yh(\mathbf{x})\}. \tag{2.11}$$

Figure 2.16 depicts the hinge loss (2.11) as a function of the hypothesis $h(\mathbf{x})$. The hinge loss (2.11) becomes minimal (equal to zero) for a correct classification ($\hat{y} = y$) with sufficient confidence $h(\mathbf{x}) \geq 1$. For a wrong classification ($\hat{y} \neq y$), the hinge loss increases monotonically with the confidence $|h(x)|$ in the wrong classification. While the hinge loss avoids the non-convexity of the 0/1 loss, it still is a non-differentiable function of $h(\mathbf{x})$. A non-differentiable loss function cannot be minimized by simple gradient-based methods (see Chapter 5) but require more advanced optimization methods.

Beside the 0/1 loss and the hinge loss, another popular loss function for binary classification problems is the logistic loss

$$L\left((\mathbf{x}, y), h\right) := \log(1 + \exp(-yh(\mathbf{x}))). \tag{2.12}$$

The logistic loss (2.12) is used in logistic regression (see Section 3.6) to measure the usefulness of a linear hypothesis $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$. Figure 2.16 depicts the logistic loss (2.12) as a function of the hypothesis $h(\mathbf{x})$. The logistic loss (2.12) is a convex and differentiable function of $h(\mathbf{x})$. For a correct classification ($\hat{y} = y$), the logistic loss (2.12) decreases monotonically with increasing confidence $h(\mathbf{x})$. For a wrong classification ($\hat{y} \neq y$), the logistic loss increases monotonically with increasing confidence $|h(\mathbf{x})|$ in the wrong classification.

Both the hinge loss (2.11) and the logistic loss (2.12) are convex functions of the weights $\mathbf{w} \in \mathbb{R}^n$ in a linear hypothesis map $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$. However, in contrast to the hinge loss, the logistic loss (2.12) is also a differentiable function of the $\mathbf{w}$. The convex and differentiable logistic loss function can be minimized using simple gradient-based methods such as gradient descent (GD) (see Chapter 5.5). In contrast, we cannot use basic gradient-based methods to minimize the hinge loss since it does not have a gradient for $h(\mathbf{x})y = 1$. However, we can apply a generalization of GD which is known as subgradient descent [16]. Subgradient descent is obtained from GD by generalizing the concept of a gradient to that of a subgradient.

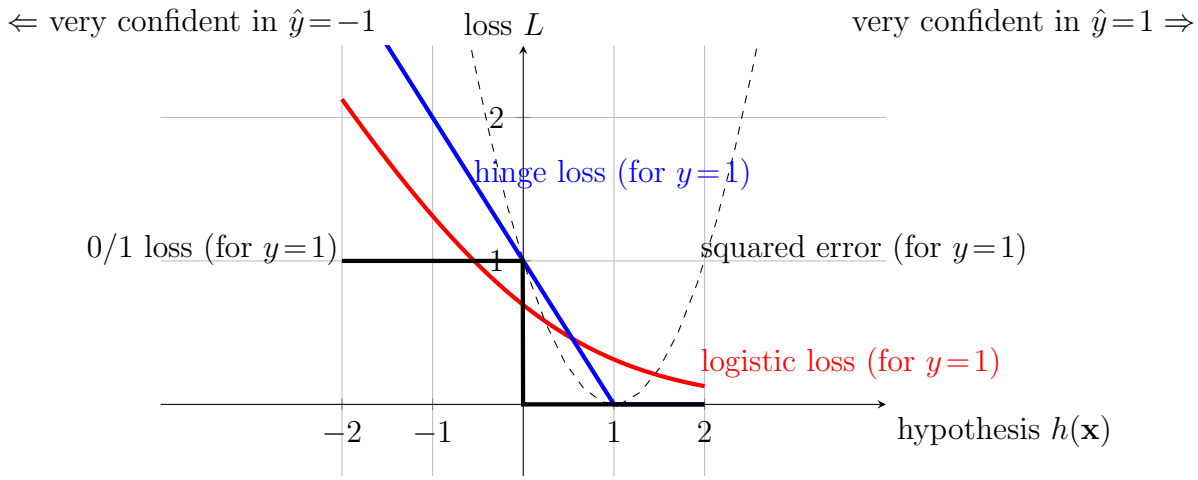


Figure 2.16: The solid curves depict three widely-used loss functions for binary classification. A data point with features $\mathbf{x}$ and label $y = 1$ is classified as $\hat{y} = 1$ if $h(\mathbf{x}) \geq 0$ and classified as $\hat{y} = -1$ if $h(\mathbf{x}) < 0$. We can interpret the absolute value $|h(\mathbf{x})|$ as the confidence in the classification $\hat{y}$. The more confident we are in a correct classification ($\hat{y} = y = 1$), i.e, the more positive $h(\mathbf{x})$, the smaller the loss. Note that each of the three loss functions for binary classification tends monotonically towards 0 for increasing $h(\mathbf{x})$. The dashed curve depicts the squared error loss (2.8), which increases for increasing $h(\mathbf{x})$.

2.3.3 Loss Functions for Ordinal Label Values

Some loss functions are particularly suited for predicting ordinal label values (see Section 2.1). Consider data points representing areal images of rectangular areas of size 1km by 1km. We characterize each data point (rectangular area) by the feature vector $\mathbf{x}$ obtained by stacking the RGB values of each image pixel (see Figure 2.5). Beside the feature vector, each rectangular area is characterized by a label $y \in \{1, 2, 3\}$ where

- $y = 1$ means that the area contains no trees.
- $y = 2$ means that the area is partially covered by trees.
- $y = 3$ means that the are is entirely covered by trees.

We might consider the label value $y = 2$ to be "larger" than label value $y = 1$ and label value $y = 3$ to be "larger" than label value $y = 2$. Let us construct a loss function that takes such an ordering of label values into account when evaluating the quality of the predictions $h(\mathbf{x})$.

Consider a data point with features $\mathbf{x}$ and label $y = 1$ as well as two different hypotheses

$h^{(a)}, h^{(b)} \in \mathcal{H}$. The hypothesis $h^{(a)}$ delivers the prediction $\hat{y}^{(a)} = h^{(a)}(\mathbf{x}) = 2$, while the other hypothesis $h^{(b)}$ delivers the prediction $\hat{y}^{(a)} = h^{(a)}(\mathbf{x}) = 3$.

Both predictions are wrong, since they are different form the true label value $y = 1$. However, it seems reasonable to consider the prediction $\hat{y}^{(a)}$ to be less wrong than the prediction $\hat{y}^{(b)}$ and therefore we would prefer the hypothesis $h^{(a)}$ over $h^{(b)}$. Unfortunately, the 0/1 loss is the same for $h^{(a)}$ and $h^{(b)}$ and therefore does not reflect our preference for $h^{(a)}$. We need to modify (or tailor) the 0/1 loss to take into account the application-specific ordering of label values. For the above application, we might define a loss function via

$$L\left((\mathbf{x}, y), h\right) := \begin{cases} 0 & \text{, when } y = h(\mathbf{x}) \\ 10 & \text{, when } |y - h(\mathbf{x})| = 1 \\ 100 & \text{otherwise.} \end{cases} \tag{2.13}$$

2.3.4 Empirical Risk

The basic idea of ML methods (including those discussed in Chapter 3) is to find (or learn) a hypothesis (out of a given hypothesis space $\mathcal{H}$) that incurs minimum loss when applied to arbitrary data points. To make this informal goal precise we need to specify what we mean by "arbitrary data point". One of the most successful approaches to define the notion of "arbitrary data point" is by probabilistic models for the observed data points.

The most basic and widely-used probabilistic model interprets data points $\left(\mathbf{x}^{(i)}, y^{(i)}\right)$ as realizations of i.i.d. RVs with a common probability distribution $p(\mathbf{x}, y)$. Given such a probabilistic model, it seems natural to measure the quality of a hypothesis h by the expected loss or Bayes risk [85]

$$\mathbb{E}\big\{L\left((\mathbf{x}, y), h\right)\big\} := \int_{\mathbf{x},y} L\left((\mathbf{x}, y), h\right) dp(\mathbf{x}, y). \tag{2.14}$$

The Bayes risk of h is the expected value of the loss $L\left((\mathbf{x}, y), h\right)$ incurred when applying the hypothesis h to (the realization of) a random data point with features $\mathbf{x}$ and label y. Note that the computation of the Bayes risk (2.15) requires the joint probability distribution $p(\mathbf{x}, y)$ of the (random) features and label of data points.

The Bayes risk (2.15) seems to be reasonable performance measure for a hypothesis h. Indeed, the Bayes risk of a hypothesis is small only if the hypothesis incurs a small loss on average for data points drawn from the probability distribution $p(\mathbf{x}, y)$. However, it might be

challenging to verify if the data points generated in a particular application domain can be accurately modelled as realizations (draws) from a probability distribution $p(\mathbf{x}, y)$. Moreover, it is also often the case that we do not know the correct probability distribution $p(\mathbf{x}, y)$.

Let us assume for the moment, that data points are generated as i.i.d. realizations of a common probability distribution $p(\mathbf{x}, y)$ which is known. It seems reasonable to learn a hypothesis h^* that incurs minimum Bayes risk,

$$\mathbb{E}\big\{L\left((\mathbf{x}, y), h^*\right)\big\} := \min_{h \in \mathcal{H}} \mathbb{E}\big\{L\left((\mathbf{x}, y), h\right)\big\}. \tag{2.15}$$

A hypothesis that solves (2.15), i.e., that achieves the minimum possible Bayes risk, is referred to as a Bayes estimator [85, Chapter 4]. The main computational challenge for learning the optimal hypothesis is the efficient (numerical) solution of the optimization problem (2.15). Efficient methods to solve the optimization problem (2.15) are studied within estimation theory [85, 153].

The focus of this book is on ML methods which do not require knowledge of the underlying probability distribution $p(\mathbf{x}, y)$. One of the most widely used principle for these ML methods is to approximate the Bayes risk by an empirical (sample) average over a finite set of labeled data points $\mathcal{D} = \big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)$. In particular, we define the empirical risk of a hypothesis $h \in \mathcal{H}$ for a dataset $\mathcal{D}$ as

$$\widehat{L}(h|\mathcal{D}) = (1/m) \sum_{i=1}^{m} L\left((\mathbf{x}^{(i)}, y^{(i)}), h\right). \tag{2.16}$$

The empirical risk of the hypothesis $h \in \mathcal{H}$ is the average loss on the data points in $\mathcal{D}$. To ease notational burden, we use $\widehat{L}(h)$ as a shorthand for $\widehat{L}(h|\mathcal{D})$ if the underlying dataset $\mathcal{D}$ is clear from the context. Note that in general the empirical risk depends on both, the hypothesis h and the (features and labels of the) data points in the dataset $\mathcal{D}$.

If the data points used to compute the empirical risk (2.16) are (can be modelled as) realizations of i.i.d. RVs whose common probability distribution is $P^{(\mathbf{x},y)}$, basic results of probability theory tell us that

$$\mathbb{E}\big\{L\left((\mathbf{x}, y), h\right)\big\} \approx (1/m) \sum_{i=1}^{m} L\left((\mathbf{x}^{(i)}, y^{(i)}), h\right) \text{ for sufficiently large sample size } m. \tag{2.17}$$

The approximation error in (2.17) can be quantified precisely by some of the most basic results of probability theory. These results are often summarized under the umbrella term

law of large numbers [11, 50, 9].

Many (if not most) ML methods are motivated by (2.17) which suggests that a hypothesis with small empirical risk (2.16) will also result in a small expected loss. The minimum possible expected loss is achieved by the Bayes estimator of the label y, given the features $\mathbf{x}$. However, to actually compute the optimal estimator we would need to know the (joint) probability distribution $p(\mathbf{x}, y)$ of features $\mathbf{x}$ and label y.

Confusion Matrix

Consider a dataset $\mathcal{D}$ with data points characterized by feature vectors $\mathbf{x}^{(i)}$ and labels $y^{(i)} \in \{1, \ldots, k\}$. We might interpret the label value of a data point as the index of a category or class to which the data point belongs to. Multi-class classification problems aim at learning a hypothesis h such that $h(\mathbf{x}) \approx y$ for any data point.

In principle, we could measure the quality of a given hypothesis h by the average 0/1 loss incurred on the labeled data points in (the training set) $\mathcal{D}$. However, if the dataset $\mathcal{D}$ contains mostly data points with one dominant label value, the average 0/1 loss might obscure the performance of h for data points having one of the rare label values (of a minority category). Indeed, even if the average 0/1 loss is very small, the hypothesis might perform poorly for data points of a minority category.

The confusion matrix generalizes the concept of the 0/1 loss to application domains where the relative frequency (fraction) of data points with a specific label value varies significantly (imbalanced data). Instead of considering only the average 0/1 loss incurred by a hypothesis on a dataset $\mathcal{D}$, we use a whole family of loss functions. In particular, for each pair of two different label values $c, c' \in \{1, \ldots, k\}$, $c \neq c'$, we define the loss function

$$L^{(c \to c')}\big((\mathbf{x}, y), h\big) := \begin{cases} 1 & \text{if } y = c \text{ and } h(\mathbf{x}) = c' \\ 0 & \text{otherwise} \end{cases}. \tag{2.18}$$

We then compute the average loss (2.18) incurred on the dataset $\mathcal{D}$,

$$\widehat{L}^{(c \to c')}(h|\mathcal{D}) := (1/m) \sum_{i=1}^{m} L^{(c \to c')}\big((\mathbf{x}^{(i)}, y^{(i)}), h\big) \text{ for } c, c' \in \{1, \ldots, k\}. \tag{2.19}$$

It is convenient to arrange the values (2.19) as a matrix which is referred to as a confusion matrix. The rows of a confusion matrix correspond to different values c of the true label of a data point. The columns of a confusion matrix correspond to different values c' delivered

by the hypothesis $h(\mathbf{x})$. The (c, c')-th entry of the confusion matrix is $\widehat{L}^{(c \to c')}(h|\mathcal{D})$.

Precision, Recall and F-Measure

Consider an object detection application where data points are images. The label of data points might indicate the presence ($y = 1$) or absence ($y = -1$) of an object, it is then customary to define the [5]

$$\text{recall } := \widehat{L}^{(1\to 1)}(h|\mathcal{D}), \text{ and the precision } := \frac{\widehat{L}^{(1\to 1)}(h|\mathcal{D})}{\widehat{L}^{(1\to 1)}(h|\mathcal{D}) + \widehat{L}^{(-1\to 1)}(h|\mathcal{D})}. \tag{2.20}$$

Clearly, we would like to find a hypothesis with both, large recall and large precision. However, these two goals are typically conflicting, a hypothesis with a high recall will have small precision. Depending on the application, we might prefer having a high recall and tolerate a lower precision.

It might be convenient to combine the recall and precision of a hypothesis into a single quantity,

$$F_1 := 2\frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \tag{2.21}$$

The F measure (2.21) is the harmonic mean [2] of the precision and recall of a hypothesis h. It is a special case of the F_β-score

$$F_\beta := \big(1 + \beta^2\big)\frac{\text{precision} \times \text{recall}}{\beta^2 \text{precision} + \text{recall}}. \tag{2.22}$$

The F measure (2.21) is obtained from (2.22) for the choice $\beta = 1$. It is therefore customary to refer to (2.21) as the F_1-score of a hypothesis h.

2.3.5 Regret

In some ML applications, we might have access to the predictions obtained from some reference methods which are referred to as experts [20, 60]. The quality of a hypothesis h is measured via the difference between the loss incurred by its predictions $h(\mathbf{x})$ and the loss incurred by predictions of the experts. This difference, which is referred to as the regret, measures by how much we regret to have used the prediction $h(\mathbf{x})$ instead of using(or following) the expert prediction. The goal of regret minimization is to learn a hypothesis with a small regret compared to given set of experts.

The concept of regret minimization is useful when we do not make any probabilistic assumptions (see Section 2.1.4) about the data. Without a probabilistic model we cannot use the Bayes risk of the (optimal) Bayes estimator as a baseline (or benchmark). The concept of regret minimization avoids the need for a probabilistic model of the data to obtain a baseline [20]. This approach replaces the Bayes risk with the regret relative to given reference methods (the experts).

2.3.6 Rewards as Partial Feedback

Some applications involve data points whose labels are so difficult or costly to determine that we cannot assume to have any labeled data available. Without any labeled data, we cannot evaluate the loss function for different choices for the hypothesis. Indeed, the evaluation of the loss function typically amounts to measuring the distance between predicted label and true label of a data point. Instead of evaluating a loss function, we must rely on some indirect feedback or "reward" that indicates the usefulness of a particular prediction [20, 137].

Consider the ML problem of predicting the optimal steering directions for an autonomous car. The prediction has to be recalculated for each new state of the car. ML methods can sense the state via a feature vector $\mathbf{x}$ whose entries are pixel intensities of a snapshot. The goal is to learn a hypothesis map from the feature vector $\mathbf{x}$ to a guess $\hat{y} = h(\mathbf{x})$ for the optimal steering direction y (true label). Unless the car circles around in small area with fixed obstacles, we have no access to labeled data points or reference driving scenes for which we already know the optimum steering direction. Instead, the car (control unit) needs to learn the hypothesis $h(\mathbf{x})$ based solely on the feedback signals obtained from various sensing devices (cameras, distance sensors).

2.4 Putting Together the Pieces

A guiding theme of this book is that ML methods are obtained by different combinations of data, model and loss. We will discuss some key principles behind these methods in depth in the following chapters. Let us develop some intuition for how ML methods operate by considering a very simple ML problem. This problem involves data points that are characterized by a single numeric feature $x \in \mathbb{R}$ and a numeric label $y \in \mathbb{R}$. We assume to have access to m labeled data points

$$\big(x^{(1)}, y^{(1)}\big), \ldots, \big(x^{(m)}, y^{(m)}\big) \tag{2.23}$$

for which we know the true label values $y^{(i)}$.

The assumption of knowing the exact true label values $y^{(i)}$ for any data point is an idealization. We might often face labelling or measurement errors such that the observed labels are noisy versions of the true label. Later on, we will discuss techniques that allow ML methods to cope with noisy labels in Chapter 7.

Our goal is to learn a (hypothesis) map $h : \mathbb{R} \to \mathbb{R}$ such that $h(x) \approx y$ for any data point. Given a data point with feature x, the function value $h(x)$ should be an accurate approximation of its label value y. We require the map to belong to the hypothesis space $\mathcal{H}$ of linear maps,

$$h^{(w_0,w_1)}(x) = w_1 x + w_0. \tag{2.24}$$

The predictor (2.24) is parameterized by the slope w_1 and the intercept (bias or offset) w_0. We indicate this by the notation $h^{(w_0,w_1)}$. A particular choice for the weights w_1, w_0 defines a linear hypothesis $h^{(w_0,w_1)}(x) = w_1 x + w_0$.

Let us use the linear hypothesis map $h^{(w_0,w_1)}(x)$ to predict the labels of training data points. In general, the predictions $\hat{y}^{(i)} = h^{(w_0,w_1)}\big(x^{(i)}\big)$ will not be perfect and incur a non-zero prediction error $\hat{y}^{(i)} - y^{(i)}$ (see Figure 2.17).

We measure the goodness of the predictor map $h^{(w_0,w_1)}$ using the average squared error loss (see (2.8))

$$\begin{aligned} f(w_0, w_1) &:= (1/m) \sum_{i=1}^{m} \big(y^{(i)} - h^{(w_0,w_1)}(x^{(i)})\big)^2 \\ &\overset{(2.24)}{=} (1/m) \sum_{i=1}^{m} \big(y^{(i)} - (w_1 x^{(i)} + w_0)\big)^2. \end{aligned} \tag{2.25}$$

The training error $f(w_0, w_1)$ is the average of the squared prediction errors incurred by the predictor $h^{(w_0,w_1)}(x)$ to the labeled data points (2.23).

It seems natural to learn a good predictor (2.24) by choosing the parameters w_0, w_1 to minimize the training error

$$\min_{w_0,w_1 \in \mathbb{R}} f(w_0, w_1) \overset{(2.25)}{=} \min_{w_1,w_0 \in \mathbb{R}} (1/m) \sum_{i=1}^{m} \big(y^{(i)} - (w_1 x^{(i)} + w_0)\big)^2. \tag{2.26}$$

The optimal parameters $\hat{w}_0, \hat{w}_1$ are characterized by the zero-gradient condition,[2]

$$\left.\frac{\partial f(w_0, w_1)}{\partial w_0}\right|_{w_0=\hat{w}_0, w_1=\hat{w}_1} = 0, \text{ and } \left.\frac{\partial f(w_0, w_1)}{\partial w_1}\right|_{w_0=\hat{w}_0, w_1=\hat{w}_1} = 0. \tag{2.27}$$

Inserting (2.25) into (2.27) and by using basic rules for calculating derivatives (see, e.g., [119]), we obtain the following optimality conditions

$$\begin{aligned} (1/m)\sum_{i=1}^{m}\big(y^{(i)} - (\hat{w}_1 x^{(i)} + \hat{w}_0)\big) = 0, \text{ and} \\ (1/m)\sum_{i=1}^{m} x^{(i)}\big(y^{(i)} - (\hat{w}_1 x^{(i)} + \hat{w}_0)\big) = 0. \end{aligned} \tag{2.28}$$

Any parameter values $\hat{w}_0, \hat{w}_1 \in \mathbb{R}$ that satisfy (2.28) define a hypothesis map $h^{(\hat{w}_0, \hat{w}_1)}(x) = \hat{w}_1 x + \hat{w}_0$ that is optimal in the sense of incurring a minimum training error,

$$f(\hat{w}_0, \hat{w}_1) = \min_{w_0, w_1 \in \mathbb{R}} f(w_0, w_1).$$

Let us rewrite the optimality condition (2.28) using matrices and vectors. To this end, we first rewrite the hypothesis (2.24) as

$$h(\mathbf{x}) = \mathbf{w}^T \mathbf{x} \text{ with } \mathbf{w} = \big(w_0, w_1\big)^T, \mathbf{x} = \big(1, x\big)^T.$$

Let us stack the feature vectors $\mathbf{x}^{(i)} = \big(1, x^{(i)}\big)^T$ and labels $y^{(i)}$ of the data points (2.23) into a feature matrix and a label vector,

$$\mathbf{X} = \big(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\big)^T \in \mathbb{R}^{m \times 2}, \mathbf{y} = \big(y^{(1)}, \ldots, y^{(m)}\big)^T \in \mathbb{R}^m. \tag{2.29}$$

We can then reformulate (2.28) as

$$\mathbf{X}^T\big(\mathbf{y} - \mathbf{X}\widehat{\mathbf{w}}\big) = \mathbf{0}. \tag{2.30}$$

The optimality conditions (2.30) and (2.28) are equivalent in the following sense. The entries of any parameter vector $\widehat{\mathbf{w}} = \big(\hat{w}_0, \hat{w}_1\big)$ that satisfies (2.30) are solutions to (2.28) and vice-versa.

[2] A necessary and sufficient condition for $\widehat{\mathbf{w}}$ to minimize a convex and differentiable function $f(\mathbf{w})$ is $\nabla f(\widehat{\mathbf{w}}) = \mathbf{0}$ [14, Sec. 4.2.3].

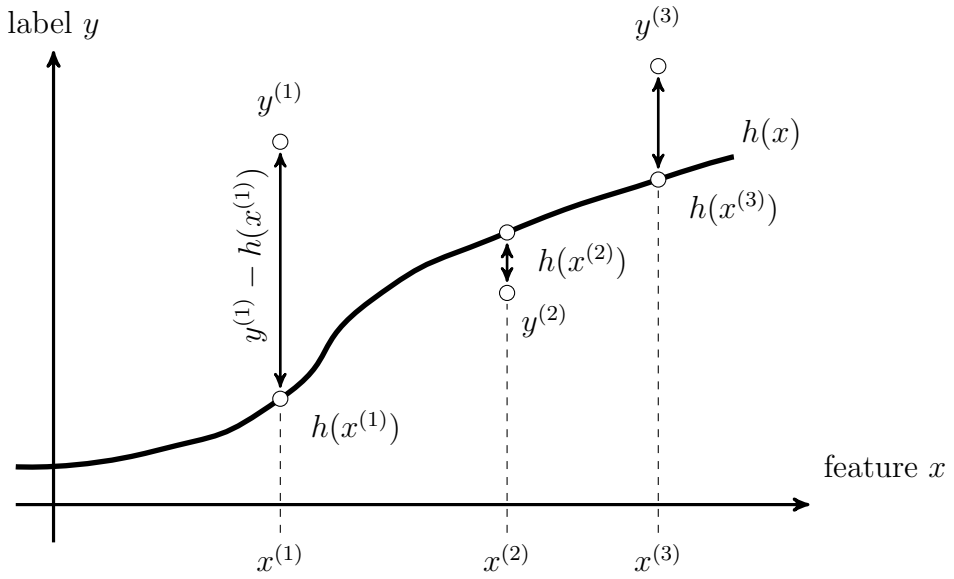


Figure 2.17: We can evaluate the quality of a particular predictor $h \in \mathcal{H}$ by measuring the prediction error $y - h(x)$ obtained for a labeled data point (x, y).

2.5 Exercises

Exercise 2.1. Perfect Prediction. Consider data points that are characterized by a single numeric feature $x \in \mathbb{R}$ and a numeric label $y \in \mathbb{R}$. We use a ML method to learn a hypothesis map $h : \mathbb{R} \to \mathbb{R}$ based on a training set consisting of three data points

$$(x^{(1)} = 1, y^{(1)} = 3), (x^{(2)} = 4, y^{(2)} = -1), (x^{(3)} = 1, y^{(3)} = 5).$$

Is there any chance for the ML method to learn a hypothesis map that perfectly fits the data points such that $h\big(x^{(i)}\big) = y^{(i)}$ for $i = 1, \ldots, 3$. Hint: Try to visualize the data points in a scatterplot and various hypothesis maps (see Figure 1.3).

Exercise 2.2. Temperature Data. Consider a features of daily air temperatures $x^{(1)}, \ldots, x^{(m)}$ measured at the FMI observation station "Utsjoki Nuorgam" during 01.12.2019 and 29.02.2020. Thus, $x^{(1)}$ is the daily temperature measured on 01.12.2019, $x^{(2)}$ is the daily temperature on 02.12.2019, and $x^{(m)}$ is the daily temperature measured on 29.02.2020. You can download this features from the link `https://en.ilmatieteenlaitos.fi/download-observations`. ML methods often determine few parameters to characterize large collections of data points. Compute, for the above temperature measurement features, the following quantities:

- the minimum $A := \min_{i=1,\ldots,m} x^{(i)}$
- the maximum $B := \max_{i=1,\ldots,m} x^{(i)}$
- the average $C := (1/m) \sum_{i=1,\ldots,m} x^{(i)}$
- the standard deviation $D := \sqrt{(1/m) \sum_{i=1,\ldots,m} \big(x^{(i)} - C\big)^2}$

Exercise 2.3. Deep Learning on Raspberry PI. Consider the tiny desktop computer "RaspberryPI" equipped with a total of 8 Gigabytes memory [31]. We want implement a ML algorithm that learns a hypothesis map that is represented by a deep ANN involving $n = 10^6$ numeric parameters. Each parameter is quantized using 8 bits (= 1 Byte). How many different hypotheses can we store at most on a RaspberryPI computer? (You can assume that 1Gigabyte = 10^9Bytes.)

Exercise 2.4. Ensembles. For some applications it can be a good idea to not learn a single hypothesis but to learn a whole ensemble of hypothesis maps $h^{(1)}, \ldots, h^{(B)}$. These hypotheses might even belong to different hypothesis spaces, $h^{(1)} \in \mathcal{H}^{(1)}, \ldots, h^{(B)} \in \mathcal{H}^{(B)}$. These hypothesis spaces can be arbitrary except that they are defined for the same feature space and label space. Given such an ensemble we can construct a new ("meta") hypothesis $\tilde{h}$ by combining (or aggregating) the individual predictions obtained from each hypothesis,

$$\tilde{h}(\mathbf{x}) := a\big(h^{(1)}(\mathbf{x}), \ldots, h^{(B)}(\mathbf{x})\big). \tag{2.31}$$

Here, $a(\cdot)$ denotes some given (fixed) combination or aggregation function. One example for such an aggregation function is the average $a\big(h^{(1)}(\mathbf{x}), \ldots, h^{(B)}(\mathbf{x})\big) := (1/B) \sum_{b=1}^{B} h^{(b)}(\mathbf{x})$. We obtain a new "meta" hypothesis space $\widetilde{\mathcal{H}}$, that consists of all hypotheses of the form (2.31) with $h^{(1)} \in \mathcal{H}^{(1)}, \ldots, h^{(B)} \in \mathcal{H}^{(B)}$. Which conditions on the aggregation function $a(\cdot)$ and the individual hypothesis spaces $\mathcal{H}^{(1)}, \ldots, \mathcal{H}^{(B)}$ ensure that $\widetilde{\mathcal{H}}$ contains each individual hypothesis space, i.e., $\mathcal{H}^{(1)}, \ldots, \mathcal{H}^{(B)} \subseteq \widetilde{\mathcal{H}}$.

Exercise 2.5. How Many Features? Consider the ML problem underlying a music information retrieval smartphone app [154]. Such an app aims at identifying a song title based on a short audio recording of a song interpretation. Here, the feature vector $\mathbf{x}$ represents the sampled audio signal and the label y is a particular song title out of a huge music database. What is the length n of the feature vector $\mathbf{x} \in \mathbb{R}^n$ whose entries are the

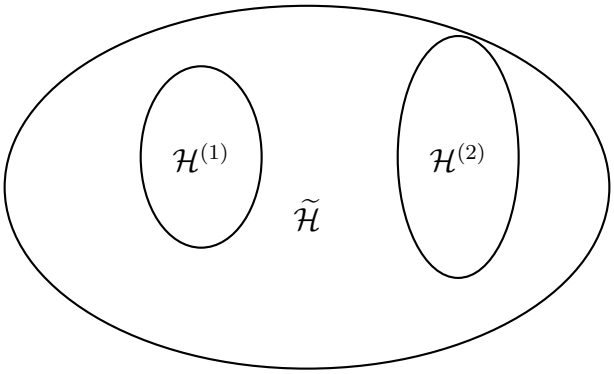


Figure 2.18: The resulting hypothesis space $\widetilde{\mathcal{H}}$ along with some of the constituent hypothesis spaces $\mathcal{H}^{(1)}, \ldots, \mathcal{H}^{(B)}$.

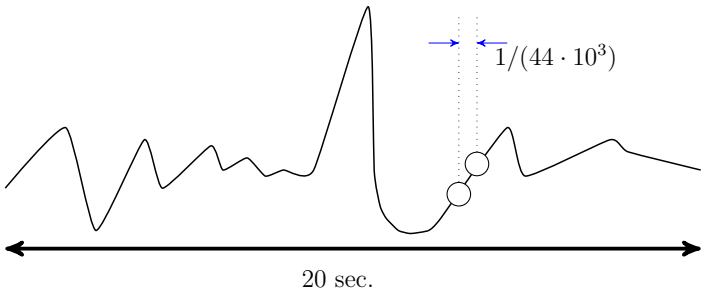


Figure 2.19: A natural choice for the features of an audio recording are the signal amplitudes measured at regular time instants.

signal amplitudes of a 20-second long recording which are sampled at a rate of 44 kHz (see Figure 2.19)?

Exercise 2.6. Multilabel Prediction. Consider data points that are characterized by the features $\mathbf{x} \in \mathbb{R}^{10}$ and a vector-valued label $\mathbf{y} \in \mathbb{R}^{30}$. Such vector-valued labels might arise in multi-label classification problems. We compute a prediction for $\mathbf{y}$ using a linear predictor map

$$\mathbf{h}(\mathbf{x}) = \mathbf{W}\mathbf{x} \text{ with some matrix } \mathbf{W} \in \mathbb{R}^{30\times 10}. \tag{2.32}$$

How many different linear predictors (2.32) are there ? 10, 30, 40, or infinite?

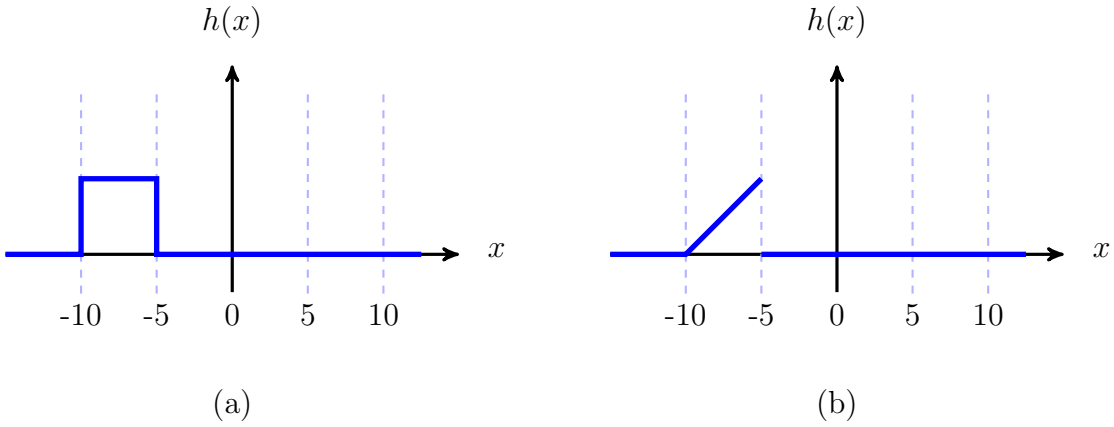
Exercise 2.7. Average squared error loss as Quadratic Form Consider the hypothesis space constituted by all linear maps $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ with some weight vector $\mathbf{w} \in \mathbb{R}^n$. We try to find the best linear map by minimizing the average squared error loss (the empirical risk) incurred on labeled data points (training set) $(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \ldots, (\mathbf{x}^{(m)}, y^{(m)})$. Is it possible to represent the resulting empirical risk as a convex quadratic function $f(\mathbf{w}) = \mathbf{w}^T\mathbf{C}\mathbf{w} + \mathbf{b}\mathbf{w} + c$? If this is possible, how are the matrix $\mathbf{C}$, vector $\mathbf{b}$ and constant c related to the features and labels of data points in the training set?

Exercise 2.8. Find Labeled Data for Given Empirical Risk. Consider linear hypothesis space consisting of linear maps $h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ that are parametrized by a weight vector $\mathbf{w}$. We learn an optimal weight vector by minimizing the average squared error loss $f(\mathbf{w}) = \widehat{L}\big(h^{(\mathbf{w})}|\mathcal{D}\big)$ incurred by $h^{(\mathbf{w})}(\mathbf{x})$ on the training set $\mathcal{D} = \big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)$. Is it possible to reconstruct the features $\mathcal{D}$ just from knowing the function $f(\mathbf{w})$?. Is the resulting labeled training data unique or are there different training sets that could have resulted in the same empirical risk function? Hint: Write down the training error $f(\mathbf{w})$ in the form $f(\mathbf{w}) = \mathbf{w}^T\mathbf{Q}\mathbf{w} + c + \mathbf{b}^T\mathbf{w}$ with some matrix $\mathbf{Q}$, vector $\mathbf{b}$ and scalar c that might depend on the features and labels of the training data points.

Exercise 2.9. Dummy Feature Instead of Intercept. Show that any hypothesis map of the form $h(x) = w_1x + w_0$ can be obtained from the concatenation of a feature map $\phi : x \mapsto \mathbf{z}$ with the linear map $\tilde{h}(\mathbf{z}) := \widetilde{\mathbf{w}}^T\mathbf{z}$ using parameter vector $\widetilde{\mathbf{w}} = \big(w_1, w_0\big)^T \in \mathbb{R}^2$.

Exercise 2.10. Approximate Non-Linear Maps Using Indicator Functions for Feature Maps. Consider an ML application generating data points characterized by a scalar feature $x \in \mathbb{R}$ and numeric label $y \in \mathbb{R}$. We construct a non-linear map by first transforming the feature x to a new feature vector $\mathbf{z} = (\phi_1(x), \phi_2(x), \phi_3(x), \phi_4(x))^T \in \mathbb{R}^4$. The

components $\phi_1(x), \ldots, \phi_4(x)$ are indicator functions of intervals $[-10,-5), [-5,0), [0,5), [5,10]$. In particular, $\phi_1(x) = 1$ for $x \in [-10,-5)$ and $\phi_1(x) = 0$ otherwise. We obtain a hypothesis space $\mathcal{H}^{(1)}$ by collecting all maps from feature x to predicted label $\hat{y}$ that can written as a a weighted linear combination $\mathbf{w}^T\mathbf{z}$ (with some parameter vector $\mathbf{w}$) of the transformed features. Which of the following hypothesis maps belong to $\mathcal{H}^{(1)}$?



Exercise 2.11. Python Hypothesis space. Consider the source codes below for five different Python functions that read in the numeric feature x, perform some computations that result in a prediction $\hat{y}$. How large is the hypothesis space that is constituted by all maps that can be represented by one of those Python functions.

```
def func1(x):
hat_y = 5*x+3
return hat_y

```

```
def func2(x):
tmp = 3*x+3
hat_y = tmp+2*x
return hat_y

```

```
def func3(x):
tmp = 3*x+3
hat_y = tmp-2*x
return hat_y

```

```
def func4(x):
tmp = 3*x+3
hat_y = tmp-2*x+4
return hat_y

```

```
def func5(x):
tmp = 3*x+3
hat_y = 4*tmp-2*x
return hat_y

```

Exercise 2.12. A Lot of Features. One important application domain for ML methods is healthcare. Here, data points represent human patients that are characterized by health-care records. These records might contain physiological parameters, CT scans along with various diagnoses provided by healthcare professionals. Is it a good idea to use every data field of a healthcare record as features of the data point ?

Exercise 2.13. Over-Parametrization. Consider data points characterized by feature vectors $\mathbf{x} \in \mathbb{R}^2$ and a numeric label $y \in \mathbb{R}$. We want to learn the best predictor out of the hypothesis space

$$\mathcal{H} = \big\{h(\mathbf{x}) = \mathbf{x}^T \mathbf{A}\mathbf{w} : \mathbf{w} \in \mathcal{S}\big\}.$$

Here, we used the matrix $\mathbf{A} = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$ and the set

$$\mathcal{S} = \big\{(1,1)^T, (2,2)^T, (-1,3)^T, (0,4)^T\big\} \subseteq \mathbb{R}^2.$$

What is the cardinality of the hypothesis space $\mathcal{H}$, i.e., how many different predictor maps does $\mathcal{H}$ contain?

Exercise 2.14. Squared Error Loss. Consider a hypothesis space $\mathcal{H}$ constituted by three predictors $h^{(1)}(\cdot), h^{(2)}(\cdot), h^{(3)}(\cdot)$. Each predictor $h^{(j)}(x)$ is a real-valued function of a real-valued argument x. Moreover, for each $j \in \{1,2,3\}$,

$$h^{(j)}(x) = \begin{cases} 0 & \text{if } x^2 \leq j \\ j & \text{otherwise.} \end{cases} \tag{2.33}$$

Can you tell which of these hypothesis maps is optimal in the sense of having smallest average squared error loss on the features

$$\big\{\, (1/10, 3)\,, (0,0)\,, (1,-1)\,\big\}.$$

Exercise 2.15. Classification Loss. The Figure 2.16 depicts different loss functions for a fixed data point with label $y = 1$ and varying hypothesis $h \in \mathcal{H}$. How would Figure 2.16 change if we evaluate the same loss functions for another data point $\mathbf{z} = (x, y)$ with label $y = -1$?

Exercise 2.16. Intercept Term. Linear regression methods model the relation between the label y and feature x of a data point as $y = h(x) + e$ with some small additive term e. The predictor map $h(x)$ is assumed to be linear $h(x) = w_1 x + w_0$. The parameter w_0 is sometimes referred to as the intercept (or bias) term. Assume we know for a given linear predictor map its values $h(x)$ for $x = 1$ and $x = 3$. Can you determine the weights w_1 and

w_0 based on $h(1)$ and $h(3)$?

Exercise 2.17. Picture Classification. Consider a huge collection of outdoor pictures you have taken during your last adventure trip. You want to organize these pictures as three categories (or classes) *dog*, *bird* and *fish*. How could you formalize this task as a ML problem?

Exercise 2.18. Maximal Hypothesis space. Consider data points characterized by a single real-valued feature x and a single real-valued label y. How large is the largest possible hypothesis space of predictor maps $h(x)$ that read in the feature value of a data point and deliver a real-valued prediction $\hat{y} = h(x)$?

Exercise 2.19. A Large but Finite Hypothesis space. Consider data points whose features are 10×10 pixel black-and-white (bw) images. Besides the pixels, each data point is also characterized by a binary label $y \in \{0, 1\}$. Consider the hypothesis space which is constituted by all maps that take a 10×10 pixel bw image as input and deliver a prediction for the label. How large is this hypothesis space?

Exercise 2.20. Size of Linear Hypothesis space. Consider a training set of m data points with feature vectors $\mathbf{x}^{(i)} \in \mathbb{R}^n$ and numeric labels $y^{(1)}, \ldots, y^{(m)}$. The feature vectors and labels of the data points in the training set are arbitrary except that we assume the feature matrix $\mathbf{X} = \big(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\big)$ is full rank. What condition on m and n guarantees that we can find a linear predictor $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ that perfectly fits the training set, i.e., $y^{(1)} = h\big(\mathbf{x}^{(1)}\big), \ldots, y^{(m)} = h\big(\mathbf{x}^{(m)}\big)$.

Exercise 2.21. Classification with Imbalanced Data. Consider a dataset $\mathcal{D}$ of m data points with feature vectors $\mathbf{x}^{(i)} \in \mathbb{R}^n$ and discrete labels $y^{(i)} \in \{1, 2, \ldots, 10\}$. The dataset is highly imbalanced, more than 90 percent of data points have a label $y = 1$. We learn a hypothesis out of the hypothesis space $\mathcal{H}'$ that is constituted by the ten maps $h^{(c)}(\mathbf{x}) = c$ for $c = 1, 2, \ldots, 10$. Is there a hypothesis $h \in \mathcal{H}'$ whose average 0/1 loss does not exceed 0.3 ?

Exercise 2.22. Accuracy and Average Logistic loss. Consider a dataset $\mathcal{D}$ that consists of m data points, indexed by $i = 1, \ldots, m$. Each data point is characterized by a feature vector $\mathbf{x}^{(i)} \in \mathbb{R}^2$ and by a binary label $y^{(i)} \in \{-1, 1\}$. We use a linear hypothesis map $h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ to classify data points according to $\hat{y} = 1$ if $h^{(\mathbf{w})}(\mathbf{x}) \geq 0$ and $\hat{y} = -1$

otherwise. Two popular quality measures of a hypothesis are the accuracy and the average logistic loss $(1/m)\sum_i L\left(\left(\mathbf{x}^{(i)}, y^{(i)}\right), h^{(\mathbf{w})}\right)$ with the logistic loss (2.12).

The accuracy of a hypothesis $h^{(\mathbf{w})}(\mathbf{x})$ is defined as $1-(1/m)\sum_i L\left(\left(\mathbf{x}^{(i)}, y^{(i)}\right), h^{(\mathbf{w})}\right)$ with the 0/1 loss (2.9). Loosely speaking, the accuracy of a hypothesis is "one minus the average 0/1 loss ". Can you construct a specific features with arbitrary but finite size m such that there are two different linear hypotheses $h^{(\mathbf{w})}$ and $h^{(\mathbf{w}')}$ with accuracy and average logistic loss of $h^{(\mathbf{w})}$, respectively, being strictly larger than accuracy and average logistic loss of $h^{(\mathbf{w}')}$.

Exercise 2.23. Achievable Accuracy for a Multi-Class Classification Problem. Consider a features consisting of data points, each characterized by some features $\mathbf{x}$ and label $y \in \{1,2,3,4,5\}$. The number of data points with label value $y=1$ is 150, those with $y=2$ is 250, with $y=3$ is 200, with $y=4$ is 300 and the number of data points with $y=5$ is 100. Is there at least one classifier that can achieve an accuracy of at least 10 %?

Exercise 2.24. Make it Easier? Consider a multi-class classification problem with data points belonging to one out of 10 different classes represented by the label space $\mathcal{Y} = \{1,2,3,\ldots,10\}$. Assume that we have found a hypothesis map (classifier) that achieves an accuracy of 70 %. Now consider a modified multi-class classification problem that is obtained by combining 4 classes of $\mathcal{Y}$ into one single new class. Can we be sure to find a classifier for this new problem that achieves an accuracy of at least 70 % ?

Chapter 3

The Landscape of ML

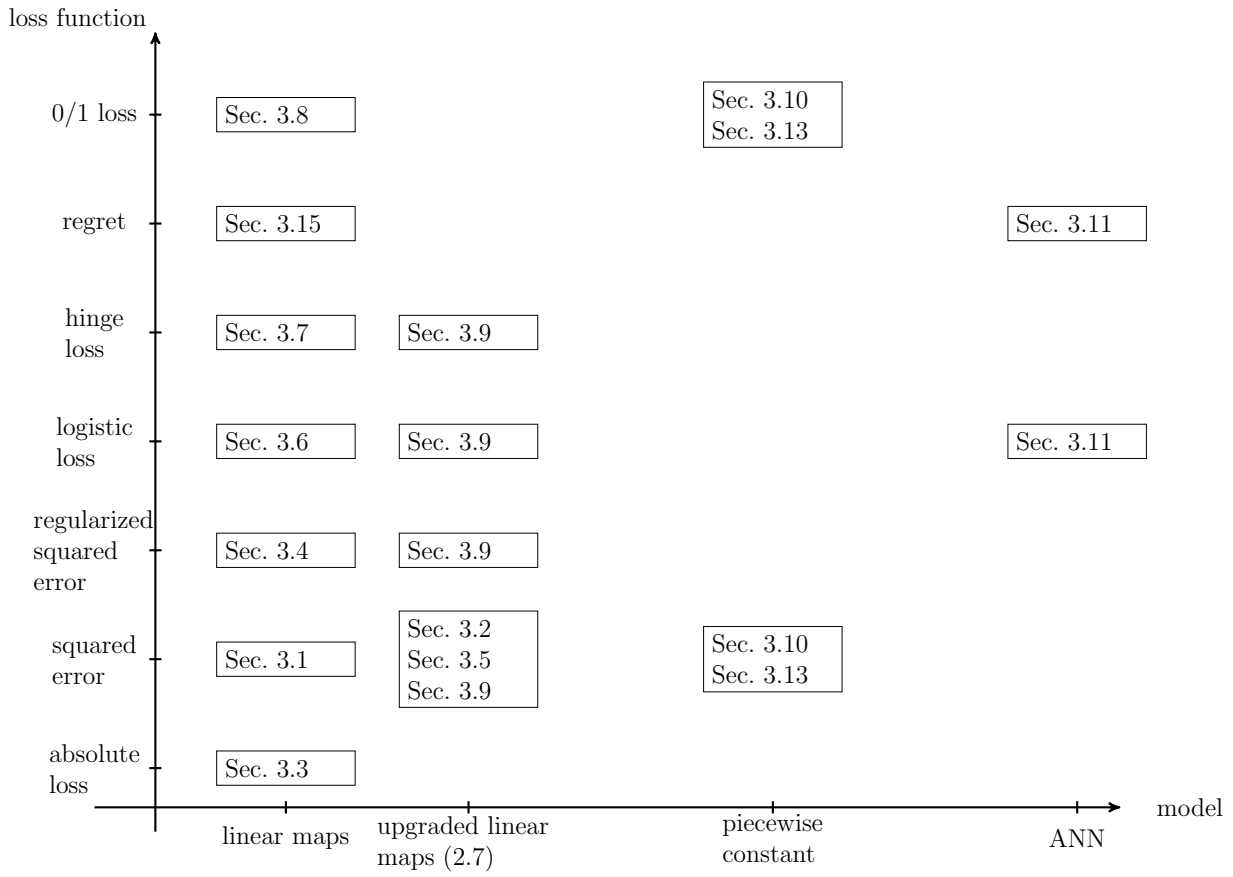


Figure 3.1: ML methods fit a model to data by minimizing a loss function. Different ML methods use different design choices for data, model and loss.

As discussed in Chapter 2, ML methods combine three main components:

- a set of data points that are characterized by features and labels
- a model or hypothesis space $\mathcal{H}$ that consists of different hypotheses $h \in \mathcal{H}$.
- a loss function to measure the quality of a particular hypothesis h.

Each of these three components involves design choices for the representation of data, their features and labels, the model and loss function. This chapter details the high-level design choices used by some of the most popular ML methods. Figure 3.1 depicts these ML methods in a two-dimensional plane whose horizontal axes represents different hypothesis spaces and the vertical axis represents different loss functions.

To obtain a practical ML method we also need to combine the above components. The basic principle of any ML method is to search the model for a hypothesis that incurs minimum loss on any data point. Chapter 4 will then discuss a principled way to turn this informal statement into actual ML algorithms that could be implemented on a computer.

3.1 Linear Regression

Consider data points characterized by feature vectors $\mathbf{x} \in \mathbb{R}^n$ and numeric label $y \in \mathbb{R}$. Linear regression methods learn a hypothesis out of the linear hypothesis space

$$\mathcal{H}^{(n)} := \{h^{(\mathbf{w})} : \mathbb{R}^n \to \mathbb{R} : h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T\mathbf{x} \text{ with some parameter vector } \mathbf{w} \in \mathbb{R}^n\}. \tag{3.1}$$

Figure 1.3 depicts the graphs of some maps from $\mathcal{H}^{(2)}$ for data points with feature vectors of the form $\mathbf{x} = (1, x)^T$. The quality of a particular predictor $h^{(\mathbf{w})}$ is measured by the squared error loss (2.8). Using labeled data $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$, linear regression learns a linear predictor $\widehat{h}$ which minimizes the average squared error loss (2.8),

$$\widehat{h} = \operatorname*{argmin}_{h \in \mathcal{H}^{(n)}} \widehat{L}(h|\mathcal{D}) \overset{(2.16)}{=} \operatorname*{argmin}_{h \in \mathcal{H}^{(n)}} (1/m) \sum_{i=1}^{m} (y^{(i)} - h(\mathbf{x}^{(i)}))^2. \tag{3.2}$$

Some authors refer to the average squared error loss in (3.2) also as the mean squared error of the hypothesis h.

Since the hypothesis space $\mathcal{H}^{(n)}$ is parametrized by the parameter vector $\mathbf{w}$ (see (3.1)),

we can rewrite (3.2) as an optimization of the parameter vector $\mathbf{w}$,

$$\widehat{\mathbf{w}} = \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^n} (1/m) \sum_{i=1}^{m} (y^{(i)} - h^{(\mathbf{w})}(\mathbf{x}^{(i)}))^2$$
$$\stackrel{h^{(\mathbf{w})}(\mathbf{x})=\mathbf{w}^T\mathbf{x}}{=} \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^n} (1/m) \sum_{i=1}^{m} (y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)})^2. \tag{3.3}$$

The optimization problems (3.2) and (3.3) are equivalent in the following sense: Any optimal parameter vector $\widehat{\mathbf{w}}$ which solves (3.3), can be used to construct an optimal predictor $\widehat{h}$, which solves (3.2), via $\widehat{h}(\mathbf{x}) = h^{(\widehat{\mathbf{w}})}(\mathbf{x}) = \big(\widehat{\mathbf{w}}\big)^T\mathbf{x}$.

3.2 Polynomial Regression

Consider an ML problem involving data points which are characterized by a single numeric feature $x \in \mathbb{R}$ (the feature space is $\mathcal{X} = \mathbb{R}$) and a numeric label $y \in \mathbb{R}$ (the label space is $\mathcal{Y} = \mathbb{R}$). We observe a bunch of labeled data points which are depicted in Figure 3.2.

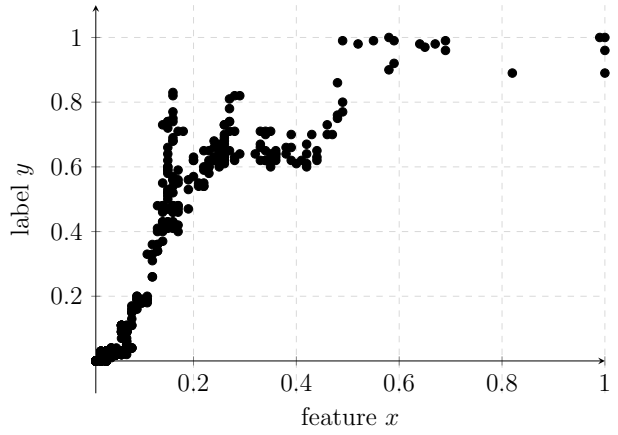


Figure 3.2: A scatterplot that depicts a set of data points $(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})$. The ith data point is depicted by a dot whose coordinates are the feature $x^{(i)}$ and label $y^{(i)}$ of that data point.

Figure 3.2 suggests that the relation $x \mapsto y$ between feature x and label y is highly non-linear. For such non-linear relations between features and labels it is useful to consider a

hypothesis space which is constituted by polynomial maps

$$\mathcal{H}^{(n)}_{\rm poly} = \{h^{(\mathbf{w})} : \mathbb{R} \to \mathbb{R} : h^{(\mathbf{w})}(x) = \sum_{j=1}^{n} w_j x^{j-1}, \\ \text{with some } \mathbf{w} = (w_1, \ldots, w_n)^T \in \mathbb{R}^n\}. \tag{3.4}$$

We can approximate any non-linear relation $y = h(x)$ with any desired level of accuracy using a polynomial $\sum_{j=1}^{n} w_j x^{j-1}$ of sufficiently large degree $n-1$.[1]

As in linear regression (see Section 3.1), we measure the quality of a hypothesis by the squared error loss (2.8) incurred on labeled data points $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$, each having a scalar feature $x^{(i)}$ and label $y^{(i)}$. Polynomial regression learns a hypothesis h by minimizing the average squared error loss (see (2.8)):

$$\min_{h^{(\mathbf{w})} \in \mathcal{H}^{(n)}_{\rm poly}} (1/m) \sum_{i=1}^{m} (y^{(i)} - h^{(\mathbf{w})}(x^{(i)}))^2. \tag{3.5}$$

We can interpret polynomial regression as a combination of a feature map (transformation) (see Section 2.1.1) and linear regression (see Section 3.1). Indeed, any polynomial predictor $h^{(\mathbf{w})} \in \mathcal{H}^{(n)}_{\rm poly}$ is obtained as a concatenation of the feature map

$$\phi(x) \mapsto \mathbf{x} := (1, x, \ldots, x^{n-1})^T \in \mathbb{R}^n \tag{3.6}$$

with some linear map $\tilde{h}^{(\mathbf{w})} : \mathbb{R}^n \to \mathbb{R} : \mathbf{x} \mapsto \mathbf{w}^T \mathbf{x}$, i.e.,

$$h^{(\mathbf{w})}(x) = \tilde{h}^{(\mathbf{w})}(\underbrace{\phi(x)}_{\overset{(3.6)}{=}\mathbf{x}}). \tag{3.7}$$

Thus, we can implement polynomial regression by first applying the feature map ϕ (see (3.6)) to the scalar features $x^{(i)}$, resulting in the transformed feature vectors

$$\mathbf{x}^{(i)} = \phi\big(x^{(i)}\big) = \big(1, x^{(i)}, \ldots, \big(x^{(i)}\big)^{n-1}\big)^T \in \mathbb{R}^n, \tag{3.8}$$

and then applying linear regression (see Section 3.1) to these transformed feature vectors $\mathbf{x}^{(i)}$.

[1] The precise formulation of this statement is known as the "Stone-Weierstrass Theorem" [119, Thm. 7.26].

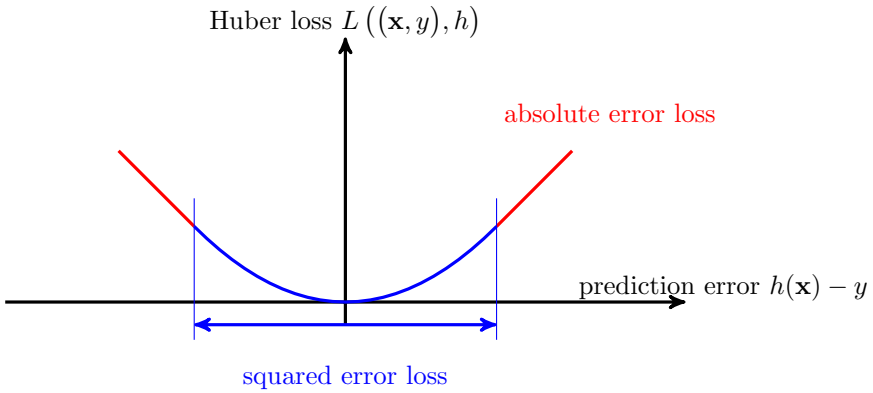


Figure 3.3: The Huber loss (3.9) resembles the squared error loss (2.8) for small prediction error and the absolute difference loss for larger prediction errors.

By inserting (3.7) into (3.5), we obtain a linear regression problem (3.3) with feature vectors (3.8). Note that while the hypothesis $h^{(\mathbf{w})} \in \mathcal{H}^{(n)}_{\rm poly}$ is a non-linear function $h^{(\mathbf{w})}(x)$ of the original feature x, it is a linear function $\tilde{h}^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ (see (3.7)), of the transformed features $\mathbf{x}$ (3.8).

3.3 Least Absolute Deviation Regression

Learning a linear predictor by minimizing the average squared error loss incurred on training data is not robust against the presence of outliers. This sensitivity to outliers is rooted in the properties of the squared error loss $(y-h(\mathbf{x}))^2$. Minimizing the average squared error forces the resulting predictor $\hat{y}$ to not be too far away from any data point. However, it might be useful to tolerate a large prediction error $y-h(\mathbf{x})$ for an unusual or exceptional data point that can be considered an outlier.

Replacing the squared loss with a different loss function can make the learning robust against outliers. One important example for such a "robustifying" loss function is the Huber loss [63]

$$L\left((\mathbf{x},y),h\right) = \begin{cases} (1/2)(y-h(\mathbf{x}))^2 & \text{for } |y-h(\mathbf{x})| \leq \varepsilon \\ \varepsilon(|y-h(\mathbf{x})| - \varepsilon/2) & \text{else.} \end{cases} \tag{3.9}$$

Figure 3.3 depicts the Huber loss as a function of the prediction error $y-h(\mathbf{x})$.

The Huber loss definition (3.9) contains a tuning parameter ϵ. The value of this tuning parameter defines when a data point is considered as an outlier. Figure 3.4 illustrates the

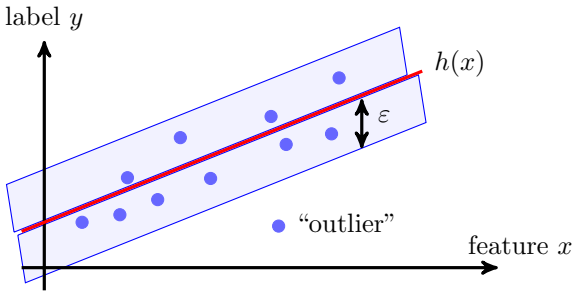


Figure 3.4: The Huber loss measures prediction errors via squared error loss for regular data points inside the band of width ε around the hypothesis map $h(\mathbf{x})$ and via the absolute difference loss for an outlier outside the band.

role of this parameter as the width of a band around a hypothesis map. The prediction error of this hypothesis map for data points within this band are measured used squared error loss (2.8). For data points outside this band (these are considered outliers), we use instead the absolute value of the prediction error as the loss value.

The Huber loss is robust to outliers since the corresponding (large) prediction errors $y - \hat{y}$ are not squared. Outliers have a smaller effect on the average Huber loss (over the entire dataset) compared to the average squared error loss. The improved robustness against outliers of the Huber loss comes at the expense of increased computational complexity. The squared error loss can be minimized using efficient gradient based methods (see Chapter 5). In contrast, for $\varepsilon = 0$, the Huber loss is non-differentiable and requires more advanced optimization methods.

The Huber loss (3.9) contains two important special cases. The first special case occurs when ε is chosen to be very large, such that the condition $|y - \hat{y}| \leq \varepsilon$ is satisfied for most data points. In this case, the Huber loss resembles the squared error loss (2.8) (up to a scaling factor $1/2$). The second special case is obtained for $\varepsilon \to 0$ when the scaled Huber loss $(1/\varepsilon)L\big((\mathbf{x}, y), h\big)$ converges to the absolute difference loss $|y - \hat{y}|$.

3.4 The Lasso

We will see in Chapter 6 that linear regression (see Section 3.1) typically requires a training set larger than the number of features used to characterized a data point. However, many important application domains generate data points with a number n of features much higher than the number m of available labeled data points in the training set.

In the high-dimensional regime, where $m \ll n$, basic linear regression methods will not be able to learn useful weights $\mathbf{w}$ for a linear hypothesis. Section 6.4 shows that for $m \ll n$, linear regression will typically learn a hypothesis that perfectly predicts labels of data points in the training set but delivers poor predictions for data points outside the training set. This phenomenon is referred to as overfitting and poses a main challenge for ML applications in the high-dimensional regime.

Chapter 7 discusses basic regularization techniques that allow to prevent ML methods from overfitting. We can regularize linear regression by augmenting the squared error loss (2.8) of a hypothesis $h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ with an additional penalty term. This penalty term depends solely on the weights $\mathbf{w}$ and serves as an estimate for the increase of the average loss on data points outside the training set. Different ML methods are obtained from different choices for this penalty term. The Lasso is obtained from linear regression by replacing the squared error loss with the regularized loss

$$L\left((\mathbf{x}, y), h^{(\mathbf{w})}\right) = \left(y - \mathbf{w}^T\mathbf{x}\right)^2 + \lambda\|\mathbf{w}\|_1. \tag{3.10}$$

Here, the penalty term is given by the scaled norm $\lambda\|\mathbf{w}\|_1$ with the regularization parameter λ. The value of λ has significant impact on the properties of the hypothesis learnt by minimizing (3.10). Roughly speaking, increasing λ results in a weight vector $\widehat{\mathbf{w}}$ with increasing number of zero coefficients. Thus, the learnt hypothesis $h^{(\widehat{\mathbf{w}})}$ ignores many features, which is a form of feature selection.

One principled approach to finding a suitable value range for λ can be based on the i.i.d. assumption (Section 2.1.4). We assume that the features and label of a data point can be modelled as

$$y = \overline{\mathbf{w}}^T\mathbf{x} + \varepsilon. \tag{3.11}$$

Here, $\overline{\mathbf{w}}$ denotes some true underlying parameter vector and ε is a realization of an a RV that is independent of the features $\mathbf{x}$. It is then possible to determine optimal values for λ in (3.10) such that the learnt weights $\widehat{\mathbf{w}}$ are close to the (unknown) true weights $\overline{\mathbf{w}}$ [151, 17]. Note that the optimal values for λ depend crucially the distribution of the noise ε and the number of non-zeros entries in $\overline{\mathbf{w}}$ in (3.11).

Instead of relying on a probabilistic model (3.11), which might be wrong, we can also tune λ by a trial-and-error approach. Here, we try out different candidate values for λ and learn (the weights of) a linear hypothesis for each value of λ. Among those, we pick the hypothesis resulting in the minimum average loss on a validation set. This approach is an

instance of the model selection techniques in Section 6.3.

3.5 Gaussian Basis Regression

Section 3.2 showed how to extend linear regression by first transforming the feature x using a vector-valued feature map $\mathbf{\Phi} : \mathbb{R} \to \mathbb{R}^n$. The output of this feature map are the transformed features $\mathbf{\Phi}(x)$ which are fed, in turn, to a linear map $h\left(\mathbf{\Phi}(x)\right) = \mathbf{w}^T\phi(x)$. Polynomial regression in Section 3.2 has been obtained for the specific feature map (3.6) whose entries are the powers x^l of the scalar original feature x. However, it is possible to use other functions, different from polynomials, to construct the feature map ϕ. We can extend linear regression using an arbitrary feature map

$$\mathbf{\Phi}(x) = (\phi_1(x), \ldots, \phi_n(x))^T \tag{3.12}$$

with the scalar maps $\phi_j : \mathbb{R} \to \mathbb{R}$ which are referred to as "basis functions". The choice of basis functions depends heavily on the particular application and the underlying relation between features and labels of the observed data points. The basis functions underlying polynomial regression are $\phi_j(x) = x^j$.

Another popular choice for the basis functions are "Gaussians"

$$\phi_{\sigma,\mu}(x) = \exp(-(1/(2\sigma^2))(x-\mu)^2). \tag{3.13}$$

The family (3.13) of maps is parameterized by the variance σ^2 and the mean μ. Gaussian basis linear regression combines the feature map

$$\mathbf{\Phi}(x) = \big(\phi_{\sigma_1,\mu_1}(x), \ldots, \phi_{\sigma_n,\mu_n}(x)\big)^T \tag{3.14}$$

with linear regression (see Figure 3.5). The resulting hypothesis space is

$$\begin{aligned}\mathcal{H}^{(n)}_{\text{Gauss}} = \{h^{(\mathbf{w})} : \mathbb{R} \to \mathbb{R} : h^{(\mathbf{w})}(x) &= \sum_{j=1}^{n} w_j \phi_{\sigma_j,\mu_j}(x) \\ &\text{with weights } \mathbf{w} = (w_1, \ldots, w_n)^T \in \mathbb{R}^n\}.\end{aligned} \tag{3.15}$$

Different choices for the variance σ_j^2 and shifts μ_j of the Gaussian function in (3.13) results in different hypothesis spaces $\mathcal{H}_{\text{Gauss}}$. Chapter 6.3 will discuss model selection techniques

that allow to find useful values for these parameters.

The hypotheses of (3.15) are parametrized by a parameter vector $\mathbf{w} \in \mathbb{R}^n$. Each hypothesis in $\mathcal{H}_{\text{Gauss}}$ corresponds to a particular choice for the parameter vector $\mathbf{w}$. Thus, instead of searching over $\mathcal{H}_{\text{Gauss}}$ to find a good hypothesis, we can search over the Euclidean space $\mathbb{R}^n$. Highly developed methods for searching over the space $\mathbb{R}^n$, for a wide range of values for n, are provided by numerical linear algebra [46].

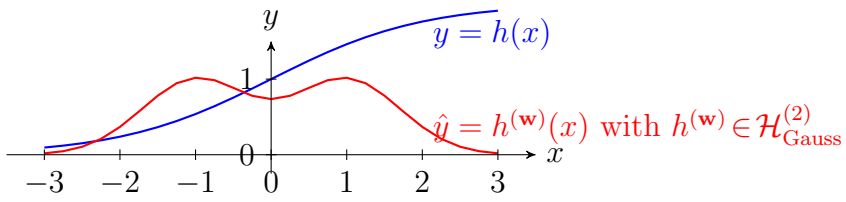


Figure 3.5: The true relation $x \mapsto y$ (blue) between feature x and label y of data points is highly non-linear. Therefore it seems reasonable to predict the label using a non-linear hypothesis map $h^{(\mathbf{w})}(x) \in \mathcal{H}^{(2)}_{\text{Gauss}}$ with some parameter vector $\mathbf{w} \in \mathbb{R}^2$.

3.6 Logistic Regression

Logistic regression is a ML method that allows to classify data points according to two categories. Thus, logistic regression is a binary classification method that can be applied to data points characterized by feature vectors $\mathbf{x} \in \mathbb{R}^n$ (feature space $\mathcal{X} = \mathbb{R}^n$) and binary labels y. These binary labels take on values from a label space that contains two different label values. Each of these two label values represents one of the two categories to which the data points can belong.

It is convenient to use the label space $\mathcal{Y} = \mathbb{R}$ and encode the two label values as $y = 1$ and $y = -1$. However, it is important to note that logistic regression can be used with an arbitrary label space which contains two different elements. Another popular choice for the label space is $\mathcal{Y} = \{0, 1\}$. Logistic regression learns a hypothesis out of the hypothesis space $\mathcal{H}^{(n)}$ (see (3.1)). Note that logistic regression uses the same hypothesis space as linear regression (see Section 3.1).

At first sight, it seems wasteful to use a linear hypothesis $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$, with some parameter vector $\mathbf{w} \in \mathbb{R}^n$, to predict a binary label y. Indeed, while the prediction $h(\mathbf{x})$ can take any real number, the label $y \in \{-1, 1\}$ takes on only one of the two real numbers 1 and -1.

It turns out that even for binary labels it is quite useful to use a hypothesis map h which can take on arbitrary real numbers. We can always obtain a predicted label $\hat{y} \in \{-1,1\}$ by comparing hypothesis value $h(\mathbf{x})$ with a threshold. A data point with feature vector $\mathbf{x}$, is classified as $\hat{y}=1$ if $h(\mathbf{x}) \geq 0$ and $\hat{y}=-1$ for $h(\mathbf{x})<0$. Thus, we use the sign of the hypothesis h to determine the classification result, which is the prediction $\hat{y} \in \mathcal{Y}$ for the true label $y \in \mathcal{Y}$. The absolute value $|h(\mathbf{x})|$ is then used to quantify the reliability of (confidence in) the classification $\hat{y}$.

Consider two data points with feature vectors $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}$ and a linear classifier map h yielding the function values $h(\mathbf{x}^{(1)})=1/10$ and $h(\mathbf{x}^{(2)})=100$. Whereas the predictions for both data points result in the same label predictions, i.e., $\hat{y}^{(1)}=\hat{y}^{(2)}=1$, the classification of the data point with feature vector $\mathbf{x}^{(2)}$ seems to be much more reliable.

Logistic regression uses the logistic loss (2.12) to assess the quality of a particular hypothesis $h^{(\mathbf{w})} \in \mathcal{H}^{(n)}$. In particular, given some labeled training set $\mathcal{D}=\{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^{m}$, logistic regression tries to minimize the empirical risk (average logistic loss)

$$\begin{aligned}\widehat{L}(\mathbf{w}|\mathcal{D}) &= (1/m)\sum_{i=1}^{m} \log(1+\exp(-y^{(i)}h^{(\mathbf{w})}(\mathbf{x}^{(i)}))) \\ &\overset{h^{(\mathbf{w})}(\mathbf{x})=\mathbf{w}^T\mathbf{x}}{=} (1/m)\sum_{i=1}^{m} \log(1+\exp(-y^{(i)}\mathbf{w}^T\mathbf{x}^{(i)})). \end{aligned} \tag{3.16}$$

Once we have found the optimal parameter vector $\widehat{\mathbf{w}}$, which minimizes (3.16), we can classify any data point solely based on its features $\mathbf{x}$. Indeed, we just need to evaluate the hypothesis $h^{(\widehat{\mathbf{w}})}$ for the features $\mathbf{x}$ to obtain the predicted label

$$\hat{y} = \begin{cases} 1 & \text{if } h^{(\widehat{\mathbf{w}})}(\mathbf{x}) \geq 0 \\ -1 & \text{otherwise.} \end{cases} \tag{3.17}$$

Since $h^{(\widehat{\mathbf{w}})}(\mathbf{x}) = \big(\widehat{\mathbf{w}}\big)^T\mathbf{x}$ (see (3.1)), the classifier (3.17) amounts to testing whether $\big(\widehat{\mathbf{w}}\big)^T\mathbf{x} \geq 0$ or not.

The classifier (3.17) partitions the feature space $\mathcal{X}=\mathbb{R}^n$ into two half-spaces $\mathcal{R}_1=\big\{\mathbf{x}: \big(\widehat{\mathbf{w}}\big)^T\mathbf{x}\geq 0\big\}$ and $\mathcal{R}_{-1}=\big\{\mathbf{x}: \big(\widehat{\mathbf{w}}\big)^T\mathbf{x}<0\big\}$ which are separated by the hyperplane $\big(\widehat{\mathbf{w}}\big)^T\mathbf{x}=0$ (see Figure 2.9). Any data point with features $\mathbf{x} \in \mathcal{R}_1$ ($\mathbf{x} \in \mathcal{R}_{-1}$) is classified as $\hat{y}=1$ ($\hat{y}=-1$).

Logistic regression can be interpreted as a statistical estimation method for a particular probabilistic model for the data points. This probabilistic model interprets the label $y \in$

$\{-1,1\}$ of a data point as a RV with the probability distribution

$$\begin{aligned} p(y=1;\mathbf{w}) &= 1/(1+\exp(-\mathbf{w}^T\mathbf{x})) \\ &\overset{h^{(\mathbf{w})}(\mathbf{x})=\mathbf{w}^T\mathbf{x}}{=} 1/(1+\exp(-h^{(\mathbf{w})}(\mathbf{x}))). \end{aligned} \tag{3.18}$$

As the notation indicates, the probability (3.18) is parametrized by the parameter vector $\mathbf{w}$ of the linear hypothesis $h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$. Given the probabilistic model (3.18), we can interpret the classification (3.17) as choosing $\hat{y}$ to maximize the probability $p(y=\hat{y};\mathbf{w})$.

Since $p(y=1)+p(y=-1)=1$,

$$\begin{aligned} p(y=-1) &= 1-p(y=1) \\ &\overset{(3.18)}{=} 1-1/(1+\exp(-\mathbf{w}^T\mathbf{x})) \\ &= 1/(1+\exp(\mathbf{w}^T\mathbf{x})). \end{aligned} \tag{3.19}$$

In practice we do not know the parameter vector in (3.18). Rather, we have to estimate the parameter vector $\mathbf{w}$ in (3.18) from observed data points. A principled approach to estimate the parameter vector is to maximize the probability (or likelihood) of actually obtaining the dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$ as realizations of i.i.d. data points whose labels are distributed according to (3.18). This yields the ML estimator

$$\begin{aligned} \widehat{\mathbf{w}} &= \operatorname*{argmax}_{\mathbf{w}\in\mathbb{R}^n} p(\{y^{(i)}\}_{i=1}^m) \\ &\overset{y^{(i)}\text{i.i.d.}}{=} \operatorname*{argmax}_{\mathbf{w}\in\mathbb{R}^n} \prod_{i=1}^m p(y^{(i)}) \\ &\overset{(3.18),(3.19)}{=} \operatorname*{argmax}_{\mathbf{w}\in\mathbb{R}^n} \prod_{i=1}^m 1/(1+\exp(-y^{(i)}\mathbf{w}^T\mathbf{x}^{(i)})). \end{aligned} \tag{3.20}$$

Note that the last expression (3.20) is only valid if we encode the binary labels using the values 1 and -1. Using different label values results in a different expression.

Maximizing a positive function $f(\mathbf{w}) > 0$ is equivalent to maximizing $\log f(w)$,

$$\operatorname*{argmax}_{\mathbf{w}\in\mathbb{R}^n} f(\mathbf{w}) = \operatorname*{argmax}_{\mathbf{w}\in\mathbb{R}^n} \log f(\mathbf{w}).$$

Therefore, (3.20) can be further developed as

$$\widehat{\mathbf{w}} \overset{(3.20)}{=} \operatorname*{argmax}_{\mathbf{w}\in\mathbb{R}^n} \sum_{i=1}^{m} -\log\big(1+\exp(-y^{(i)}\mathbf{w}^T\mathbf{x}^{(i)})\big)$$
$$= \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^n} (1/m)\sum_{i=1}^{m} \log\big(1+\exp(-y^{(i)}\mathbf{w}^T\mathbf{x}^{(i)})\big). \tag{3.21}$$

Comparing (3.21) with (3.16) reveals that logistic regression is nothing but ML estimation of the parameter vector $\mathbf{w}$ in the probabilistic model (3.18).

3.7 Support Vector Machines

SVMs are a family of ML methods for learning a hypothesis to predict a binary label y of a data point based on its features $\mathbf{x}$. Without loss of generality we consider binary labels taking values in the label space $\mathcal{Y} = \{-1, 1\}$. A SVM uses the linear hypothesis space (3.1) which consists of linear maps $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ with some parameter vector $\mathbf{w} \in \mathbb{R}^n$. Thus, the SVM uses the same hypothesis space as linear regression and logistic regression which we have discussed in Section 3.1 and Section 3.6, respectively. What sets the SVM apart from these other methods is the choice of loss function.

Different instances of a SVM are obtained by using different constructions for the features of a data point. Kernel SVMs use the concept of a kernel map to construct (typically high-dimensional) features (see Section 3.9 and [82]). In what follows, we assume the feature construction has been solved and we have access to a feature vector $\mathbf{x} \in \mathbb{R}^n$ for each data point.

Figure 3.6 depicts a dataset $\mathcal{D}$ of labeled data points, each characterized by a feature vector $\mathbf{x}^{(i)} \in \mathbb{R}^2$ (used as coordinates of a marker) and a binary label $y^{(i)} \in \{-1, 1\}$ (indicated by different marker shapes). We can partition dataset $\mathcal{D}$ into two classes

$$\mathcal{C}^{(y=1)} = \{\mathbf{x}^{(i)} : y^{(i)} = 1\}, \text{ and } \mathcal{C}^{(y=-1)} = \{\mathbf{x}^{(i)} : y^{(i)} = -1\}. \tag{3.22}$$

The SVM tries to learn a linear map $h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ that perfectly separates the two classes in the sense of

$$\underbrace{h\big(\mathbf{x}^{(i)}\big)}_{\mathbf{w}^T\mathbf{x}^{(i)}} > 0 \text{ for } \mathbf{x}^{(i)} \in \mathcal{C}^{(y=1)} \text{ and } \underbrace{h\big(\mathbf{x}^{(i)}\big)}_{\mathbf{w}^T\mathbf{x}^{(i)}} < 0 \text{ for } \mathbf{x}^{(i)} \in \mathcal{C}^{(y=-1)}. \tag{3.23}$$

We refer to a dataset, whose data points have binary labels. as linear separable if we can find at least one linear map that separates in the sense of (3.23). The dataset in Figure 3.6 is linearly separable.

As can be verified easily, any linear map $h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ achieving zero average hinge loss (2.11) on the dataset $\mathcal{D}$ perfectly satisfies this dataset (3.23). It seems reasonable to learn a linear map by minimizing the average hinge loss (2.11). However, one drawback of this approach is that there might be (infinitely) many different linear maps that achieve zero average hinge loss and, in turn, perfectly separate the data points in Figure 3.6. Indeed, consider a linear map $h^{(\mathbf{w})}$ that achieves zero average hinge loss for the $\mathcal{D}$ in Figure 3.6 (and therefore perfectly separates it). Then, any other linear map $h^{(\mathbf{w}')}$ with weights $\mathbf{w}' = \lambda\mathbf{w}$, using an arbitrary number $\lambda > 1$ also achieves zero average hinge loss (and perfectly separates the dataset).

Neither the separability requirement (3.23) nor the hinge loss (2.11) are sufficient as a sole training criterion. Indeed, there are many (if not most) datasets that are not linearly separable. Even for a linearly separable dataset (such as the one Figure 3.6), there are infinitely many linear maps with zero average hinge loss. Which one of these infinitely many different maps should we use? To settle these issues, the SVM uses a "regularized" hinge loss,

$$\begin{aligned} L\left((\mathbf{x},y),h^{(\mathbf{w})}\right) &:= \max\{0,1-y\cdot h^{(\mathbf{w})}(\mathbf{x})\} + \lambda\left\|\mathbf{w}\right\|_2^2 \\ &\overset{h^{(\mathbf{w})}(\mathbf{x})=\mathbf{w}^T\mathbf{x}}{=} \max\{0,1-y\cdot \mathbf{w}^T\mathbf{x}\} + \lambda\left\|\mathbf{w}\right\|_2^2. \end{aligned} \tag{3.24}$$

The loss (3.7) augments the hinge loss (2.11) by the term $\lambda\left\|\mathbf{w}\right\|_2^2$. This term is the scaled (by $\lambda > 0$) squared Euclidean norm of the weights $\mathbf{w}$ of the linear hypothesis h used to classify data points. Adding the penalty term $\lambda\left\|\mathbf{w}\right\|_2^2$ to the hinge loss (2.11) is an instance of regularization (see Chapter 7).

The loss favours linear maps $h^{(\mathbf{w})}$ that are robust against (small) perturbations of the data points. The tuning parameter λ in (3.7) controls the strength of this regularization effect and might therefore also be referred to as a regularization parameter. We will discuss regularization on a more general level in Chapter 7.

Let us now develop a useful geometric interpretation of the linear hypothesis obtained by minimizing the loss function (3.7). According to [82, Chapter 2], a classifier $h^{(\mathbf{w}_{\text{SVM}})}$ that minimizes the average loss (3.7), maximizes the distance (margin) ξ between its decision boundary and each of the two classes $\mathcal{C}^{(y=1)}$ and $\mathcal{C}^{(y=-1)}$ (see (3.22)). The decision boundary

is given by the set of feature vectors $\mathbf{x}$ satisfying $\mathbf{w}_{\rm SVM}^T \mathbf{x} = 0$,

Making the margin as large as possible is reasonable as it ensures that the resulting classifications are robust against small perturbations of the features (see Section 7.2). As depicted in Figure 3.6, the margin between the decision boundary and the classes $\mathcal{C}_1$ and $\mathcal{C}_2$ is typically determined by few data points (such as $\mathbf{x}^{(6)}$ in Figure 3.6) which are closest to the decision boundary. These data points have minimum distance to the decision boundary and are referred to as support vectors.

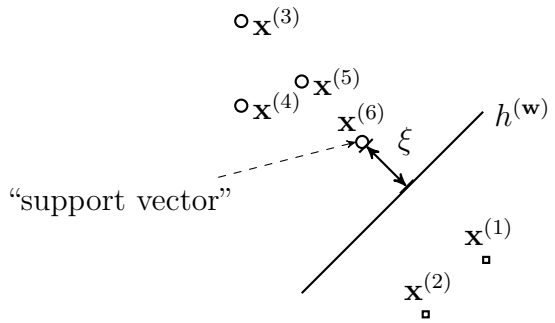


Figure 3.6: The SVM learns a hypothesis (or classifier) $h^{(\mathbf{w})}$ with minimum average soft-margin hinge loss (3.7). Minimizing this loss is equivalent to maximizing the margin ξ between the decision boundary of $h^{(\mathbf{w})}$ and each class of the training set.

We highlight that both, the SVM and logistic regression use the same hypothesis space of linear maps. Both methods learn a linear classifier $h^{(\mathbf{w})} \in \mathcal{H}^{(n)}$ (see (3.1)) whose decision boundary is a hyperplane in the feature space $\mathcal{X} = \mathbb{R}^n$ (see Figure 2.9). The difference between SVM and logistic regression is in their choice for the loss function used to evaluate the quality of a hypothesis $h^{(\mathbf{w})} \in \mathcal{H}^{(n)}$.

The hinge loss (2.11) is a (in some sense optimal) convex approximation to the 0/1 loss (2.9). Thus, we expect the classifier obtained by the SVM to yield a smaller classification error probability $p(\hat{y} \neq y)$ (with $\hat{y} = 1$ if $h(\mathbf{x}) \geq 0$ and $\hat{y} = -1$ otherwise) compared to logistic regression which uses the logistic loss (2.12). Note that using hinge loss ensures some level of robustness of the resulting ML method. Indeed, a hypothesis with maximum margin is maximally robust against perturbations of the feature vectors of data points. Section 7.2 discusses the importance of robustness in ML methods in more detail.

The statistical superiority (in terms of robustness) of the SVM comes at the cost of increased computational complexity. The hinge loss (2.11) is non-differentiable which prevents the use of simple gradient-based methods (see Chapter 5) and requires more advanced optimization methods. In contrast, the logistic loss (2.12) is convex and differentiable. Logistic

regression allows for the use of gradient-based methods to minimize the average logistic loss incurred on a training set (see Chapter 5).

3.8 Bayes Classifier

Consider data points characterized by a feature vector $\mathbf{x} \in \mathcal{X}$ and some binary label $y \in \mathcal{Y}$. We can use any two different label values but let us assume that the two possible label values are $y = -1$ or $y = 1$. We would like to find (or learn) a classifier $h : \mathcal{X} \rightarrow \mathcal{Y}$ such that the predicted (or estimated) label $\hat{y} = h(\mathbf{x})$ agrees with the true label $y \in \mathcal{Y}$ as much as possible. Thus, it is reasonable to assess the quality of a classifier h using the 0/1 loss (2.9). We could then learn a classifier using the ERM with the loss function (2.9). However, the resulting optimization problem is typically intractable since the loss (2.9) is non-convex and non-differentiable.

Instead of solving the (intractable) ERM for 0/1 loss (2.9), we can take a different route to construct a classifier. This construction is based on a simple probabilistic model for data. Using this model, we can interpret the average 0/1 loss incurred by a hypothesis on a training set as an approximation to the probability $p_{\rm err} = p(y \neq h(\mathbf{x}))$. Any classifier $\widehat{h}$ that minimizes the error probability $p_{\rm err}$, which is the expected 0/1 loss, is referred to as a Bayes estimator. Section 4.5 will discuss ML methods using Bayes estimator in more detail.

Let us derive the Bayes estimator for a the special case of a binary classification problem. Here, data points are characterized by features $\mathbf{x}$ and label $y \in \{-1, 1\}$. Elementary probability theory allows to derive the Bayes estimator, which is the hypothesis minimizing the expected 0/1 loss, as

$$\widehat{h}(\mathbf{x}) = \begin{cases} 1 & \text{if } p(y=1|\mathbf{x}) > p(y=-1|\mathbf{x}) \\ -1 & \text{otherwise.} \end{cases} . \tag{3.25}$$

Note that the Bayes estimator (3.25) depends on the probability distribution $p(\mathbf{x}, y)$ underlying the data points.[2] We obtain different Bayes estimators for different probabilistic models. One widely used probabilistic model results in a Bayes estimator that belongs to the linear hypothesis space (3.1). Note that this hypothesis space underlies also logistic regression (see Section 3.6) and the SVM (see Section 3.7). Thus, logistic regression, SVM and Bayes estimator are all examples of a linear classifier (see Figure 2.9).

[2]Remember that we interpret data points as realizations of i.i.d. RVs with common probability distribution $p(\mathbf{x}, y)$.

A linear classifier partitions the feature space $\mathcal{X}$ into two half-spaces. One half-space consists of all feature vectors $\mathbf{x}$ which result in the predicted label $\hat{y} = 1$ and the other half-space constituted by all feature vectors $\mathbf{x}$ which result in the predicted label $\hat{y} = -1$. The family of ML methods that learn a linear classifier differ in their choices for the loss functions used to assess the quality of these half-spaces.

3.9 Kernel Methods

Consider a ML (classification or regression) problem with an underlying feature space $\mathcal{X}$. In order to predict the label $y \in \mathcal{Y}$ of a data point based on its features $\mathbf{x} \in \mathcal{X}$, we apply a predictor h selected out of some hypothesis space $\mathcal{H}$. Let us assume that the available computational infrastructure only allows us to use a linear hypothesis space $\mathcal{H}^{(n)}$ (see (3.1)).

For some applications, using a linear hypothesis $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ is not suitable since the relation between features $\mathbf{x}$ and label y might be highly non-linear. One approach to extend the capabilities of linear models is to transform the raw features of a data point before applying a linear hypothesis h.

The family of kernel methods is based on transforming the features $\mathbf{x}$ to new features $\hat{\mathbf{x}} \in \mathcal{X}'$ which belong to a (typically very) high-dimensional space $\mathcal{X}'$ [82]. While the original feature space might be a low-dimensional Euclidean space (e.g., $\mathcal{X} = \mathbb{R}^2$), the transformed feature space $\mathcal{X}'$ is an infinite-dimensional function space.

The rationale behind transforming the original features into a new (higher-dimensional) feature space $\mathcal{X}'$ is to reshape the geometry of the features $\mathbf{x}^{(i)} \in \mathcal{X}$ such that the transformed feature vectors $\hat{\mathbf{x}}^{(i)}$ have a "more linear" geometry (see Figure 3.7).

Kernel methods are obtained by formulating ML problems (such as linear regression or logistic regression) using the transformed features $\hat{\mathbf{x}} = \phi(\mathbf{x})$. A key challenge within kernel methods is the choice of a suitable feature map $\phi : \mathcal{X} \to \mathcal{X}'$ which maps the original features vector $\mathbf{x}$ to new features $\hat{\mathbf{x}} = \phi(\mathbf{x})$.

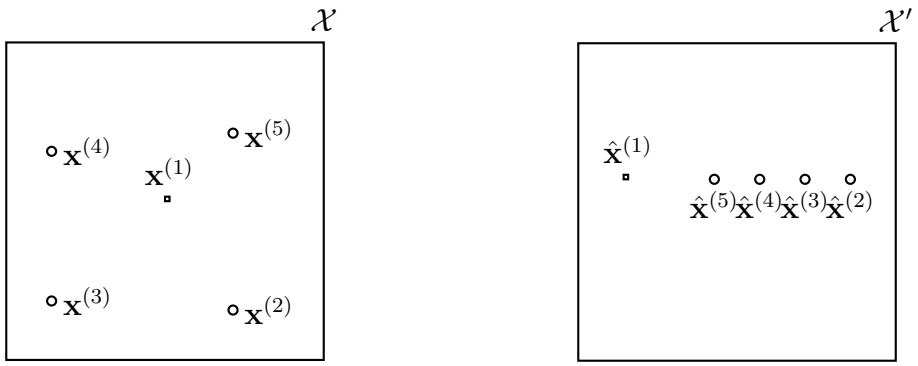


Figure 3.7: The data set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{5}$ consists of 5 data points with features $\mathbf{x}^{(i)}$ and binary labels $y^{(i)}$. Left: In the original feature space $\mathcal{X}$, the data points cannot be separated perfectly by any linear classifier Right: The feature map $\phi : \mathcal{X} \to \mathcal{X}'$ transforms the features $\mathbf{x}^{(i)}$ to the new features $\hat{\mathbf{x}}^{(i)} = \phi\big(\mathbf{x}^{(i)}\big)$ in the new feature space $\mathcal{X}'$. In the new feature space $\mathcal{X}'$ the data points can be separated perfectly by a linear classifier.

3.10 Decision Trees

A decision tree is a flowchart-like description of a map $h : \mathcal{X} \to \mathcal{Y}$ which maps the features $\mathbf{x} \in \mathcal{X}$ of a data point to a predicted label $h(\mathbf{x}) \in \mathcal{Y}$ [58]. While we can use decision trees for an arbitrary feature space $\mathcal{X}$ and label space $\mathcal{Y}$, we will discuss them for the particular feature space $\mathcal{X} = \mathbb{R}^2$ and label space $\mathcal{Y} = \mathbb{R}$.

Figure 3.8 depicts an example for a decision tree. A decision tree, consists of nodes which are connected by directed edges. We can think of a decision tree,as a step-by-step instruction, or a "recipe", for how to compute the function value $h(\mathbf{x})$ given the features $\mathbf{x} \in \mathcal{X}$ of a data point. This computation starts at the "root" node and ends at one of the "leaf" nodes of the decision tree.

A leaf node $\hat{y}$, which does not have any outgoing edges, represents a decision region $\mathcal{R}_{\hat{y}} \subseteq \mathcal{X}$ in the feature space. The hypothesis h associated with a decision tree, is constant over the regions $\mathcal{R}_{\hat{y}}$, such that $h(\mathbf{x}) = \hat{y}$ for all $\mathbf{x} \in \mathcal{R}_{\hat{y}}$ and some label value $\hat{y} \in \mathbb{R}$.

The nodes in a decision tree are of two different types,

- decision (or test) nodes, which represent particular "tests" about the feature vector $\mathbf{x}$, e.g., "is the norm of $\mathbf{x}$ larger than 10?").
- leaf nodes, which correspond to subsets of the feature space.

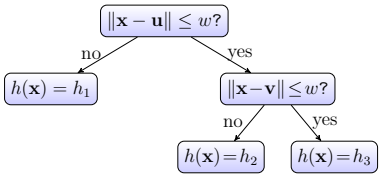
The particular decision tree,depicted in Figure 3.8 consists of two decision nodes (including

the root node) and three leaf nodes.

Given limited computational resources, we can only use decision trees with a limited depth. The depth of a decision tree, is the maximum number of hops it takes to reach a leaf node starting from the root and following the arrows. The decision tree,depicted in Figure 3.8 has depth 2. We obtain an entire hypothesis space by collecting all hypothesis maps that are obtained from the decision tree in Figure 3.8 with some vectors $\mathbf{u}$ and $\mathbf{v}$, some positive radius $w > 0$. The resulting hypothesis space is parametrized by the vectors $\mathbf{u}, \mathbf{v}$ and the number w.

To assess the quality of a particular decision tree,we can use various loss functions. Examples of loss functions used to measure the quality of a decision tree, are the squared error loss for numeric labels (regression) or the impurity of individual decision region for categorical labels (classification).

Decision tree methods use as a hypothesis space the set of all hypotheses which represented by a family of decision trees. Figure 3.9 depicts a collection of decision trees which are characterized by having depth at most two. More generally, we can construct a collection of decision trees using a fixed set of "elementary tests" on the input feature vector such as $\|\mathbf{x}\| > 3$, $x_3 < 1$. These tests might also involve a continuous (real-valued) parameter such as $\{x_2 > w\}_{w \in [0,10]}$. We then build a hypothesis space by considering all decision trees not exceeding a maximum depth and whose decision nodes carry out one of the elementary tests.



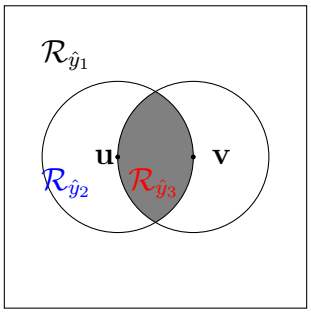


Figure 3.8: A decision tree represents a hypothesis h which is constant on the decision region $\mathcal{R}_{\hat{y}}$, i.e., $h(\mathbf{x}) = \hat{y}$ for all $\mathbf{x} \in \mathcal{R}_{\hat{y}}$. Each decision region $\mathcal{R}_{\hat{y}} \subseteq \mathcal{X}$ corresponds to a specific leaf node in the decision tree.

A decision tree represents a map $h : \mathcal{X} \to \mathcal{Y}$, which is piecewise-constant over regions of the feature space $\mathcal{X}$. These non-overlapping decision regions partition the feature space into subsets that are all mapped to the same predicted label. Each leaf node of a decision tree corresponds to one particular decision region. Using large decision trees with many test nodes, we can learn a hypothesis with many different decision regions. These decision regions

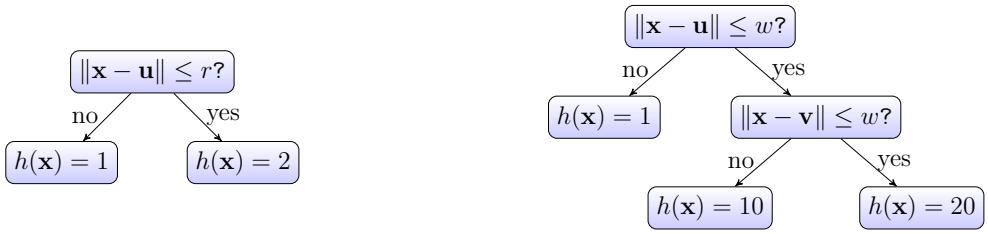


Figure 3.9: A hypothesis space $\mathcal{H}$ which consists of two decision trees with depth 2 and using the tests $\|\mathbf{x}-\mathbf{u}\| \leq w$ and $\|\mathbf{x}-\mathbf{v}\| \leq w$ with a fixed radius w and vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$.

can be chosen such they perfectly align with a given labeled dataset (see Figure 3.10).

The hypothesis space spanned by sufficiently large (deep) decision tree allows to approximate any given non-linear map (under mild technical conditions such as Lipschitz continuity). This is quite different from ML methods using the linear hypothesis space (3.1), such as linear regression, logistic regression or the SVM. These methods learn linear hypothesis maps with a rather simple geometry. Indeed, a linear map is constant along hyperplanes. Moreover, the decision regions obtained from linear classifiers are always entire half-spaces (see Figure 2.9).

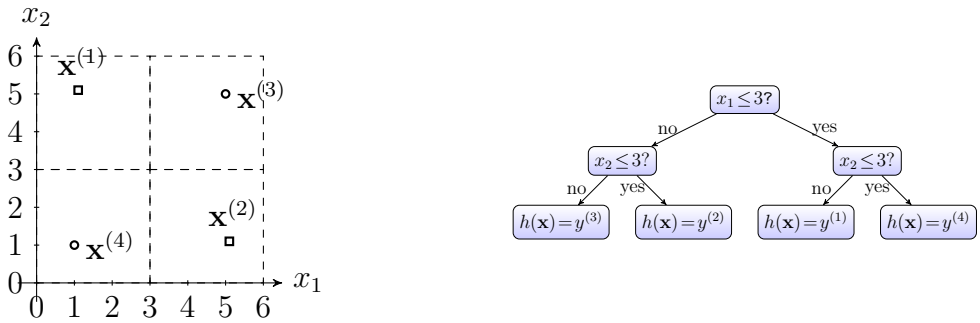


Figure 3.10: Using a sufficiently large (deep) decision tree, we can construct a map h that perfectly fits any given labeled dataset $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$ such that $h(\mathbf{x}^{(i)}) = y^{(i)}$ for $i = 1, \ldots, m$.

3.11 Deep Learning

Another example of a hypothesis space uses a signal-flow representation of a hypothesis $h : \mathbb{R}^n \to \mathbb{R}$. This signal-flow representation is referred to as a ANN. As the name indicates, an ANN is a network of interconnected elementary computational units. These computational units might be referred to as artificial neurons or just neurons.

Figure 3.11 depicts the simplest possible ANN that consists of a single neuron. The neuron computes a weighted sum of the inputs and then applies an activation function $\sigma(z)$ to produce the output $\sigma(z)$. The j-th input of a neuron is assigned a parameter or weight w_j. For a given choice of weights the ANN in Figure 3.11 represents a hypothesis map $h^{(\mathbf{w})}(\mathbf{x}) = \sigma(z) = \sigma\big(\sum_j w_j x_j\big)$.

The ANN in Figure 3.11 defines a hypothesis space that is constituted by all maps $h^{(\mathbf{w})}$ obtained for different choices for the weights $\mathbf{w}$ in Figure 3.11. Note that the single-neuron ANN in Figure 3.11 reduces to a linear map when we use the activation function $\sigma(z) = z$. However, even when we use a non-linear activation function in Figure 3.11, the resulting hypothesis space is essentially the same as the space of linear maps (3.1). In particular, if we threshold the output of the ANN in Figure 3.11 to obtain a binary label, we will always obtain a linear classifier like logistic regression and SVM (see Section 3.6 and Section 3.7).

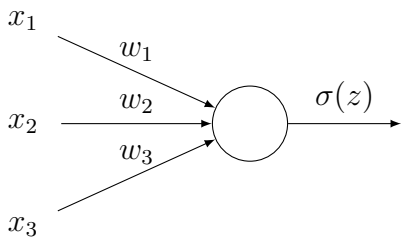


Figure 3.11: ANN consisting of a single neuron that implements a weighted summation $z = \sum_j w_j x_j$ of its inputs x_j followed by applying a non-linear activation function $\sigma(z)$.

Deep learning methods use ANN consisting of many (thousands to millions) interconnected neurons [48]. The interconnections between neurons can be arbitrary in general. However, it turns out to be beneficial to organize neurons as layers and only connect neurons in consecutive layers [48]. Figure 3.12 depicts an example for a ANN consisting of

one hidden layer to represent a (parametrized) hypothesis $h^{(\mathbf{w})} : \mathbb{R}^n \to \mathbb{R}$. The first layer

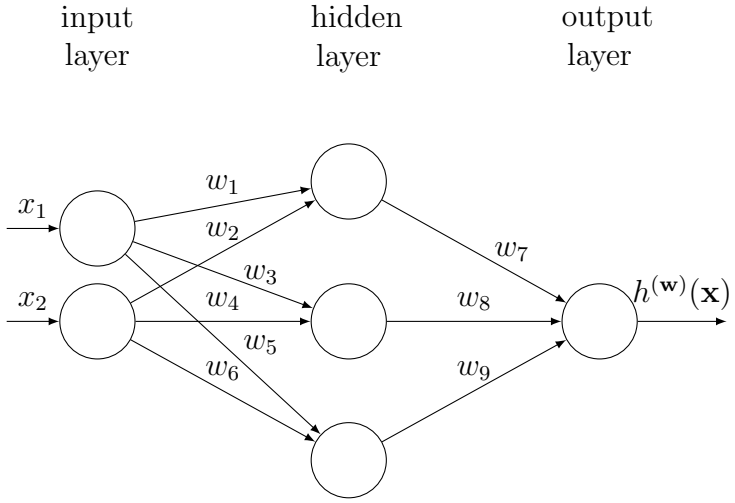


Figure 3.12: ANN representation of a hypothesis $h^{(\mathbf{w})}(\mathbf{x})$ which maps the features (input) $\mathbf{x} = (x_1, x_2)^T$ to a predicted label (output) $h^{(\mathbf{w})}(\mathbf{x})$. The depicted ANN defines a hypothesis space consisting of all maps $h^{(\mathbf{w})}(x)$ obtained from all possible choices for the weights $\mathbf{w} = (w_1, \ldots, w_9)^T$.

of the ANN in Figure 3.12 is referred to as the input layer. The input layer reads in the feature vector $\mathbf{x} \in \mathbb{R}^n$ of a data point. The features x_j are then multiplied with the weights $w_{j,j'}$ associated with the link between the jth input node ("neuron") with the j'th node in the middle (hidden) layer. The output of the j'-th node in the hidden layer is given by $s_{j'} = \sigma(\sum_{j=1}^{n} w_{j,j'} x_j)$ with some (typically non-linear) activation function $\sigma : \mathbb{R} \to \mathbb{R}$. The input to the activation function is the weighted combination $\sum_{j=1}^{n} w_{j,j'} s_{j'}$ of the outputs s_j of the nodes in the previous layer. For the ANN depicted in Figure 3.12, the output of neuron s_1 is $\sigma(z)$ with $z = w_{1,1}x_1 + w_{1,2}x_2$.

The hypothesis map represented by an ANN is parametrized by the weights of the connections between neurons. Moreover, the resulting hypothesis map depends also on the choice for the activation functions of the individual neurons. These activation function are a design choice that can be adapted to the statistical properties of the data. However, a few particular choices for the activation function have proven useful in many important application domains. Two popular choices for the activation function used within ANNs are the sigmoid function $\sigma(z) = \frac{1}{1+\exp(-z)}$ or the rectified linear unit (ReLU) $\sigma(z) = \max\{0, z\}$. ANNs using many, say 10, hidden layers, is often referred to as a "deep net". ML methods

using hypothesis spaces obtained from deep nets are known as deep learning methods [48].

It can be shown that an ANN with only one single (but arbitrarily large) hidden layer can approximate any given map $h : \mathcal{X} \to \mathcal{Y} = \mathbb{R}$ to any desired accuracy [28]. Deep learning methods often use a ANN with a relatively large number (more than hundreds) of hidden layers. We refer to a ANN with a relatively large number of hidden layers as a deep net.

There is empirical and theoretical evidence that using many hidden layers, instead of few but wide layers, is computationally and statistically favourable [33, 114] and [48, Ch. 6.4.1.]. The hypothesis map $h^{(\mathbf{w})}$ represented by an ANN can be evaluated (to obtain the predicted label at the output) efficiently using message passing over the ANN. This message passing can be implemented using parallel and distributed computers. Moreover, the graphical representation of a parametrized hypothesis in the form of a ANN allows us to efficiently compute the gradient of the loss function via a (highly scalable) message passing procedure known as back-propagation [48]. Being able to quickly compute gradients is instrumental for the efficiency of gradient based methods for learning a good choice for the ANN weights (see Chapter 5).

3.12 Maximum Likelihood

For many applications it is useful to model the observed data points $\mathbf{z}^{(i)}$, with $i = 1, \ldots, m$, as i.i.d. realizations of a RV $\mathbf{z}$ with probability distribution $p(\mathbf{z}; \mathbf{w})$. This probability distribution is parametrized by a parameter vector $\mathbf{w} \in \mathbb{R}^n$. A principled approach to estimating the parameter vector $\mathbf{w}$ based on a set of i.i.d. realizations $\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(m)} \sim p(\mathbf{z}; \mathbf{w})$ is ML estimation [85].

ML estimation can be interpreted as an ML problem with a hypothesis space parametrized by the parameter vector $\mathbf{w}$. Each element $h^{(\mathbf{w})}$ of the hypothesis space $\mathcal{H}$ corresponds to one particular choice for the parameter vector $\mathbf{w}$. ML methods use the loss function

$$L\left(\mathbf{z}, h^{(\mathbf{w})}\right) := -\log p(\mathbf{z}; \mathbf{w}). \tag{3.26}$$

A widely used choice for the probability distribution $p\big(\mathbf{z}; \mathbf{w}\big)$ is a multivariate normal (Gaussian) distribution with mean $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$, both of which constitute the parameter vector $\mathbf{w} = (\boldsymbol{\mu}, \boldsymbol{\Sigma})$ (we have to reshape the matrix $\boldsymbol{\Sigma}$ suitably into a vector form). Given the i.i.d. realizations $\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(m)} \sim p\left(\mathbf{z}; \mathbf{w}\right)$, the ML estimates $\hat{\boldsymbol{\mu}}$, $\widehat{\boldsymbol{\Sigma}}$ of the mean

vector and the covariance matrix are obtained via

$$\hat{\boldsymbol{\mu}}, \widehat{\boldsymbol{\Sigma}} = \operatorname*{argmin}_{\boldsymbol{\mu} \in \mathbb{R}^n, \boldsymbol{\Sigma} \in \mathbb{S}_+^n} (1/m) \sum_{i=1}^{m} - \log p\big(\mathbf{z}^{(i)}; (\boldsymbol{\mu}, \boldsymbol{\Sigma})\big). \tag{3.27}$$

The optimization in (3.27) is over all possible choices for the mean vector $\boldsymbol{\mu} \in \mathbb{R}^n$ and the covariance matrix $\boldsymbol{\Sigma} \in \mathbb{S}_+^n$. Here, $\mathbb{S}_+^n$ denotes the set of all positive semi-definite (psd) Hermitian $n \times n$ matrices.

The ML problem (3.27) can be interpreted as an instance of ERM (4.3) using the particular loss function (3.26). The resulting estimates are given explicitly as

$$\hat{\boldsymbol{\mu}} = (1/m) \sum_{i=1}^{m} \mathbf{z}^{(i)}, \text{ and } \widehat{\boldsymbol{\Sigma}} = (1/m) \sum_{i=1}^{m} (\mathbf{z}^{(i)} - \hat{\boldsymbol{\mu}})(\mathbf{z}^{(i)} - \hat{\boldsymbol{\mu}})^T. \tag{3.28}$$

Note that the expressions (3.28) are only valid when the probability distribution of the data points is modelled as a multivariate normal distribution.

3.13 Nearest Neighbour Methods

Nearest neighbor (NN) methods are a family of ML methods that are characterized by a specific construction of the hypothesis space. NN methods can be applied to regression problems involving numeric labels (e.g., using label space $\mathcal{Y} = \mathbb{R}$) as well as for classification problems involving categorical labels (e.g., with label space $\mathcal{Y} = \{-1, 1\}$).

While NN methods can be combined with arbitrary label spaces, they require the feature space to have a specific structure. NN methods require the feature space be a metric space [119] that provides a measure for the distance between different feature vectors. We need a metric or distance measure to determine the nearest neighbour of a data point. A prominent example for a metric feature space is the Euclidean space $\mathbb{R}^n$. The metric of $\mathbb{R}^n$ given by the Euclidean distance $\|\mathbf{x} - \mathbf{x}'\|$ between two vectors $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^n$.

Consider a training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$ that consists of labeled data points. Thus, for each data point we know the features and the label value. Given such a training set, NN methods construct a hypothesis space that consist of piece-wise constant maps $h : \mathcal{X} \to \mathcal{Y}$. For any hypothesis h in that space, the function value $h(\mathbf{x})$ depends only on the (labels of the) k nearest data points (smallest distance to $\mathbf{x}$) in the training set $\mathcal{D}$. The number k of NNs used to determine the function value $h(\mathbf{x})$ is a hyper-parameter of the NN method. NN methods are also referred to as k-NN methods to make their dependence on the parameter

k explicit.

Let us illustrate NN methods by considering a binary classification problem using an uneven number for k (e.g., $k = 3$ or $k = 5$). The goal is to learn a hypothesis that predicts the binary label $y \in \{-1, 1\}$ of a data point based on its feature vector $\mathbf{x} \in \mathbb{R}^n$. This learning task can make use of a training set $\mathcal{D}$ containing $m > k$ data points with known labels. Given a data point with features $\mathbf{x}$, denote by $\mathcal{N}^{(k)}$ a set of k data points in $\mathcal{D}$ whose feature vectors have smallest distance to $\mathbf{x}$. The number of data points in $\mathcal{N}^{(k)}$ whose label is 1 is denoted $m_1^{(k)}$ and those with label value -1 is denoted $m_{-1}^{(k)}$. The k-NN method "learns" a hypothesis $\widehat{h}$ given by

$$\widehat{h}(\mathbf{x}) = \begin{cases} 1 & \text{if } m_1^{(k)} > m_{-1}^{(k)} \\ -1 & \text{otherwise.} \end{cases} \tag{3.29}$$

It is important to note that, in contrast to the ML methods in Section 3.1 - Section 3.11, the hypothesis space of k-NN depends on a labeled dataset (training set) $\mathcal{D}$. As a consequence, k-NN methods need to access (and store) the training set whenever the compute a prediction (evaluate $h(\mathbf{x})$). To compute the prediction $h(\mathbf{x})$ for a data point with features $\mathbf{x}$, k-NN needs to determine its NNs in the training set. When using a large training set this implies a large storage requirement for k-NN methods. Moreover, k-NN methods might be prone to revealing sensitive information with its predictions (see Exercise 3.9).

For a fixed k, NN methods do not require any parameter tuning. Such parameter tuning (or learning) is required linear regression, logistic regression and deep learning methods. In contrast, the hypothesis "learnt" by NN methods is characterized point-wise, for each possible value of features $\mathbf{x}$, by the NN in the training set. Compared to the ML methods in Section 3.1 - Section 3.11, NN methods do not require to solve (challenging) optimization problems for model parameters. Beside their low computational requirements (put aside the memory requirements), NN methods are also conceptually appealing as natural approximations of Bayes estimators (see [26] and Exercise 3.10).

3.14 Deep Reinforcement Learning

Deep reinforcement learning (DRL) refers to a subset of ML problems and methods that revolve around the control of dynamic systems such as autonomous driving cars or cleaning robots [129, 137, 104]. A DRL problem involves data points that represent the states of a dynamic system at different time instants $t = 0, 1, \ldots$. The data points representing the state at some time instant t is characterized by the feature vector $\mathbf{x}^{(t)}$. The entries of this

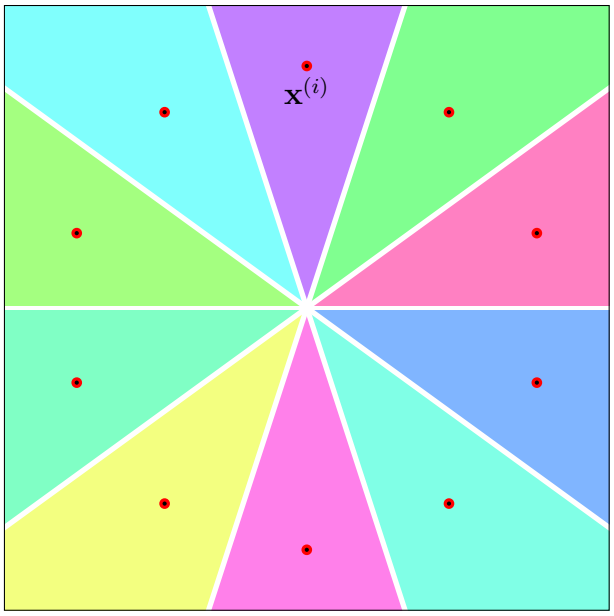


Figure 3.13: A hypothesis map h for k-NN with $k = 1$ and feature space $\mathcal{X} = \mathbb{R}^2$. The hypothesis map is constant over regions (indicated by the coloured areas) located around feature vectors $\mathbf{x}^{(i)}$ (indicated by a dot) of some data $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}$.

feature vector are the individual features of the state at time t. These features might be obtained via sensors, onboard-cameras or other ML methods (that predict the location of the dynamic system). The label $y^{(t)}$ of a data point might represent the optimal steering angle at time t.

DRL methods learn a hypothesis h that delivers optimal predictions $\hat{y}^{(t)} := h\big(\mathbf{x}^{(t)}\big)$ for the optimal steering angle $y^{(t)}$. As their name indicates, DRL methods use hypothesis spaces obtained from a deep net (see Section 3.11). The quality of the prediction $\hat{y}^{(t)}$ obtained from a hypothesis is measured by the loss $L\left(\big(\mathbf{x}^{(t)}, y^{(t)}\big), h\right) := -r^{(t)}$ with a reward signal $r^{(t)}$. This reward signal might be obtained from a distance (collision avoidance) sensor or low-level characteristics of an on-board camera snapshot.

The (negative) reward signal $-r^{(t)}$ typically depends on the feature vector $\mathbf{x}^{(t)}$ and the discrepancy between optimal steering direction $y^{(t)}$ (which is unknown) and its prediction $\hat{y}^{(t)} := h\big(\mathbf{x}^{(t)}\big)$. However, what sets DRL methods apart from other ML methods such as linear regression (see Section 3.1) or logistic regression (see Section 3.6) is that they can evaluate the loss function only point-wise $L\left(\big(\mathbf{x}^{(t)}, y^{(t)}\big), h\right)$ for the specific hypothesis h that has been used to compute the prediction $\hat{y}^{(t)} := h\big(\mathbf{x}^{(t)}\big)$ at time instant t. This is fundamentally different from linear regression that uses the squared error loss (2.8) which can be evaluated for every possible hypothesis $h \in \mathcal{H}$.

3.15 LinUCB

ML methods are instrumental for various recommender systems [86]. A basic form of a recommender system amount to chose at some time instant t the most suitable item (product, song, movie) among a finite set of alternatives $a = 1, \ldots, A$. Each alternative is characterized by a feature vector $\mathbf{x}^{(t,a)}$ that varies between different time instants.

The data points arising in recommender systems might represent time instants t at which recommendations are computed. The data point at time t is characterized by a feature vector

$$\mathbf{x}^{(t)} = \big(\big(\mathbf{x}^{(t,1)}\big)^T, \ldots, \big(\mathbf{x}^{(t,A)}\big)^T \big)^T. \tag{3.30}$$

The feature vector $\mathbf{x}^{(t)}$ is obtained by stacking the feature vectors of alternatives at time t into a single long feature vector. The label of the data point t is a vector of rewards $\mathbf{y}^{(t)} := \big(r_1^{(t)}, \ldots, r_A^{(t)}\big)^T \in \mathbb{R}^A$. The entry $r_a^{(t)}$ represents the reward obtained by choosing (recommending) alternative a (with features $\mathbf{x}^{(t,a)}$) at time t. We might interpret the reward $r^{(t,a)}$ as an indicator if the costumer actually buys the product corresponding to the recommended alternative a.

The ML method LinUCB (the name seems to be inspired by the terms "linear" and "upper confidence bound" (UCB)) aims at learning a hypothesis h that allows to predict the rewards $\mathbf{y}^{(i)}$ based on the feature vector $\mathbf{x}^{(t)}$ (3.30). As its hypothesis space $\mathcal{H}$, LinUCB uses the space of linear maps from the stacked feature vectors $\mathbb{R}^{nA}$ to the space of reward vectors $\mathbb{R}^A$. This hypothesis space can be parametrized by matrices $\mathbf{W} \in \mathbb{R}^{A \times nA}$. Thus, LinUCB learns a hypothesis that computes predicted rewards via

$$\widehat{\mathbf{y}}^{(t)} := \mathbf{W}\mathbf{x}^{(t)}. \tag{3.31}$$

The entries of $\widehat{\mathbf{y}}^{(t)} = \big(\hat{r}_1^{(t)}, \ldots, \hat{r}_A^{(t)}\big)$ are predictions of the individual rewards $r^{(t,a)}$. It seems natural to recommend at time t the alternative a whose predicted reward is maximum. However, it turns out that this approach is sub-optimal as it prevents the recommender system from learning the optimal predictor map $\mathbf{W}$.

Loosely speaking, LinUCB tries out (explores) each alternative $a \in \{1, \ldots, A\}$ sufficiently often to obtain a sufficient amount of training data for learning a good weight matrix $\mathbf{W}$. At time t, LinUCB chooses the alternative $a^{(t)}$ that maximizes the quantity

$$\hat{r}_a^{(t)} + R(t,a) \ , \ a = 1, \ldots, A. \tag{3.32}$$

We can think of the component $R(t,a)$ as a measure for the uncertainty in the reward obtained by action a. It is constructed such that (3.32) upper bounds the actual reward $r_a^{(t)}$ with a prescribed level of confidence (or probability). The confidence term $R(t,a)$ depends on the feature vectors $\mathbf{x}^{(t',a)}$ of the alternative a at previous time instants $t' < t$. Thus, at each time instant t, LinUCB chooses the alternative a that results in the largest upper confidence bound (UCB) (3.32) on the reward (hence the "UCB" in LinUCB). We refer to the relevant literature on sequential learning and decision-making for more details on the LinUCB method [86].

3.16 Exercises

Exercise 3.1. Logistic loss and Accuracy. Section 3.6 discussed logistic regression as a ML method that learns a linear hypothesis map by minimizing the logistic loss (3.16). The logistic loss has computationally pleasant properties as it is smooth and convex. However, in some applications we might be ultimately interested in the accuracy or (equivalently) the average 0/1 loss (2.9). Can we upper bound the average 0/1 loss using the average logistic loss incurred by a given hypothesis on a given training set?

Exercise 3.2. Robustness Against Outliers. Consider the dataset

$$\mathcal{D} = \big\{\big(x^{(1)}, y^{(1)}\big), \ldots, \big(x^{(m)}, y^{(m)}\big)\big\}.$$

The i-th data point is characterized by the feature $x^{(i)} = i/m$ and label $y^{(i)} = i/m$. Linear regression learns a linear hypothesis $h^{(\mathbf{w})}(x) = w_1 x + w_2$, with parameter vector $\mathbf{w} = \big(w_1, w_2\big)^T$ such that the average squared error loss $(1/m)\sum_{i=1}^{m}\big(y^{(i)} - h(x^{(i)})\big)^2$ is minimized. Let us denote the resulting (learnt) parameter vector as $\widehat{\mathbf{w}} = \big(\hat{w}_1, \hat{w}_2\big)^T$. Consider now that the original dataset is perturbed, resulting in a new dataset $\widetilde{\mathcal{D}}$. The dataset $\widetilde{\mathcal{D}}$ is obtained from $\mathcal{D}$ by replacing the first data point $\big(1/m, 1/m\big)$ with an outlier with feature value $x^{(o)} = c$ and label value $y^{(o)} = 0$. We then apply linear regression to this perturbed dataset which results in a parameter vector $\widehat{\mathbf{w}}^{(o)} := \big(\hat{w}_1^{(o)}, \hat{w}_2^{(o)}\big)^T$. Discuss the deviation between the parameter vectors $\widehat{\mathbf{w}}$ and $\widehat{\mathbf{w}}^{(o)}$ as a function of the outlier feature value c.

Exercise 3.3. How Many Neurons? Consider a predictor map $h(x)$ which is piecewise linear and consisting of 1000 pieces. Assume we want to represent this map by an ANN using neurons with one hidden layer of neurons having a ReLU activation function. The

output layer consists of a single neuron with linear activation function. How many neurons must the ANN contain at least ?

Exercise 3.4. Effective Dimension of ANN. Consider a ANN with $n = 10$ input neurons following by three hidden layers consisting of 4, 9 and 3 nodes. The three hidden layers are followed by the output layer consisting of a single neuron. Assume that all neurons use a linear activation function and no bias term. What is the effective dimension $d_{\text{eff}}(\mathcal{H})$ of the hypothesis space $\mathcal{H}$ that consists of all hypothesis maps that can be obtained from this ANN.

Exercise 3.5. Linear Classifiers. Consider data points characterized by feature vectors $\mathbf{x} \in \mathbb{R}^n$ and binary labels $y \in \{-1, 1\}$. We are interested in finding a good linear classifier which is such that the feature vectors resulting in $h(\mathbf{x}) = 1$ is a half-space. Which of the methods discussed in this chapter aim at learning a linear classifier?

Exercise 3.6. Data Dependent Hypothesis space. Consider a ML application involving data points with features $\mathbf{x} \in \mathbb{R}^6$ and a numeric label $y \in \mathbb{R}$. We learn a hypothesis by minimizing the average loss incurred on a training set $\mathcal{D} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}$. Which of the following ML methods uses a hypothesis space that depends on the dataset $\mathcal{D}$?

- logistic regression
- linear regression
- k-NN

Exercise 3.7. Triangle. Consider the ANN in Figure 3.12 using the ReLU activation function (see Figure 3.11). Show that there is a particular choice for the weights $\mathbf{w} = (w_1, \ldots, w_9)^T$ such that the resulting hypothesis map $h^{(\mathbf{w})}(x)$ is a triangle as depicted in Figure 3.14. Can you also find a choice for the weights $\mathbf{w} = (w_1, \ldots, w_9)^T$ that produce the same triangle shape if we replace the ReLU activation function with the linear function $\sigma(z) = 10 \cdot z$?

Exercise 3.8. Approximate Triangle with Gaussians Try to approximate the hypothesis map depicted in Figure 3.14 by an element of $\mathcal{H}_{\text{Gauss}}$ (see (3.15)) using $\sigma = 1/10$, $n = 10$ and $\mu_j = -1 + (2j/10)$.

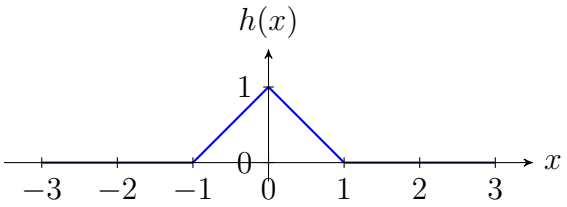


Figure 3.14: A hypothesis map $h : \mathbb{R} \to \mathbb{R}$ with the shape of a triangle.

Exercise 3.9. Privacy Leakage in k-NN Consider a k-NN method for a binary classification problem. We use $k = 1$ and a given training set whose data points characterize humans. Each human is characterized by a feature vector and label that indicates sensitive information (e.g., some sickness). Assume that you have access to the feature vectors of the data points in the training set but not to their labels. Can you infer the label value of a data point in the training set based on the prediction that you obtained based on your feature vector?

Exercise 3.10. k-NN Approximates Bayes estimator Consider a binary classification problem involving data points that are characterized by feature vectors $\mathbf{x} \in \mathbb{R}^n$ and binary labels $y \in \{-1, 1\}$. We have access to a labeled training set $\mathcal{D}$ of size m. Show that the k-NN hypothesis (3.29) is obtained from the Bayes estimator (3.25) by approximating or estimating the conditional probability distribution $p(\mathbf{x}|y)$ via the density estimator [12, Sec. 2.5.2.]

$$\hat{p}(\mathbf{x}\ |y) := (k/m)\frac{1}{\mathrm{vol}(R_k)}. \tag{3.33}$$

Here, $\mathrm{vol}(R)$ denotes the volume of a ball with radius R and R_k is the distance between $\mathbf{x}$ and the kth nearest feature vector of a data point in $\mathcal{D}$.

Chapter 4

Empirical Risk Minimization

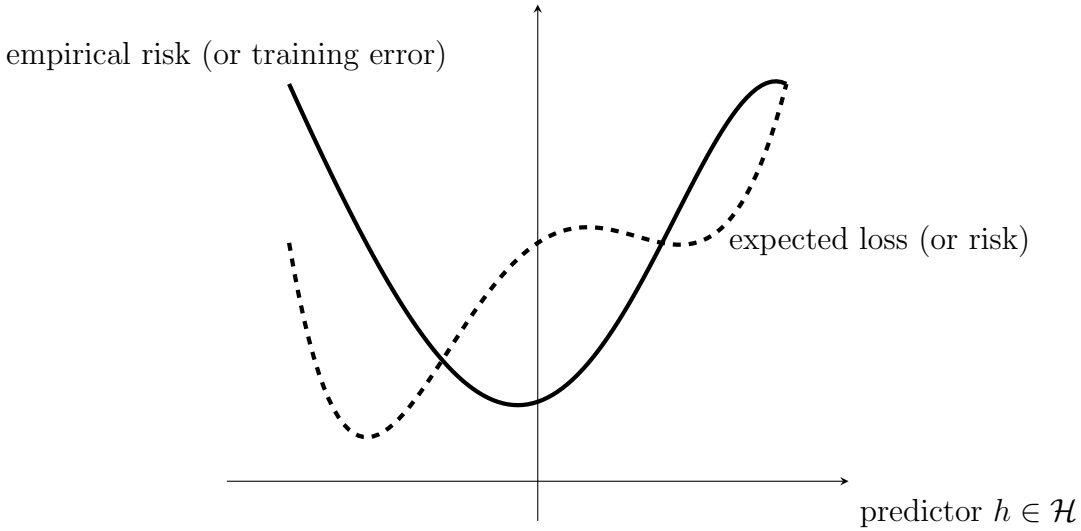


Figure 4.1: ML methods learn a hypothesis $h \in \mathcal{H}$ that incur small loss when predicting the label y of a data point based on its features $\mathbf{x}$. ERM approximates the expected loss or risk by the empirical risk (solid curve) incurred on a finite set of labeled data points (the training set). Note that we can compute the empirical risk based on the observed data points. However, to compute the risk we would need to know the underlying probability distribution which is rarely the case.

Chapter 2 discussed three components (see Figure 2.1):

- data points characterized by features $\mathbf{x} \in \mathcal{X}$ and labels $y \in \mathcal{Y}$,
- a hypothesis space $\mathcal{H}$ of computationally feasible maps $h : \mathcal{X} \to \mathcal{Y}$,

- and a loss function $L\left((\mathbf{x}, y), h\right)$ that measures the discrepancy between the predicted label $h(\mathbf{x})$ and the true label y.

Ideally we would like to learn a hypothesis $h \in \mathcal{H}$ such that $L\left((\mathbf{x}, y), h\right)$ is small for any data point $(\mathbf{x}, y)$. However, to implement this informal goal we need to define what is meant precisely by "any data point". Maybe the most widely used approach to the define the concept of "any data point" is the i.i.d. assumption.

The i.i.d. assumption interprets data points as realizations of i.i.d. RVs with a common probability distribution $p(\mathbf{x}, y)$. The probability distribution $p(\mathbf{x}, y)$ allows us to define the risk of a hypothesis h as the expectation of the loss incurred by h on (the realizations of) a random data point. We can interpret the risk of a hypothesis as a measure for its quality in predicting the label of"any data point".

If we know the probability distribution $p(\mathbf{x}, y)$ from which data points are drawn (i.i.d.), we can precisely determine the hypothesis with minimum risk. This optimal hypothesis, which is referred to as a Bayes estimator, can be read off almost directly from the posterior probability distribution $p(y|\mathbf{x})$ of the label y given the features $\mathbf{x}$ of a data point. The precise form of the Bayes estimator depends on the choice for the loss function. When using the squared error loss, the optimal hypothesis (or Bayes estimator) is given by the posterior mean $h(\mathbf{x}) = \mathbb{E}\big\{y|\mathbf{x}\big\}$ [85].

In most ML application, we do not know the true underlying probability distribution $p(\mathbf{x}, y)$ from which data points are generated. Therefore, we cannot compute the Bayes estimator exactly. However, we can approximately compute this estimator by replacing the exact probability distribution with an estimate or approximation. Section 4.5 discusses a specific ML method that implements this approach.

The risk of the Bayes estimator (which is the Bayes risk) provides a useful baseline against which we can compare the average loss incurred by a ML method on a set of data points. Section 6.6 shows how to diagnose ML methods by comparing its average loss of a hypothesis on a training set and its average loss on a validation set with a baseline.

Section (4.1) motivates ERM by approximating the risk using the empirical risk (or average loss) computed for a set of labeled (training) data points (see Figure 4.1). This approximation is justified by the law of large numbers which characterizes the deviation between averages of RVs and their expectation. Section 4.2 discusses the statistical and computational aspects of ERM. We then specialize the ERM for three particular ML methods arising from different combinations of hypothesis space and loss function. Section 4.3 discusses ERM for linear regression (see Section 3.1). Here, ERM amounts to minimizing a

differentiable convex function, which can be done efficiently using gradient-based methods (see Chapter 5).

We then discuss in Section 4.4 the ERM obtained for decision tree models. The resulting ERM problems becomes a discrete optimization problem which are typically much harder than convex optimization problems. We cannot apply gradient-based methods to solve the ERM for decision trees. To solve ERM for a decision tree, we essentially must try out all possible choices for the tree structure [64].

Section 4.5 considers the ERM obtained when learning a linear hypothesis using the 0/1 loss for classification problems. The resulting ERM amounts to minimizing a non-differentiable and non-convex function. Instead of applying optimization methods to solve this ERM instance, we will instead directly construct approximations of the Bayes estimator.

Section 4.6 decomposes the operation of ML methods into training periods and inference periods. The training period amounts to learning a hypothesis by solving the ERM on a given training set. The resulting hypothesis is then applied to new data points, which are not contained in the training set. This application of a learnt hypothesis to data points outside the training set is referred to as the inference period. Section 4.7 demonstrates how an online learning method can be obtained by solving the ERM sequentially as new data points come in. Online learning methods alternate between training and inference periods whenever new data is collected.

4.1 Approximating Risk by Empirical Risk

The data points arising in many important application domains can be modelled (or approximated) as realizations of i.i.d. RVs with a common (joint) probability distribution $p(\mathbf{x}, y)$ for the features $\mathbf{x}$ and label y. The probability distribution $p(\mathbf{x}, y)$ used in this i.i.d. assumption allows us to define the expected loss or risk of a hypothesis $h \in \mathcal{H}$ as

$$\mathbb{E}\big\{ L\left((\mathbf{x}, y), h\right)\big\}. \tag{4.1}$$

It seems reasonable to learn a hypothesis h such that its risk (4.1) is minimal,

$$h^* := \operatorname*{argmin}_{h \in \mathcal{H}} \mathbb{E}\big\{ L\left((\mathbf{x}, y), h\right)\big\}. \tag{4.2}$$

We refer to any hypothesis h^* that achieves the minimum risk (4.2) as a Bayes estimator [85]. Note that the Bayes estimator h^* depends on both, the probability distribution $p(\mathbf{x}, y)$

and the loss function. When using the squared error loss (2.8) in (4.2), the Bayes estimator h^* is given by the posterior mean of y given the features $\mathbf{x}$ (see [109, Ch. 7]).

Risk minimization (4.2) cannot be used for the design of ML methods whenever we do not know the probability distribution $p(\mathbf{x}, y)$. If we do not know the probability distribution $p(\mathbf{x}, y)$, which is the rule for many ML applications, we cannot evaluate the expectation in (4.1). One exception to this rule is if the data points are synthetically generated by drawing realizations from a given probability distribution $p(\mathbf{x}, y)$.

The idea of ERM is to approximate the expectation in (4.2) with an average loss (the empirical risk) incurred on a given set of data points (the "training set"),

$$\mathcal{D} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}.$$

As discussed in Section 2.3.4, this approximation is justified by the law of large numbers. We obtain ERM by replacing the risk in the minimization problem (4.2) with the empirical risk (2.16),

$$\begin{aligned} \widehat{h} &\in \operatorname*{argmin}_{h \in \mathcal{H}} \widehat{L}(h|\mathcal{D}) \\ &\overset{(2.16)}{=} \operatorname*{argmin}_{h \in \mathcal{H}} (1/m) \sum_{i=1}^{m} L\left((\mathbf{x}^{(i)}, y^{(i)}), h\right). \end{aligned} \tag{4.3}$$

As the notation in (4.3) indicates there might be several different hypotheses that minimize $\widehat{L}(h|\mathcal{D})$. We denote by $\widehat{h}$ any of them. Mathematically, ERM (4.3) is just an instance of an optimization problem [14]. The optimization domain in (4.3) is the hypothesis space $\mathcal{H}$ of a ML method, the objective function (or cost function) is the empirical risk (2.16). ML methods that learn a hypothesis via ERM (4.3) are instances of optimization algorithms [133].

ERM (4.3) is a form of "learning by trial and error". A (hypothetical) instructor (or supervisor) provides us the labels $y^{(i)}$ for the data points in $\mathcal{D}$ which are characterized by features $\mathbf{x}^{(i)}$. This dataset serves as a training set in the following sense. We use a current guess for a good hypothesis h to predict the labels $y^{(i)}$ of the data points in $\mathcal{D}$ only from their features $\mathbf{x}^{(i)}$. We then determine average loss $\widehat{L}(h|\mathcal{D})$ that is incurred by the predictions $\hat{y}^{(i)} = h\big(\mathbf{x}^{(i)}\big)$. If the training error $\widehat{L}(h|\mathcal{D})$ is too large, we try out another hypothesis map h' different from h with the hope of achieving a smaller training error $\widehat{L}(h'|\mathcal{D})$.

We highlight that ERM (4.3) is motivated by the law of large numbers. The law of large numbers, in turn, is only useful if the data points generated within an ML application can be

well modelled as realizations of i.i.d. RVs. This i.i.d. assumption is one of the most widely used working assumptions for the design and analysis of ML methods. However, there are many important application domains involving data points that clearly violate this i.i.d. assumption.

One example for non-i.i.d. data is time series data that consists of temporally ordered (consecutive) data points [15, 88]. Each data point in a time series might represent a specific time interval or single time instants. Another example for non-i.i.d. data arises in active learning where ML methods actively choose (or query) new data points [25]. For a third example of non-i.i.d. data, we refer to FL applications that involve collections (networks) of data generators with different statistical properties [93, 70, 74, 142, 124]. A detailed discussion of ML methods for non-i.i.d. data is beyond the scope of this book.

4.2 Computational and Statistical Aspects of ERM

Solving the optimization problem (4.3) provides two things. First, the minimizer $\widehat{h}$ is a predictor which performs optimal on the training set $\mathcal{D}$. Second, the corresponding objective value $\widehat{L}(\widehat{h}|\mathcal{D})$ (the "training error") can be used to estimate for the risk or expected loss of $\widehat{h}$. However, as we will discuss in Chapter 7, for some datasets $\mathcal{D}$, the training error $\widehat{L}(\widehat{h}|\mathcal{D})$ obtained for $\mathcal{D}$ can be very different from the expected loss (risk) of $\widehat{h}$ when applied to new data points which are not contained in $\mathcal{D}$.

The i.i.d. assumption implies that the training error $\widehat{L}(h|\mathcal{D})$ is only a noisy approximation of the risk $\mathbb{E}\big\{L\left((\mathbf{x},y),h\right)\big\}$. The ERM solution $\widehat{h}$ is a minimizer of this noisy approximation and therefore in general different from the Bayes estimator which minimizes the risk itself. Even if the hypothesis $\widehat{h}$ delivered by ERM (4.3) has small training error $\widehat{L}(\widehat{h}|\mathcal{D})$, it might have unacceptably large risk $\mathbb{E}\big\{L\left((\mathbf{x},y),\widehat{h}\right)\big\}$. We refer to such a situation as overfitting and will discuss techniques for detecting and avoiding it in Chapter 6.

Many important ML methods use hypotheses that are parametrized by a parameter vector $\mathbf{w}$. For each possible parameter vector, we obtain a hypothesis $h^{(\mathbf{w})}(\mathbf{x})$. Such a parametrization is used in linear regression methods which learn a linear hypothesis $h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ with some parameter vector $\mathbf{w}$. Another example for such a parametrization is obtained from ANNs with the weights assigned to inputs of individual (artificial) neurons (see Figure 3.12).

For ML methods that use a parametrized hypothesis $h^{(\mathbf{w})}(\mathbf{x})$, we can reformulate the

optimization problem (4.3) as an optimization of the parameter vector,

$$\widehat{\mathbf{w}} = \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^n} f(\mathbf{w}) \text{ with } f(\mathbf{w}) := \underbrace{(1/m)\sum_{i=1}^{m} L\left((\mathbf{x}^{(i)}, y^{(i)}), h^{(\mathbf{w})}\right)}_{\widehat{L}\left(h^{(\mathbf{w})}|\mathcal{D}\right)}. \tag{4.4}$$

The objective function $f(\mathbf{w})$ in (4.4) is the empirical risk $\widehat{L}\left(h^{(\mathbf{w})}|\mathcal{D}\right)$ incurred by the hypothesis $h^{(\mathbf{w})}$ when applied to the data points in the dataset $\mathcal{D}$. The optimization problems (4.4) and (4.3) are fully equivalent. Given the optimal parameter vector $\widehat{\mathbf{w}}$ solving (4.4), the hypothesis $h^{(\widehat{\mathbf{w}})}$ solves (4.3).

We highlight that the precise shape of the objective function $f(\mathbf{w})$ in (4.4) depends on the parametrization of the hypothesis space $\mathcal{H}$. The parametrization is the precise rule that assigns a hypothesis map $h^{(\mathbf{w})}$ to a given parameter vector $\mathbf{w}$. The shape of $f(\mathbf{w})$ depends also on the choice for the loss function $L\left((\mathbf{x}^{(i)}, y^{(i)}), h^{(\mathbf{w})}\right)$. As depicted in Figure 4.2, the different combinations of parametrized hypothesis space and loss function can result in objective functions with fundamentally different properties such that their optimization is more or less difficult.

The objective function $f(\mathbf{w})$ for the ERM obtained for linear regression (see Section 3.1) is differentiable and convex and can therefore be minimized using simple gradient-based methods (see Chapter 5). In contrast, the objective function $f(\mathbf{w})$ of ERM obtained for least absolute deviation regression or the SVM (see Section 3.3 and 3.7) is non-differentiable but still convex. The minimization of such functions is more challenging but still tractable as there exist efficient convex optimization methods which do not require differentiability of the objective function [110].

The objective function $f(\mathbf{w})$ obtained for ANN are typically highly non-convex with many local minima (see Figure 4.2). The optimization of non-convex objective function is in general more difficult than optimizing a convex objective function (see Fig. 4.2). However, it turns out that gradient-based methods can still be successfully applied to instances of ERM that result in a non-convex objective function [48]. The implementation of ERM might be even more challenging for ML methods that use decision trees or the 0/1 loss (2.9). Indeed, the ERM instances obtained when using decision trees or the 0/1 loss (2.9)involve non-differentiable objective functions which are even harder to minimize compared to non-convex smooth functions (see Section 4.4).

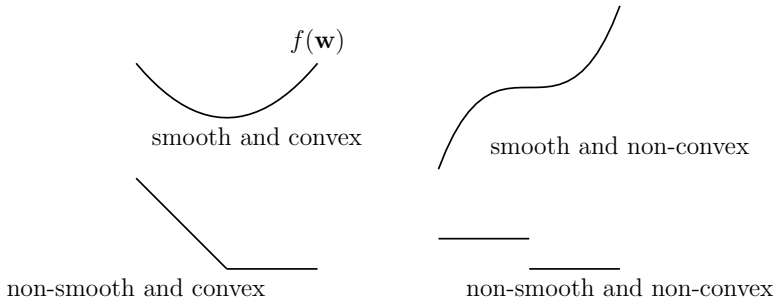


Figure 4.2: Different types of objective function in ERM arising from different combinations of hypothesis space and loss function.

4.3 ERM for Linear Regression

As discussed in Section 3.1, linear regression learns a linear hypothesis $h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ with minimum squared error loss (2.8). For linear regression, the ERM problem (4.4) becomes

$$\widehat{\mathbf{w}} = \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^n} (1/m) \sum_{i=1}^{m} \left(y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)}\right)^2. \tag{4.5}$$

Here, $m = |\mathcal{D}|$ denotes the sample size of the training set $\mathcal{D}$. The objective function $f(\mathbf{w})$ in (4.5) is convex and smooth. Such a function can be minimized using the gradient-based methods discussed in Chapter 5. Variants of these gradient-based methods are widely used in current ML methods to solve the underlying optimization problems [48, Ch. 8].

We can rewrite the ERM problem (4.5) more concisely by stacking the labels $y^{(i)}$ and feature vectors $\mathbf{x}^{(i)}$, for $i = 1, \ldots, m$, into a "label vector" $\mathbf{y}$ and "feature matrix" $\mathbf{X}$,

$$\begin{aligned} \mathbf{y} &= (y^{(1)}, \ldots, y^{(m)})^T \in \mathbb{R}^m, \text{ and} \\ \mathbf{X} &= \left(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\right)^T \in \mathbb{R}^{m\times n}. \end{aligned} \tag{4.6}$$

This allows us to rewrite the objective function in (4.5) as

$$(1/m) \sum_{i=1}^{m} \left(y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)}\right)^2 = (1/m)\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2. \tag{4.7}$$

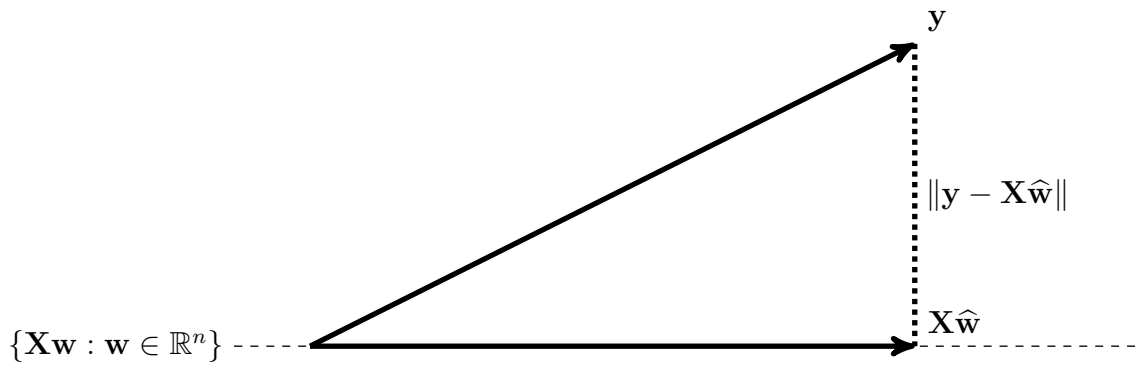


Figure 4.3: The ERM (4.8) for linear regression amounts to an orthogonal projection of the label vector $\mathbf{y} = \big(y^{(1)}, \ldots, y^{(m)}\big)^T$ on the subspace spanned by the columns of the feature matrix $\mathbf{X} = \big(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\big)^T$.

Inserting (4.7) into (4.5), allows to rewrite the ERM problem for linear regression as

$$\widehat{\mathbf{w}} = \operatorname*{argmin}_{\mathbf{w} \in \mathbb{R}^n} (1/m) \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2. \tag{4.8}$$

The formulation (4.8) allows for an interesting geometric interpretation of linear regression. Solving (4.8) amounts to finding a vector $\mathbf{X}\mathbf{w}$ (with feature matrix $\mathbf{X}$ (4.6)), that is closest (in the Euclidean norm) to the label vector $\mathbf{y} \in \mathbb{R}^m$ (4.6). The solution to this approximation problem is precisely the orthogonal projection of the vector $\mathbf{y}$ onto the subspace of $\mathbb{R}^m$ that is spanned by the columns of the feature matrix $\mathbf{X}$ (see Figure 4.3).

To solve the optimization problem (4.8), it is convenient to rewrite it as the quadratic problem

$$\min_{\mathbf{w} \in \mathbb{R}^n} \underbrace{(1/2)\mathbf{w}^T \mathbf{Q} \mathbf{w} - \mathbf{q}^T \mathbf{w}}_{=f(\mathbf{w})}$$
$$\text{with } \mathbf{Q} = (1/m)\mathbf{X}^T\mathbf{X}, \mathbf{q} = (1/m)\mathbf{X}^T\mathbf{y}. \tag{4.9}$$

Since $f(\mathbf{w})$ is a differentiable and convex function, a necessary and sufficient condition for $\widehat{\mathbf{w}}$ to be a minimizer $f(\widehat{\mathbf{w}}) = \min_{\mathbf{w} \in \mathbb{R}^n} f(\mathbf{w})$ is the **zero-gradient condition** [14, Sec. 4.2.3]

$$\nabla f(\widehat{\mathbf{w}}) = \mathbf{0}. \tag{4.10}$$

Combining (4.9) with (4.10), yields the following necessary and sufficient condition for a

parameter vector $\widehat{\mathbf{w}}$ to solve the ERM (4.5),

$$(1/m)\mathbf{X}^T\mathbf{X}\widehat{\mathbf{w}} = (1/m)\mathbf{X}^T\mathbf{y}. \tag{4.11}$$

This condition can be rewritten as

$$(1/m)\mathbf{X}^T\big(\mathbf{y} - \mathbf{X}\widehat{\mathbf{w}}\big) = \mathbf{0}. \tag{4.12}$$

As indicated in Figure 4.3, the optimality condition (4.12) requires the vector

$$\big(\mathbf{y} - \mathbf{X}\widehat{\mathbf{w}}\big) = \big(\big(y^{(1)} - \hat{y}^{(1)}\big), \ldots, \big(y^{(m)} - \hat{y}^{(m)}\big)\big)^T,$$

whose entries are the prediction errors for the data points in the training set, to be orthogonal (or normal) to the subspace spanned by the columns of the feature matrix $\mathbf{X}$. In view of this geometric interpretation, we refer to (4.12) as a "normal equation".

It can be shown that, for any given feature matrix $\mathbf{X}$ and label vector $\mathbf{y}$, there always exists at least one optimal parameter vector $\widehat{\mathbf{w}}$ which solves (4.11). The optimal parameter vector might not be unique, i.e., there might be several different parameter vectors achieving the minimum in (4.5). However, every vector $\widehat{\mathbf{w}}$ which solves (4.11) achieves the same minimum empirical risk

$$\widehat{L}(h^{(\widehat{\mathbf{w}})} \mid \mathcal{D}) = \min_{\mathbf{w}\in\mathbb{R}^n} \widehat{L}(h^{(\mathbf{w})} \mid \mathcal{D}) = \|(\mathbf{I} - \mathbf{P})\mathbf{y}\|^2. \tag{4.13}$$

Here, we used the orthogonal projection matrix $\mathbf{P} \in \mathbb{R}^{m\times m}$ on the linear span of the feature matrix $\mathbf{X} = (\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)})^T \in \mathbb{R}^{m\times n}$ (see (4.6)). The linear span of a matrix $\mathbf{A} = (\mathbf{a}^{(1)}, \ldots, \mathbf{a}^{(m)}) \in \mathbb{R}^{n\times m}$, denoted as $\mathrm{span}\big\{\mathbf{A}\big\}$, is the subspace of $\mathbb{R}^n$ consisting of all linear combinations of the columns $\mathbf{a}^{(r)} \in \mathbb{R}^n$ of $\mathbf{A}$.

If the columns of the feature matrix $\mathbf{X}$ (see (4.6)) are linearly independent, which implies that the matrix $\mathbf{X}^T\mathbf{X}$ is invertible, the projection matrix $\mathbf{P}$ is given explicitly as

$$\mathbf{P} = \mathbf{X}\big(\mathbf{X}^T\mathbf{X}\big)^{-1}\mathbf{X}^T.$$

Moreover, the solution of (4.11) is then unique and given by

$$\widehat{\mathbf{w}} = \big(\mathbf{X}^T\mathbf{X}\big)^{-1}\mathbf{X}^T\mathbf{y}. \tag{4.14}$$

The closed-form solution (4.14) requires the inversion of the $n \times n$ matrix $\mathbf{X}^T\mathbf{X}$.

Note that formula (4.14) is only valid if the matrix $\mathbf{X}^T\mathbf{X}$ is invertible. The feature matrix $\mathbf{X}$ is determined by the data points obtained in a ML application. Its properties are therefore not under the control of a ML method and it might well happen that the matrix $\mathbf{X}^T\mathbf{X}$ is not invertible. As a point in case, the matrix $\mathbf{X}^T\mathbf{X}$ cannot be invertible for any dataset containing fewer data points than the number of features used to characterize data points (this is referred to as high-dimensional data).

Moreover, the matrix $\mathbf{X}^T\mathbf{X}$ is not invertible if there are two co-linear features $x_j, x_{j'}$ such that $x_j = \beta x_{j'}$ holds for any data point with some constant $\alpha \in \mathbb{R}$. Indeed, the existence of two co-linear features implies that the (feature) matrix $\mathbf{X}$ has a non-trivial null-space which, in turn, implies that the matrix $\mathbf{X}^T\mathbf{X}$ cannot be invertible (see Exercise 4.9).

Let us now consider a dataset such that the feature matrix $\mathbf{X}$ is not full column-rank and, in turn, the matrix $\mathbf{X}^T\mathbf{X}$ is not invertible. In this case we cannot use (4.14) to compute the optimal parameter vector since the inverse of $\mathbf{X}^T\mathbf{X}$ does not exist. Moreover, in this case, there are infinitely many different weight vectors that solve (4.11). Each of these solutions result in a linear hypothesis with minimum average squared error loss on the training set. Which one of these optimal weight vectors should we pick ? Section 7.3 explains the benefits of using weight vectors with a small Euclidean norm. The optimal weight vector $\widehat{\mathbf{w}}$ solving the linear regression optimality condition (4.11) and having minimum Euclidean norm among all such vectors is given by

$$\widehat{\mathbf{w}} = \big(\mathbf{X}^T\mathbf{X}\big)^{\dagger}\mathbf{X}^T\mathbf{y}. \tag{4.15}$$

Here, $\big(\mathbf{X}^T\mathbf{X}\big)^{\dagger}$ denotes the pseudoinverse (or the Moore–Penrose inverse) of $\mathbf{X}^T\mathbf{X}$ (see [46, 45]).

Computing the (pseudo-)inverse of $\mathbf{X}^T\mathbf{X}$ can be computationally challenging for large number n of features. Figure 2.5 depicts a simple ML problem where the number of features is already in the millions. The computational complexity of inverting the matrix $\mathbf{X}^T\mathbf{X}$ depends crucially on its condition number. We refer to a matrix as ill-conditioned if its condition number is much larger than 1. In general, ML methods do not have any control on the condition number of the matrix $\mathbf{X}^T\mathbf{X}$. Indeed, this matrix is determined solely by the (features of the) data points fed into the ML method.

Section 5.4 will discuss a method for computing the optimal parameter vector $\widehat{\mathbf{w}}$ that does not require any matrix inversion. This method, referred to as GD constructs a sequence $\mathbf{w}^{(0)}, \mathbf{w}^{(1)}, \ldots$ of increasingly accurate approximations of $\widehat{\mathbf{w}}$. This iterative method has two major benefits compared to evaluating the formula (4.14) using direct matrix inversion, such

as Gauss-Jordan elimination [46].

First, GD typically requires significantly fewer arithmetic operations compared to direct matrix inversion. This is crucial in modern ML applications involving large feature matrices. Second, GD does not break when the matrix $\mathbf{X}$ is not full rank and the formula (4.14) cannot be used any more.

4.4 ERM for Decision Trees

Consider ERM (4.3) for a regression problem with label space $\mathcal{Y} = \mathbb{R}$ and feature space $\mathcal{X} = \mathbb{R}^n$ and the hypothesis space defined by decision trees(see Section 3.10). In stark contrast to ERM for linear regression or logistic regression, ERM for decision trees amounts to a discrete optimization problem. Consider the particular hypothesis space $\mathcal{H}$ depicted in Figure 3.9. This hypothesis space contains a finite number of different hypothesis maps. Each individual hypothesis map corresponds to a particular decision tree.

For the small hypothesis space $\mathcal{H}$ in Figure 3.9, ERM is easy. Indeed, we just have to evaluate the empirical risk ("training error") $\widehat{L}(h)$ for each hypothesis in $\mathcal{H}$ and pick the one yielding the smallest empirical risk. However, when allowing for a very large (deep) decision tree, the computational complexity of exactly solving the ERM becomes intractable [65]. A popular approach to learn a decision tree is to use greedy algorithms which try to expand (grow) a given decision tree by adding new branches to leaf nodes in order to reduce the average loss on the training set (see [66, Chapter 8] for more details).

> The core idea of many decision tree learning methods is simple: expand the tree by replacing a leaf node with a decision node, i.e., by adding a new test on the feature vector, whenever this reduces the overall empirical risk as much as possible.

Consider the labeled dataset $\mathcal{D}$ depicted in Figure 4.4 and a given decision tree for predicting the label y based on the features $\mathbf{x}$. We might first try a hypothesis obtained from the simple tree shown in the top of Figure 4.4. This hypothesis does not allow to achieve a small average loss on the training set $\mathcal{D}$. Therefore, we might grow the tree by replacing a leaf node with a decision node. According to Figure 4.4, to so obtained larger decision tree provides a hypothesis that is able to perfectly predict the labels of the training set (it achieves zero empirical risk).

An important question for methods that learn a decision tree by sequentially growing the tree is when to stop the growth. A straightforward stopping criterion is imposed by

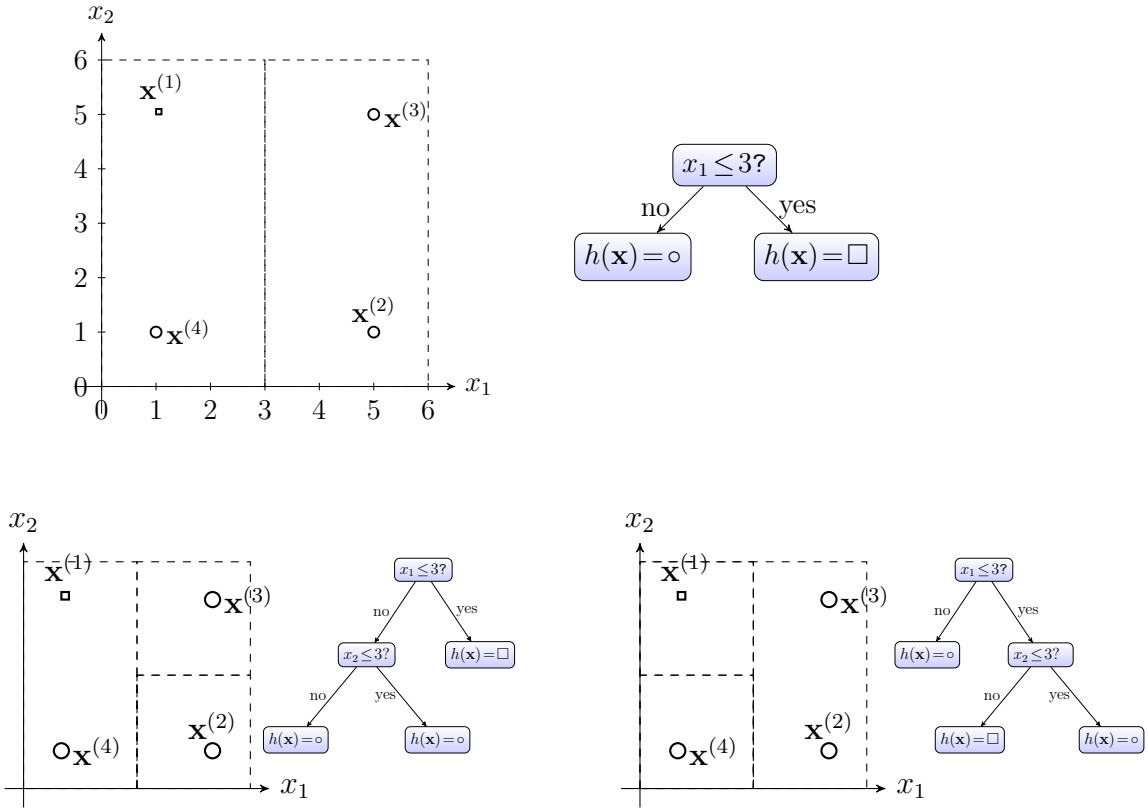


Figure 4.4: Consider a given labeled dataset and the decision tree in the top row. We then grow the decision tree by expanding one of its two leaf nodes. The bottom row shows the resulting decision trees, along with their decision boundaries. Each decision tree in the bottom row is obtained by expanding a different leaf node of the decision tree in the top row.

computational constraints: we may only be able to afford decision trees up to a certain maximum depth. Beyond these computational limits, there are also statistical limits on tree size. Very deep decision trees can represent extremely complex mappings, but this flexibility often leads to overfitting the training set (see Figure 3.10 and Chapter 7). In such cases, a deep tree may achieve a very small average loss on the training set, yet still suffer from poor generalization, incurring unacceptably high loss when predicting the labels of new data points (outside the training set)

4.5 ERM for Bayes Classifiers

The family of ML methods referred to as Bayes estimator uses the 0/1 loss (2.9) to measuring the quality of a classifier h. The resulting ERM is

$$\begin{aligned}\widehat{h} &= \operatorname*{argmin}_{h\in\mathcal{H}} (1/m) \sum_{i=1}^{m} L\left((\mathbf{x}^{(i)}, y^{(i)}), h\right) \\ &\overset{(2.9)}{=} \operatorname*{argmin}_{h\in\mathcal{H}} (1/m) \sum_{i=1}^{m} \mathcal{I}(h(\mathbf{x}^{(i)}) \neq y^{(i)}). \end{aligned} \tag{4.16}$$

The objective function in this optimization problem is non-differentiable and non-convex (see Figure 4.2). This prevents us from using gradient-based methods (see Chapter 5) to solve (4.16).

We will now approach the ERM (4.16) via a different route by interpreting the data points $(\mathbf{x}^{(i)}, y^{(i)})$ as realizations of i.i.d. RVs with the common probability distribution $p(\mathbf{x}, y)$.

As discussed in Section 2.3, the empirical risk obtained using 0/1 loss approximates the error probability $p(\hat{y} \neq y)$ with the predicted label $\hat{y} = 1$ for $h(\mathbf{x}) > 0$ and $\hat{y} = -1$ otherwise (see (2.10)). Thus, we can approximate the ERM (4.16) as

$$\widehat{h} \overset{(2.10)}{\approx} \operatorname*{argmin}_{h\in\mathcal{H}} p(\hat{y} \neq y). \tag{4.17}$$

Note that the hypothesis h, which is the optimization variable in (4.17), enters into the objective function of (4.17) via the definition of the predicted label $\hat{y}$, which is $\hat{y} = 1$ if $h(\mathbf{x}) > 0$ and $\hat{y} = -1$ otherwise.

It turns out that if we would know the probability distribution $p(\mathbf{x}, y)$, which is required to compute $p(\hat{y} \neq y)$, the solution of (4.17) can be found via elementary Bayesian decision theory [115]. In particular, the optimal classifier $h(\mathbf{x})$ is such that $\hat{y}$ achieves the maximum

"a-posteriori" probability $p(\hat{y}|\mathbf{x})$ of the label being $\hat{y}$, given (or conditioned on) the features $\mathbf{x}$.

Since we typically do not know the probability distribution $p(\mathbf{x}, y)$, we have to estimate (or approximate) it from the observed data points $(\mathbf{x}^{(i)}, y^{(i)})$. This estimation is feasible if the data points can be considered (approximated) as realizations of i.i.d. RVs with a common probability distribution $p(\mathbf{x}, y)$. We can then estimate (the parameters) of the probability distribution $p(\mathbf{x}, y)$ using ML methods (see Section 3.12). For numeric features and labels, a widely-used parametric probability distribution $p(\mathbf{x}, y)$ is the multivariate normal (Gaussian) distribution. In particular, conditioned on the label y, the feature vector $\mathbf{x}$ is a Gaussian random vector with mean $\boldsymbol{\mu}_y$ and covariance $\boldsymbol{\Sigma}$,

$$p(\mathbf{x}|y) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_y, \boldsymbol{\Sigma}).^{1} \tag{4.18}$$

The conditional expectation of the features $\mathbf{x}$, given (conditioned on) the label y of a data point, is $\boldsymbol{\mu}_1$ if $y = 1$, while for $y = -1$ the conditional mean of $\mathbf{x}$ is $\boldsymbol{\mu}_{-1}$. In contrast, the conditional covariance matrix $\boldsymbol{\Sigma} = \mathbb{E}\{(\mathbf{x} - \boldsymbol{\mu}_y)(\mathbf{x} - \boldsymbol{\mu}_y)^T|y\}$ of $\mathbf{x}$ is the same for both values of the label $y \in \{-1, 1\}$. The conditional probability distribution $p(\mathbf{x}|y)$ of the feature vector, given the label y, is multivariate normal. In contrast, the marginal distribution of the features $\mathbf{x}$ is a Gaussian mixture model (GMM). We will revisit GMMs later in Section 8.2 where we will see that they are a great tool for soft clustering.

For this probabilistic model of features and labels, the optimal classifier minimizing the error probability $p(\hat{y} \neq y)$ is $\hat{y} = 1$ for $h(\mathbf{x}) > 0$ and $\hat{y} = -1$ for $h(\mathbf{x}) \leq 0$ using the classifier map

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} \text{ with } \mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_{-1}). \tag{4.19}$$

Carefully note that this expression is only valid if the matrix $\boldsymbol{\Sigma}$ is invertible.

We cannot implement the classifier (4.19) directly, since we do not know the true values of the class-specific mean vectors $\boldsymbol{\mu}_1$, $\boldsymbol{\mu}_{-1}$ and covariance matrix $\boldsymbol{\Sigma}$. Therefore, we have to replace those unknown parameters with some estimates $\hat{\boldsymbol{\mu}}_1$, $\hat{\boldsymbol{\mu}}_{-1}$ and $\widehat{\boldsymbol{\Sigma}}$. A principled

[1] We use the shorthand $\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$ to denote the probability density function (pdf)

$$p(\mathbf{x}) = \frac{1}{\sqrt{\det(2\pi\boldsymbol{\Sigma})}} \exp\big(-(1/2)(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\big)$$

of a Gaussian random vector $\mathbf{x}$ with mean $\boldsymbol{\mu} = \mathbb{E}\{\mathbf{x}\}$ and covariance matrix $\boldsymbol{\Sigma} = \mathbb{E}\big\{(\mathbf{x}-\boldsymbol{\mu})(\mathbf{x}-\boldsymbol{\mu})^T\big\}$.

approach is to use the maximum likelihood estimates (see (3.28))

$$\begin{aligned}
\hat{\boldsymbol{\mu}}_1 &= (1/m_1)\sum_{i=1}^{m} \mathcal{I}(y^{(i)}=1)\mathbf{x}^{(i)}, \\
\hat{\boldsymbol{\mu}}_{-1} &= (1/m_{-1})\sum_{i=1}^{m} \mathcal{I}(y^{(i)}=-1)\mathbf{x}^{(i)}, \\
\hat{\boldsymbol{\mu}} &= (1/m)\sum_{i=1}^{m} \mathbf{x}^{(i)}, \\
\text{and } \widehat{\boldsymbol{\Sigma}} &= (1/m)\sum_{i=1}^{m} (\mathbf{z}^{(i)}-\hat{\boldsymbol{\mu}})(\mathbf{z}^{(i)}-\hat{\boldsymbol{\mu}})^T, \qquad (4.20)
\end{aligned}$$

with $m_1 = \sum_{i=1}^{m} \mathcal{I}(y^{(i)} = 1)$ denoting the number of datapoints with label $y = 1$ (m_{-1} is defined similarly). Inserting the estimates (4.20) into (4.19) yields the implementable classifier

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} \text{ with } \mathbf{w} = \widehat{\boldsymbol{\Sigma}}^{-1}(\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_{-1}). \qquad (4.21)$$

We highlight that the classifier (4.21) is only well-defined if the estimated covariance matrix $\widehat{\boldsymbol{\Sigma}}$ (4.20) is invertible. This requires to use a sufficiently large number of training datapoints such that $m \geq n$.

We derived the classifier (4.21) as an approximate solution to the ERM (4.16). The classifier (4.21) partitions the feature space $\mathbb{R}^n$ into two half-spaces. One half-space consists of feature vectors $\mathbf{x}$ for which the hypothesis (4.21) is non-negative and, in turn, $\hat{y} = 1$. The other half-space is constituted by feature vectors $\mathbf{x}$ for which the hypothesis (4.21) is negative and, in turn, $\hat{y} = -1$. Figure 2.9 illustrates these two half-spaces and the decision boundary between them.

The Bayes estimator (4.21) is another instance of a linear classifier like logistic regression and the SVM. Each of these methods learns a linear hypothesis $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$, whose decision boundary (vectors $\mathbf{x}$ with $h(\mathbf{x}) = 0$) is a hyperplane (see Figure 2.9). However, these methods use different loss functions for assessing the quality of a particular linear hypothesis $h(\mathbf{x}) = \mathbf{wx}$ (which defined the decision boundary via $h(\mathbf{x}) = 0$). Therefore, these three methods typically learn classifiers with different decision boundaries.

For the estimator $\widehat{\boldsymbol{\Sigma}}$ (3.28) to be accurate (close to the unknown covariance matrix) we need a number of datapoints (sample size) which is at least of the order n^2. This sample size requirement might be infeasible for applications with only few datapoints available.

The maximum likelihood estimate $\widehat{\boldsymbol{\Sigma}}$ (4.20) is not invertible whenever $m < n$. In this case,

the expression (4.21) becomes useless. To cope with small sample size $m < n$ we can simplify the model (4.18) by requiring the covariance to be diagonal $\mathbf{\Sigma} = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_n^2)$. This is equivalent to modelling the individual features $x_1, \ldots, x_n$ of a data point as conditionally independent, given its label y. The resulting special case of a Bayes estimator is often referred to as a "naive Bayes" classifier.

We finally highlight that the classifier (4.21) is obtained using the generative model (4.18) for the data. Therefore, Bayes estimator belong to the family of generative ML methods which involve modelling the data generation. In contrast, logistic regression and the SVM do not require a generative model for the data points but aim directly at finding the relation between features $\mathbf{x}$ and label y of a data point. These methods belong therefore to the family of discriminative ML methods.

Generative methods such as those learning a Bayes estimator are preferable for applications with only very limited amounts of labeled data. Indeed, having a generative model such as (4.18) allows us to synthetically generate more labeled data by generating random features and labels according to the probability distribution (4.18). We refer to [105] for a more detailed comparison between generative and discriminative methods.

4.6 Training and Inference Periods

Some ML methods repeat the cycle in Figure 1 in a highly irregular fashion. Consider a large image collection which we use to learn a hypothesis about how cat images look like. It might be reasonable to adjust the hypothesis by fitting a model to the image collection. This fitting or training amounts to repeating the cycle in Figure 1 during some specific time period (the "training time") for a large number.

After the training period, we only apply the hypothesis to predict the labels of new images. This second phase is also known as inference period and might be much longer compared to the training period. Ideally, we would like to only have a very short training period to learn a good hypothesis and then only use the hypothesis for inference.

4.7 Online Learning

In it most basic form, ERM requires a given set of labeled data points, which we refer to as the training set. However, some ML methods can access data only in a sequential fashion. As a point in case, consider time series data such as daily minimum and maximum temperatures

recorded by a FMI weather station. Such a time series consists of a sequence of data points that are generated at successive time instants.

Online learning studies ML methods that learn (or optimize) a hypothesis incrementally as new data arrives. This mode of operation is quite different from ML methods that learn a hypothesis at once by solving an ERM problem. These different operation modes corresponds to different frequencies of iterating the basic ML cycle depicted in Figure 1. Online learning methods start a new cycle in Figure 1 whenever a new data point arrives (e.g., we have recorded the minimum and maximum temperate of a day that just ended).

We now present an online learning variant of linear regression (see Section 3.1) which is suitable for time series data with data points $\big(\mathbf{x}^{(t)}, y^{(t)}\big)$ gathered sequentially (over time). In particular, the data points $\big(\mathbf{x}^{(t)}, y^{(t)}\big)$ become available (are gathered) at time instants $t = 1, 2, 3 \ldots$.

Let us stack the feature vectors and labels of all data points available at time t into feature matrix $\mathbf{X}^{(t)}$ and label vector $\mathbf{y}^{(t)}$, respectively. The feature matrix and label vector for the first three time instants are

$$\begin{aligned}
&t = 1: && \mathbf{X}^{(1)} := \big(\mathbf{x}^{(1)}\big)^T, && \mathbf{y}^{(1)} = \big(y^{(1)}\big)^T, \\
&t = 2: && \mathbf{X}^{(2)} := \big(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}\big)^T, && \mathbf{y}^{(2)} = \big(y^{(1)}, y^{(2)}\big)^T, \\
&t = 3: && \mathbf{X}^{(3)} := \big(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}\big)^T, && \mathbf{y}^{(3)} = \big(y^{(1)}, y^{(2)}, y^{(3)}\big)^T. \qquad (4.22)
\end{aligned}$$

As detailed in Section 3.1, linear regression learns the weights $\mathbf{w}$ of a linear map $h(\mathbf{x}) := \mathbf{w}^T\mathbf{x}$ by minimizing the squared error loss $\big(y - h(\mathbf{x})\big)^2$. This informal goal of linear regression is made precise by the ERM problem (4.5) which defines the optimal weights via incurring minimum average squared error loss (empirical risk) on a given training set $\mathcal{D}$. These optimal weights are given by the solutions of (4.12). When the feature vectors of data points in $\mathcal{D}$ are linearly independent, we obtain the closed-form expression (4.14) for the optimal weights.

Inserting the feature matrix $\mathbf{X}^{(t)}$ and label vector $\mathbf{y}^{(t)}$ (4.22) into (4.14), yields

$$\widehat{\mathbf{w}}^{(t)} = \big(\big(\mathbf{X}^{(t)}\big)^T\mathbf{X}^{(t)}\big)^{-1}\big(\mathbf{X}^{(t)}\big)^T\mathbf{y}^{(t)}. \qquad (4.23)$$

For each time instant we can evaluate the RHS of (4.23) to obtain the parameter vector $\widehat{\mathbf{w}}^{(t)}$ that minimizes the average squared error loss over all data points gathered up to time t. However, computing $\widehat{\mathbf{w}}^{(t)}$ via direct evaluation of the RHS in (4.23) for each new time instant t misses an opportunity for recycling computations done already at earlier time instants.

Let us now show how to (partially) reuse the computations used to evaluate (4.23) for

time t in the evaluation of (4.23) for the next time instant $t+1$. To this end, we first rewrite the matrix $\mathbf{Q}^{(t)} := \big(\mathbf{X}^{(t)}\big)^T \mathbf{X}^{(t)}$ as

$$\mathbf{Q}^{(t)} = \sum_{r=1}^{t} \mathbf{x}^{(r)} \big(\mathbf{x}^{(r)}\big)^T. \tag{4.24}$$

Since $\mathbf{Q}^{(t+1)} = \mathbf{Q}^{(t)} + \mathbf{x}^{(t+1)} \big(\mathbf{x}^{(t+1)}\big)^T$, we can use a well-known identity for matrix inverses (see [7, 94]) to obtain

$$\big(\mathbf{Q}^{(t+1)}\big)^{-1} = \big(\mathbf{Q}^{(t)}\big)^{-1} + \frac{\big(\mathbf{Q}^{(t)}\big)^{-1} \mathbf{x}^{(t+1)} \big(\mathbf{x}^{(t+1)}\big)^T \big(\mathbf{Q}^{(t)}\big)^{-1}}{1 - \big(\mathbf{x}^{(t+1)}\big)^T \big(\mathbf{Q}^{(t)}\big)^{-1} \mathbf{x}^{(t+1)}}. \tag{4.25}$$

Inserting (4.25) into (4.23) yields the following relation between optimal parameter vectors at consecutive time instants t and $t+1$,

$$\widehat{\mathbf{w}}^{(t+1)} = \widehat{\mathbf{w}}^{(t)} - \big(\mathbf{Q}^{(t+1)}\big)^{-1} \mathbf{x}^{(t+1)} \big(\big(\mathbf{x}^{(t+1)}\big)^T \widehat{\mathbf{w}}^{(t)} - y^{(t+1)}\big). \tag{4.26}$$

Note that neither evaluating the RHS of (4.26) nor evaluating the RHS of (4.25) requires to actually invert a matrix of with more than one entry (we can think of a scalar number as 1×1 matrix). In contrast, evaluating the RHS (4.23) requires to invert the matrix $\mathbf{Q}^{(t)} \in \mathbb{R}^{n \times n}$. We obtain an online algorithm for linear regression via computing the updates (4.26) and (4.25) for each new time instant t. Another online method for linear regression will be discussed at the end of Section 5.7.

4.8 Weighted ERM

Consider a ML method that uses some hypothesis space $\mathcal{H}$ and loss function L to measure the quality predictions obtained from a specific hypothesis when applied to a data point. A principled approach to learn a useful hypothesis is via ERM (4.3) using a training set

$$\mathcal{D} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}.$$

.

For some applications it might be useful to modify ERM (4.3) by weighting the data points. In particular, for each data point $\big(\mathbf{x}^{(i)}, y^{(i)}\big)$ we specify a non-negative weight $q^{(i)} \in \mathbb{R}_+$. Weighted ERM is obtained from ERM (4.3) by replacing the average loss over the

training set with a weighted average loss,

$$\widehat{h} \in \operatorname*{argmin}_{h \in \mathcal{H}} \sum_{i=1}^{m} q^{(i)} L\left((\mathbf{x}^{(i)}, y^{(i)}), h\right). \tag{4.27}$$

Note that we obtain ERM (4.3) as the special case of weighted ERM (4.27) for the weights $q^{(i)} = 1/m$.

We might interpret the sample weight $q^{(i)}$ as a measure for the importance or relevance of the data point $\left(\mathbf{x}^{(i)}, y^{(i)}\right)$ for the hypothesis $\hat{h}$ learnt via (4.27). The extreme case $q^{(i)} = 0$ means that the data point $\left(\mathbf{x}^{(i)}, y^{(i)}\right)$ becomes irrelevant for learning a hypothesis via (4.27). This could be useful if the data point $\left(\mathbf{x}^{(i)}, y^{(i)}\right)$ represents an outlier that violates the i.i.d. assumption which is satisfied by most of the other data points. Thus, using suitable weights in (4.27) could make the resulting ML method robust against outliers in the training set. Note that we have discussed another strategy (via the choice for the loss function) to achieve robustness against outliers in Section 3.3.

Another use-case of weighted ERM (4.27) is for applications where the risk of a hypothesis is defined using a probability distribution that is different form the probability distribution of the data points in the training set. Thus, the data points conform to an i.i.d. assumption with underlying probability distribution $p(\mathbf{x}, y)$. However, we would like to measure the quality of a hypothesis via the expected loss or risk using a different probability distribution $p'(\mathbf{x}, y)$,

$$\mathbb{E}_{p'}\left\{L\left((\mathbf{x}, y), h\right)\right\} = \int L\left((\mathbf{x}, y), h\right) dp'(\mathbf{x}, y). \tag{4.28}$$

Having a different probability distribution $p'(\mathbf{x}, y) (\neq p(\mathbf{x}, y)))$ to define the overall quality (risk) of a hypothesis might be beneficial for binary classification problems with imbalanced data. Indeed, using the average loss (which approximates the risk under $p(\mathbf{x}, y)$) might not be a useful quality measure if one class is over-represented in the training set (see Section 2.3.4). It can be shown that, under mild conditions, the weighted average loss in (4.27) approximates (4.28) when using the weights $q^{(i)} = p'\left(\mathbf{x}^{(i)}, y^{(i)}\right)/p\left(\mathbf{x}^{(i)}, y^{(i)}\right)$ [12, Sec. 11.1.4].

4.9 Exercise

Exercise 4.1. Uniqueness in Linear Regression What conditions on a training set ensure that there is a unique optimal linear hypothesis map for linear regression?

Exercise 4.2. Uniqueness in Linear Regression II Linear regression uses the squared error loss (2.8) to measure the quality of a linear hypothesis map. We learn the weights $\mathbf{w}$ of a linear map via ERM using a training set $\mathcal{D}$ that consists of $m = 100$ data points. Each data point is characterized by $n = 5$ features and a numeric label. Is there a unique choice for the weights $\mathbf{w}$ that results in a linear predictor with minimum average squared error loss on the training set $\mathcal{D}$)?

Exercise 4.3. A Simple Linear Regression Problem. Consider a training set of m data points, each characterized by a single numeric feature x and numeric label y. We learn a hypothesis of the form $h(x) = x + b$ with some bias $b \in \mathbb{R}$. Can you write down a formula for the optimal b, that minimizes the average squared error loss on training data $\big(x^{(1)}, y^{(1)}\big), \ldots, \big(x^{(m)}, y^{(m)}\big)$.

Exercise 4.4. Simple Least Absolute Deviation Problem. Consider data points characterized by single numeric feature x and label y. We learn a hypothesis map of the form $h(x) = x + b$ with some bias $b \in \mathbb{R}$. Can you write down a formula for the optimal b, that minimizes the average absolute error on training data $\big(x^{(1)}, y^{(1)}\big), \ldots, \big(x^{(m)}, y^{(m)}\big)$.

Exercise 4.5. Polynomial Regression. Consider polynomial regression for data points with a single numeric feature $x \in \mathbb{R}$ and numeric label y. Here, polynomial regression is equivalent to linear regression using the transformed feature vectors $\mathbf{x} = \big(x^0, x^1, \ldots, x^{n-1}\big)^T$. Given a dataset $\mathcal{D} = \big(x^{(1)}, y^{(1)}\big), \ldots, \big(x^{(m)}, y^{(m)}\big)$, we construct the feature matrix $\mathbf{X} = \big(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\big) \in \mathbb{R}^{m \times m}$ with its ith column given by the feature vector $\mathbf{x}^{(i)}$. Verify that this feature matrix is a Vandermonde matrix [44]? How is the determinant of the feature matrix related to the features and labels of data points in the dataset $\mathcal{D}$?

Exercise 4.6. Training Error is not Expected Loss. Consider a training set that consists of data points $\big(x^{(i)}, y^{(i)}\big)$, for $i = 1, \ldots, m = 100$, that are obtained as realizations of i.i.d. RVs. The common probability distribution of these RVs is defined by a random data point (x, y). The feature x of this random data point is a standard Gaussian RV with zero mean and unit variance. The label of a data point is modelled as $y = x + e$ with Gaussian noise $e \sim \mathcal{N}(0, 1)$. The feature x and noise e are statistically independent. We evaluate the

specific hypothesis $h(x) = 0$ (which outputs 0 no matter what the feature value x is) by the training error $E_t = (1/m) \sum_{i=1}^{m} \big(y^{(i)} - h\big(x^{(i)}\big)\big)^2$. Note that E_t is the average squared error loss (2.8) incurred by hypothesis h on the data points $\big(x^{(i)}, y^{(i)}\big)$, for $i = 1, \ldots, m = 100$. What is the probability that the training error E_t is at least 20 % larger than the expected (squared error) loss $\mathbb{E}\big\{\big(y - h(x)\big)^2\big\}$? What is the mean (expected value) and variance of the training error ?

Exercise 4.7. Optimization Methods as Filters. Let us consider a fictional (ideal) optimization method that can be represented as a filter $\mathcal{F}$. This filter $\mathcal{F}$ reads in a real-valued objective function $f(\cdot)$, defined for all parameter vectors $\mathbf{w} \in \mathbb{R}^n$. The output of the filter $\mathcal{F}$ is another real-valued function $\hat{f}(\mathbf{w})$ that is defined point-wise as

$$\hat{f}(\mathbf{w}) = \begin{cases} 1 & \text{, if } \mathbf{w} \text{ is a local minimum of } f(\cdot) \\ 0 & \text{, otherwise.} \end{cases} \tag{4.29}$$

Verify that the filter $\mathcal{F}$ is shift or translation invariant, i.e., $\mathcal{F}$ commutes with a translation $f'(\mathbf{w}) := f(\mathbf{w} + \mathbf{w}^{(o)})$ with an arbitrary but fixed (reference) vector $\mathbf{w}^{(o)} \in \mathbb{R}^n$.

Exercise 4.8. Linear Regression with Sample Weighting. Consider a linear regression method that uses ERM to learn weights $\widehat{\mathbf{w}}$ of a linear hypothesis $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$. The weights are learnt by minimizing the average squared error loss incurred by h on a training set that is constituted by the data points $\big(\mathbf{x}^{(i)}, y^{(i)}\big)$ for $i = 1, \ldots, 100$.

Sometimes it is useful to assign sample-weights $q^{(i)}$ to the data points and learn $\widehat{\mathbf{w}}$. These sample-weights reflect varying levels of importance or relevance of different data points. For simplicity we use the sample weights $q^{(i)} = 2\alpha \in [0, 1]$ for $i = 1, \ldots, 50$ and $q^{(i)} = 2(1 - \alpha)$ for $i = 51, \ldots, 100$. Can you find a closed-form expression (similar to (4.14)) for the weights $\widehat{\mathbf{w}}^{(\alpha)}$ that minimize the weighted average squared error loss $f(\mathbf{w}) := (1/50) \sum_{i=1}^{50} \alpha\big(y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)}\big)^2 + (1/50) \sum_{i=51}^{100} (1 - \alpha)\big(y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)}\big)^2$ for different α?

Exercise 4.9. Co-Linear Features. Provide a rigorous proof of the fact that the $\mathbf{X}$ (see (4.6)) of a dataset cannot have full column rank if the data points have two co-linear features.

Chapter 5

Gradient-Based Learning

This chapter discusses an important family of optimization methods for solving ERM (4.4) with a parametrized hypothesis space (see Chapter 4.2). The common theme of these methods is to construct local approximations of the objective function in (4.4). These local approximations are obtained from the gradients of the objective function. Gradient-based methods have gained popularity recently as an efficient technique for tuning the parameters of deep nets within deep learning methods [48].

Section 5.1 discusses GD as the most basic form of gradient-based methods. The idea of GD is to update the weights by locally optimizing a linear approximation of the objective function. This update is referred to as a GD step and provides the main algorithmic primitive of gradient-based methods. One key challenge for a good use of gradient-based methods is the appropriate extend of the local approximations. This extent is controlled by a step size parameter that is used in the basic GD step. Section 5.2 discusses some approaches for choosing this step size. Section 5.3 discusses a second main challenge in using gradient-based methods which is to decide when to stop repeating the GD steps.

Section 5.4 and Section 5.5 spell out GD for two instances of ERM arising from linear regression and logistic regression, respectively. The beneficial effect of data normalization on the convergence speed of gradient-based methods is briefly discussed in Section 5.6. As explained in Section 5.7, the use of stochastic approximations enables gradient-based methods for applications involving massive amounts of data ("big data"). Section 5.8 develops intuition for advanced gradient-based methods that exploit the information gathered during previous iterations.

5.1 The Basic Gradient Step

Let us rewrite ERM (4.4) as the optimization problem

$$\min_{\mathbf{w}\in\mathbb{R}^n} f(\mathbf{w}) := (1/m)\sum_{i=1}^{m} L\left((\mathbf{x}^{(i)}, y^{(i)}), h^{(\mathbf{w})}\right). \tag{5.1}$$

From now on we tacitly assume that each individual loss

$$f_i(\mathbf{w}) := L\left((\mathbf{x}^{(i)}, y^{(i)}), h^{(\mathbf{w})}\right) \tag{5.2}$$

arising in (5.1) represents a differentiable function of the parameter vector $\mathbf{w}$. Trivially, differentiability of the components (5.2) implies differentiability of the overall objective function $f(\mathbf{w})$ (5.1).

Two important examples of ERM involving such differentiable loss functions are linear regression and logistic regression. In contrast, the hinge loss (2.11) used by the SVM results in a non-differentiable objective function $f(\mathbf{w})$ (5.1). However, it is possible to (significantly) extend the scope of gradient-based methods to non-differentiable functions by replacing the concept of a gradient with that of a subgradient.

Gradient based methods are iterative. They construct a sequence of parameter vectors $\mathbf{w}^{(0)} \rightarrow \mathbf{w}^{(1)} \ldots$ that hopefully converge to a minimizer $\overline{\mathbf{w}}$ of $f(\mathbf{w})$,

$$f(\overline{\mathbf{w}}) = \bar{f} := \min_{\mathbf{w}\in\mathbb{R}^n} f(\mathbf{w}). \tag{5.3}$$

Note that there might be several different optimal parameter vectors $\overline{\mathbf{w}}$ that satisfy the optimality condition (5.3). We want the sequence generated by a gradient based method to converge towards any of them. The vectors $\mathbf{w}^{(r)}$ are (hopefully) increasingly, with increasing iteration r, more accurate approximation for a minimizer $\overline{\mathbf{w}}$ of (5.3).

Since the objective function $f(\mathbf{w})$ is differentiable, we can approximate it locally around the vector $\mathbf{w}^{(r)}$ using a tangent hyperplane that passes through the point $\left(\mathbf{w}^{(r)}, f\left(\mathbf{w}^{(r)}\right)\right) \in \mathbb{R}^{n+1}$. The normal vector of this hyperplane is given by $\mathbf{n} = (\nabla f\left(\mathbf{w}^{(r)}\right), -1)$ (see Figure 5.1). The first component of the normal vector is the gradient $\nabla f(\mathbf{w})$ of the objective function $f(\mathbf{w})$ evaluated at the vector $\mathbf{w}^{(r)}$. A main use of the gradient $\nabla f\left(\mathbf{w}^{(r)}\right)$ is to construct the linear approximation [119]

$$f(\mathbf{w}) \approx f\left(\mathbf{w}^{(r)}\right) + \left(\mathbf{w} - \mathbf{w}^{(r)}\right)^T \nabla f\left(\mathbf{w}^{(r)}\right) \text{ for } \mathbf{w} \text{ sufficiently close to } \mathbf{w}^{(r)}. \tag{5.4}$$

Requiring the objective function $f(\mathbf{w})$ in (5.4) to be differentiable is the same as requiring the validity of the local linear approximation (5.4) at every possible vector $\mathbf{w}^{(r)}$. It turns out that differentiability alone is not very helpful for the design and analysis of gradient based methods.

Gradient based methods are most useful for finding the minimum of differentiable functions $f(\mathbf{w})$ that are also smooth. Informally, a differentiable function $f(\mathbf{w})$ is smooth if the gradient $\nabla f(\mathbf{w})$ does not change too rapidly as a function of the argument $\mathbf{w}$. A quantitative version of the smoothness concept refers to a function as β-smooth if its gradient is Lipschitz continuous with Lipschitz constant $\beta > 0$ [16, Sec. 3.2],

$$\|\nabla f(\mathbf{w}) - \nabla f(\mathbf{w}')\| \leq \beta \|\mathbf{w} - \mathbf{w}'\|. \tag{5.5}$$

Note that if a function $f(\mathbf{w})$ is β smooth, it is also β' smooth for any $\beta' > \beta$. The smallest β such that (5.5) is satisfied depends on the features and labels of data points used in (5.1) as well as on the choice for the loss function.

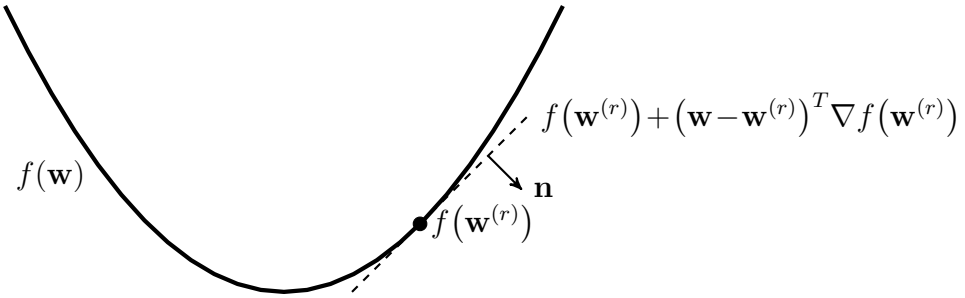


Figure 5.1: A differentiable function $f(\mathbf{w})$ can be approximated locally around a point $\mathbf{w}^{(r)}$ using a hyperplane whose normal vector $\mathbf{n} = (\nabla f\big(\mathbf{w}^{(r)}\big), -1)$ is determined by the gradient $\nabla f\big(\mathbf{w}^{(r)}\big)$ [119].

Consider a current guess or approximation $\mathbf{w}^{(r)}$ for the optimal parameter vector $\overline{\mathbf{w}}$ (5.3). We would like to find a new (better) parameter vector $\mathbf{w}^{(r+1)}$ that has smaller objective value $f(\mathbf{w}^{(r+1)}) < f\big(\mathbf{w}^{(r)}\big)$ than the current guess $\mathbf{w}^{(r)}$. The approximation (5.4) suggests to choose the next guess $\mathbf{w} = \mathbf{w}^{(r+1)}$ such that $\big(\mathbf{w}^{(r+1)} - \mathbf{w}^{(r)}\big)^T \nabla f\big(\mathbf{w}^{(r)}\big)$ is negative. We can achieve this by the GD step

$$\mathbf{w}^{(r+1)} = \mathbf{w}^{(r)} - \alpha \nabla f(\mathbf{w}^{(r)}) \tag{5.6}$$

with a sufficiently small step size $\alpha > 0$. Figure 5.2 illustrates the GD step (5.6) which is the elementary computation of gradient based methods.

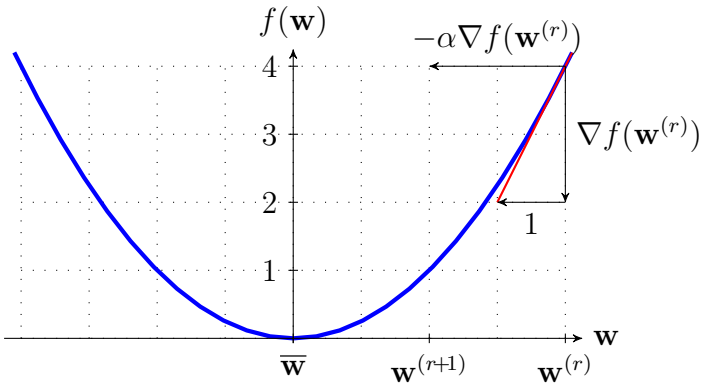


Figure 5.2: A GD step (5.6) updates a current guess or approximation $\mathbf{w}^{(r)}$ for the optimal parameter vector $\overline{\mathbf{w}}$ (5.3) by adding the correction term $-\alpha\nabla f(\mathbf{w}^{(r)})$. The updated parameter vector $\mathbf{w}^{(r+1)}$ is (typically) an improved approximation of the minimizer $\overline{\mathbf{w}}$.

The step size α in (5.6) must be sufficiently small to ensure the validity of the linear approximation (5.4). In the context of ML, the GD step size parameter α is also referred to as learning rate. Indeed, the step size α determines the amount of progress during a GD step towards learning the optimal parameter vector $\overline{\mathbf{w}}$.

We need to emphasize that the interpretation of the step size α as a learning rate is only useful when the step size is sufficiently small. Indeed, when increasing the step size α in (5.6) beyond a critical value (that depends on the properties of the objective function $f(\mathbf{w})$), the iterates (5.6) move away from the optimal parameter vector $\overline{\mathbf{w}}$. Nevertheless, from now on we will consequently use the term learning rate for α.

The idea of gradient-based methods is to repeat the GD step (5.6) for a sufficient number of iterations (repetitions) to obtain a sufficiently accurate approximation of the optimal parameter vector $\overline{\mathbf{w}}$ (5.3). It turns out that this is feasible for a sufficiently small learning rate and if the objective function is convex and smooth. Section 5.2 discusses precise conditions on the learning rate such that the iterates produced by the GD step converge to the optimum parameter vector, i.e., $\lim_{r\to\infty} f(\mathbf{w}^{(r)}) = f\big(\overline{\mathbf{w}}\big)$.

To implement the GD step (5.6) we need to choose a useful learning rate α. Moreover, executing the GD step (5.6) requires to compute the gradient $\nabla f(\mathbf{w}^{(r)})$. Both tasks can be computationally challenging as discussed in Section 5.2 and 5.7. For the objective function (5.1) obtained in linear regression and logistic regression, we can obtain closed-form expressions for the gradient $\nabla f(\mathbf{w})$ (see Sections 5.4 and 5.5).

In general, we do not have closed-form expressions for the gradient of the objective func-

tion (5.1) arising from a non-linear hypothesis space. One example for such a hypothesis space is obtained from a ANN, which is used by deep learning methods (see Section 3.11). The empirical success of deep learning methods might be partially attributed to the availability of an efficient algorithm for computing the gradient $\nabla f(\mathbf{w}^{(r)})$. This algorithm is known as back-propagation [48].

5.2 Choosing the Learning Rate

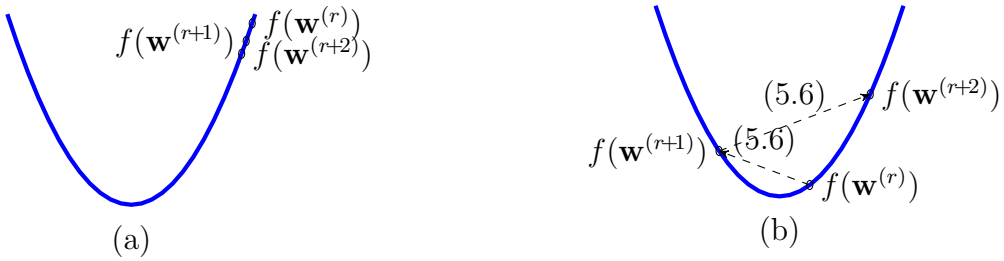


Figure 5.3: Effect of choosing bad values for the learning rate α in the GD step(5.6). (a) If the learning rate α in the GD step (5.6) is chosen too small, the iterations make very little progress towards the optimum or even fail to reach the optimum at all. (b) If the learning rate α is chosen too large, the iterates $\mathbf{w}^{(r)}$ might not converge at all (it might happen that $f(\mathbf{w}^{(r+1)}) > f(\mathbf{w}^{(r)})$!).

The choice of the learning rate α in the GD step (5.6) has a significant impact on the performance of the resulting ML methods. If we choose the learning rate α too large, the GD steps (5.6) diverge (see Figure 5.3-(b)) and, in turn, fail to converge towards the optimal weights $\overline{\mathbf{w}}$.

If we choose the learning rate α too small (see Figure 5.3-(a)), the updates (5.6) make only very little progress towards approximating the optimal parameter vector $\overline{\mathbf{w}}$. In applications that require real-time processing of data streams, it might be possible to repeat the GD steps only for a moderate number. If the learning rate is chosen too small, the updates (5.6) will fail to arrive at a good approximation of $\overline{\mathbf{w}}$ within an acceptable number of iterations.

Finding a (nearly) optimal choice for the learning rate α of GD can be a challenging task. Many sophisticated approaches for tuning the learning rate of gradient-based methods have been proposed [48, Chapter 8]. A detailed discussion of these approaches is beyond the scope of this book. We will instead discuss two sufficient conditions on the learning rate which

guarantee the convergence of the GD iterations to the optimum of a convex and smooth objective function (5.1).

The first condition applies to an objective function that is β-smooth (see (5.5)) with known constant β (not necessarily the smallest constant such that (5.5) holds). Then, the iterates $\mathbf{w}^{(r)}$ generated by the GD step (5.6) with a learning rate

$$\alpha < 2/\beta, \tag{5.7}$$

satisfy [102, Thm. 2.1.13]

$$f\big(\mathbf{w}^{(r)}\big) - \bar{f} \leq \frac{2(f\big(\mathbf{w}^{(0)}\big) - \bar{f}) \left\|\mathbf{w}^{(0)} - \overline{\mathbf{w}}\right\|_2^2}{2 \left\|\mathbf{w}^{(0)} - \overline{\mathbf{w}}\right\|_2^2 + r(f\big(\mathbf{w}^{(0)}\big) - \bar{f})\alpha(2 - \beta\alpha)}. \tag{5.8}$$

The bound (5.8) not only tells us that GD iterates converge to an optimal parameter vector but also characterize the convergence speed or rate. The sub-optimality $f\big(\mathbf{w}^{(r)}\big) - \min_{\mathbf{w}} f(\mathbf{w})$ in terms of objective function value decreases inversely (like "$1/r$") with the number r of GD steps (5.6). Convergence bounds like (5.8) can be used to specify a stopping criterion, i.e., to determine the number of GD steps to be computed (see Section 5.3).

The condition (5.7) and the bound (5.8) is only useful if we can verify β smoothness (5.5) assumption for a reasonable constant β. Verifying (5.8) only for a very large β results in the bound (5.8) being too loose (pessimistic). When we use a loose bound (5.8) to determine the number of GD steps, we might compute an unnecessary large number of GD steps (5.6).

One elegant approach to verify if a differentiable function $f(\mathbf{w})$ is β smooth (5.5) is via the Hessian matrix $\nabla^2 f(\mathbf{w}) \in \mathbb{R}^{n \times n}$ if it exists. The entries of this Hessian matrix are the second-order partial derivatives $\frac{\partial f(\mathbf{w})}{\partial w_j \partial w_{j'}}$ of the function $f(\mathbf{w})$.

Consider an objective function $f(\mathbf{w})$ (5.1) that is convex and twice-differentiable with psd Hessian $\nabla^2 f(\mathbf{w})$. If the maximum eigenvalue $\lambda_{\max}\big(\nabla^2 f(\mathbf{w})\big)$ of the Hessian is upper bounded uniformly (for all $\mathbf{w}$) by the constant $\beta > 0$, then $f(\mathbf{w})$ is β smooth (5.5) [16]. This implies, in turn via (5.7), the sufficient condition

$$\alpha < \frac{2}{\lambda_{\max}\big(\nabla^2 f(\mathbf{w})\big)} \text{ for all } \mathbf{w} \in \mathbb{R}^n \tag{5.9}$$

for the GD learning rate such that the GD steps converge to the minimum of the objective function $f(\mathbf{w})$.

It is important to note that the condition (5.9) guarantees convergence of the GD steps

for any possible initialization $\mathbf{w}^{(0)}$. Note that the usefulness of the condition (5.9) depends on the difficulty of computing the Hessian matrix $\nabla^2 f(\mathbf{w})$. Section 5.4 and Section 5.5 will present closed-form expressions for the Hessian of the objective function (5.1) obtained for linear regression and logistic regression. These closed-form expressions involve the feature vectors and labels of the data points in the training set $\mathcal{D} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}$ used in (5.1).

While it might be computationally challenging to determine the maximum (in absolute value) eigenvalue $\lambda_{\max}\big(\nabla^2 f(\mathbf{w})\big)$ for arbitrary $\mathbf{w}$, it might still be feasible to find an upper bound U for it. If we know such an upper bound $U \geq \lambda_{\max}\big(\nabla^2 f(\mathbf{w})\big)$ (valid for all $\mathbf{w} \in \mathbb{R}^n$), the learning rate $\alpha = 1/U$ still ensures convergence of the GD steps (5.6).

Up to know we have assumed a fixed (constant) learning rate α that is used for each repetition of the GD steps (5.6). However, it might be useful to vary or adjust the learning rate as the GD steps (5.6) proceed. Thus, we might use a different learning rate α_r for each iteration r of (5.6). Such a varying learning rate is useful for a variant of GD that uses stochastic approximation (see Section 5.7). However, we might use a varying learning rate also to avoid the burden of verifying β smoothness (5.5) with a tight (small) β. The GD steps (5.6) with the learning rate $\alpha_r := 1/r$ converge to the optimal parameter vector $\overline{\mathbf{w}}$ as long as we can ensure a bounded gradient $\|\nabla f(\mathbf{w})\| \leq U$ for a sufficiently large neighbourhood of $\overline{\mathbf{w}}$ [102].

5.3 When To Stop?

One main challenge in the successful application of GD is to decide when to stop iterating (or repeating) the GD step (5.6). Maybe the most simple approach is to monitor the decrease in the objective function $f(\mathbf{w}^{(r)})$ and to stop if the decrease $f(\mathbf{w}^{(r-1)}) - f(\mathbf{w}^{(r)})$ falls below a threshold. However, the ultimate goal of a ML method is not to minimize the objective function $f(\mathbf{w})$ in (5.1). Indeed, the objective function represents the average loss of a hypothesis $h^{(\mathbf{w})}$ incurred on a training set. However, the ultimate goal of a ML method is to learn a parameter vector $\mathbf{w}$ such that the resulting hypothesis accurately predicts any data point, including those outside the training set.

We will see in Chapter 6 how to use validation techniques to probe a hypothesis outside the training set. These validation techniques provide a validation error $\tilde{f}(\mathbf{w})$ that estimates the average loss of a hypothesis with parameter vector $\mathbf{w}$. Early stopping techniques monitor the validation error $\tilde{f}(\mathbf{w}^{(r)})$ as the GD iterations r proceed to decide when to stop iterating.

Another possible stopping criterion is to use a fixed number of iterations or GD steps. This fixed number of iterations can be chosen based on convergence bounds such as (5.8) in order to guarantee a prescribed sub-optimality of the final iterate $\mathbf{w}^{(r)}$. A slightly more convenient convergence bound can be obtained from (5.8) when using the the learning rate $\alpha = 1/\beta$ in the GD step (5.6) [16],

$$f\big(\mathbf{w}^{(r)}\big) - \bar{f} \leq \frac{2\beta \left\|\mathbf{w}^{(0)} - \overline{\mathbf{w}}\right\|_2^2}{r} \text{ for } r = 1, 2, \ldots. \tag{5.10}$$

5.4 GD for Linear Regression

We now present a GD method for linear regression (see Section 3.1). This method learns the parameter vector for a linear hypothesis

$$h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T \mathbf{x}. \tag{5.11}$$

The ERM principle from Section 4.3 tells us to choose the parameter vector $\mathbf{w}$ in (5.11) by minimizing the average squared error loss (2.8)

$$\widehat{L}(h^{(\mathbf{w})}|\mathcal{D}) \stackrel{(4.4)}{=} (1/m) \sum_{i=1}^{m} (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2. \tag{5.12}$$

The average squared error loss (5.12) is computed by applying the predictor $h^{(\mathbf{w})}(\mathbf{x})$ to labeled data points in a training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$. An optimal parameter vector $\overline{\mathbf{w}}$ for (5.11) is obtained as

$$\overline{\mathbf{w}} = \operatorname*{argmin}_{\mathbf{w} \in \mathbb{R}^n} f(\mathbf{w}) \text{ with } f(\mathbf{w}) = (1/m) \sum_{i=1}^{m} \big(y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)}\big)^2. \tag{5.13}$$

The objective function $f(\mathbf{w})$ in (5.13) is convex and smooth. We can therefore use GD (5.6) to solve (5.13) iteratively, i.e., by constructing a sequence of parameter vectors that converge to an optimal parameter vector $\overline{\mathbf{w}}$. To implement GD, we need to compute the gradient $\nabla f(\mathbf{w})$.

The gradient of the objective function in (5.13) is given by

$$\nabla f(\mathbf{w}) = -(2/m) \sum_{i=1}^{m} \big(y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)}\big) \mathbf{x}^{(i)}. \tag{5.14}$$

By inserting (5.14) into the basic GD iteration (5.6), we obtain Algorithm 1.

Algorithm 1 Linear regression via GD

Input: dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$; learning rate $\alpha > 0$.
Initialize: set $\mathbf{w}^{(0)} := \mathbf{0}$; set iteration counter $r := 0$
1: **repeat**
2: $r := r + 1$ (increase iteration counter)
3: $\mathbf{w}^{(r)} := \mathbf{w}^{(r-1)} + \alpha(2/m) \sum_{i=1}^{m} \big(y^{(i)} - \big(\mathbf{w}^{(r-1)}\big)^T \mathbf{x}^{(i)}\big)\mathbf{x}^{(i)}$ (do a GD step (5.6))
4: **until** stopping criterion met
Output: $\mathbf{w}^{(r)}$ (which approximates $\overline{\mathbf{w}}$ in (5.13))

Let us have a closer look on the update in step 3 of Algorithm 1, which is

$$\mathbf{w}^{(r)} := \mathbf{w}^{(r-1)} + \alpha(2/m) \sum_{i=1}^{m} \big(y^{(i)} - \big(\mathbf{w}^{(r-1)})^T \mathbf{x}^{(i)}\big)\mathbf{x}^{(i)}. \tag{5.15}$$

The update (5.15) has an appealing form as it amounts to correcting the previous guess (or approximation) $\mathbf{w}^{(r-1)}$ for the optimal parameter vector $\overline{\mathbf{w}}$ by the correction term

$$(2\alpha/m) \sum_{i=1}^{m} \underbrace{\big(y^{(i)} - \big(\mathbf{w}^{(r-1)})^T \mathbf{x}^{(i)}\big)}_{e^{(i)}} \mathbf{x}^{(i)}. \tag{5.16}$$

The correction term (5.16) is a weighted average of the feature vectors $\mathbf{x}^{(i)}$ using weights $(2\alpha/m) \cdot e^{(i)}$. These weights consist of the global factor $(2\alpha/m)$ (that applies equally to all feature vectors $\mathbf{x}^{(i)}$) and a sample-specific factor $e^{(i)} = (y^{(i)} - \big(\mathbf{w}^{(r-1)})^T \mathbf{x}^{(i)})$, which is the prediction (approximation) error obtained by the linear predictor $h^{(\mathbf{w}^{(r-1)})}(\mathbf{x}^{(i)}) = \big(\mathbf{w}^{(r-1)})^T \mathbf{x}^{(i)}$ when predicting the label $y^{(i)}$ from the features $\mathbf{x}^{(i)}$.

We can interpret the GD step (5.15) as an instance of "learning by trial and error". Indeed, the GD step amounts to first "trying out" (trial) the predictor $h(\mathbf{x}^{(i)}) = \big(\mathbf{w}^{(r-1)})^T \mathbf{x}^{(i)}$. The predicted values are then used to improve the weight vector $\mathbf{w}^{(r-1)}$ based on the error $e^{(i)} = y^{(i)} - \big(\mathbf{w}^{(r-1)})^T \mathbf{x}^{(i)}$.

The choice of the learning rate α used for Algorithm 1 can be based on the condition (5.9) with the Hessian $\nabla^2 f(\mathbf{w})$ of the objective function $f(\mathbf{w})$ underlying linear regression (see (5.13)). This Hessian is given explicitly as

$$\nabla^2 f(\mathbf{w}) = (1/m)\mathbf{X}^T \mathbf{X}, \tag{5.17}$$

with the feature matrix $\mathbf{X} = \big(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\big)^T \in \mathbb{R}^{m \times n}$ (see (4.6)). Note that the Hessian (5.17) does not depend on the parameter vector $\mathbf{w}$.

Comparing (5.17) with (5.9), one particular strategy for choosing the learning rate in Algorithm 1 is to (i) compute the matrix product $\mathbf{X}^T\mathbf{X}$, (ii) compute the maximum eigenvalue $\lambda_{\max}\big((1/m)\mathbf{X}^T\mathbf{X}\big)$ of this product and (iii) set the learning rate to $\alpha = 1/\lambda_{\max}\big((1/m)\mathbf{X}^T\mathbf{X}\big)$.

While it might be computationally difficult to compute the maximum eigenvalue of the psd matrix $(1/m)\mathbf{X}^T\mathbf{X}$ exactly, we might be able to find an upper bound U such that[1]

$$\lambda_{\max}\big((1/m)\mathbf{X}^T\mathbf{X}\big) \leq U. \tag{5.18}$$

In some applications, we might have prior knowledge about the data points and their features which allows us to find a reasonable upper bound U with little computational effort. Given an upper bound U in (5.18), we can ensure convergence of the GD steps when using learning rate $\alpha = 1/U$.

Consider a dataset $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$ with normalized features, i.e., $\|\mathbf{x}^{(i)}\| = 1$ for all $i = 1, \ldots, m$. It can then be shown that $\lambda_{\max}\big((1/m)\mathbf{X}^T\mathbf{X}\big) \leq 1$, i.e., we can use the upper bound $U = 1$ in (5.18). Thus, in this case, we can ensure convergence of the iterates $\mathbf{w}^{(r)}$ (see (5.15)) by choosing the learning rate $\alpha = 1$.

Time-Data Tradeoffs. The number of GD steps required by Algorithm 1 to ensure a prescribed sub-optimality depends crucially on the condition number of $\mathbf{X}^T\mathbf{X}$. What can we say about the condition number? In general, we have not control over this quantity as the matrix $\mathbf{X}$ consists of the feature vectors of arbitrary data points. However, it is often useful to model the feature vectors as realizations of i.i.d. random vectors. It is then possible to bound the probability of the feature matrix having a sufficiently small condition number. These bounds can then be used to choose the step-size such that convergence is guaranteed with sufficiently large probability. The usefulness of these bounds typically depends on the ratio n/m. For increasing sample-size, these bounds allow to use larger step-sizes and, in turn, result in faster convergence of GD algorithm. Thus, we obtain a trade-off between the runtime of Algorithm 1 and the number of data points that we feed into it [107].

[1]The problem of computing a full eigenvalue decomposition of $\mathbf{X}^T\mathbf{X}$ has essentially the same complexity as ERM via directly solving (4.11), which we want to avoid by using the "cheaper" GD Algorithm 1.

5.5 GD for Logistic Regression

Logistic regression learns a linear hypothesis $h^{(\mathbf{w})}$ that is used to classify data points by predicting their binary label. The quality of such a linear classifier is measured by the logistic loss (2.12). The ERM principle suggest to learn the parameter vector $\mathbf{w}$ by minimizing the average logistic loss (3.16) obtained for a training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$. The training set consists of data points with features $\mathbf{x}^{(i)} \in \mathbb{R}^n$ and binary labels $y^{(i)} \in \{-1, 1\}$.

We can rewrite ERM for logistic regression as the optimization problem

$$\overline{\mathbf{w}} = \operatorname*{argmin}_{\mathbf{w} \in \mathbb{R}^n} f(\mathbf{w})$$
$$\text{with } f(\mathbf{w}) = (1/m) \sum_{i=1}^{m} \log \big(1+\exp\big(-y^{(i)} \mathbf{w}^T \mathbf{x}^{(i)}\big)\big). \tag{5.19}$$

The objective function $f(\mathbf{w})$ is differentiable and therefore we can use GD (5.6) to solve (5.19). We can write down the gradient of the objective function in (5.19) in closed-form as

$$\nabla f(\mathbf{w}) = (1/m) \sum_{i=1}^{m} \frac{-y^{(i)}}{1+\exp(y^{(i)} \mathbf{w}^T \mathbf{x}^{(i)})} \mathbf{x}^{(i)}. \tag{5.20}$$

Inserting (5.20) into the GD step (5.6) yields Algorithm 2.

Algorithm 2 Logistic regression via GD

Input: labeled dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$ containing feature vectors $\mathbf{x}^{(i)} \in \mathbb{R}^n$ and labels $y^{(i)} \in \mathbb{R}$; GD learning rate $\alpha > 0$.

Initialize: set $\mathbf{w}^{(0)} := \mathbf{0}$; set iteration counter $r := 0$

1: **repeat**

2: $r := r+1$ (increase iteration counter)

3: $\mathbf{w}^{(r)} := \mathbf{w}^{(r-1)} + \alpha(1/m) \sum_{i=1}^{m} \frac{y^{(i)}}{1+\exp\left(y^{(i)} \left(\mathbf{w}^{(r-1)}\right)^T \mathbf{x}^{(i)}\right)} \mathbf{x}^{(i)}$ (do a GD step (5.6))

4: **until** stopping criterion met

Output: $\mathbf{w}^{(r)}$, which approximates a solution $\overline{\mathbf{w}}$ of (5.19))

Let us have a closer look on the update in step (3) of Algorithm 2. This step amounts to computing

$$\mathbf{w}^{(r)} := \mathbf{w}^{(r-1)} + \alpha(1/m) \sum_{i=1}^{m} \frac{y^{(i)}}{1+\exp\left(y^{(i)} \left(\mathbf{w}^{(r-1)}\right)^T \mathbf{x}^{(i)}\right)} \mathbf{x}^{(i)}. \tag{5.21}$$

Similar to the GD step (5.15) for linear regression, also the GD step (5.21) for logistic regression can be interpreted as an implementation of the trial-and-error principle. Indeed, (5.21) corrects the previous guess (or approximation) $\mathbf{w}^{(r-1)}$ for the optimal parameter vector $\overline{\mathbf{w}}$ by the correction term

$$(\alpha/m)\sum_{i=1}^{m}\underbrace{\frac{y^{(i)}}{1+\exp(y^{(i)}\mathbf{w}^{T}\mathbf{x}^{(i)})}}_{e^{(i)}}\mathbf{x}^{(i)}. \tag{5.22}$$

The correction term (5.22) is a weighted average of the feature vectors $\mathbf{x}^{(i)}$. The feature vector $\mathbf{x}^{(i)}$ is weighted by the factor $(\alpha/m)\cdot e^{(i)}$. These weighting factors are a product of the global factor (α/m) that applies equally to all feature vectors $\mathbf{x}^{(i)}$. The global factor is multiplied by a data point-specific factor $e^{(i)}=\frac{y^{(i)}}{1+\exp(y^{(i)}\mathbf{w}^{T}\mathbf{x}^{(i)})}$, which quantifies the error of the classifier $h^{(\mathbf{w}^{(r-1)})}(\mathbf{x}^{(i)})=\big(\mathbf{w}^{(r-1)}\big)^{T}\mathbf{x}^{(i)}$ for a single data point with true label $y^{(i)}\in\{-1,1\}$ and features $\mathbf{x}^{(i)}\in\mathbb{R}^{n}$.

We can use the sufficient condition (5.9) for the convergence of GD steps to guide the choice of the learning rate α in Algorithm 2. To apply condition (5.9), we need to determine the Hessian $\nabla^{2}f(\mathbf{w})$ matrix of the objective function $f(\mathbf{w})$ underlying logistic regression (see (5.19)). Some basic calculus reveals (see [58, Ch. 4.4.])

$$\nabla^{2}f(\mathbf{w})=(1/m)\mathbf{X}^{T}\mathbf{D}\mathbf{X}. \tag{5.23}$$

Here, we used the feature matrix $\mathbf{X}=\big(\mathbf{x}^{(1)},\ldots,\mathbf{x}^{(m)}\big)^{T}\in\mathbb{R}^{m\times n}$ (see (4.6)) and the diagonal matrix $\mathbf{D}=\mathrm{diag}\{d_{1},\ldots,d_{m}\}\in\mathbb{R}^{m\times m}$ with diagonal elements

$$d_{i}=\frac{1}{1+\exp(-\mathbf{w}^{T}\mathbf{x}^{(i)})}\left(1-\frac{1}{1+\exp(-\mathbf{w}^{T}\mathbf{x}^{(i)})}\right). \tag{5.24}$$

We highlight that, in contrast to the Hessian (5.17) of the objective function arising in linear regression, the Hessian (5.23) of logistic regression varies with the parameter vector $\mathbf{w}$. This makes the analysis of Algorithm 2 and the optimal choice for the learning rate α more difficult compared to Algorithm 1. At least, we can ensure convergence of (5.21) (towards a solution of (5.19)) for the learning rate $\alpha=1$ if we normalize feature vectors such that $\|\mathbf{x}^{(i)}\|=1$. This follows from the fact the diagonal entries (5.24) take values in the interval $[0,1]$.

5.6 Data Normalization

The number of GD steps (5.6) required to reach the minimum (within a prescribed accuracy) of the objective function (4.5) depends crucially on the condition number [16, 68]

$$\kappa(\mathbf{X}^T\mathbf{X}) := \lambda_{\max}/\lambda_{\min}. \tag{5.25}$$

Here, we use the largest and smallest eigenvalue of the matrix $\mathbf{X}^T\mathbf{X}$, denoted as $\lambda_{\max}$ and $\lambda_{\min}$, respectively. The condition number (5.25) is only well-defined if the columns of the feature matrix $\mathbf{X}$ (4.6) (which are the feature vectors $\mathbf{x}^{(i)}$), are linearly independent. In this case the condition number is lower bounded as $1 \leq \kappa(\mathbf{X}^T\mathbf{X})$.

It can be shown that the GD steps (5.6) converge faster for smaller condition number $\kappa(\mathbf{X}^T\mathbf{X})$ [68]. Thus, GD will be faster for datasets with a feature matrix $\mathbf{X}$ such that $\kappa(\mathbf{X}^T\mathbf{X}) \approx 1$. It is therefore often beneficial to pre-process the feature vectors using a normalization (or standardization) procedure as detailed in Algorithm 3.

Algorithm 3 "Data Normalization"

Input: labeled dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$

1: remove sample means $\widehat{\mathbf{x}} = (1/m)\sum_{i=1}^m \mathbf{x}^{(i)}$ from features, i.e.,

$$\mathbf{x}^{(i)} := \mathbf{x}^{(i)} - \widehat{\mathbf{x}} \text{ for } i = 1, \ldots, m$$

2: normalise features to have unit variance,

$$\hat{x}_j^{(i)} := x_j^{(i)}/\hat{\sigma} \text{ for } j = 1, \ldots, n \text{ and } i = 1, \ldots, m$$

with the empirical (sample) variance $\hat{\sigma}_j^2 = (1/m)\sum_{i=1}^m \left(x_j^{(i)}\right)^2$

Output: normalized feature vectors $\{\hat{\mathbf{x}}^{(i)}\}_{i=1}^m$

Algorithm 3 transforms the original feature vectors $\mathbf{x}^{(i)}$ into new feature vectors $\widehat{\mathbf{x}}^{(i)}$ such that the new feature matrix $\widehat{\mathbf{X}} = (\widehat{\mathbf{x}}^{(1)}, \ldots, \widehat{\mathbf{x}}^{(m)})^T$ is better conditioned than the original feature matrix, i.e., $\kappa(\widehat{\mathbf{X}}^T\widehat{\mathbf{X}}) < \kappa(\mathbf{X}^T\mathbf{X})$.

5.7 Stochastic GD

Consider the GD steps (5.6) for minimizing the empirical risk (5.1). The gradient $\nabla f(\mathbf{w})$ of the objective function (5.1) has a particular structure. Indeed, this gradient is a sum

$$\nabla f(\mathbf{w}) = (1/m) \sum_{i=1}^{m} \nabla f_i(\mathbf{w}) \text{ with } f_i(\mathbf{w}) := L\left((\mathbf{x}^{(i)}, y^{(i)}), h^{(\mathbf{w})}\right). \tag{5.26}$$

Each component of the sum (5.26) corresponds to one particular data points $(\mathbf{x}^{(i)}, y^{(i)})$, for $i = 1, \ldots, m$. We need to compute a sum of the form (5.26) for each new GD step (5.6).

Computing the sum in (5.26) can be computationally challenging for at least two reasons. First, computing the sum is challenging for very large datasets with m in the order of billions. Second, for datasets which are stored in different data centres located all over the world, the summation would require a huge amount of network resources. Moreover, the finite transmission rate of communication networks limits the rate by which the GD steps (5.6) can be executed.

The idea of stochastic gradient descent (SGD) is to replace the exact gradient $\nabla f(\mathbf{w})$ (5.26) by an approximation that is easier to compute than a direct evaluation of (5.26). The word "stochastic" in the name SGD hints already at the use of a stochastic approximation $g(\mathbf{w}) \approx \nabla f(\mathbf{w})$. It turns out that using a gradient approximation $g(\mathbf{w})$ can result in significant savings in computational complexity while incurring a graceful degradation in the overall optimization accuracy. The optimization accuracy (distance to minimum of $f(\mathbf{w})$) depends crucially on the "gradient noise"

$$\varepsilon := \nabla f(\mathbf{w}) - g(\mathbf{w}). \tag{5.27}$$

The elementary step of most SGD methods is obtained from the GD step (5.6) by replacing the exact gradient $\nabla f(\mathbf{w})$ with some stochastic approximation $g(\mathbf{w})$,

$$\mathbf{w}^{(r+1)} = \mathbf{w}^{(r)} - \alpha_r g\big(\mathbf{w}^{(r)}\big), \tag{5.28}$$

As the notation in (5.28) indicates, SGD methods use a learning rate α_r that varies between different iterations.

To avoid accumulation of the gradient noise (5.27) during the SGD updates (5.28), SGD methods typically decrease the learning rate α_r as the iterations proceed. The precise dependence of the learning rate α_r on the iteration index r is referred to as a learning rate

schedule [48, Chapter 8]. One possible choice for the learning rate schedule is $\alpha_r := 1/r$ [101]. Exercise 5.3 discusses conditions on the learning rate schedule that guarantee convergence of the updates SGD to the minimum of $f(\mathbf{w})$.

The approximate ("noisy") gradient $g(\mathbf{w})$ can be obtained by different randomization strategies. The most basic form of SGD constructs the gradient approximation $g(\mathbf{w})$ by replacing the sum (5.26) with a randomly selected component,

$$g(\mathbf{w}) := \nabla f_{\hat{i}}(\mathbf{w}). \tag{5.29}$$

The index $\hat{i}$ is chosen randomly from the set $\{1, \ldots, m\}$. The resulting SGD method then repeats the update

$$\mathbf{w}^{(r+1)} = \mathbf{w}^{(r)} - \alpha \nabla f_{\hat{i}_r}(\mathbf{w}^{(r)}), \tag{5.30}$$

sufficiently often. Every update (5.30) uses a "fresh" randomly chosen (drawn) index $\hat{i}_r$. Formally, the indices $\hat{i}_r$ are realizations of i.i.d. RVs whose common probability distribution is the uniform distribution over the index set $\{1, \ldots, m\}$.

Note that (5.30) replaces the summation over the training set during the GD step (5.6) by randomly selecting a single component of this sum. The resulting savings in computational complexity can be significant when the training set consists of a large number of data points that might be stored in a distributed fashion (in the "cloud"). The saving in computational complexity of SGD comes at the cost of introducing a non-zero gradient noise

$$\begin{aligned} \varepsilon_r &\overset{(5.27)}{=} \nabla f(\mathbf{w}^{(r)}) - g\big(\mathbf{w}^{(r)}\big) \\ &= \nabla f(\mathbf{w}^{(r)}) - \nabla f_{\hat{i}_r}(\mathbf{w}). \end{aligned} \tag{5.31}$$

Mini-Batch SGD. Let us now discuss a variant of SGD that tries to reduce the approximation error (gradient noise) (5.31) arising in the SGD step (5.30). The idea behind this variant, referred to as mini-batch SGD, is quite simple. Instead of using only a single randomly selected component $\nabla f_i(\mathbf{w})$ (see (5.26)) for constructing a gradient approximation, mini-batch SGD uses several randomly chosen components.

We summarize mini-batch SGD in Algorithm 4 which requires an integer batch size B as input parameter. Algorithm 4 repeats the SGD step (5.28) using a gradient approximation that is constructed from a randomly selected subset $\mathcal{B} = \{i_1, \ldots, i_B\}$ (a "batch"),

$$g\big(\mathbf{w}\big) = (1/B) \sum_{i' \in \mathcal{B}} \nabla f_{i'}(\mathbf{w}). \tag{5.32}$$

For each new iteration of Algorithm 4, a new batch $\mathcal{B}$ is generated by a random generator. Note that Algorithm 4 includes the basic SGD variant (5.30) as a special case for the batch

Algorithm 4 Mini-Batch SGD

Input: components $f_i(\mathbf{w})$, for $i = 1, \ldots, m$ of objective function $f(\mathbf{w}) = \sum_{i=1}^{m} f_i(\mathbf{w})$; batch size B; learning rate schedule $\alpha_r > 0$.

Initialize: set $\mathbf{w}^{(0)} := \mathbf{0}$; set iteration counter $r := 0$

1: **repeat**
2: randomly select a batch $\mathcal{B} = \{i_1, \ldots, i_B\} \subseteq \{1, \ldots, m\}$ of indices that select a subset of components f_i
3: compute an approximate gradient $g\big(\mathbf{w}^{(r)}\big)$ using (5.32)
4: $r := r + 1$ (increase iteration counter)
5: $\mathbf{w}^{(r)} := \mathbf{w}^{(r-1)} - \alpha_r g\big(\mathbf{w}^{(r-1)}\big)$
6: **until** stopping criterion met

Output: $\mathbf{w}^{(r)}$ (which approximates $\operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^n} f(\mathbf{w})$))

size $B = 1$. Another special case is $B = m$, where the SGD step 5 in Algorithm 4 becomes an ordinary GD step (5.6).

Online Learning. A main motivation for the SGD step (5.30) is that a training set is already collected but so large that the sum in (5.26) is computationally intractable. Another variant of SGD is obtained for sequential (time-series) data. In particular, consider data points that are gathered sequentially, one new data point $\big(\mathbf{x}^{(t)}, y^{(t)}\big)$ at each time instant $t = 1, 2, \ldots$. With each new data point $\big(\mathbf{x}^{(t)}, y^{(t)}\big)$ we can access a new component $f_t(\mathbf{w}) = L\left(\big(\mathbf{x}^{(t)}, y^{(t)}\big), h^{(\mathbf{w})}\right)$ (see (5.1)). For sequential data, we can use a slight modification of the SGD step (5.28) to obtain an online learning method (see Section 4.7). This online variant of SGD computes, at each time instant t,

$$\mathbf{w}^{(t)} := \mathbf{w}^{(t-1)} - \alpha_t \nabla f_t(\mathbf{w}^{(t-1)}). \tag{5.33}$$

5.8 Advanced Gradient-Based Methods

The main idea underlying GD and SGD is to approximate the objective function (5.1) locally around a current guess or approximation $\mathbf{w}^{(r)}$ for the optimal weights. This local approximation is a tangent hyperplane whose normal vector is determined by the gradient $\nabla f(\mathbf{w}^{(r)}$. We then obtain an updated (improved) approximation by minimizing this local approximation, i.e., by doing a GD step (5.6).

The idea of advanced gradient methods [48, Ch. 8] is to exploit the information provided

by the gradients $\nabla f\big(\mathbf{w}^{(r')}\big)$ at previous iterations $r' = 1, \ldots, r$, to build an improved local approximation of $f(\mathbf{w})$ around a current iterate $\mathbf{w}^{(r)}$. Figure 5.4 indicates such an improved local approximation of $f(\mathbf{w})$ which is non-linear (e.g., quadratic). These improved local approximations can be used to adapt the learning rate during the GD steps (5.6).

Advanced gradient-based methods use improved local approximations to modify the gradient $\nabla f\big(\mathbf{w}^{(r')}\big)$ to obtain an improved update direction. Figure 5.5 depicts the contours of an objective function $f(\mathbf{w})$ for which the gradient $\nabla f\big(\mathbf{w}^{(r)}\big)$ points only weakly towards the optimal parameter vector $\overline{\mathbf{w}}$ (minimizer of $f(\mathbf{w})$). The gradient history $\nabla f\big(\mathbf{w}^{(r')}\big)$, for$r' = 1, \ldots, r$, allows to detect such an unfavourable geometry of the objective function. Moreover, the gradient history can be used to "correct" the update direction $\nabla f\big(\mathbf{w}^{(r)}\big)$ to obtain an improved update direction towards the optimum parameter vector $\overline{\mathbf{w}}$.

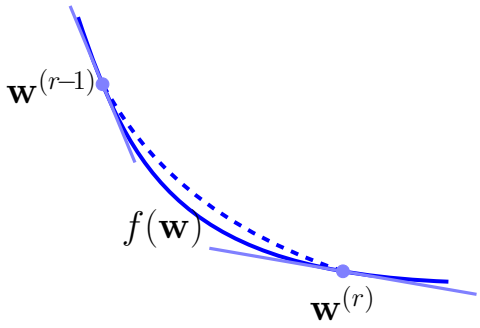


Figure 5.4: Advanced gradient-based methods use the gradients at previous iterations to construct an improved (non-linear) local approximation (dashed) of the objective function $f(\mathbf{w})$ (solid).

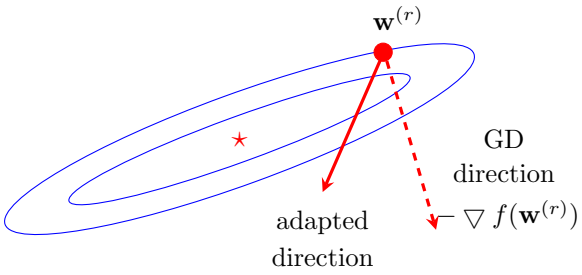


Figure 5.5: Advanced gradient-based methods use improved (non-linear) local approximations of the objective function $f(\mathbf{w})$ to "nudge" the update direction towards the optimal parameter vector $\overline{\mathbf{w}}$. The update direction of plain vanilla GD (5.6) is the negative gradient $-\nabla f\big(\mathbf{w}^{(r)}\big)$. For some objective functions the negative gradient might be only weakly correlated with the straight direction from $\mathbf{w}^{(r)}$ towards the optimal parameter vector ($\star$).

5.9 Exercises

Exercise 5.1. Use Knowledge About Problem Class. Consider the space $\mathcal{P}$ of sequences $f = (f[0], f[1], \ldots)$ that have the following properties

- for each sequence there is an index $i^{(f)}$ such that f is monotone increasing for indices $i' \geq i^{(f)}$ and monotone decreasing for indices $i' \geq i^{(f)}$
- any change point r of f, where $f[r] \neq f[r+1]$ can only occur at integer multiples of 100, e.g., $r = 100$ or $r = 300$.

Given a function $f \in \mathcal{P}$ and starting point r_0 our goal is to find the minimum value of $\min_r f[r] = f[r^{(f)}]$ as quickly as possible. Can you construct an iterative algorithm that can access a given function f only by querying the values $f[r], f[r-1]$ and $f[r+1]$ for any given index r.

Exercise 5.2. Maximum Eigenvalue of Flipped Product. Show that $\lambda_{\max}\big(\mathbf{A}\mathbf{A}^T\big) = \lambda_{\max}\big(\mathbf{A}^T\mathbf{A}\big)$ for any given matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$.

Exercise 5.3. Learning rate Schedule for SGD Let us learn a linear hypothesis $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ using data points that arrive sequentially at discrete time instants $t = 0, 1, \ldots$. At time t, we gather a new data point $\big(\mathbf{x}^{(r)}, y^{(r)}\big)$. The data points can be modelled as realizations of i.i.d. copies of a RV $\big(\mathbf{x}, y\big)$. The probability distribution of the features $\mathbf{x}$ is

a standard multivariate normal distribution $\mathcal{N}(\mathbf{0}, \mathbf{I})$. The label of a random[2] data point is related to its features via $y = \overline{\mathbf{w}}^T \mathbf{x} + \varepsilon$ with some fixed but unknown true parameter vector $\overline{\mathbf{w}}$. The additive noise $\varepsilon \sim \mathcal{N}(0, 1)$ follows a standard normal distribution. We use SGD to learn the parameter vector $\mathbf{w}$ of a linear hypothesis,

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha_t \big(\big(\mathbf{w}^{(t)}\big)^T \mathbf{x}^{(t)} - y^{(t)} \big) \mathbf{x}^{(t)}. \tag{5.34}$$

with learning rate schedule $\alpha_t = \beta / t^\gamma$. Note that we compute a new SGD iteration (5.34) for each new time instant t. What conditions on the hyper-parameters β, γ ensure that $\lim_{t \to \infty} \mathbf{w}^{(t)} = \overline{\mathbf{w}}$ in distribution?

Exercise 5.4. ImageNet. The "ImageNet" database contains more than 10^6 images [80]. These images are labeled according to their content (e.g., does the image show a dog?) and stored as a file of size at least 4 kilobytes. We want to learn a classifier that allows to predict if an image shows a dog or not. To learn this classifier we run GD for logistic regression on a small computer that has 32 kilobytes memory and is connected to the internet with bandwidth of 1 Mbit/s. Therefore, for each single GD update (5.6) it must essentially download all images in ImageNet. How long would such a single GD update take ?

Exercise 5.5. Apple or No Apple? Consider data points being images of size of 1024×1024 pixels. Each image is characterized by the RGB pixel color intensities (value range $0, \ldots, 255$ resuling in 24 bits for each pixel), which we stack into a feature vector $\mathbf{x} \in \mathbb{R}^n$. We assign each image the label $y = 1$ if it shows an apple and $y = -1$ if it does not show an apple. We use logistic regression to learn a linear hypothesis $h(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$ to classify an image according to $\hat{y} = 1$ if $h(\mathbf{x}) \geq 0$. The training set consists of $m = 10^{10}$ labeled images which are stored in the cloud. We implement the ML method on our own laptop which is connected to the internet with a bandwidth of at most 100 Mbps. Unfortunately we can only store at most five images on our computer. How long does it take at least to complete one single GD step ?

Exercise 5.6. Feature Normalization To Speed Up GD Consider the dataset with feature vectors $\mathbf{x}^{(1)} = (100, 0)^T \in \mathbb{R}^2$ and $\mathbf{x}^{(2)} = (0, 1/10)^T \in \mathbb{R}^2$ which we stack into the matrix $\mathbf{X} = (\mathbf{x}^{(1)}, \mathbf{x}^{(2)})^T$. What is the condition number of $\mathbf{X}^T \mathbf{X}$? What is the condition number of $\big(\widehat{\mathbf{X}}\big)^T \widehat{\mathbf{X}}$ with the matrix $\widehat{\mathbf{X}} = (\widehat{\mathbf{x}}^{(1)}, \widehat{\mathbf{x}}^{(2)})^T$ constructed from the normalized feature vectors $\widehat{\mathbf{x}}^{(i)}$ delivered by Algorithm 3.

[2]More precisely, a data point that is obtained as the realization of RV.

Exercise 5.7. Convergence of GD Steps Consider a differentiable objective function $f(\mathbf{w})$ whose argument is a parameter vector $\mathbf{w} \in \mathbb{R}^n$. We make no assumption about smoothness or convexity. Thus, the function $f(\mathbf{w})$ might be non-convex and might also be not β smooth. However, the gradient $\nabla f(\mathbf{w})$ is uniformly upper bounded $\|\nabla f(\mathbf{w})\| \leq 100$ for every $\mathbf{w}$. Starting from some initial vector $\mathbf{w}^{(0)}$ we construct a sequence of parameter vectors using GD steps,

$$\mathbf{w}^{(r+1)} = \mathbf{w}^{(r)} - \alpha_r \nabla f\big(\mathbf{w}^{(r)}\big). \tag{5.35}$$

The learning rate α_r in (5.35) is allowed to vary between different iterations. Can you provide sufficient conditions on the evolution of the learning rate α_r, as iterations proceed, that ensure convergence of the sequence $\mathbf{w}^{(0)}, \mathbf{w}^{(1)}, \ldots$.

Chapter 6

Model Validation and Selection

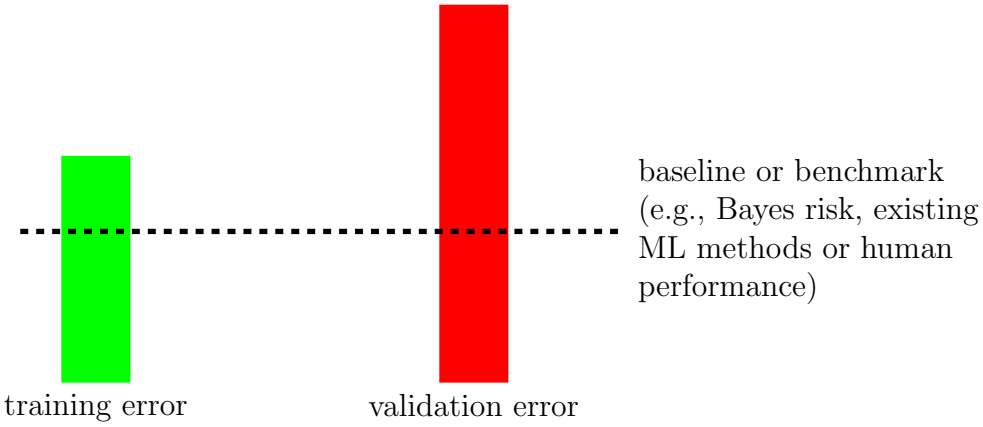


Figure 6.1: We can diagnose a ML method by comparing its training error with its validation error. Ideally both are on the same level as a baseline (or benchmark error level).

Chapter 4 discussed ERM as a principled approach to learning a useful hypothesis out of a hypothesis space or model. In particular, ERM-based ML methods learn a hypothesis $\widehat{h} \in \mathcal{H}$ that incurs minimum average loss on a set of labeled data points that serve as the training set. We refer to the average loss incurred by a hypothesis on the training set as the training error. The minimum average loss achieved by a hypothesis that solves the ERM might be referred to as the training error of the overall ML method. Note that a specific ML method consists of specific design choices for the hypothesis space (or model) and loss function (see Chapter 3).

ERM is sensible only if the training error of a hypothesis is an reliable approximation

for its (average) loss incurred on data points outside the training set. We say that a learnt hypothesis generalizes well if the loss incurred outside the training set is not significantly larger than the average loss on the training set. Whether the training error of a hypothesis is a reliable approximation for its loss on data points outside the training set depends on both, the statistical properties of the data points and on the hypothesis space used by the ML method.

Modern ML methods typically use hypothesis spaces with large effective dimension (see Section 2.2). As an example consider linear regression (see Section 3.1) with data points having a large number n of features (this setting is referred to as the high-dimensional regime). The effective dimension of the linear hypothesis space (3.1), which is used by linear regression, is equal to the number n of features. Modern technology allows to collect a huge number of features about individual data points which implies, in turn, that the effective dimension of (3.1) is large. Another example of a high-dimensional hypothesis space arises in deep learning methods using a hypothesis space are constituted by all maps represented by an ANN with billions of tunable parameters.

A high-dimensional hypothesis space is very likely to contain a hypothesis that perfectly fits any given training set. Such a hypothesis achieves a very small training error but might incur a large loss when predicting the labels of a data point that is not included in training set. Thus, the (minimum) training error achieved by a hypothesis learnt by ERM can be misleading. We say that a ML method, such as linear regression using too many features, overfits the training set when it learns a hypothesis (e.g., via ERM) that has small training error but incurs much larger loss outside the training set.

Section 6.1 shows that linear regression is likely to overfit a training set if the number of features of a data point exceeds the size of the training set. Section 6.2 demonstrates how to validate a learnt hypothesis by computing its average loss on data points outside the training set. We refer to the set of data points used to validate the learnt hypothesis as a validation set. If a ML method overfits the training set, it learns a hypothesis whose training error is much smaller than its validation error. We can detect if a ML method overfits by comparing its training error with its validation error (see Figure 6.1).

We can use the validation error not only to detect if a ML method overfits. The validation error can also be used as a quality measure for the hypothesis space or model used by the ML method. This is analogous to the concept of a loss function that allows us to evaluate the quality of a hypothesis $h \in \mathcal{H}$. Section 6.3 shows how to select between ML methods using different models by comparing their validation errors.

Section 6.4 uses a simple probabilistic model for the data to study the relation between the training error of a learnt hypothesis and its expected loss (see (4.1)). This probabilistic analysis reveals the interplay between the data, the hypothesis space and the resulting training error and validation error of a ML method.

Section 6.5 discusses the bootstrap as a simulation based alternative to the probabilistic analysis of Section 6.4. While Section 6.4 assumes a specific probability distribution of the data points, the bootstrap does not require the specification of a probability distribution underlying the data.

As indicated in Figure 6.1, for some ML applications, we might have a baseline (or benchmark) for the achievable performance of ML methods. Such a baseline might be obtained from existing ML methods, human performance levels or from a probabilistic model (see Section 6.4). Section 6.6 details how the comparison between training error, validation error and (if available) a baseline informs possible improvements for a ML method. These improvements might be obtained by collecting more data points, using more features of data points or by changing the hypothesis space (or model).

Having a baseline for the expected loss, such as the Bayes risk, allows to tell if a ML method already provides satisfactory results. If the training error and the validation error of a ML method are close to the baseline, there might be little point in trying to further improve the ML method.

6.1 Overfitting

We now have a closer look at the occurrence of overfitting in linear regression methods. As discussed in Section 3.1, linear regression methods learn a linear hypothesis $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ which is parametrized by the parameter vector $\mathbf{w} \in \mathbb{R}^n$. The learnt hypothesis is then used to predict the numeric label $y \in \mathbb{R}$ of a data point based on its feature vector $\mathbf{x} \in \mathbb{R}^n$. Linear regression aims at finding a parameter vector $\widehat{\mathbf{w}}$ with minimum average squared error loss incurred on a training set

$$\mathcal{D} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}.$$

The training set $\mathcal{D}$ consists of m data points $\big(\mathbf{x}^{(i)}, y^{(i)}\big)$, for $i = 1, \ldots, m$, with known label values $y^{(i)}$. We stack the feature vectors $\mathbf{x}^{(i)}$ and labels $y^{(i)}$, respectively, of the data points in the training set into the feature matrix $\mathbf{X} = (\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)})^T$ and label vector $\mathbf{y} = (y^{(1)}, \ldots, y^{(m)})^T$.

The ERM (4.13) of linear regression is solved by any parameter vector $\widehat{\mathbf{w}}$ that solves (4.11). The (minimum) training error of the hypothesis $h^{(\widehat{\mathbf{w}})}$ is obtained as

$$\widehat{L}(h^{(\widehat{\mathbf{w}})} \mid \mathcal{D}) \stackrel{(4.4)}{=} \min_{\mathbf{w} \in \mathbb{R}^n} \widehat{L}(h^{(\mathbf{w})}|\mathcal{D}) \stackrel{(4.13)}{=} \|(\mathbf{I}-\mathbf{P})\mathbf{y}\|_2^2. \tag{6.1}$$

Here, we used the orthogonal projection matrix $\mathbf{P}$ on the linear span

$$\text{span}\{\mathbf{X}\} = \big\{\mathbf{X}\mathbf{a} : \mathbf{a} \in \mathbb{R}^n\big\} \subseteq \mathbb{R}^m,$$

of the feature matrix $\mathbf{X} = (\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)})^T \in \mathbb{R}^{m \times n}$.

In many ML applications we have access to a huge number of individual features to characterize a data point. As a point in case, consider a data point which is a snapshot obtained from a modern smartphone camera. These cameras have a resolution of several megapixels. Here, we can use millions of pixel colour intensities as its features. For such applications, it is common to have more features for data points than the size of the training set,

$$n \geq m. \tag{6.2}$$

Whenever (6.2) holds, the feature vectors $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)} \in \mathbb{R}^n$ of the data points in $\mathcal{D}$ are typically linearly independent. As a case in point, if the feature vectors $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)} \in \mathbb{R}^n$ are realizations of i.i.d. RVs with a continuous probability distribution, these vectors are linearly independent with probability one [100].

If the feature vectors $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)} \in \mathbb{R}^n$ are linearly independent, the span of the feature matrix $\mathbf{X} = (\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)})^T$ coincides with $\mathbb{R}^m$ which implies, in turn, $\mathbf{P} = \mathbf{I}$. Inserting $\mathbf{P} = \mathbf{I}$ into (4.13) yields

$$\widehat{L}(h^{(\widehat{\mathbf{w}})} \mid \mathcal{D}) = 0. \tag{6.3}$$

As soon as the number $m = |\mathcal{D}|$ of training data points does not exceed the number n of features that characterize data points, there is (with probability one) a linear predictor $h^{(\widehat{\mathbf{w}})}$ achieving zero training error(!).

While the hypothesis $h^{(\widehat{\mathbf{w}})}$ achieves zero training error, it will typically incur a non-zero average prediction error $y - h^{(\widehat{\mathbf{w}})}(\mathbf{x})$ on data points $(\mathbf{x}, y)$ outside the training set (see Figure 6.2). Section 6.4 will make this statement more precise by using a probabilistic model for the data points within and outside the training set.

Note that (6.3) also applies if the features $\mathbf{x}$ and labels y of data points are completely

unrelated. Consider an ML problem with data points whose labels y and features are realizations of a RV that are statistically independent. Thus, in a very strong sense, the features $\mathbf{x}$ contain no information about the label of a data point. Nevertheless, as soon as the number of features exceeds the size of the training set, such that (6.2) holds, linear regression methods will learn a hypothesis with zero training error.

We can easily extend the above discussion about the occurrence of overfitting in linear regression to other methods that combine linear regression with a feature map. Polynomial regression, using data points with a single feature z, combines linear regression with the feature map $z \mapsto \mathbf{\Phi}(z) := \big(z^0, \ldots, z^{n-1}\big)^T$ as discussed in Section 3.2.

It can be shown that whenever (6.2) holds and the features $z^{(1)}, \ldots, z^{(m)}$ of the training set are all different, the feature vectors $\mathbf{x}^{(1)} := \mathbf{\Phi}\big(z^{(1)}\big), \ldots, \mathbf{x}^{(m)} := \mathbf{\Phi}\big(z^{(m)}\big)$ are linearly independent. This implies, in turn, that polynomial regression is guaranteed to find a hypothesis with zero training error whenever $m \leq n$ and the data points in the training set have different feature values.

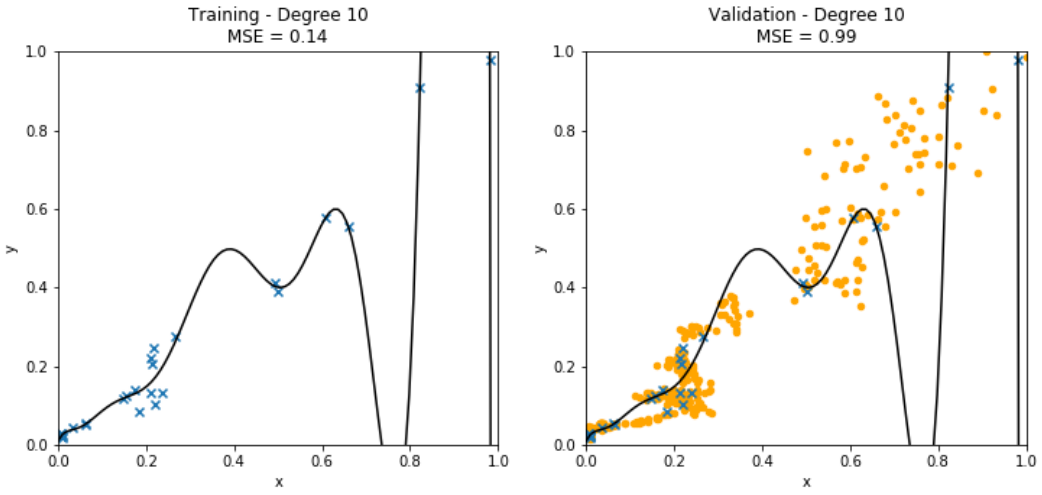


Figure 6.2: Polynomial regression learns a polynomial map with degree $n-1$ by minimizing its average loss on a training set (blue crosses). Using high-degree polynomials (large n) results in a small training error. However, the learnt high-degree polynomial performs poorly on data points outside the training set (orange dots).

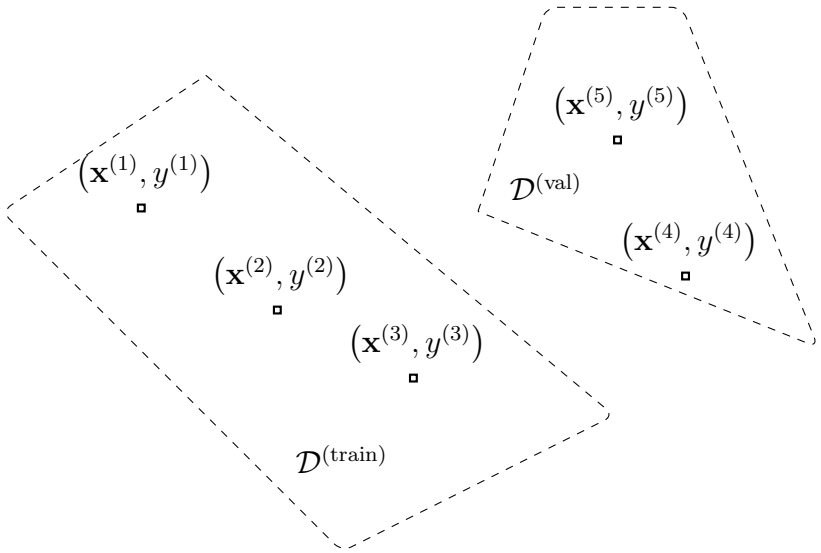


Figure 6.3: We split the dataset $\mathcal{D}$ into two subsets, a training set $\mathcal{D}^{(\text{train})}$ and a validation set $\mathcal{D}^{(\text{val})}$. We use the training set to learn (find) the hypothesis $\widehat{h}$ with minimum empirical risk $\widehat{L}(\widehat{h}|\mathcal{D}^{(\text{train})})$ on the training set (4.3). We then validate $\widehat{h}$ by computing its average loss $\widehat{L}(\widehat{h}|\mathcal{D}^{(\text{val})})$ on the validation set $\mathcal{D}^{(\text{val})}$. The average loss $\widehat{L}(\widehat{h}|\mathcal{D}^{(\text{val})})$ obtained on the validation set is the validation error. Note that $\widehat{h}$ depends on the training set $\mathcal{D}^{(\text{train})}$ but is completely independent of the validation set $\mathcal{D}^{(\text{val})}$.

6.2 Validation

Consider an ML method that uses ERM (4.3) to learn a hypothesis $\widehat{h} \in \mathcal{H}$ out of the hypothesis space $\mathcal{H}$. The discussion in Section 6.1 revealed that the training error of a learnt hypothesis $\widehat{h}$ can be a poor indicator for the performance of $\widehat{h}$ for data points outside the training set. The hypothesis $\widehat{h}$ tends to "look better" on the training set over which it has been tuned within ERM. The basic idea of validating the predictor $\widehat{h}$ is simple:

- first we learn a hypothesis $\widehat{h}$ using ERM on a training set and
- then we compute the average loss of $\widehat{h}$ on data points that do not belong to the training set.

Thus, validation means to compute the average loss of a hypothesis using data points that have not been used in ERM to learn that hypothesis.

Assume we have access to a dataset of m data points,

$$\mathcal{D} = \big\{ \big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big) \big\}.$$

Each data point is characterized by a feature vector $\mathbf{x}^{(i)}$ and a label $y^{(i)}$. Algorithm 5 outlines how to learn and validate a hypothesis $h \in \mathcal{H}$ by splitting the dataset $\mathcal{D}$ into a training set and a validation set. The random shuffling in step 1 of Algorithm 5 ensures the i.i.d. assumption for the shuffled data. Section 6.2.1 shows next how the i.i.d. assumption ensures that the validation error (6.6) approximates the expected loss of the hypothesis $\widehat{h}$. The hypothesis $\widehat{h}$ is learnt via ERM on the training set during step 4 of Algorithm 5.

6.2.1 The Size of the Validation Set

The choice of the split ratio $\rho \approx m_t/m$ in Algorithm 5 is often based on trial and error. We try out different choices for the split ratio and pick the one with the smallest validation error. It is difficult to make a precise statement on how to choose the split ratio which applies broadly [83]. This difficulty stems from the fact that the optimal choice for ρ depends on the precise statistical properties of the data points.

One approach to determine the required size of the validation set is to use a probabilistic model for the data points. The i.i.d. assumption is maybe the most widely used probabilistic model within ML. Here, we interpret data points as the realizations of i.i.d. RVs. These i.i.d. RVs have a common (joint) probability distribution $p(\mathbf{x}, y)$ over possible features $\mathbf{x}$

Algorithm 5 Validated ERM

Input: model $\mathcal{H}$, loss function L, dataset $\mathcal{D} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}$; split ratio ρ

1: randomly shuffle the data points in $\mathcal{D}$

2: create the training set $\mathcal{D}^{(\text{train})}$ using the first $m_t = \lceil \rho m \rceil$ data points,

$$\mathcal{D}^{(\text{train})} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m_t)}, y^{(m_t)}\big)\big\}.$$

3: create the validation set $\mathcal{D}^{(\text{val})}$ by the $m_v = m - m_t$ remaining data points,

$$\mathcal{D}^{(\text{val})} = \big\{\big(\mathbf{x}^{(m_t+1)}, y^{(m_t+1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}.$$

4: learn hypothesis $\widehat{h}$ via ERM on the training set,

$$\widehat{h} := \operatorname*{argmin}_{h \in \mathcal{H}} \widehat{L}\big(h | \mathcal{D}^{(\text{train})}\big) \tag{6.4}$$

5: compute the training error

$$E_t := \widehat{L}\big(\widehat{h} | \mathcal{D}^{(\text{train})}\big) = (1/m_t) \sum_{i=1}^{m_t} L\left((\mathbf{x}^{(i)}, y^{(i)}), \widehat{h}\right). \tag{6.5}$$

6: compute the validation error

$$E_v := \widehat{L}\big(\widehat{h} | \mathcal{D}^{(\text{val})}\big) = (1/m_v) \sum_{i=m_t+1}^{m} L\left((\mathbf{x}^{(i)}, y^{(i)}), \widehat{h}\right). \tag{6.6}$$

Output: learnt hypothesis $\widehat{h}$, training error E_t, validation error E_v

and labels y of a data point. Under the i.i.d. assumption, the validation error E_v (6.6) also becomes a realization of a RV. The expectation (or mean) $\mathbb{E}\{E_v\}$ of this RV is precisely the risk $\mathbb{E}\{L\left((\mathbf{x},y),\widehat{h}\right)\}$ of $\widehat{h}$ (see (4.1)).

Within the above i.i.d. assumption, the validation error E_v becomes a realization of a RV that fluctuates around its mean $\mathbb{E}\{E_v\}$. We can quantify this fluctuation using the variance

$$\sigma^2_{E_v} := \mathbb{E}\big\{\big(E_v - \mathbb{E}\{E_v\}\big)^2\big\}.$$

Note that the validation error is the average of the realizations $L\left((\mathbf{x}^{(i)},y^{(i)}),\widehat{h}\right)$ of i.i.d. RVs. The probability distribution of the RV $L\left((\mathbf{x},y),\widehat{h}\right)$ is determined by the probability distribution $p(\mathbf{x},y)$, the choice of loss function and the hypothesis $\widehat{h}$. In general, we do not know $p(\mathbf{x},y)$ and, in turn, also do not know the probability distribution of $L\left((\mathbf{x},y),\widehat{h}\right)$.

If we know an upper bound U on the variance of the (random) loss $L\left((\mathbf{x}^{(i)},y^{(i)}),\widehat{h}\right)$, we can bound the variance of E_v as

$$\sigma^2_{E_v} \leq U/m_v.$$

We can then, in turn, ensure that the variance $\sigma^2_{E_v}$ of the validation error E_v does not exceed a given threshold η, say $\eta = (1/100)E_t^2$, by using a validation set of size

$$m_v \geq U/\eta. \tag{6.7}$$

The lower bound (6.7) is only useful if we can determine an upper bound U on the variance of the RV $L\left((\mathbf{x},y),\widehat{h}\right)$ where $(\mathbf{x},y)$ is a RV with probability distribution $p(\mathbf{x},y)$. An upper bound on the variance of $L\left((\mathbf{x},y),\widehat{h}\right)$ can be derived using probability theory if we know an accurate probabilistic model $p(\mathbf{x},y)$ for the data points. Such a probabilistic model might be provided by application-specific scientific fields such as biology or psychology. Another option is to estimate the variance of $L\left((\mathbf{x},y),\widehat{h}\right)$ using the sample variance of the actual loss values $L\left((\mathbf{x}^{(1)},y^{(1)}),\widehat{h}\right),\ldots,L\left((\mathbf{x}^{(m)},y^{(m)}),\widehat{h}\right)$ obtained for the dataset $\mathcal{D}$.

6.2.2 k-Fold Cross Validation

Algorithm 5 uses the most basic form of splitting a given dataset $\mathcal{D}$ into a training set and a validation set. Many variations and extensions of this basic splitting approach have been proposed and studied (see [32] and Section 6.5). One very popular extension of the single split into training set and validation set is known as k-fold cross-validation (k-fold CV) [58,

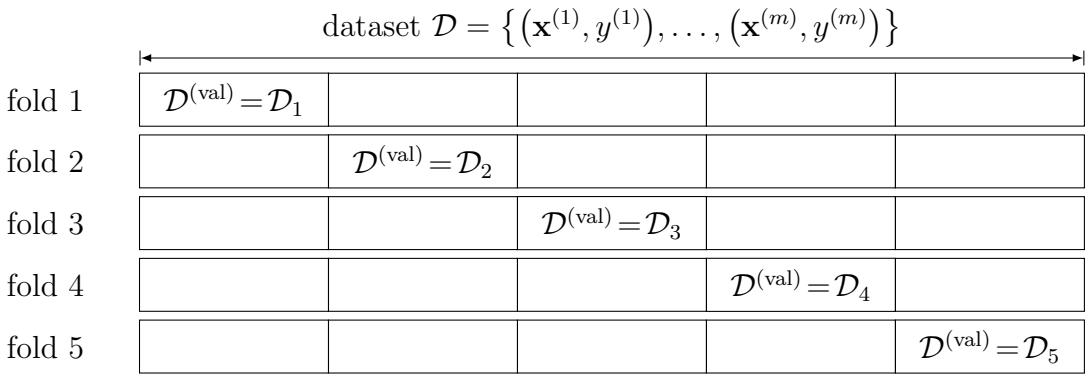


Figure 6.4: Illustration of k-fold CV for $k = 5$. We evenly partition the entire dataset $\mathcal{D}$ into $k = 5$ subsets (or folds) $\mathcal{D}_1, \ldots, \mathcal{D}_5$. We then repeat the validated ERM Algorithm 5 for $k = 5$ times. The bth repetition uses the bth fold $\mathcal{D}_b$ as the validation set and the remaining $k-1 (= 4)$ folds as the training set for ERM (4.3).

Sec. 7.10]. We summarize k-fold CV in Algorithm 6 below.

Figure 6.4 illustrates the key principle behind k-fold CV. First, we divide the entire dataset evenly into k subsets which are referred to as "folds". The learning (via ERM) and validation of a hypothesis out of a given hypothesis space $\mathcal{H}$ is then repeated k times. During each repetition, we use one fold as the validation set and the remaining $k-1$ folds as a training set. We then average the values of the training error and validation error obtained for each repetition (fold).

The average (over all k folds) validation error delivered by k-fold CV tends to better estimate the expected loss or risk (4.1) compared to the validation error obtained from a single split in Algorithm 5. Consider a dataset that consists of a relatively small number of data points. If we use a single split of this small dataset into a training set and validation set, we might be very unlucky and choose data points for the validation set which are outliers and not representative for the statistical properties of most data points. The effect of such an unlucky split is typically averaged out when using k-fold CV.

6.2.3 Imbalanced Data

The simple validation approach discussed above requires the validation set to be a good representative for the overall statistical properties of the data. This might not be the case in applications with discrete valued labels and some of the label values being very rare. We might then be interested in having a good estimate of the conditional risks $\mathbb{E}\{L\left((\mathbf{x}, y), h\right) | y =$

Algorithm 6 k-fold CV ERM

Input: model $\mathcal{H}$, loss function L, dataset $\mathcal{D} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}$; number k of folds

1: randomly shuffle the data points in $\mathcal{D}$

2: divide the shuffled dataset $\mathcal{D}$ into k folds $\mathcal{D}_1, \ldots, \mathcal{D}_k$ of size $B = \lceil m/k \rceil$,

$$\mathcal{D}_1 = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(B)}, y^{(B)}\big)\big\}, \ldots, \mathcal{D}_k = \big\{\big(\mathbf{x}^{((k-1)B+1)}, y^{((k-1)B+1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\} \quad (6.8)$$

3: **for** fold index $b = 1, \ldots, k$ **do**

4: use bth fold as the validation set $\mathcal{D}^{(\text{val})} = \mathcal{D}_b$

5: use rest as the training set $\mathcal{D}^{(\text{train})} = \mathcal{D} \setminus \mathcal{D}_b$

6: learn hypothesis $\widehat{h}$ via ERM on the training set,

$$\widehat{h}^{(b)} := \operatorname*{argmin}_{h \in \mathcal{H}} \widehat{L}\big(h | \mathcal{D}^{(\text{train})}\big) \quad (6.9)$$

7: compute the training error

$$E_t^{(b)} := \widehat{L}\big(\widehat{h} | \mathcal{D}^{(\text{train})}\big) = (1/\big|\mathcal{D}^{(\text{train})}\big|) \sum_{i \in \mathcal{D}^{(\text{train})}} L\left((\mathbf{x}^{(i)}, y^{(i)}), h\right). \quad (6.10)$$

8: compute **validation error**

$$E_v^{(b)} := \widehat{L}\big(\widehat{h} | \mathcal{D}^{(\text{val})}\big) = (1/\big|\mathcal{D}^{(\text{val})}\big|) \sum_{i \in \mathcal{D}^{(\text{val})}} L\left((\mathbf{x}^{(i)}, y^{(i)}), \widehat{h}\right). \quad (6.11)$$

9: **end for**

10: compute average training and validation errors

$$E_t := (1/k) \sum_{b=1}^{k} E_t^{(b)}, \text{ and } E_v := (1/k) \sum_{b=1}^{k} E_v^{(b)}$$

11: pick a learnt hypothesis $\widehat{h} := \widehat{h}^{(b)}$ for some $b \in \{1, \ldots, k\}$

Output: learnt hypothesis $\widehat{h}$; average training error E_t; average validation error E_v

$y'\}$ where y' is one of the rare label values. This is more than requiring a good estimate for the risk $\mathbb{E}\{L\left((\mathbf{x}, y), h\right)\}$.

Consider data points characterized by a feature vector $\mathbf{x}$ and binary label $y \in \{-1, 1\}$. Assume we aim at learning a hypothesis $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ to classify data points as $\hat{y} = 1$ if $h(\mathbf{x}) \geq 0$ while $\hat{y} = -1$ otherwise. The learning is based on a dataset $\mathcal{D}$ which contains only one single (!) data point with $y = -1$. If we then split the dataset into training and validation set, it is with high probability that the validation set does not include any data point with label value $y = -1$. This cannot happen when using k-fold CV since the single data point must be in one of the validation folds. However, even the applicability of k-fold CV for such an imbalanced dataset is limited since we evaluate the performance of a hypothesis $h(\mathbf{x})$ using only one single data point with $y = -1$. The resulting validation error will be dominated by the loss of $h(\mathbf{x})$ incurred on data points from the majority class (those with true label value $y = 1$).

To learn and validate a hypothesis with imbalanced data, it might be useful to to generate synthetic data points to enlarge the minority class. This can be done using data augmentation techniques which we discuss in Section 7.3. Another option is to choose a loss function that takes the different frequencies of label values into account. Let us illustrate this approach in what follows by an illustrative example.

Consider an imbalanced dataset of size $m = 100$, which contains 90 data points with label $y = 1$ but only 10 data points with label $y = -1$. We might want to put more weight on wrong predictions obtained for data points from the minority class (with true label value $y = -1$). This can be done by using a much larger value for the loss $L\left((\mathbf{x}, y = -1), h(\mathbf{x}) = 1\right)$ than for the loss $L\left((\mathbf{x}, y = 1), h(\mathbf{x}) = -1\right)$ incurred by incorrectly predicting the label of a data point from the majority class (with true label value $y = 1$).

6.3 Model Selection

Chapter 3 illustrated how many well-known ML methods are obtained by different combinations of a hypothesis space or model, loss function and data representation. While for many ML applications there is often a natural choice for the loss function and data representation, the right choice for the model is typically less obvious. We now discuss how to use the validation methods of Section 6.2 to choose between different candidate models.

Consider data points characterized by a single numeric feature $x \in \mathbb{R}$ and numeric label $y \in \mathbb{R}$. If we suspect that the relation between feature x and label y is non-linear, we might

use polynomial regression which is discussed in Section 3.2. Polynomial regression uses the hypothesis space $\mathcal{H}^{(n)}_{\rm poly}$ with some maximum degree n. Different choices for the maximum degree n yield a different hypothesis space: $\mathcal{H}^{(1)} = \mathcal{H}^{(0)}_{\rm poly}, \mathcal{H}^{(2)} = \mathcal{H}^{(1)}_{\rm poly}, \ldots, \mathcal{H}^{(M)} = \mathcal{H}^{(M)}_{\rm poly}$.

Another ML method that learns non-linear hypothesis map is Gaussian basis regression (see Section 3.5). Here, different choices for the variance σ and shifts μ of the Gaussian basis function (3.13) result in different hypothesis spaces. For example, $\mathcal{H}^{(1)} = \mathcal{H}^{(2)}_{\rm Gauss}$ with $\sigma = 1$ and $\mu_1 = 1$ and $\mu_2 = 2$, $\mathcal{H}^{(2)} = \mathcal{H}^{(2)}_{\rm Gauss}$ with $\sigma = 1/10$, $\mu_1 = 10$, $\mu_2 = 20$.

Algorithm 7 summarizes a simple method to choose between different candidate models $\mathcal{H}^{(1)}, \mathcal{H}^{(2)}, \ldots, \mathcal{H}^{(M)}$. The idea is to first learn and validate a hypothesis $\widehat{h}^{(l)}$ separately for each model $\mathcal{H}^{(l)}$ using Algorithm 6. For each model $\mathcal{H}^{(l)}$, we learn the hypothesis $\widehat{h}^{(l)}$ via ERM (6.4) and then compute its validation error $E_v^{(l)}$ (6.6). We then choose the hypothesis $\widehat{h}^{(\hat{l})}$ from those model $\mathcal{H}^{(\hat{l})}$ which resulted in the smallest validation error $E_v^{(\hat{l})} = \min_{l=1,\ldots,M} E_v^{(l)}$.

The workflow of Algorithm 7 is similar to the workflow of ERM. Remember that the idea of ERM is to learn a hypothesis out of a set of different candidates (the hypothesis space). The quality of a particular hypothesis h is measured using the (average) loss incurred on some training set. We use the same principle for model selection but on a higher level. Instead of learning a hypothesis within a hypothesis space, we choose (or learn) a hypothesis space within a set of candidate hypothesis spaces. The quality of a given hypothesis space is measured by the validation error (6.6). To determine the validation error of a hypothesis space, we first learn the hypothesis $\widehat{h} \in \mathcal{H}$ via ERM (6.4) on the training set. Then, we obtain the validation error as the average loss of $\widehat{h}$ on the validation set.

The final hypothesis $\widehat{h}$ delivered by the model selection Algorithm 7 not only depends on the training set used in ERM (see (6.9)). This hypothesis $\widehat{h}$ has also been chosen based on its validation error which is the average loss on the validation set in (6.11). Indeed, we compared this validation error with the validation errors of other models to pick the model $\mathcal{H}^{(\hat{l})}$ (see step 10) which contains $\widehat{h}$. Since we used the validation error (6.11) of $\widehat{h}$ to learn it, we cannot use this validation error as a good indicator for the general performance of $\widehat{h}$.

To estimate the general performance of the final hypothesis $\widehat{h}$ delivered by Algorithm 7 we must try it out on a test set. The test set, which is constructed in step 3 of Algorithm 7, consists of data points that are neither contained in the training set (6.9) nor the validation set (6.11) used for training and validating the candidate models $\mathcal{H}^{(1)}, \ldots, \mathcal{H}^{(M)}$. The average loss of the final hypothesis on the test set is referred to as the test error. The test error is computed in the step 12 of Algorithm 7.

Sometimes it is beneficial to use different loss functions for the training and the validation

Algorithm 7 Model Selection

Input: list of candidate models $\mathcal{H}^{(1)}, \ldots, \mathcal{H}^{(M)}$, loss function L, dataset $\mathcal{D} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}$; number k of folds, test set fraction ρ

1: randomly shuffle the data points in $\mathcal{D}$
2: determine size $m' := \lceil \rho m \rceil$ of test set
3: construct a test set

$$\mathcal{D}^{(\text{test})} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m')}, y^{(m')}\big)\big\}$$

4: construct a training set and a validation set,

$$\mathcal{D}^{(\text{trainval})} = \big\{\big(\mathbf{x}^{(m'+1)}, y^{(m'+1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}$$

5: **for** model index $l = 1, \ldots, M$ **do**
6: run Algorithm 6 using $\mathcal{H} = \mathcal{H}^{(l)}$, dataset $\mathcal{D} = \mathcal{D}^{(\text{trainval})}$, loss function L and k folds
7: Algorithm 6 delivers hypothesis $\widehat{h}$ and validation error E_v
8: store learnt hypothesis $\widehat{h}^{(l)} := \widehat{h}$ and validation error $E_v^{(l)} := E_v$
9: **end for**
10: pick model $\mathcal{H}^{(\hat{l})}$ with minimum validation error $E_v^{(\hat{l})} = \min_{l=1,\ldots,M} E_v^{(l)}$
11: define optimal hypothesis $\widehat{h} = \widehat{h}^{(\hat{l})}$
12: compute **test error**

$$E^{(\text{test})} := \widehat{L}\big(\widehat{h} | \mathcal{D}^{(\text{test})}\big) = (1/\big|\mathcal{D}^{(\text{test})}\big|) \sum_{i \in \mathcal{D}^{(\text{test})}} L\left((\mathbf{x}^{(i)}, y^{(i)}), \widehat{h}\right). \tag{6.12}$$

Output: hypothesis $\widehat{h}$; training error $E_t^{(\hat{l})}$; validation error $E_v^{(\hat{l})}$, test error $E^{(\text{test})}$.

of a hypothesis. As an example, consider logistic regression and the SVM which have been discussed in Sections 3.6 and 3.7, respectively. Both methods use the same model which is the space of linear hypothesis maps $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$. The main difference between these two methods is in their choice for the loss function. Logistic regression minimizes the (average) logistic loss (2.12) on the training set to learn the hypothesis $h^{(1)}(\mathbf{x}) = \left(\mathbf{w}^{(1)}\right)^T\mathbf{x}$ with a parameter vector $\mathbf{w}^{(1)}$. The SVM instead minimizes the (average) hinge loss (2.11) on the training set to learn the hypothesis $h^{(2)}(\mathbf{x}) = \left(\mathbf{w}^{(2)}\right)^T\mathbf{x}$ with a parameter vector $\mathbf{w}^{(2)}$. It is inconvenient to compare the usefulness of the two hypotheses $h^{(1)}(\mathbf{x})$ and $h^{(2)}(\mathbf{x})$ using different loss functions to compute their validation errors. This comparison is more convenient if we instead compute the validation errors for $h^{(1)}(\mathbf{x})$ and $h^{(2)}(\mathbf{x})$ using the average 0/1 loss (2.9).

Algorithm 7 requires as one of its inputs a given list of candidate models. The longer this list, the more computation is required from Algorithm 7. Sometimes it is possible to prune the list of candidate models by removing models that are very unlikely to have minimum validation error.

Consider polynomial regression which uses as the model the space $\mathcal{H}^{(r)}_{\rm poly}$ of polynomials with maximum degree r (see (3.4)). For $r = 1$, $\mathcal{H}^{(r)}_{\rm poly}$ is the space of polynomials with maximum degree one (which are linear maps), $h(x) = w_2x + w_1$. For $r = 2$, $\mathcal{H}^{(r)}_{\rm poly}$ is the space of polynomials with maximum degree two, $h(x) = w_3x^2 + w_2x + w_1$.

The polynomial degree r parametrizes a nested set of models,

$$\mathcal{H}^{(r)}_{\rm poly} \subset \mathcal{H}^{(r)}_{\rm poly} \subset \ldots.$$

For each degree r, we learn a hypothesis $h^{(r)} \in \mathcal{H}^{(r)}_{\rm poly}$ with minimum average loss (training error) $E^{(r)}_t$ on a training set (see (6.5)). To validate the learnt hypothesis $h^{(r)}$, we compute its average loss (validation error) $E^{(r)}_v$ on a validation set (see (6.6)).

Figure 6.5 depicts the typical dependency of the training and validation errors on the polynomial degree r. The training error $E^{(r)}_t$ decreases monotonically with increasing polynomial degree r. To illustrate this monotonic decrease, we consider the two specific choices $r = 3$ and $r = 5$ with corresponding models $\mathcal{H}^{(r)}_{\rm poly}$ and $\mathcal{H}^{(r)}_{\rm poly}$. Note that $\mathcal{H}^{(3)}_{\rm poly} \subset \mathcal{H}^{(5)}_{\rm poly}$ since any polynomial with degree not exceeding 3 is also a polynomial with degree not exceeding 5. Therefore, the training error (6.5) obtained when minimizing over the larger model $\mathcal{H}^{(5)}_{\rm poly}$ can only decrease but never increase compared to (6.5) using the smaller model $\mathcal{H}^{(3)}_{\rm poly}$

Figure 6.5 indicates that the validation error $E^{(r)}_v$ (see (6.6)) behaves different compared to the training error $E^{(r)}_t$. Starting with degree $r = 0$, the validation error first decreases with increasing degree r. As soon as the degree r is increased beyond a critical value, the

validation error starts to increase with increasing r. For very large values of r, the training error becomes almost negligible while the validation error becomes very large. In this regime, polynomial regression overfits the training set.

Figure 6.6 illustrates the overfitting of polynomial regression when using a maximum degree that is too large. In particular, Figure 6.6 depicts a learnt hypothesis which is a degree 9 polynomial that fits quite well the training set, resulting in a small training error. To achieve such a low training error, requires the learnt polynomial to have an unreasonable high rate of change for feature values $x \approx 0$. This results in large prediction errors for data points with feature values $x \approx 0$.

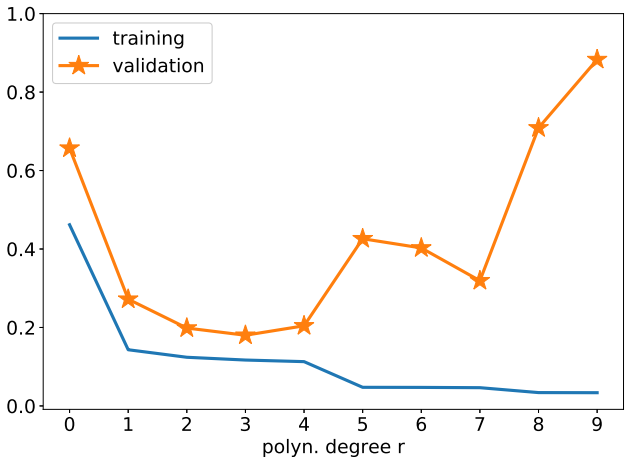


Figure 6.5: The training error and validation error obtained from polynomial regression using different values r for the maximum polynomial degree.

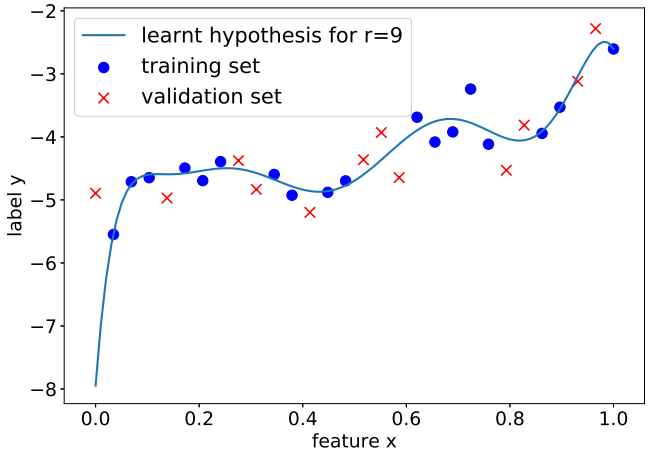


Figure 6.6: A hypothesis $\widehat{h}$ which is a polynomial with degree not larger than $r = 9$. The hypothesis has been learnt by minimizing the average loss on the training set. Note the rapid change of $\widehat{h}$ for feature values $x \approx 0$.

6.4 A Probabilistic Analysis of Generalization

More Data Beats Clever Algorithms ?; More Data Beats Clever Feature Selection?

A key challenge in ML is to ensure that a hypothesis that predicts well the labels on a training set (which has been used to learn that hypothesis) will also predict well the labels of data points outside the training set. We say that a ML method generalizes well if it learns a hypothesis $\widehat{h}$ that performs not significantly worse on data points outside the training set. In other words, the loss incurred by $\widehat{h}$ for data points outside the training set is not much larger than the average loss of $\widehat{h}$ incurred on the training set.

We now study the generalization of linear regression methods (see Section 3.1) using an i.i.d. assumption. In particular, we interpret data points as i.i.d. realizations of RVs that have the same distribution as a random data point $\mathbf{z} = (\mathbf{x}, y)$. The feature vector $\mathbf{x}$ is then a realization of a standard Gaussian RV with zero mean and covariance being the identity matrix, i.e., $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

The label y of a random data point is related to its features $\mathbf{x}$ via a linear Gaussian model

$$y = \overline{\mathbf{w}}^{T}\mathbf{x} + \varepsilon, \text{ with noise } \varepsilon \sim \mathcal{N}(0, \sigma^2). \tag{6.13}$$

We assume the noise variance σ^2 fixed and known. This is a simplifying assumption and in

practice we would need to estimate the noise variance from data [24]. Note that, within our probabilistic model, the error component ε in (6.13) is intrinsic to the data and cannot be overcome by any ML method. We highlight that the probabilistic model for the observed data points is just a modelling assumption. This assumption allows us to study some fundamental behaviour of ML methods. There are principled methods ("statistical tests") that allow to determine if a given dataset can be accurately modelled using (6.13) [62].

We predict the label y from the features $\mathbf{x}$ using a linear hypothesis $h(\mathbf{x})$ that depends only on the first l features $x_1, \ldots, x_l$. Thus, we use the hypothesis space

$$\mathcal{H}^{(l)} = \{h^{(\mathbf{w})}(\mathbf{x}) = (\mathbf{w}^T, \mathbf{0}^T)\mathbf{x} \text{ with } \mathbf{w} \in \mathbb{R}^l\}. \tag{6.14}$$

Note that each element $h^{(\mathbf{w})} \in \mathcal{H}^{(l)}$ corresponds to a particular choice of the parameter vector $\mathbf{w} \in \mathbb{R}^l$.

The model parameter $l \in \{0, \ldots, n\}$ coincides with the effective dimension of the hypothesis space $\mathcal{H}^{(l)}$. For $l < n$, the hypothesis space $\mathcal{H}^{(l)}$ is a proper (strict) subset of the space of linear hypothesis maps (2.4) used within linear regression (see Section 3.1). Moreover, the parameter l indexes a nested sequence of models,

$$\mathcal{H}^{(0)} \subseteq \mathcal{H}^{(1)} \subseteq \ldots \subseteq \mathcal{H}^{(n)}.$$

The quality of a particular predictor $h^{(\mathbf{w})} \in \mathcal{H}^{(l)}$ is measured via the average squared error $\widehat{L}(h^{(\mathbf{w})} \mid \mathcal{D}^{(\text{train})})$ incurred on the labeled training set

$$\mathcal{D}^{(\text{train})} = \{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m_t)}, y^{(m_t)}\big)\}. \tag{6.15}$$

We interpret data points in the training set $\mathcal{D}^{(\text{train})}$ as well as any other data point outside the training set as realizations of i.i.d. RVs with a common probability distribution. This common probability distribution is a multivariate normal (Gaussian) distribution,

$$\mathbf{x}, \mathbf{x}^{(i)} \text{ i.i.d. with } \mathbf{x}, \mathbf{x}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{6.16}$$

The labels $y^{(i)}, y$ are related to the features of data points via (see (6.13))

$$y^{(i)} = \overline{\mathbf{w}}^T \mathbf{x}^{(i)} + \varepsilon^{(i)}, \text{ and } y = \overline{\mathbf{w}}^T \mathbf{x} + \varepsilon. \tag{6.17}$$

Here, the noise terms $\varepsilon, \varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$ are realizations of i.i.d. Gaussian RVs with zero mean and variance σ^2.

Chapter 4 showed that the training error $\widehat{L}(h^{(\mathbf{w})} \mid \mathcal{D}^{(\text{train})})$ is minimized by the predictor $h^{(\widehat{\mathbf{w}})}(\mathbf{x}) = \widehat{\mathbf{w}}^T \mathbf{I}_{l\times n}\mathbf{x}$, that uses the parameter vector

$$\widehat{\mathbf{w}} = \big(\big(\mathbf{X}^{(l)}\big)^T \mathbf{X}^{(l)}\big)^{-1} \big(\mathbf{X}^{(l)}\big)^T \mathbf{y}. \tag{6.18}$$

Here we used the (restricted) feature matrix $\mathbf{X}^{(l)}$ and the label vector $\mathbf{y}$ defined as, respectively,

$$\begin{aligned} \mathbf{X}^{(l)} &= (\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m_{\rm t})})^T \mathbf{I}_{n\times l} \in \mathbb{R}^{m_{\rm t}\times l}, \text{ and} \\ \mathbf{y} &= \big(y^{(1)}, \ldots, y^{(m_{\rm t})}\big)^T \in \mathbb{R}^{m_{\rm t}}. \end{aligned} \tag{6.19}$$

It will be convenient to tolerate a slight abuse of notation and denote both, the length-l vector (6.18) as well as the zero-padded parameter vector $\big(\widehat{\mathbf{w}}^T, \mathbf{0}^T\big)^T \in \mathbb{R}^n$, by $\widehat{\mathbf{w}}$. This allows us to write

$$h^{(\widehat{\mathbf{w}})}(\mathbf{x}) = \widehat{\mathbf{w}}^T \mathbf{x}. \tag{6.20}$$

We highlight that the formula (6.18) for the optimal weight vector $\widehat{\mathbf{w}}$ is only valid if the matrix $\big(\mathbf{X}^{(l)}\big)^T \mathbf{X}^{(l)}$ is invertible. Within our toy model (see (6.16)), this is true with probability one whenever $m_{\rm t} \geq l$. Indeed, for $m_{\rm t} \geq l$ the truncated feature vectors $\mathbf{I}_{l\times n}\mathbf{x}^{(1)}, \ldots, \mathbf{I}_{l\times n}\mathbf{x}^{(m_t)}$, which are i.i.d. realizations of a Gaussian RV, are linearly independent with probability one [9, 41].

In what follows, we consider the case $m_{\rm t} > l$ such that the formula (6.18) is valid (with probability one). The more challenging high-dimensional regime $m_{\rm t} \leq l$ will be studied in Chapter 7.

The optimal parameter vector $\widehat{\mathbf{w}}$ (see (6.18)) depends on the training set $\mathcal{D}^{(\text{train})}$ via the feature matrix $\mathbf{X}^{(l)}$ and label vector $\mathbf{y}$ (see (6.19)). Therefore, since we model the data points in the training set as realizations of RVs, the parameter vector $\widehat{\mathbf{w}}$ (6.18) is the realization of a RV. For each specific realization of the training set $\mathcal{D}^{(\text{train})}$, we obtain a specific realization of the optimal parameter vector $\widehat{\mathbf{w}}$.

The probabilistic model (6.13) relates the features $\mathbf{x}$ of a data point to its label y via some (unknown) true parameter vector $\overline{\mathbf{w}}$. Intuitively, the best linear hypothesis would be $h(\mathbf{x}) = \widehat{\mathbf{w}}^T \mathbf{x}$ with parameter vector $\widehat{\mathbf{w}} = \overline{\mathbf{w}}$. However, in general this will not be achievable since we have to compute $\widehat{\mathbf{w}}$ based on the features $\mathbf{x}^{(i)}$ and noisy labels $y^{(i)}$ of the data points in the training set $\mathcal{D}$.

The parameter vector $\widehat{\mathbf{w}}$ delivered by ERM (4.5) typically results in a non-zero estimation

error

$$\Delta \mathbf{w} := \widehat{\mathbf{w}} - \overline{\mathbf{w}}. \tag{6.21}$$

The estimation error (6.21) is the realization of a RV since the learnt parameter vector $\widehat{\mathbf{w}}$ (see (6.18)) is itself a realization of a RV.

The Bias and Variance Decomposition. The prediction accuracy of $h^{(\widehat{\mathbf{w}})}$, using the learnt parameter vector (6.18), depends crucially on the mean squared estimation error (MSEE)

$$E_{\rm est} := \mathbb{E}\{\|\Delta \mathbf{w}\|_2^2\} \stackrel{(6.21)}{=} \mathbb{E}\big\{ \|\widehat{\mathbf{w}} - \overline{\mathbf{w}}\|_2^2 \big\}. \tag{6.22}$$

We will next decompose the MSEE $E_{\rm est}$ into two components, which are referred to as a variance term and a bias term. The variance term quantifies the fluctuation of the parameter vector obtained from ERM on the training set (6.15). The bias term characterizes the systematic (or average) deviation between the true parameter vector $\overline{\mathbf{w}}$ (see (6.13)) and the expectation of the learnt parameter vector $\widehat{\mathbf{w}}$.

Let us start with rewriting (6.22) using elementary manipulations as

$$\begin{aligned} E_{\rm est} &\stackrel{(6.22)}{=} \mathbb{E}\big\{ \|\widehat{\mathbf{w}} - \overline{\mathbf{w}}\|_2^2 \big\} \big\} \\ &= \mathbb{E}\Big\{ \big\| \big(\widehat{\mathbf{w}} - \mathbb{E}\big\{\widehat{\mathbf{w}}\big\}\big) - \big(\overline{\mathbf{w}} - \mathbb{E}\big\{\widehat{\mathbf{w}}\big\}\big) \big\|_2^2 \Big\}. \end{aligned}$$

We can develop the last expression further by expanding the squared Euclidean norm,

$$\begin{aligned} E_{\rm est} &= \mathbb{E}\big\{ \|\widehat{\mathbf{w}} - \mathbb{E}\{\widehat{\mathbf{w}}\}\|_2^2 \big\} - 2\mathbb{E}\big\{ \big(\widehat{\mathbf{w}} - \mathbb{E}\big\{\widehat{\mathbf{w}}\big\}\big)^T \big(\overline{\mathbf{w}} - \mathbb{E}\big\{\widehat{\mathbf{w}}\big\}\big) \big\} + \mathbb{E}\big\{ \big\|\overline{\mathbf{w}} - \mathbb{E}\big\{\widehat{\mathbf{w}}\big\}\big\|_2^2 \big\} \\ &= \mathbb{E}\big\{ \|\widehat{\mathbf{w}} - \mathbb{E}\{\widehat{\mathbf{w}}\}\|_2^2 \big\} - 2\big(\underbrace{\mathbb{E}\big\{\widehat{\mathbf{w}}\big\} - \mathbb{E}\big\{\widehat{\mathbf{w}}\big\}}_{=\mathbf{0}} \big)^T \big(\overline{\mathbf{w}} - \mathbb{E}\big\{\widehat{\mathbf{w}}\big\}\big) \big\} + \mathbb{E}\big\{ \big\|\overline{\mathbf{w}} - \mathbb{E}\big\{\widehat{\mathbf{w}}\big\}\big\|_2^2 \big\} \\ &= \underbrace{\mathbb{E}\big\{ \|\widehat{\mathbf{w}} - \mathbb{E}\{\widehat{\mathbf{w}}\}\|_2^2 \big\}}_{\text{variance } \ V} + \underbrace{\mathbb{E}\big\{ \|\overline{\mathbf{w}} - \mathbb{E}\{\widehat{\mathbf{w}}\}\|_2^2 \big\}}_{\text{bias } \ B^2}. \end{aligned} \tag{6.23}$$

The first component in (6.23) represents the (expected) variance of the learnt parameter vector $\widehat{\mathbf{w}}$ (6.18). Note that, within our probabilistic model, the training set (6.15) is the realization of a RV since it is constituted by data points that are i.i.d. realizations of RVs (see (6.16) and (6.13)).

The second component in (6.23) is referred to as a bias term. The parameter vector $\widehat{\mathbf{w}}$ is computed from a randomly fluctuating training set via (6.18) and is therefore itself

fluctuating around its expectation $\mathbb{E}\{\widehat{\mathbf{w}}\}$. The bias term is the Euclidean distance between this expectation $\mathbb{E}\{\widehat{\mathbf{w}}\}$ and the true parameter vector $\overline{\mathbf{w}}$ relating features and label of a data point via (6.13).

The bias term B^2 and the variance V in (6.23) both depend on the model complexity parameter l but in a fundamentally different manner. The bias term B^2 typically decreases with increasing l while the variance V increases with increasing l. In particular, the bias term is given as

$$B^2 = \|\overline{\mathbf{w}} - \mathbb{E}\{\widehat{\mathbf{w}}\}\|_2^2 = \sum_{j=l+1}^{n} \overline{w}_j^2, \tag{6.24}$$

The bias term (6.24) is zero if and only if

$$\overline{w}_j = 0 \text{ for any index } j = l+1, \ldots, n. \tag{6.25}$$

The necessary and sufficient condition (6.25) for zero bias is equivalent to $h^{(\overline{\mathbf{w}})} \in \mathcal{H}^{(l)}$. Note that the condition (6.25) depends on both, the model parameter l and the true parameter vector $\overline{\mathbf{w}}$. While the model parameter l is under control, the true parameter vector $\overline{\mathbf{w}}$ is not under our control but determined by the underlying data generation process. The only way to ensure (6.25) for every possible parameter vector $\overline{\mathbf{w}}$ in (6.13) is to use $l = n$, i.e., to use all available features $x_1, \ldots, x_n$ of a data point.

When using the model $\mathcal{H}^{(l)}$ with $l < n$, we cannot guarantee a zero bias term since we have no control over the true underlying parameter vector $\overline{\mathbf{w}}$ in (6.13). In general, the bias term decreases with an increasing model size l (see Figure 6.7). We highlight that the bias term does not depend on the variance σ^2 of the noise ε in our toy model (6.13).

Let us now consider the variance term in (6.23). Using the statistical independence of the features and labels of data points (see (6.13), (6.16) and (6.17)), one can show that[1]

$$V = \mathbb{E}\big\{ \|\widehat{\mathbf{w}} - \mathbb{E}\{\widehat{\mathbf{w}}\}\|_2^2 \big\} = \big(B^2 + \sigma^2\big) \mathrm{tr}\left\{ \mathbb{E}\left\{ \left(\left(\mathbf{X}^{(l)}\right)^T \mathbf{X}^{(l)} \right)^{-1} \right\} \right\}. \tag{6.26}$$

By (6.16), the matrix $\left(\left(\mathbf{X}^{(l)}\right)^T \mathbf{X}^{(l)} \right)^{-1}$ is a realization of a (matrix-valued) RV with an inverse Wishart distribution [91]. For $m_{\mathrm{t}} > l+1$, its expectation is given as

$$\mathbb{E}\{ \left(\left(\mathbf{X}^{(l)}\right)^T \mathbf{X}^{(l)} \right)^{-1} \} = 1/(m_{\mathrm{t}} - l - 1)\mathbf{I}. \tag{6.27}$$

[1]This derivation is not very difficult but rather lengthy. For more details about the derivation of (6.26) we refer to the literature [9, 88].

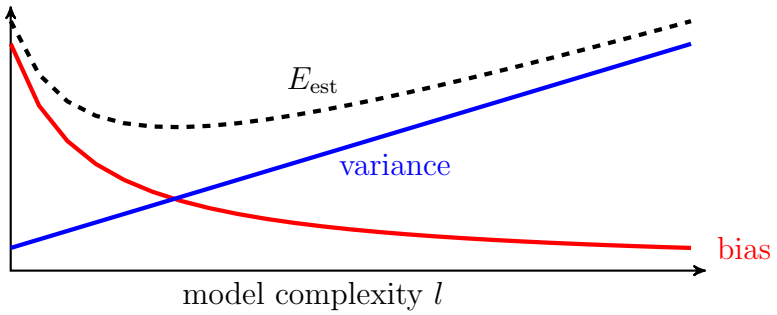


Figure 6.7: The MSEE $E_{\rm est}$ incurred by linear regression can be decomposed into a bias term B^2 and a variance term V (see (6.23)). These two components depend on the model complexity l in an opposite manner which results in a bias-variance trade-off.

By inserting (6.27) and $\mathrm{tr}\{\mathbf{I}\} = l$ into (6.26),

$$V = \mathbb{E}\left\{\|\widehat{\mathbf{w}} - \mathbb{E}\left\{\widehat{\mathbf{w}}\right\}\|_2^2\right\} = \big(B^2 + \sigma^2\big) l/(m_{\rm t} - l - 1). \tag{6.28}$$

The variance (6.28) typically increases with increasing model complexity l (see Figure 6.7). In contrast, the bias term (6.24) decreases with increasing l.

The opposite dependence of variance and bias on the model complexity results in a bias-variance trade-off. Choosing a model (hypothesis space) with small bias will typically result in large variance and vice versa. In general, the choice of model must balance between a small variance and a small bias.

Generalization. Consider a linear regression method that learns the linear hypothesis $h(\mathbf{x}) = \widehat{\mathbf{w}}^T\mathbf{x}$ using the parameter vector (6.18). The parameter vector $\widehat{\mathbf{w}}^T$ (6.18) results in a linear hypothesis with minimum training error, i.e., minimum average loss on the training set. However, the ultimate goal of ML is to find a hypothesis that predicts well the label of any data point. In particular, we want the hypothesis $h(\mathbf{x}) = \widehat{\mathbf{w}}^T\mathbf{x}$ to generalize well to data points outside the training set.

We quantify the generalization capability of $h(\mathbf{x}) = \widehat{\mathbf{w}}^T\mathbf{x}$ by its expected prediction loss

$$E_{\rm pred} = \mathbb{E}\big\{\big(y - \underbrace{\widehat{\mathbf{w}}^T\mathbf{x}}_{=\hat{y}}\big)^2\big\}. \tag{6.29}$$

Note that $E_{\rm pred}$ is a measure for the performance of a ML method and not of a specific hypothesis. Indeed, the learnt parameter vector $\widehat{\mathbf{w}}$ is not fixed but depends on the data points in the training set. These data points are modelled as realizations of i.i.d. RVs

and, in turn, the learnt parameter vector $\widehat{\mathbf{w}}$ becomes a realization of a RV. Thus, in some sense, the expected prediction loss (6.29) characterizes the overall ML method that reads in a training set and delivers (learn) a linear hypothesis with parameter vector $\widehat{\mathbf{w}}$ (6.18). In contrast, the risk (4.1) introduced in Chapter 4 characterizes the performance of a specific (fixed) hypothesis h without taking into account a learning process that delivered h based on data.

Let us now relate the expected prediction loss (6.29) of the linear hypothesis $h(\mathbf{x}) = \widehat{\mathbf{w}}^T\mathbf{x}$ to the bias and variance of (6.18),

$$\begin{aligned} E_{\rm pred} &\overset{(6.13)}{=} \mathbb{E}\{\Delta\mathbf{w}^T\mathbf{x}\mathbf{x}^T\Delta\mathbf{w}\} + \sigma^2 \\ &\overset{(a)}{=} \mathbb{E}\{\mathbb{E}\{\Delta\mathbf{w}^T\mathbf{x}\mathbf{x}^T\Delta\mathbf{w} \mid \mathcal{D}^{(\rm train)}\}\} + \sigma^2 \\ &\overset{(b)}{=} \mathbb{E}\{\Delta\mathbf{w}^T\Delta\mathbf{w}\} + \sigma^2 \\ &\overset{(6.21),(6.22)}{=} E_{\rm est} + \sigma^2 \\ &\overset{(6.23)}{=} B^2 + V + \sigma^2. \end{aligned} \tag{6.30}$$

Here, step (a) uses the law of iterated expectation (see, e.g., [9]). Step (b) uses that the feature vector $\mathbf{x}$ of a "new" data point is a realization of a RV which is statistically independent of the data points in the training set $\mathcal{D}^{(\rm train)}$. We also used our assumption that $\mathbf{x}$ is the realization of a RV with zero mean and covariance matrix $\mathbb{E}\{\mathbf{x}\mathbf{x}^T\} = \mathbf{I}$ (see (6.16)).

According to (6.30), the average (expected) prediction error $E_{\rm pred}$ is the sum of three components: (i) the bias B^2, (ii) the variance V and (iii) the noise variance σ^2. Figure 6.7 illustrates the typical dependency of the bias and variance on the model (6.14), which is parametrized by the model complexity l. Note that the model complexity parameter l in (6.14) coincides with the effective model dimension $d_{\rm eff}\left(\mathcal{H}^{(l)}\right)$ (see Section 2.2).

The bias and variance, whose sum is the estimation error $E_{\rm est}$, can be influenced by varying the model complexity l which is a design parameter. The noise variance σ^2 is the intrinsic accuracy limit of our toy model (6.13) and is not under the control of the ML engineer. It is impossible for any ML method (no matter how computationally expensive) to achieve, on average, a prediction error smaller than the noise variance σ^2. Carefully note that this statement only applies if the data points arising in a ML application can be (reasonably well) modelled as realizations of i.i.d. RVs.

We highlight that our statistical analysis, resulting in the formulas for the bias (6.24), variance (6.28) and the average prediction error (6.30), applies only if the observed data

points can be reasonably well modelled using the probabilistic model specified by (6.13), (6.16) and (6.17). If this is the case, i.e., the validity of this probabilistic model can be verified by principled statistical methods for model validation [159, 146].

Section 6.5 discusses a fundamentally different approach for characterizating the statistical properties of a ML method. Instead of carefully analyzing a (possibly inaccurate) probabilistic model using the rigorous calculus of probability theory, this approach uses random sampling techniques to synthesize i.i.d. copies of given data points. We can then approximate the expectation of relevant quantities, such as the loss $L\big(\big(\mathbf{x},y\big),h\big)$, using an average over the additional synthetic data points [58].

The qualitative behaviour of estimation error in Figure 6.7 depends on the definition for the model complexity. Our concept of effective dimension (see Section 2.2) coincides with some widely-used notions of model complexity for the linear hypothesis space (6.14). However, for more complicated models such as deep nets it is often not obvious how effective dimension is related to more tangible quantities such as the total number of tunable weights or the number of artificial neurons. Indeed, the effective dimension might also depend on the specific learning algorithm such as SGD. Depending on the actual measure of model complextiy, we might observe a behaviour of estimation error fundamentally different from the shape in Figure 6.7. One specific instance for such counter-intuitive behaviour is the "double descent phenomena" [8].

6.5 The Bootstrap

basic idea of bootstrap: use histogram of dataset as the underlying probability distribution; generate new data points by random sampling (with replacement) from that distribution.

Consider learning a hypothesis $\widehat{h} \in \mathcal{H}$ by minimizing the average loss incurred on a dataset $\mathcal{D} = \{\big(\mathbf{x}^{(1)},y^{(1)}\big),\ldots,\big(\mathbf{x}^{(m)},y^{(m)}\big)\}$. The data points $\big(\mathbf{x}^{(i)},y^{(i)}\big)$ are modelled as realizations of i.i.d. RVs. Let use denote the (common) probability distribution of these RVs by $p(\mathbf{x},y)$.

If we interpret the data points $\big(\mathbf{x}^{(i)},y^{(i)}\big)$ as realizations of RVs, also the learnt hypothesis $\widehat{h}$ is a realization of a RV. Indeed, the hypothesis $\widehat{h}$ is obtained by solving an optimization problem (4.3) that involves realizations of RVs. The bootstrap is a method for estimating (parameters of) the probability distribution $p(\widehat{h})$ [58].

Section 6.4 used a probabilistic model for data points to derive (the parameters of) the probability distribution $p(\widehat{h})$. Note that the analysis in Section 6.4 only applies to the specific

probabilistic model (6.16), (6.17). In contrast, the bootstrap can be used for data points drawn from an arbitrary probability distribution.

The core idea behind the bootstrap is to use the histogram $\hat{p}(\mathbf{z})$ of the data points in $\mathcal{D}$ to generate B new datasets $\mathcal{D}^{(1)}, \ldots, \mathcal{D}^{(B)}$. Each dataset is constructed such that is has the same size as the original dataset $\mathcal{D}$. For each dataset $\mathcal{D}^{(b)}$, we solve a separate ERM (4.3) to obtain the hypothesis $\widehat{h}^{(b)}$. The hypothesis $\widehat{h}^{(b)}$ is a realization of a RV whose distribution is determined by the histogram $\hat{p}(\mathbf{z})$ as well as the hypothesis space and the loss function used in the ERM (4.3).

6.6 Diagnosing ML

diagnose ML methods by comparing training error with validation error and (if available) some baseline; baseline can be obtained via the Bayes risk when using a probabilistic model (such as the i.i.d. assumption) or human performance or the performance of existing ML methods ("experts" in regret framework)

In what follows, we tacitly assume that data points can (to a good approximation) be interpreted as realizations of i.i.d. RVs (see Section 2.1.4). This "i.i.d. assumption" underlies ERM (4.3) as the guiding principle for learning a hypothesis with small risk (4.1). This assumption also motivates to use the average loss (6.6) on a validation set as an estimate for the risk. More fundamentally, we need the i.i.d. assumption to define the concept of risk as a measure for how well a hypothesis predicts the labels of arbitrary data points.

Consider a ML method which uses Algorithm 5 (or Algorithm 6) to learn and validate the hypothesis $\widehat{h} \in \mathcal{H}$. Besides the learnt hypothesis $\widehat{h}$, these algorithms also deliver the training error E_t and the validation error E_v. As we will see shortly, we can diagnose ML methods to some extent just by comparing training with validation errors. This diagnosis is further enabled if we know a baseline $E^{(\mathrm{ref})}$.

One important source for a baseline $E^{(\mathrm{ref})}$ are probabilistic models for the data points (see Section 6.4). Given a probabilistic model, which specifies the probability distribution $p(\mathbf{x}, y)$ of the features and label of data points, we can compute the minimum achievable risk (4.1). Indeed, the minimum achievable risk is precisely the expected loss of the Bayes estimator $\widehat{h}(\mathbf{x})$ of the label y, given the features $\mathbf{x}$ of a data point. The Bayes estimator $\widehat{h}(\mathbf{x})$ is fully determined by the probability distribution $p(\mathbf{x}, y)$ of the features and label of a (random) data point [85, Chapter 4]. When using the squared error loss (2.8) to define the risk (4.1), the Bayes estimator is given by the posterior mean $\widehat{h}(\mathbf{x}) = \mathbb{E}\{y|\mathbf{x}\}$.

A further potential source for a baseline $E^{(\text{ref})}$ is an existing, but for some reason unsuitable, ML method. This existing ML method might be computationally too expensive to be used for the ML application at end. However, we might still use its statistical properties as a benchmark. The

We might also use the performance of human experts as a baseline. If we want to develop a ML method that detects certain type of skin cancers from images of the skin, a benchmark might be the current classification accuracy achieved by experienced dermatologists [36].

We can diagnose a ML method by comparing the training error E_t with the validation error E_v and (if available) the benchmark $E^{(\text{ref})}$.

- $E_t \approx E_v \approx E^{(\text{ref})}$: The training error is on the same level as the validation error and the benchmark error. There is not much to improve here since the validation error is already on the desired error level. Moreover, the training error is not much smaller than the validation error which indicates that there is no overfitting. It seems we have obtained a ML method that achieves the benchmark error level.

- $E_v \gg E_t$: The validation error is significantly larger than the training error. It seems that the ERM (4.3) results in a hypothesis $\widehat{h}$ that overfits the training set. The loss incurred by $\widehat{h}$ on data points outside the training set, such as those in the validation set, is significantly worse. This is an indicator for overfitting which can be addressed either by reducing the effective dimension of the hypothesis space or by increasing the size of the training set. To reduce the effective dimension of the hypothesis space we have different options depending on the used model. We might use a small number of features in a linear model (3.1), a smaller maximum depth of decision trees (Section 3.10) or a fewer layers in an ANN (Section 3.11). One very elegant means for reducing the effective dimension of a hypothesis space is by limiting the number of GD steps used in gradient-based methods. This optimization based shrinking of a hypothesis space is referred to as early stopping. More generally, we can reduce the effective dimension of a hypothesis space via regularization techniques (see Chapter 7).

- $E_t \approx E_v \gg E^{(\text{ref})}$: The training error is on the same level as the validation error and both are significantly larger than the baseline. Since the training error is not much smaller than the validation error, the learnt hypothesis seems to not overfit the training set. However, the training error achieved by the learnt hypothesis is significantly larger than the benchmark error level. There can be several reasons for this to happen. First, it might be that the hypothesis space used by the ML method is too small, i.e., it

does not include a hypothesis that provides a good approximation for the relation between features and label of a data point. The remedy for this situation is to use a larger hypothesis space, e.g., by including more features in a linear model, using higher polynomial degrees in polynomial regression, using deeper decision trees or having larger ANNs (deep nets). Another reason for the training error being too large is that the optimization algorithm used to solve ERM (4.3) is not working properly.

When using gradient-based methods (see Section 5.4) to solve ERM, one reason for $E_t \gg E^{(\text{ref})}$ could be that the learning rate α in the GD step (5.6) is chosen too small or too large (see Figure 5.3-(b)). This can be solved by adjusting the learning rate by trying out several different values and using the one resulting in the smallest training error. Another option is derive optimal values for the learning rate based on a probabilistic model for how the data points are generated. One example for such a probabilistic model is the i.i.d. assumption that has been used in Section 6.4 to analyze linear regression methods.

- $E_t \gg E_v$: The training error is significantly larger than the validation error (see Exercise 6.2). The idea of ERM (4.3) is to approximate the risk (4.1) of a hypothesis by its average loss on a training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$. The mathematical underpinning for this approximation is the law of large numbers which characterizes the average of (realizations of) i.i.d. RVs. The quality and usefulness of this approximation depends on the validity of two conditions. First, the data points used for computing the average loss should be such that they would be typically obtained as realizations of i.i.d. RVs with a common probability distribution. Second, the number of data points used for computing the average loss must be sufficiently large.

  Whenever the data points behave different than the the realizations of i.i.d. RVs or if the size of the training set or validation set is too small, the interpretation (and comparison) of the training error and the validation error of a learnt hypothesis becomes more difficult. As an extreme case, it might then be that the validation error consists of data points for which every hypothesis incurs small average loss. Here, we might try to increase the size of the validation set by collecting more labeled data points or by using data augmentation (see Section 7.3). If the size of training set and validation set are large but we still obtain $E_t \gg E_v$, one should verify if data points in these sets conform to the i.i.d. assumption. There are principled statistical test for the validity of the i.i.d. assumption for a given dataset (see [88] and references therein).

6.7 Exercises

Exercise 6.1. Validation Set Size. Consider a linear regression problem with data points (x, y) characterized by a scalar feature x and a numeric label y. Assume data points are realizations of i.i.d. RVs whose common probability distribution is multivariate normal with zero-mean and covariance matrix $\mathbf{C} = \begin{pmatrix} \sigma_x^2 & \sigma_{x,y} \\ \sigma_{x,y} & \sigma_y^2 \end{pmatrix}$. The entries of this covariance matrix are the variance σ_x^2 of the (zero-mean) feature, the variance σ_x^2 of the (zero-mean) label and the covariance between feature and label of a random data point. How many data points do we need to include in a validation set such that with probability of at least 0.8 the validation error of a given hypothesis h does not deviate by more than 20 percent from its expected loss?

Exercise 6.2. Validation Error Smaller Than Training Error? Linear regression learns a linear hypothesis map $\widehat{h}$ having minimal average squared error on a training set. The learnt hypothesis $\widehat{h}$ is then validated on a validation set which is different from the training set. Can you construct a training set and validation set such that the validation error of $\widehat{h}$ is strictly smaller than the training error of $\widehat{h}$?

Exercise 6.3. When is Validation Set Too Small? The usefulness of the validation error as an indicator for the performance of a hypothesis depends on the size of the validation set. Experiment with different ML methods and datasets to find out the minimum required size for the validation set.

Exercise 6.4. Too many Features? Consider data points that are characterized by $n = 1000$ numeric features $x_1, \ldots, x_n \in \mathbb{R}$ and a numeric label $y \in \mathbb{R}$. We want to learn a linear hypothesis map $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ for predicting the label of a data point based on its features. Could it be beneficial to constrain the learnt hypothesis by requiring it to only depend on the first 5 features of a data point?

Exercise 6.5. Benchmark via Probability Theory. Consider data points that are characterized with single numeric feature x and label y. We model the feature and label of a data point as i.i.d. realizations of a Gaussian random vector $\mathbf{z} \sim \mathcal{N}(0, \mathbf{C})$ with zero mean and covariance matrix $\mathbf{C}$. The optimal hypothesis $\widehat{h}(x)$ to predict the label y given the feature x is the conditional expectation of the (unobserved) label y given the (observed)

feature x. How is the expected squared error loss of this optimal hypothesis (which is the Bayes estimator) related to the covariance matrix $\mathbf{C}$ of the Gaussian random vector $\mathbf{z}$.

Chapter 7

Regularization

Keywords: Data Augmentation. Robustness. Semi-Supervised Learning. Transfer Learning. Multitask Learning.

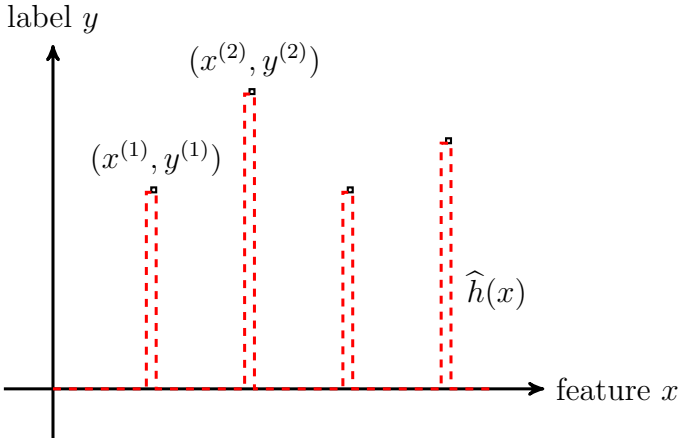


Figure 7.1: The non-linear hypothesis map $\widehat{h}$ perfectly predicts the labels of four data points in a training set and therefore has vanishing training error. Despite perfectly fitting the training set, the hypothesis $\widehat{h}$ delivers the trivial (and useless) prediction $\hat{y} = \widehat{h}(x) = 0$ for data points outside the training set. Regularization techniques help to prevent ML methods from learning such a map $\widehat{h}$.

Many ML methods use the principle of ERM (see Chapter 4) to learn a hypothesis out of a hypothesis space by minimizing the average loss (training error) on a set of labeled data points (which constitute a training set). Using ERM as a guiding principle for ML methods makes sense only if the training error is a good indicator for its loss incurred outside the training set.

Figure 7.1 illustrates a typical scenario for a modern ML method which uses a large hypothesis space. This large hypothesis space includes highly non-linear maps which can perfectly resemble any dataset of modest size. However, there might be non-linear maps for which a small training error does not guarantee accurate predictions for the labels of data points outside the training set.

Chapter 6 discussed validation techniques to verify if a hypothesis with small training error will predict also well the labels of data points outside the training set. These validation techniques, including Algorithm 5 and Algorithm 6, probe the hypothesis $\widehat{h} \in \mathcal{H}$ delivered by ERM on a validation set. The validation set consists of data points which have not been used in the training set of ERM (4.3). The validation error, which is the average loss of the hypothesis on the data points in the validation set, serves as an estimate for the average error or risk (4.1) of the hypothesis $\widehat{h}$.

This chapter discusses regularization as an alternative to validation techniques. In contrast to validation, regularization techniques do not require having a separate validation set which is not used for the ERM (4.3). This makes regularization attractive for applications where obtaining a separate validation set is difficult or costly (where labelled data is scarce).

Instead of probing a hypothesis $\widehat{h}$ on a validation set, regularization techniques estimate (or approximate) the loss increase when applying $\widehat{h}$ to data points outside the training set. The loss increase is estimated by adding a regularization term to the training error in ERM (4.3).

Section 7.1 discusses the resulting regularized ERM, which we will refer to as structural risk minimization (SRM). It turns out that the SRM is equivalent to ERM using a smaller (pruned) hypothesis space. The amount of pruning depends on the weight of the regularization term relative to the training error. For an increasing weight of the regularization term, we obtain a stronger pruning resulting in a smaller effective hypothesis space.

Section 7.2 constructs regularization terms by requiring the resulting ML method to be robust against (small) random perturbations of the data points in a training set. Here, we replace each data point of a training set by the realization of a RV that fluctuates around this data point. This construction allows to interpret regularization as a (implicit) form of data augmentation.

Section 7.3 discusses data augmentation methods as a simulation-based implementation of regularization. Data augmentation adds a certain number of perturbed copies to each data point in the training set. One way to construct perturbed copies of a data point is to add the realization of a RV to its features.

Section 7.4 analyzes the effect of regularization for linear regression using a simple probabilistic model for data points. This analysis parallels our previous study of the validation error of linear regression in Section 6.4. Similar to Section 6.4, we reveal a trade-off between the bias and variance of the hypothesis learnt by regularized linear regression. This trade- off was traced out by a discrete model parameter (the effective dimension) in Section 6.4. In contrast, regularization offers a continuous trade-off between bias and variance via a continuous regularization parameter.

Semi-supervised learning (SSL) uses (large amounts of) unlabeled data points to support the learning of a hypothesis from (a small number of) labeled data points [21]. Section 7.5 discusses SSL methods that use the statistical properties of unlabeled data points to construct useful regularization terms. These regularization terms are then used in SRM with a (typically small) set of labeled data points.

Multitask learning exploits similarities between different but similar learning tasks [19]. We can formally define a learning task by a particular choice for the loss function (see Section 2.3) . The primary role of a loss function is to score the quality of a hypothesis map. However, the loss function also encapsulates the choice for the label of a data point. For learning tasks defined for a single underlying data generation process it is reasonable to assume that the same subset of features is relevant for those learning tasks. One example for a ML application involving several similar learning tasks is multi-label classification (see Section 2.1.2). Each individual label of a data point represents a separate learning task. Section 7.6 shows how multitask learning can be implemented using regularization methods. The loss incurred in different learning tasks serves mutual regularization terms in a joint SRM for all learning tasks.

Section 7.7 shows how regularization can be used for transfer learning. Like multitask learning, also transfer learning exploits relations between different but similar learning tasks. In contrast to multitask learning, which jointly solves the individual learning tasks, transfer learning solves the learning tasks sequentially. One example of transfer learning is to fine tune a pre-trained model. A pre-trained model can be obtained via ERM (4.3) in a ("source") learning task for which we have a large amount of labeled training data. The fine-tuning is then obtained via ERM (4.3) in the ("target") learning task of interest for which we might have only a small amount of labeled training data.

7.1 Structural Risk Minimization

Section 2.2 defined the effective dimension $d_{\rm eff}(\mathcal{H})$ of a hypothesis space $\mathcal{H}$ as the maximum number of data points that can be perfectly fit by some hypothesis $h \in \mathcal{H}$. As soon as the effective dimension of the hypothesis space in (4.3) exceeds the number m of training data points, we can find a hypothesis that perfectly fits the training data. However, a hypothesis that perfectly fits the training data might deliver poor predictions for data points outside the training set (see Figure 7.1).

Modern ML methods typically use a hypothesis space with large effective dimension [151, 17]. Two well-known examples for such methods is linear regression (see Section 3.1) using a large number of features and deep learning with ANNs using a large number (billions) of artificial neurons (see Section 3.11). The effective dimension of these methods can be easily on the order of billions (10^9) if not larger [126]. To avoid overfitting during the naive use of ERM (4.3) we would require a training set containing at least as many data points as the effective dimension of the hypothesis space. However, in practice we often do not have access to a training set consisting of billions of labeled data points. The challenge is typically in the labelling process which often requires human labour.

It seems natural to combat overfitting of a ML method by pruning its hypothesis space $\mathcal{H}$. We prune $\mathcal{H}$ by removing some of the hypothesis in $\mathcal{H}$ to obtain the smaller hypothesis space $\mathcal{H}' \subset \mathcal{H}$. We then replace ERM (4.3) with the restricted (or pruned) ERM

$$\widehat{h} = \operatorname*{argmin}_{h \in \mathcal{H}'} \widehat{L}(h|\mathcal{D}) \text{ with pruned hypothesis space } \mathcal{H}' \subset \mathcal{H}. \tag{7.1}$$

The effective dimension of the pruned hypothesis space $\mathcal{H}'$ is typically much smaller than the effective dimension of the original (large) hypothesis space $\mathcal{H}$, $d_{\rm eff}(\mathcal{H}') \ll d_{\rm eff}(\mathcal{H})$. For a given size m of the training set, the risk of overfitting in (7.1) is much smaller than the risk of overfitting in (4.3).

Let us illustrate the idea of pruning for linear regression using the hypothesis space (3.1) constituted by linear maps $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$. The effective dimension of (3.1) is equal to the number of features, $d_{\rm eff}(\mathcal{H}) = n$. The hypothesis space $\mathcal{H}$ might be too large if we use a large number n of features, leading to overfitting. We prune (3.1) by retaining only linear hypothesis maps $h(\mathbf{x}) = \big(\mathbf{w}'\big)^T\mathbf{x}$ whose weight vector $\mathbf{w}'$ satisfies $w'_3 = w'_4 = \ldots = w'_n = 0$. Thus, the hypothesis space $\mathcal{H}'$ is constituted by all linear maps that only depend on the first two features x_1, x_2 of a data point. The effective dimension of $\mathcal{H}'$ is $d_{\rm eff}(\mathcal{H}') = 2$ instead of

$d_{\rm eff}\left(\mathcal{H}\right) = n$.

Pruning the hypothesis space is a special case of a more general strategy which some authors refer to as SRM [145]. The idea behind SRM is to modify the training error in ERM (4.3) to favour a hypothesis that is more smooth or regular. By steering ML methods towards learning a smooth hypothesis, we make it less sensitive, or more robust, to small perturbations of data points in the training set. Section 7.2 discusses the intimate relation between the robustness (against perturbations of the data points in the training set) of a ML method and its ability to generalize to data points outside the training set.

We measure the smoothness of a hypothesis using a regularizer $\mathcal{R}(h) \in \mathbb{R}_+$. The value $\mathcal{R}(h)$ is a quantitative measure for the non-smoothness or irregularity of the hypothesis h. The (design) choice for the regularizer depends on the precise definition of what is meant by regularity or variation of a hypothesis. Section 7.3 discusses how a particular choice for the regularizer $\mathcal{R}(h)$ arises naturally from a probabilistic model for data points.

We obtain SRM by adding the scaled regularizer $\lambda \mathcal{R}(h)$ to the ERM (4.3) ,

$$\begin{aligned} \widehat{h} &= \operatorname*{argmin}_{h \in \mathcal{H}} \big[\widehat{L}(h|\mathcal{D}) + \lambda \mathcal{R}(h)\big] \\ &\stackrel{(2.16)}{=} \operatorname*{argmin}_{h \in \mathcal{H}} \big[(1/m) \sum_{i=1}^{m} L\left((\mathbf{x}^{(i)}, y^{(i)}), h\right) + \lambda \mathcal{R}(h)\big]. \end{aligned} \tag{7.2}$$

We can interpret the penalty term $\lambda \mathcal{R}(h)$ in (7.2) as an estimate (or approximation) for the increase, relative to the training error on $\mathcal{D}$, of the average loss of a hypothesis $\widehat{h}$ when applied to data points outside $\mathcal{D}$. Another interpretation of the term $\lambda \mathcal{R}(h)$ will be discussed in Section 7.3.

The regularization parameter λ allows us to trade between a small training error $\widehat{L}(h^{(\mathbf{w})}|\mathcal{D})$ and small regularization term $\mathcal{R}(h)$, which enforces smoothness or regularity of h. If we choose a large value for λ, we heavily punish any "irregular" hypothesis h with large $\mathcal{R}(h)$ (see (7.2)). Thus, increasing the value of λ results in the solution (minimizer) of (7.2) having smaller $\mathcal{R}(h)$. On the other hand, choosing a small value for λ in (7.2) puts more emphasis on obtaining a hypothesis h incurring a small training error. For the extreme case $\lambda = 0$, the SRM (7.2) reduces to ERM (4.3).

The pruning approach (7.1) is intimately related to the SRM (7.2). They are, in a certain sense, **dual** to each other. First, note that (7.2) reduces to the pruning approach (7.1) when using the regularizer $\mathcal{R}(h) = 0$ for all $h \in \mathcal{H}'$, and $\mathcal{R}(h) = \infty$ otherwise, in (7.2). In the other direction, for many important choices for the regularizer $\mathcal{R}(h)$, there is a restriction

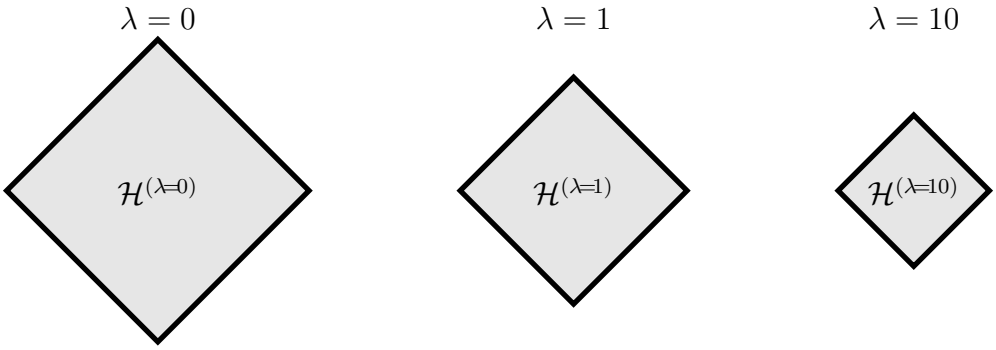


Figure 7.2: Adding the scaled regularizer $\lambda\mathcal{R}(h)$ to the training error in the objective function of SRM (7.2) is equivalent to solving ERM (7.1) with a pruned hypothesis space $\mathcal{H}^{(\lambda)}$.

$\mathcal{H}^{(\lambda)} \subset \mathcal{H}$ such that the solutions of (7.1) and (7.2) coincide (see Figure 7.2). The relation between the optimization problems (7.1) and (7.2) can be made precise using the theory of convex duality (see [14, Ch. 5] and [10]).

For a hypothesis space $\mathcal{H}$ whose elements $h \in \mathcal{H}$ are parametrized by a parameter vector $\mathbf{w} \in \mathbb{R}^n$, we can rewrite SRM (7.2) as

$$\begin{aligned}\widehat{\mathbf{w}}^{(\lambda)} &= \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^n} \big[\widehat{L}(h^{(\mathbf{w})}|\mathcal{D}) + \lambda\mathcal{R}(\mathbf{w})\big] \\ &= \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^n} \big[(1/m)\sum_{i=1}^{m} L\left((\mathbf{x}^{(i)}, y^{(i)}), h^{(\mathbf{w})}\right) + \lambda\mathcal{R}(\mathbf{w})\big]. \end{aligned} \tag{7.3}$$

For the particular choice of squared error loss (2.8), linear hypothesis space (3.1) and regularizer $\mathcal{R}(\mathbf{w}) = \|\mathbf{w}\|_2^2$, SRM (7.3) specializes to

$$\widehat{\mathbf{w}}^{(\lambda)} = \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^n} \big[(1/m)\sum_{i=1}^{m} \left(y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)}\right)^2 + \lambda\|\mathbf{w}\|_2^2\big]. \tag{7.4}$$

The special case (7.4) of SRM (7.3) is known as ridge regression [58].

Ridge regression (7.4) is equivalent to (see [10, Ch. 5])

$$\widehat{\mathbf{w}}^{(\lambda)} = \operatorname*{argmin}_{h^{(\mathbf{w})}\in\mathcal{H}^{(\lambda)}} (1/m)\sum_{i=1}^{m} \left(y^{(i)} - h^{(\mathbf{w})}(\mathbf{x}^{(i)})\right)^2 \tag{7.5}$$

with the restricted hypothesis space

$$\mathcal{H}^{(\lambda)} := \{h^{(\mathbf{w})} : \mathbb{R}^n \to \mathbb{R} : h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}, \text{ with some } \mathbf{w} \in \mathbb{R}^n, \|\mathbf{w}\|_2^2 \leq C(\lambda)\} \subset \mathcal{H}^{(n)}. \quad (7.6)$$

For any given value λ of the regularization parameter in (7.4), there is a number $C(\lambda)$ such that solutions of (7.4) coincide with the solutions of (7.5). Thus, ridge regression (7.4) is equivalent to linear regression with a pruned version $\mathcal{H}^{(\lambda)}$ of the linear hypothesis space (3.1). The size of the pruned hypothesis space $\mathcal{H}^{(\lambda)}$ (7.6) varies continuously with λ.

Another popular special case of ERM (7.3) is obtained for the regularizer $\mathcal{R}(\mathbf{w}) = \|\mathbf{w}\|_1$ and known as the Lasso [59]

$$\widehat{\mathbf{w}}^{(\lambda)} = \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^n} \big[(1/m)\sum_{i=1}^{m} \big(y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)}\big)^2 + \lambda\|\mathbf{w}\|_1\big]. \quad (7.7)$$

Ridge regression (7.4) and the Lasso (7.7) have fundamentally different computational and statistical properties. Ridge regression (7.4) uses a smooth and convex objective function that can be minimized using efficient GD methods. The objective function of Lasso (7.7) is also convex but non-smooth and therefore requires more advanced optimization methods. The increased computational complexity of Lasso (7.7) comes at the benefit of typically delivering a hypothesis with a smaller expected loss than those obtained from ridge regression [17, 59].

7.2 Robustness

Section 7.1 motivates regularization as a soft variant of model selection. Indeed, the regularization term in SRM (7.2) is equivalent to ERM (7.1) using a pruned hypothesis space. We now discuss an alternative view on regularization as a means to make ML methods robust.

The ML methods discussed in Chapter 4 rest on the idealizing assumption that we have access to the true label values and feature values of labeled data points (that form a training set). These methods learn a hypothesis $h \in \mathcal{H}$ with minimum average loss (training error) incurred for data points in the training set. In practice, the acquisition of label and feature values might be prone to errors. These errors might stem from the measurement device itself (hardware failures or thermal noise in electronic devices) or might be due to human mistakes such as labelling errors.

Let us assume for the sake of exposition that the label values $y^{(i)}$ in the training set are accurate but that the features $\mathbf{x}^{(i)}$ are a perturbed version of the true features of the ith

data point. Thus, instead of having observed the data point $\big(\mathbf{x}^{(i)}, y^{(i)}\big)$ we could have equally well observed the data point $\big(\mathbf{x}^{(i)} + \boldsymbol{\varepsilon}, y^{(i)}\big)$ in the training set. Here, we have modelled the perturbations in the features using a RV $\boldsymbol{\varepsilon}$. The probability distribution of the perturbation $\boldsymbol{\varepsilon}$ is a design parameter that controls robustness properties of the overall ML method. We will study a particular choice for this distribution in Section 7.3.

A robust ML method should learn a hypothesis that incurs a small loss not only for a specific data point $\big(\mathbf{x}^{(i)}, y^{(i)}\big)$ but also for perturbed data points $\big(\mathbf{x}^{(i)} + \boldsymbol{\varepsilon}, y^{(i)}\big)$. Therefore, it seems natural to replace the loss $L\big(\big(\mathbf{x}^{(i)}, y^{(i)}\big), h\big)$, incurred on the ith data point in the training set, with the expectation

$$\mathbb{E}\big\{ L\big(\big(\mathbf{x}^{(i)} + \boldsymbol{\varepsilon}, y^{(i)}\big), h\big) \big\}. \tag{7.8}$$

The expectation (7.8) is computed using the probability distribution of the perturbation $\boldsymbol{\varepsilon}$. We will show in Section 7.3 that minimizing the average of the expectation (7.8), for $i = 1, \ldots, m$, is equivalent to the SRM (7.2).

Using the expected loss (7.8) is not the only possible approach to make a ML method robust. Another approach to make a ML method robust is known as bagging (or bootstrap aggregation). The idea of bagging is to use the bootstrap method (see Section 6.5 and [58, Ch. 8]) to construct a finite number of perturbed copies $\mathcal{D}^{(1)}, \ldots, \mathcal{D}^{(B)}$ of the original training set $\mathcal{D}$.

We then learn (e.g, using ERM) a separate hypothesis $h^{(b)}$ for each perturbed copy $\mathcal{D}^{(b)}$, $b = 1, \ldots, B$. This results in a whole ensemble of different hypotheses $h^{(b)}$ which might even belong to different hypothesis spaces. For example, one the hypothesis $h^{(1)}$ could be a linear map (see Section 3.1) and the hypothesis $h^{(2)}$ could be obtained from an ANN (see Section 3.11).

The final hypothesis delivered by bagging is obtained by combining or aggregating (e.g., using the average) the predictions $h^{(b)}\big(\mathbf{x}\big)$ delivered by each hypothesis $h^{(b)}$, for $b = 1, \ldots, B$ in the ensemble. The ML method referred to as random forest uses bagging to learn an ensemble of decision trees (see Chapter 3.10). The individual predictions obtained from the different decision trees forming a random forest are then combined (e.g., using an average for numeric labels or a majority vote for finite-valued labels), to obtain a final prediction [58].

7.3 Data Augmentation

ML methods using ERM (4.3) are prone to overfitting as soon as the effective dimension of the hypothesis space $\mathcal{H}$ exceeds the number m of data points in the training set. Section 6.3 and Section 7.1 approached this by modifying either the model or the loss function by adding a regularization term. Both approaches prune the hypothesis space $\mathcal{H}$ underlying a ML method to reduce the effective dimension $d_{\text{eff}}(\mathcal{H})$. Model selection does this reduction in a discrete fashion while regularization implements a soft "shrinking" of the hypothesis space.

Instead of trying to reduce the effective dimension we could also try to increase the number m of data points in the training set used for ERM (4.3). We now discuss how to synthetically generate new labeled data points by exploiting statistical symmetries of data.

The data arising in many ML applications exhibit intrinsic symmetries and invariances at least in some approximation. The rotated image of a cat still shows a cat. The temperature measurement taken at a given location will be similar to another measurement taken 10 milliseconds later. Data augmentation exploits such symmetries and invariances to augment the raw data with additional synthetic data.

Let us illustrate data augmentation using an application that involves data points characterized by features $\mathbf{x} \in \mathbb{R}^n$ and number labels $y \in \mathbb{R}$. We assume that the data generating process is such that data points with close feature values have the same label. Equivalently, this assumption is requiring the resulting ML method to be robust against small perturbations of the feature values (see Section 7.2). This suggests to augment a data point $\big(\mathbf{x}, y\big)$ by several synthetic data points

$$\big(\mathbf{x} + \boldsymbol{\varepsilon}^{(1)}, y\big), \ldots, \big(\mathbf{x} + \boldsymbol{\varepsilon}^{(B)}, y\big), \tag{7.9}$$

with $\boldsymbol{\varepsilon}^{(1)}, \ldots, \boldsymbol{\varepsilon}^{(B)}$ being realizations of i.i.d. random vectors with the same probability distribution $p(\boldsymbol{\varepsilon})$.

Given a (raw) dataset $\mathcal{D} = \big\{\big(\mathbf{x}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big)\big\}$ we denote the associated aug-

mented dataset by

$$\begin{aligned}\mathcal{D}' = \big\{ &\big(\mathbf{x}^{(1,1)}, y^{(1)}\big), \ldots, \big(\mathbf{x}^{(1,B)}, y^{(1)}\big), \\ &\big(\mathbf{x}^{(2,1)}, y^{(2)}\big), \ldots, \big(\mathbf{x}^{(2,B)}, y^{(2)}\big), \\ &\ldots \\ &\big(\mathbf{x}^{(m,1)}, y^{(m)}\big), \ldots, \big(\mathbf{x}^{(m,B)}, y^{(m)}\big)\big\}. \end{aligned} \tag{7.10}$$

The size of the augmented dataset $\mathcal{D}'$ is $m' = B \times m$. For a sufficiently large augmentation parameter B, the augmented sample size m' is larger than the effective dimension n of the hypothesis space $\mathcal{H}$. We then learn a hypothesis via ERM on the augmented dataset,

$$\begin{aligned}\widehat{h} &= \operatorname*{argmin}_{h \in \mathcal{H}} \widehat{L}(h|\mathcal{D}') \\ &\overset{(7.10)}{=} \operatorname*{argmin}_{h \in \mathcal{H}} (1/m') \sum_{i=1}^{m} \sum_{b=1}^{B} L\big((\mathbf{x}^{(i,b)}, y^{(i,b)}), h\big) \\ &\overset{(7.9)}{=} \operatorname*{argmin}_{h \in \mathcal{H}} (1/m) \sum_{i=1}^{m} (1/B) \sum_{b=1}^{B} L\big((\mathbf{x}^{(i)} + \boldsymbol{\varepsilon}^{(b)}, y^{(i)}), h\big). \end{aligned} \tag{7.11}$$

We can interpret data-augmented ERM (7.11) as a data-driven form of regularization (see Section 7.1): For each data point $\big(\mathbf{x}^{(i)}, y^{(i)}\big) \in \mathcal{D}$ in $\mathcal{D}$, we replace the corresponding loss $L\big((\mathbf{x}^{(i)}, y^{(i)}), h\big)$ with the average loss $(1/B)\sum_{b=1}^{B} L\big((\mathbf{x}^{(i)} + \boldsymbol{\varepsilon}^{(b)}, y^{(i)}), h\big)$. This average loss is computed over the augmented data points arising from $\big(\mathbf{x}^{(i)}, y^{(i)}\big) \in \mathcal{D}$.

The actual implementation of (7.11) require to first generate B realizations $\boldsymbol{\varepsilon}^{(b)} \in \mathbb{R}^n$ of i.i.d. RVs with common probability distribution $p(\boldsymbol{\varepsilon})$. This generation might be computationally costly for large values of B or n. However, when using a large augmentation parameter B, we might use the approximation

$$(1/B) \sum_{b=1}^{B} L\big((\mathbf{x}^{(i)} + \boldsymbol{\varepsilon}^{(b)}, y^{(i)}), h\big) \approx \mathbb{E}\big\{ L\big((\mathbf{x}^{(i)} + \boldsymbol{\varepsilon}, y^{(i)}), h\big) \big\}. \tag{7.12}$$

This approximation is made precise by a key result of probability theory, known as the law of large numbers. We obtain an instance of ERM by inserting (7.12) into (7.11),

$$\widehat{h} = \operatorname*{argmin}_{h \in \mathcal{H}} (1/m) \sum_{i=1}^{m} \mathbb{E}\big\{ L\big((\mathbf{x}^{(i)} + \boldsymbol{\varepsilon}, y^{(i)}), h\big) \big\}. \tag{7.13}$$

The usefulness of (7.13) as an approximation to the augmented ERM (7.11) depends on the difficulty of computing the expectation $\mathbb{E}\big\{ L\left((\mathbf{x}^{(i)}+\boldsymbol{\varepsilon}, y^{(i)}), h\right)\big\}$. The complexity of computing this expectation depends on the choice of loss function and the choice for the probability distribution $p(\boldsymbol{\varepsilon})$.

Let us study (7.13) for the special case linear regression with squared error loss (2.8) and linear hypothesis space (3.1),

$$\widehat{h} = \underset{h(\mathbf{w})\in\mathcal{H}^{(n)}}{\operatorname{argmin}} (1/m) \sum_{i=1}^{m} \mathbb{E}\big\{ \big(y^{(i)} - \mathbf{w}^T\big(\mathbf{x}^{(i)}+\boldsymbol{\varepsilon}\big)\big)^2\big\}. \tag{7.14}$$

We use perturbations $\boldsymbol{\varepsilon}$ drawn from a multivariate normal distribution with zero mean and covariance matrix $\sigma^2\mathbf{I}$,

$$\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I}). \tag{7.15}$$

We develop (7.14) further by using

$$\mathbb{E}\{\big(y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)}\big)\boldsymbol{\varepsilon}\} = \mathbf{0}. \tag{7.16}$$

The identity (7.16) uses that the data points $\big(\mathbf{x}^{(i)}, y^{(i)}\big)$ are fixed and known (deterministic) while $\boldsymbol{\varepsilon}$ is a zero-mean random vector. Combining (7.16) with (7.14),

$$\begin{aligned}\mathbb{E}\big\{ \big(y^{(i)} - \mathbf{w}^T\big(\mathbf{x}^{(i)}+\boldsymbol{\varepsilon}\big)\big)^2\big\} &= \big(y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)}\big)^2 + \|\mathbf{w}\|_2^2\, \mathbb{E}\big\{\, \|\boldsymbol{\varepsilon}\|_2^2\,\big\} \\ &= \big(y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)}\big)^2 + n\,\|\mathbf{w}\|_2^2\,\sigma^2. \end{aligned} \tag{7.17}$$

where the last step used $\mathbb{E}\big\{\, \|\boldsymbol{\varepsilon}\|_2^2\,\big\} \overset{(7.15)}{=} n\sigma^2$. Inserting (7.17) into (7.14),

$$\widehat{h} = \underset{h(\mathbf{w})\in\mathcal{H}^{(n)}}{\operatorname{argmin}} (1/m) \sum_{i=1}^{m} \big(y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)}\big)^2 + n\,\|\mathbf{w}\|_2^2\,\sigma^2. \tag{7.18}$$

We have obtained (7.18) as an approximation of the augmented ERM (7.11) for the special case of squared error loss (2.8) and the linear hypothesis space (3.1). This approximation uses the law of large numbers (7.12) and becomes more accurate for increasing augmentation parameter B.

Note that (7.18) is nothing but ridge regression (7.4) using the regularization parameter $\lambda = n\sigma^2$. Thus, we can interpret ridge regression as implicit data augmentation (7.10) by applying random perturbations (7.9) to the feature vectors in the original training set $\mathcal{D}$.

The regularizer $\mathcal{R}(\mathbf{w}) = \|\mathbf{w}\|_2^2$ in (7.18) arose naturally from the specific choice for the probability distribution (7.15) of the random perturbation $\boldsymbol{\varepsilon}^{(i)}$ in (7.9) and using the squared error loss. Other choices for this probability distribution or the loss function result in different regularizers.

Augmenting data points with random perturbations distributed according (7.15) treat the features of a data point independently. For application domains that generate data points with highly correlated features it might be useful to augment data points using random perturbations $\boldsymbol{\varepsilon}$ (see (7.9)) distributed as

$$\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{C}). \tag{7.19}$$

The covariance matrix $\mathbf{C}$ of the perturbation $\boldsymbol{\varepsilon}$ can be chosen using domain expertise or estimated (see Section 7.5). Inserting the distribution (7.19) into (7.13),

$$\widehat{h} = \operatorname*{argmin}_{h(\mathbf{w}) \in \mathcal{H}^{(n)}} \Big[(1/m) \sum_{i=1}^{m} \big(y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)}\big)^2 + \mathbf{w}^T \mathbf{C} \mathbf{w} \Big]. \tag{7.20}$$

Note that (7.20) reduces to ordinary ridge regression (7.18) for the choice $\mathbf{C} = \sigma^2 \mathbf{I}$.

7.4 Statistical and Computational Aspects of Regularization

The goal of this section is to develop a better understanding for the effect of the regularization term in SRM (7.3). We will analyze the solutions of ridge regression (7.4) which is the special case of SRM using the linear hypothesis space (3.1) and squared error loss (2.8). Using the feature matrix $\mathbf{X} = \big(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\big)^T$ and label vector $\mathbf{y} = (y^{(1)}, \ldots, y^{(m)})^T$, we can rewrite (7.4) more compactly as

$$\widehat{\mathbf{w}}^{(\lambda)} = \operatorname*{argmin}_{\mathbf{w} \in \mathbb{R}^n} \big[(1/m) \, \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \, \|\mathbf{w}\|_2^2 \big]. \tag{7.21}$$

The solution of (7.21) is given by [134, Sec. 3.3.]

$$\widehat{\mathbf{w}}^{(\lambda)} = (1/m)\big((1/m)\mathbf{X}^T\mathbf{X} + \lambda \mathbf{I}\big)^{-1} \mathbf{X}^T \mathbf{y}. \tag{7.22}$$

For $\lambda = 0$, (7.22) reduces to the formula (6.18) for the optimal weights in linear regression (see (7.4) and (4.5)). Note that for $\lambda > 0$, the formula (7.22) is always valid, even when $\mathbf{X}^T\mathbf{X}$ is singular (not invertible). For $\lambda > 0$ the optimization problem (7.21) (and (7.4)) has the unique solution (7.22).

To study the statistical properties of the predictor $h^{(\widehat{\mathbf{w}}^{(\lambda)})}(\mathbf{x}) = \big(\widehat{\mathbf{w}}^{(\lambda)}\big)^T\mathbf{x}$ (see (7.22)) we use the probabilistic toy model (6.13), (6.16) and (6.17) that we used already in Section 6.4. We interpret the training data $\mathcal{D}^{(\text{train})} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$ as realizations of i.i.d. RVs whose distribution is defined by (6.13), (6.16) and (6.17).

We can then define the average prediction error of ridge regression as

$$E_{\text{pred}}^{(\lambda)} := \mathbb{E}\Bigg\{\bigg(y - h^{(\widehat{\mathbf{w}}^{(\lambda)})}(\mathbf{x})\bigg)^2\Bigg\}. \tag{7.23}$$

As shown in Section 6.4, the error $E_{\text{pred}}^{(\lambda)}$ is the sum of three components: the bias, the variance and the noise variance σ^2 (see (6.30)). The bias of $\widehat{\mathbf{w}}^{(\lambda)}$ is

$$B^2 = \mathbb{E}\Bigg\{\Big\|(\mathbf{I} - (\mathbf{X}^T\mathbf{X} + m\lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{X})\overline{\mathbf{w}}\Big\|_2^2\Bigg\}. \tag{7.24}$$

For sufficiently large size m of the training set, we can use the approximation

$$\mathbf{X}^T\mathbf{X} \approx m\mathbf{I} \tag{7.25}$$

such that (7.24) can be approximated as

$$\begin{aligned} B^2 &\approx \Big\|(\mathbf{I} - (\mathbf{I} + \lambda\mathbf{I})^{-1})\overline{\mathbf{w}}\Big\|_2^2 \\ &= \sum_{j=1}^n \frac{\lambda}{1+\lambda}\overline{w}_j^2. \end{aligned} \tag{7.26}$$

Let us compare the (approximate) bias term (7.26) of ridge regression with the bias term (6.24) of ordinary linear regression (which is the extreme case of ridge regression with $\lambda = 0$). The bias term (7.26) increases with increasing regularization parameter λ in ridge regression (7.4). Sometimes the increase in bias is outweighed by the reduction in variance. The variance typically decreases with increasing λ as shown next.

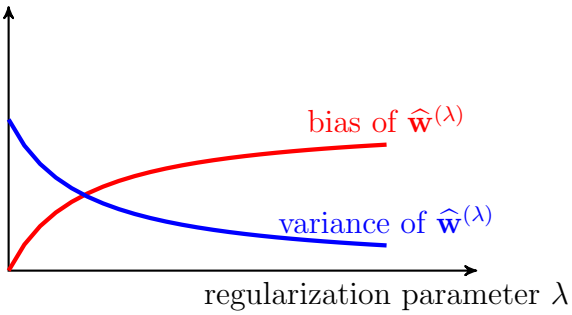


Figure 7.3: The bias and variance of ridge regression (7.4) depend on the regularization parameter λ in an opposite manner resulting in a bias-variance trade-off.

The variance of ridge regression (7.4) satisfies

$$V = (\sigma^2/m^2)\times \\ \mathrm{tr}\big\{\mathbb{E}\{((1/m)\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{X}((1/m)\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})^{-1}\}\big\}. \tag{7.27}$$

Inserting the approximation (7.25) into (7.27),

$$V \approx (\sigma^2/m^2)\mathrm{tr}\big\{\mathbb{E}\{(\mathbf{I}+\lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{X}(\mathbf{I}+\lambda\mathbf{I})^{-1}\}\big\} = \sigma^2(1/m)(n/(1+\lambda)^2). \tag{7.28}$$

According to (7.28), the variance of $\widehat{\mathbf{w}}^{(\lambda)}$ decreases with increasing regularization parameter λ of ridge regression (7.4). This is the opposite behaviour as observed for the bias (7.26), which increases with increasing λ. By comparing the variance approximation (7.28) with the variance (6.28) of linear regression suggests to interpret the ratio $n/(1+\lambda)^2$ as an effective number of features used by ridge regression. Increasing the regularization parameter λ decreases the effective number of features.

Figure 7.3 illustrates the trade-off between the bias B^2 (7.26) of ridge regression, which increases for increasing λ, and the variance V (7.28) which decreases with increasing λ. Note that we have seen another example for a bias-variance trade-off in Section 6.4. This trade-off was traced out by a discrete (model complexity) parameter $l \in \{1, 2, \ldots\}$ (see (6.14)). In stark contrast to discrete model selection, the bias-variance trade-off for ridge regression is traced out by the continuous regularization parameter $\lambda \in \mathbb{R}_+$.

The main statistical effect of the regularization term in ridge regression is to balance the bias with the variance to minimize the average prediction error of the learnt hypothesis. There is also a computational effect or adding a regularization term. Roughly speaking, the

regularization term serves as a pre-conditioning of the optimization problem and, in turn, reduces the computational complexity of solving ridge regression (7.21).

The objective function in (7.21) is a smooth (infinitely often differentiable) convex function. We can therefore use GD to solve (7.21) efficiently (see Chapter 5). Algorithm 8 summarizes the application of GD to (7.21). The computational complexity of Algorithm 8 depends crucially on the number of GD iterations required to reach a sufficiently small neighbourhood of the solutions to (7.21). Adding the regularization term $\lambda\|\mathbf{w}\|_2^2$ to the objective function of linear regression speeds up GD. To verify this claim, we first rewrite (7.21) as the quadratic problem

$$\min_{\mathbf{w}\in\mathbb{R}^n} \underbrace{(1/2)\mathbf{w}^T\mathbf{Q}\mathbf{w} - \mathbf{q}^T\mathbf{w}}_{=f(\mathbf{w})} \text{ with } \mathbf{Q} = (1/m)\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}, \mathbf{q} = (1/m)\mathbf{X}^T\mathbf{y}. \tag{7.29}$$

This is similar to the quadratic optimization problem (4.9) underlying linear regression but with a different matrix $\mathbf{Q}$. The computational complexity (number of iterations) required by GD (see (5.6)) to solve (7.29) up to a prescribed accuracy depends crucially on the condition number $\kappa(\mathbf{Q}) \geq 1$ of the psd matrix $\mathbf{Q}$ [68]. The smaller the condition number $\kappa(\mathbf{Q})$, the fewer iterations are required by GD. We refer to a matrix with a small condition number as being "well-conditioned".

The condition number of the matrix $\mathbf{Q}$ in (7.29) is given by

$$\kappa(\mathbf{Q}) = \frac{\lambda_{\max}((1/m)\mathbf{X}^T\mathbf{X}) + \lambda}{\lambda_{\min}((1/m)\mathbf{X}^T\mathbf{X}) + \lambda}. \tag{7.30}$$

According to (7.30), the condition number $\kappa(\mathbf{Q})$ tends to one for increasing regularization parameter λ,

$$\lim_{\lambda\to\infty} \frac{\lambda_{\max}((1/m)\mathbf{X}^T\mathbf{X}) + \lambda}{\lambda_{\min}((1/m)\mathbf{X}^T\mathbf{X}) + \lambda} = 1. \tag{7.31}$$

Thus, the number of required GD iterations in Algorithm 8 decreases with increasing regularization parameter λ.

7.5 Semi-Supervised Learning

Consider the task of predicting the numeric label y of a data point $\mathbf{z} = \big(\mathbf{x}, y\big)$ based on its feature vector $\mathbf{x} = \big(x_1, \ldots, x_n\big)^T \in \mathbb{R}^n$. At our disposal are two datasets $\mathcal{D}^{(u)}$ and $\mathcal{D}^{(l)}$.

Algorithm 8 Regularized Linear regression via GD

Input: dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$; GD learning rate $\alpha > 0$.
Initialize:set $\mathbf{w}^{(0)} := \mathbf{0}$; set iteration counter $r := 0$
1: **repeat**
2: $r := r + 1$ (increase iteration counter)
3: $\mathbf{w}^{(r)} := (1 - \alpha\lambda)\mathbf{w}^{(r-1)} + \alpha(2/m)\sum_{i=1}^{m}(y^{(i)} - \big(\mathbf{w}^{(r-1)}\big)^{T}\mathbf{x}^{(i)})\mathbf{x}^{(i)}$ (do a GD step (5.6))
4: **until** stopping criterion met
Output: $\mathbf{w}^{(r)}$ (which approximates $\widehat{\mathbf{w}}^{(\lambda)}$ in (7.21))

For each datapoint in $\mathcal{D}^{(u)}$ we only know the feature vector. We therefore refer to $\mathcal{D}^{(u)}$ as "unlabelled data". For each datapoint in $\mathcal{D}^{(l)}$ we know both, the feature vector $\mathbf{x}$ and the label y. We therefore refer to $\mathcal{D}^{(l)}$ as "labeled data".

SSL methods exploit the information provided by unlabelled data $\mathcal{D}^{(u)}$ to support the learning of a hypothesis based on minimizing its empirical risk on the labelled (training) data $\mathcal{D}^{(l)}$. The success of SSL methods depends on the statistical properties of the data generated within a given application domain. Loosely speaking, the information provided by the probability distribution of the features must be relevant for the ultimate task of predicting the label y from the the features $\mathbf{x}$ [21].

Let us design a SSL method, summarized in Algorithm 9 below, using the data augmentation perspective from Section 7.3. The idea is to augment the (small) labeled dataset $\mathcal{D}^{(l)}$ by adding random perturbations fo the features of each data point in $\mathcal{D}^{(l)}$. This is reasonable for applications where features are subject to inherent measurement or modelling errors. Given a data point with vector $\mathbf{x}$ we could have equally well observed a feature vector $\mathbf{x} + \boldsymbol{\varepsilon}$ with some small random perturbation $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$. To estimate the covariance matrix matrix $\mathbf{C}$, we use the sample covariance matrix of the feature vectors in the (large) unlabelled dataset $\mathcal{D}^{(u)}$. We then learn a hypothesis using the augmented (regularized) ERM (7.20).

7.6 Multitask Learning

Consider a specific learning task of finding a hypothesis h with minimum (expected) loss $L\left((\mathbf{x}, y), h\right)$. Note that the loss incurred by h for a specific data point depends on the definition for the label of a data point. We can obtain different learning tasks for the same data points by using different choices or definitions for the label of a data point. Multitask learning exploits the similarities between different learning tasks to jointly solve them. Let us next discuss a simple example of a multitask learning problem.

Algorithm 9 A Semi-Supervised Learning Algorithm

Input: labeled dataset $\mathcal{D}^{(l)} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$; unlabeled dataset $\mathcal{D}^{(u)} = \{\widetilde{\mathbf{x}}^{(i)}\}_{i=1}^{m'}$

1: compute $\mathbf{C}$ via sample covariance on $\mathcal{D}^{(u)}$,

$$\mathbf{C} := (1/m') \sum_{i=1}^{m'} \big(\widetilde{\mathbf{x}}^{(i)} - \widehat{\mathbf{x}}\big)\big(\widetilde{\mathbf{x}}^{(i)} - \widehat{\mathbf{x}}\big)^T \text{ with } \widehat{\mathbf{x}} := (1/m') \sum_{i=1}^{m'} \widetilde{\mathbf{x}}^{(i)}. \tag{7.32}$$

2: compute (e.g. using GD)

$$\widehat{\mathbf{w}} := \operatorname*{argmin}_{\mathbf{w} \in \mathbb{R}^n} \Big[(1/m) \sum_{i=1}^{m} \big(y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)}\big)^2 + \mathbf{w}^T \mathbf{C} \mathbf{w} \Big]. \tag{7.33}$$

Output: hypothesis $\widehat{h}(\mathbf{x}) = \big(\widehat{\mathbf{w}}\big)^T \mathbf{x}$

Consider a data point $\mathbf{z}$ representing a hand-drawing that is collected via the online game `https://quickdraw.withgoogle.com/`. The features of a data point are the pixel intensities of the bitmap which is used to store the hand-drawing. As label we could use the fact if a hand-drawing shows an apple or not. This results in the learning task $\mathcal{T}^{(1)}$. Another choice for the label of a hand-drawing could be the fact if a hand-drawing shows a fruit at all or not. This results in another learning task $\mathcal{T}^{(2)}$ which is similar but different from the task $\mathcal{T}^{(1)}$.

The idea of multitask learning is that a reasonable hypothesis h for some learning task should also do well for a similar learning task. Thus, we can use the loss incurred on similar learning tasks as a regularization term for learning a hypothesis for the learning task at hand. Algorithm 10 is a straightforward implementation of this idea for a given dataset that gives rise to T related learning tasks $\mathcal{T}^{(1)}, \ldots, \mathcal{T}^{(T)}$. For each individual learning task $\mathcal{T}^{(t')}$ it uses the loss on the remaining learning tasks $\mathcal{T}^{(t)}$, with $t \neq t'$, as regularization term in (7.34).

The applicability of Algorithm 10 is somewhat limited as it aims at finding a single hypothesis that does well for all T learning tasks simultaneously. For certain application domains it might be more reasonable to not learn a single hypothesis for all learning tasks but to learn a separate hypothesis $h^{(t)}$ for each learning task $t = 1, \ldots, T$. However, these separate hypothesis maps typically might still share some structural similarities.[1] We can

[1] One important example for such a structural similarity in the case of linear predictors $h^{(t)}(\mathbf{x}) = \big(\mathbf{w}^{(t)}\big)^T \mathbf{x}$ is that the parameter vectors $\mathbf{w}^{(T)}$ have a small joint support. Requiring the parameter vectors to have a small joint support is equivalent to requiring the stacked vector $\widetilde{\mathbf{w}} = \big(\mathbf{w}^{(1)}, \ldots, \mathbf{w}^{(T)}\big)$ to be block (group)

Algorithm 10 A Multitask Learning Algorithm

Input: dataset $\mathcal{D} = \{\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(m)}\}$; T learning tasks with loss functions $L^{(1)}, \ldots, L^{(T)}$, hypothesis space $\mathcal{H}$

1: learn a hypothesis $\widehat{h}$ via

$$\widehat{h} := \operatorname*{argmin}_{h \in \mathcal{H}} \sum_{t=1}^{T} \sum_{i=1}^{m} L^{(t)}\big(\mathbf{z}^{(i)}, h\big). \tag{7.34}$$

Output: hypothesis $\widehat{h}$

enforce different notion of similarity between hypothesis maps $h^{(t)}$ by adding a regularization term to the individual loss functions of the learning tasks.

Algorithm 11 generalizes Algorithms 10 by learning a separate hypothesis for each task t while requiring these hypotheses to be structurally similar. The structural (dis-)similarity between the hypothesis maps is measured by a regularization term $\mathcal{R}$ in (7.35).

Algorithm 11 A Multitask Learning Algorithm

Input: dataset $\mathcal{D} = \{\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(m)}\}$ with T associated learning tasks with loss functions $L^{(1)}, \ldots, L^{(T)}$, hypothesis space $\mathcal{H}$

1: learn a hypothesis $\widehat{h}$ via

$$\widehat{h}^{(1)}, \ldots, \widehat{h}^{(T)} := \operatorname*{argmin}_{h^{(1)}, \ldots, h^{(T)} \in \mathcal{H}} \sum_{t=1}^{T} \sum_{i=1}^{m} L^{(t)}\big(\mathbf{z}^{(i)}, h^{(t)}\big) + \lambda \mathcal{R}\big(h^{(1)}, \ldots, h^{(T)}\big). \tag{7.35}$$

Output: hypotheses $\widehat{h}^{(1)}, \ldots, \widehat{h}^{(T)}$

7.7 Transfer Learning

Regularization is also instrumental for transfer learning to capitalize on synergies between different related learning tasks [108, 61]. Transfer learning is enabled by constructing regularization terms for a learning task by using the result of a previous learning task. While multitask learning methods solve many related learning tasks simultaneously, transfer learning methods operate in a more sequential fashion.

Let us illustrate the idea of transfer learning using two learning tasks which differ significantly in their intrinsic difficulty. Informally, we consider a learning task to be easy if

sparse [34].

we can easily gather large amounts of labeled (training) data for that task. Consider the learning task $\mathcal{T}^{(1)}$ of predicting whether an image shows a cat or not. For this learning task we can easily gather a large training set $\mathcal{D}^{(1)}$ using via image collections of animals. Another (related) learning task $\mathcal{T}^{(2)}$ is to predict whether an image shows a cat of a particular breed, with a particular body height and with a specific age. The learning task $\mathcal{T}^{(2)}$ is more dificult than $\mathcal{T}^{(1)}$ since we have only a very limited amount of cat images for which we know the particular breed, body height and precise age of the depicted cat.

7.8 Exercises

Exercise 7.1. Ridge Regression is a Quadratic Problem. Consider the linear hypothesis space consisting of linear maps parameterized by weights $\mathbf{w}$. We try to find the best linear map by minimizing the regularized average squared error loss (empirical risk) incurred on a training set

$$\mathcal{D} := \big\{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \ldots, (\mathbf{x}^{(m)}, y^{(m)})\big\}.$$

Ridge reression augments the average squared error loss on $\mathcal{D}$ by the regularizer $\|\mathbf{w}\|^2$, yielding the following learning problem

$$\min_{\mathbf{w}\in\mathbb{R}^n} f(\mathbf{w}) = (1/m)\sum_{i=1}^{m}\big(y^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)}\big) + \lambda\|\mathbf{w}\|_2^2.$$

Is it possible to rewrite the objective function $f(\mathbf{w})$ as a convex quadratic function $f(\mathbf{w}) = \mathbf{w}^T\mathbf{C}\mathbf{w} + \mathbf{b}^T\mathbf{w} + c$? If this is possible, how are the matrix $\mathbf{C}$, vector $\mathbf{b}$ and constant c related to the feature vectors and labels of the training data ?

Exercise 7.2. Regularization or Model Selection. Consider data points, each characterized by $n = 10$ features $\mathbf{x} \in \mathbb{R}^n$ and a single numeric label y. We want to learn a linear hypothesis $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ by minimizing the average squared error on the training set $\mathcal{D}$ of size $m = 4$. We could learn such a hypothesis by two approaches. The first approach is to split the dataset into a training set and a validation set. Then we consider all models that consists of linear hypotheses with weight vectors having at most two non-zero weights. Each of these models corresponds to a different subset of two weights that might be non-zero. Find the model resulting in the smallest validation errors (see Algorithm 5). Compute the

average loss of the resulting optimal linear hypothesis on some data points that have neither been used in the training set nor the validation set. Compare this average loss ("test error") with the average loss obtained on the same data points by the hypothesis learnt by ridge regression (7.4).

Chapter 8

Clustering

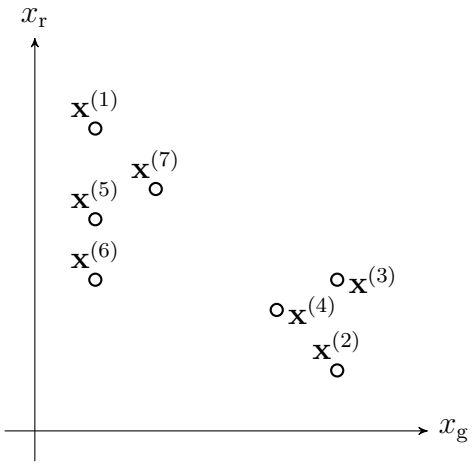


Figure 8.1: Each circle represents an image which is characterized by its average redness x_{r} and average greenness x_{g}. The i-th image is depicted by a circle located at the point $\mathbf{x}^{(i)} = \big(x_{\mathrm{r}}^{(i)}, x_{\mathrm{g}}^{(i)}\big)^T \in \mathbb{R}^2$. It seems that the images can be grouped into two clusters.

So far we focused on ML methods that use the ERM principle and lean a hypothesis by minimizing the discrepancy between its predictions and the true labels on a training set. These methods are referred to as supervised methods as they require labeled data points for which the true label values have been determined by some human (who serves as a "supervisor"). This and the following chapter discus ML methods which do not require to know the label of any data point. These methods are often referred to as "unsupervised" since they do not require a "supervisor" to provide the label values for any data point.

One important family of unsupervised ML methods aim at clustering a given set of data

points such as those depicted in Figure 8.1. The basic idea of clustering is to decompose a set of data points into few subsets or clusters that consist of similar data points. For the dataset in Figure 8.1 it seems reasonable to define two clusters, one cluster $\{\mathbf{x}^{(1)}, \mathbf{x}^{(5)}, \mathbf{x}^{(6)}, \mathbf{x}^{(7)}\}$ and a second cluster $\{\mathbf{x}^{(2)}, \mathbf{x}^{(3)}, \mathbf{x}^{(4)}, \mathbf{x}^{(8)}\}$.

Formally, clustering methods learn a hypothesis that assign each data point either to precisely one cluster (see Section 8.1) or several clusters with different degrees of belonging (see Section 8.2). Different clustering methods use different measures for the similarity between data points. For data points characterized by (numeric) Euclidean feature vectors, the similarity between data points can be naturally defined in terms of the Euclidean distance between feature vectors. Section 8.3 discusses clustering methods that use notions of similarity that are not based on a Euclidean space.

There is a strong conceptual link between clustering methods and the classification methods discussed in Chapter 3. Both type of methods learn a hypothesis that reads in the features of a data point and delivers a prediction for some quantity of interest. In classification methods, this quantity of interest is some generic label of a data point. For clustering methods, this quantity of interest for a data point is the cluster assignment (for hard clustering) of the degree of belonging (for soft clustering). A main difference between clustering and classification is that clustering methods do not require the true label (cluster assignment or degree of belonging) of a single data point.

Classification methods learn a good hypothesis via minimizing their average loss incurred on a training set of labeled data points. In contrast, clustering methods do not have access to a single labeled data point. To find the correct labels (cluster assignments), clustering methods must rely solely on the intrinsic geometry of the data points. We will see that clustering methods use this intrinsic geometry to determine an empirical risk incurred by a candidate hypothesis (which maps data points to cluster assignments or degree of belonging). Thus, much like classification methods, also clustering implement the generic ERM principle (see Chapter 4) to find a good hypothesis (clustering).

This chapter discusses two main flavours of clustering methods:

- hard clustering (see Section 8.1)
- and soft clustering methods (see Section 8.2).

Hard clustering methods learn a hypothesis h that reads in the feature vector $\mathbf{x}$ of a data point and delivers a predicted cluster assignment $\hat{y} = h(\mathbf{x}) \in \{1, \ldots, k\}$. Thus, assigns each

data point to one single cluster. Section 8.1 will discuss one of the most widely-used hard clustering algorithms which is known as k-means.

In contrast to hard clustering methods, soft clustering methods assign each data point to several clusters with varying degree of belonging. These methods learn a hypothesis that delivers a vector $\hat{\mathbf{y}} = \big(\hat{y}_1, \ldots, \hat{y}_k\big)^T$ with entry $\hat{y}_c \in [0,1]$ being the predicted degree by which the data point belongs to the c-th cluster. Hard clustering is an extreme case of soft clustering where we enforce each degree of belonging to take only values in $\{0,1\}$. Moreover, hard clustering requires that for each data point only of the corresponding degree of belonging (one for each cluster) is non-zero.

The main focus of this chapter is on methods that require data points being represented by numeric feature vectors (see Sections 8.1 and 8.2). These methods define the similarity between data points using the Euclidean distance between their feature vectors. Some applications generate data points for which it is not obvious how to obtain numeric feature vectors such that their Euclidean distances reflect the similarity between data points. It is then desirable to use a more flexible notion of similarity which does not require to determine (useful) numeric feature vectors of data points.

Maybe the most fundamental concept to represent similarities between data points is a similarity graph. The nodes of the similarity graph are the individual data points of a dataset. Similar data points are connected by edges (links) that might be assigned some weight that quantities the amount of similarity. Section 8.3 discusses clustering methods that use a graph to represent similarities between data points.

8.1 Hard Clustering with k-means

Consider a dataset $\mathcal{D}$ which consists of m data points that are indexed by $i = 1, \ldots, m$. The data points are characterized via their numeric feature vectors $\mathbf{x}^{(i)} \in \mathbb{R}^n$, for $i = 1, \ldots, m$. It will be convenient for the following discussion if we identify a data point with its feature vector. In particular, we refer by $\mathbf{x}^{(i)}$ to the i-th data point. Hard clustering methods decompose (or cluster) the dataset into a given number k of different clusters $\mathcal{C}^{(1)}, \ldots, \mathcal{C}^{(k)}$. These methods assign each data point $\mathbf{x}^{(i)}$ to one and only one cluster $\mathcal{C}^{(c)}$ with the cluster index $c \in \{1, \ldots, k\}$.

Let us define for each data point its label $y^{(i)} \in \{1, \ldots, k\}$ as the index of the cluster to which the ith data point actually belongs to. The c-th cluster consists of all data points with $y^{(i)} = c$,

$$\mathcal{C}^{(c)} := \big\{ i \in \{1, \ldots, m\} : y^{(i)} = c \big\}. \tag{8.1}$$

We can interpret hard clustering methods as ML methods that compute predictions $\hat{y}^{(i)}$ for the ("correct") cluster assignments $y^{(i)}$. The predicted cluster assignments result in the predicted clusters

$$\widehat{\mathcal{C}}^{(c)} := \big\{ i \in \{1, \ldots, m\} : \hat{y}^{(i)} = c \big\}, \text{ for } c = 1, \ldots, k. \tag{8.2}$$

We now discuss a hard clustering method which is known as k-means. This method does not require the knowledge of the label or (true) cluster assignment $y^{(i)}$ for any data point in $\mathcal{D}$. This method computes predicted cluster assignments $\hat{y}^{(i)}$ based solely from the intrinsic geometry of the feature vectors $\mathbf{x}^{(i)} \in \mathbb{R}^n$ for all $i = 1, \ldots, m$. Since it does not require any labeled data points, k-means is often referred to as being an unsupervised method. However, note that k-means requires the number k of clusters to be given as an input (or hyper-) parameter.

The k-means method represents the c-th cluster $\widehat{\mathcal{C}}^{(c)}$ by a representative feature vector $\boldsymbol{\mu}^{(c)} \in \mathbb{R}^n$. It seems reasonable to assign data points in $\mathcal{D}$ to clusters $\widehat{\mathcal{C}}^{(c)}$ such that they are well concentrated around the cluster representatives $\boldsymbol{\mu}^{(c)}$. We make this informal requirement precise by defining the clustering error

$$\widehat{L}\big(\{\boldsymbol{\mu}^{(c)}\}_{c=1}^{k}, \{\hat{y}^{(i)}\}_{i=1}^{m} \mid \mathcal{D}\big) = (1/m) \sum_{i=1}^{m} \Big\| \mathbf{x}^{(i)} - \boldsymbol{\mu}^{(\hat{y}^{(i)})} \Big\|^2. \tag{8.3}$$

Note that the clustering error $\widehat{L}$ (8.3) depends on both, the cluster assignments $\hat{y}^{(i)}$, which

define the cluster (8.2), and the cluster representatives $\boldsymbol{\mu}^{(c)}$, for $c = 1, \ldots, k$.

Finding the optimal cluster means $\{\boldsymbol{\mu}^{(c)}\}_{c=1}^{k}$ and cluster assignments $\{\hat{y}^{(i)}\}_{i=1}^{m}$ that minimize the clustering error (8.3) is computationally challenging. The difficulty stems from the fact that the clustering error is a non-convex function of the cluster means and assignments. While jointly optimizing the cluster means and assignments is hard, separately optimizing either the cluster means for given assignments or vice-versa is easy. In what follows, we present simple closed-form solutions for these sub-problems. The k-means method simply combines these solutions in an alternating fashion.

It can be shown that for given predictions (cluster assignments) $\hat{y}^{(i)}$, the clustering error (8.3) is minimized by setting the cluster representatives equal to the **cluster means** [12]

$$\boldsymbol{\mu}^{(c)} := \big(1/|\widehat{\mathcal{C}}^{(c)}|\big) \sum_{\hat{y}^{(i)}=c} \mathbf{x}^{(i)}. \tag{8.4}$$

To evaluate (8.4) we need to know the predicted cluster assignments $\hat{y}^{(i)}$. The crux is that the optimal predictions $\hat{y}^{(i)}$, in the sense of minimizing clustering error (8.3), depend themselves on the choice for the cluster representatives $\boldsymbol{\mu}^{(c)}$. In particular, for given cluster representative $\boldsymbol{\mu}^{(c)}$ with $c = 1, \ldots, k$, the clustering error is minimized by the cluster assignments

$$\hat{y}^{(i)} \in \operatorname*{argmin}_{c \in \{1,\ldots,k\}} \big\|\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(c)}\big\|. \tag{8.5}$$

Here, we denote by $\operatorname*{argmin}_{c' \in \{1,\ldots,k\}} \|\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(c')}\|$ the set of all cluster indices $c \in \{1, \ldots, k\}$ such that $\|\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(c)}\| = \min_{c' \in \{1,\ldots,k\}} \|\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(c')}\|$.

Note that (8.5) assigns the ith datapoint to those cluster $\mathcal{C}^{(c)}$ whose cluster mean $\boldsymbol{\mu}^{(c)}$ is nearest (in Euclidean distance) to $\mathbf{x}^{(i)}$. Thus, if we knew the optimal cluster representatives, we could predict the cluster assignments using (8.5). However, we do not know the optimal cluster representatives unless we have found good predictions for the cluster assignments $\hat{y}^{(i)}$ (see (8.4)).

To recap: We have characterized the optimal choice (8.4) for the cluster representatives for given cluster assignments and the optimal choice (8.5) for the cluster assignments for given cluster representatives. It seems natural, starting from some initial guess for the cluster representatives, to alternate between the cluster assignment update (8.5) and the update (8.4) for the cluster means. This alternating optimization strategy is illustrated in Figure 8.2 and summarized in Algorithm 12. Note that Algorithm 12, which is maybe the most basic variant of k-means, simply alternates between the two updates (8.4) and (8.5)

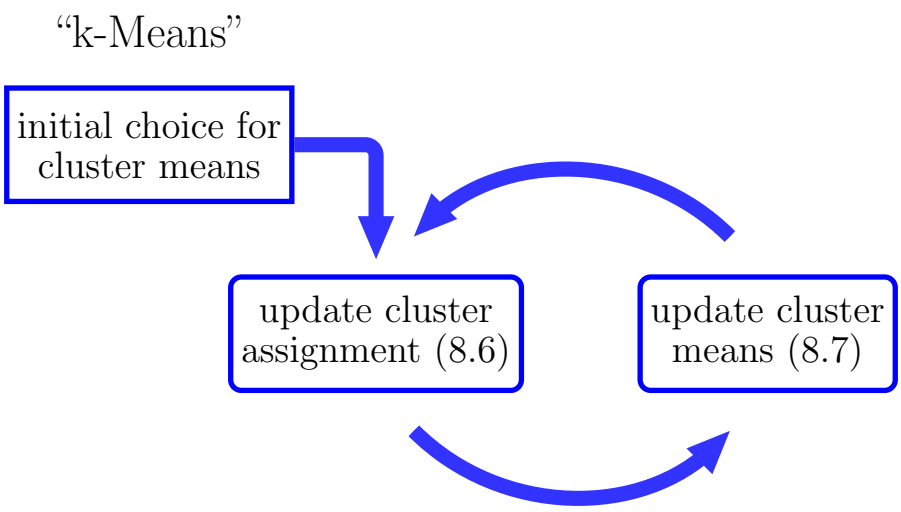


Figure 8.2: The workflow of k-means. Starting from an initial guess or estimate for the cluster means, the cluster assignments and cluster means are updated (improved) in an alternating fashion.

until some stopping criterion is satisfied.

Algorithm 12 requires the specification of the number k of clusters and initial choices for the cluster means $\boldsymbol{\mu}^{(c)}$, for $c = 1, \ldots, k$. Those quantities are hyper-parameters that must be tuned to the specific geometry of the given dataset $\mathcal{D}$. This tuning can be based on probabilistic models for the dataset and its cluster structure (see Section 2.1.4 and [81, 150]). Alternatively, if Algorithm 12 is used as pre-processing within an overall supervised ML method (see Chapter 3), the validation error (see Section 6.3) of the overall method might guide the choice of the number k of clusters.

Choosing Number of Clusters. The choice for the number k of clusters typically depends on the role of the clustering method within an overall ML application. If the clustering method serves as a pre-processing for a supervised ML problem, we could try out different values of the number k and determine, for each choice k, the corresponding validation error. We then pick the value of k which results in the smallest validation error. If the clustering method is mainly used as a tool for data visualization, we might prefer a small number of clusters. The choice for the number k of clusters can also be guided by the so-called "elbow-method". Here, we run the k-means Algorithm 12 for several different choices of k. For each value of k, Algorithm 12 delivers a clustering with clustering error

$$E^{(k)} = \widehat{L}\big(\{\boldsymbol{\mu}^{(c)}\}_{c=1}^{k}, \{\hat{y}^{(i)}\}_{i=1}^{m} \mid \mathcal{D}\big).$$

Algorithm 12 "k-means"

Input: dataset $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^{m}$; number k of clusters; initial cluster means $\boldsymbol{\mu}^{(c)}$ for $c = 1, \ldots, k$.

1: **repeat**

2: for each datapoint $\mathbf{x}^{(i)}$, $i=1, \ldots, m$, do

$$\hat{y}^{(i)} := \operatorname*{argmin}_{c' \in \{1,\ldots,k\}} \|\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(c')}\| \quad \text{(update cluster assignments)} \tag{8.6}$$

3: for each cluster $c=1, \ldots, k$ do

$$\boldsymbol{\mu}^{(c)} := \frac{1}{|\{i : \hat{y}^{(i)} = c\}|} \sum_{i:\hat{y}^{(i)}=c} \mathbf{x}^{(i)} \quad \text{(update cluster means)} \tag{8.7}$$

4: **until** stopping criterion is met

5: compute final clustering error $E^{(k)} := (1/m) \sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}^{(\hat{y}^{(i)})} \right\|^2$

Output: cluster means $\boldsymbol{\mu}^{(c)}$, for $c = 1, \ldots, k$, cluster assignments $\hat{y}^{(i)} \in \{1, \ldots, k\}$, for $i = 1, \ldots, m$, final clustering error $E^{(k)}$

We then plot the minimum empirical error $E^{(k)}$ as a function of the number k of clusters. Figure 8.3 depicts an example for such a plot which typically starts with a steep decrease for increasing k and then flattening out for larger values of k. Note that for $k \geq m$ we can achieve zero clustering error since each datapoint $\mathbf{x}^{(i)}$ can be assigned to a separate cluster $\mathcal{C}^{(c)}$ whose mean coincides with that datapoint, $\mathbf{x}^{(i)} = \boldsymbol{\mu}^{(c)}$.

Cluster-Means Initialization. We briefly mention some popular strategies for choosing the initial cluster means in Algorithm 12. One option is to initialize the cluster means with realizations of i.i.d. random vectors whose probability distribution is matched to the dataset $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^{m}$ (see Section 3.12). For example, we could use a multivariate normal distribution $\mathcal{N}(\mathbf{x}; \widehat{\boldsymbol{\mu}}, \widehat{\boldsymbol{\Sigma}})$ with the sample mean $\widehat{\boldsymbol{\mu}} = (1/m) \sum_{i=1}^{m} \mathbf{x}^{(i)}$ and the sample covariance $\widehat{\boldsymbol{\Sigma}} = (1/m) \sum_{i=1}^{m} (\mathbf{x}^{(i)} - \widehat{\boldsymbol{\mu}})(\mathbf{x}^{(i)} - \widehat{\boldsymbol{\mu}})^T$. Alternatively, we could choose the initial cluster means $\boldsymbol{\mu}^{(c)}$ by selecting k different data points $\mathbf{x}^{(i)}$ from $\mathcal{D}$. This selection process might combine random choices with an optimization of the distances between cluster means [3]. Finally, the cluster means might also be chosen by evenly partitioning the principal component of the dataset (see Chapter 9).

Interpretation as ERM. For a practical implementation of Algorithm 12 we need to decide when to stop updating the cluster means and assignments (see (8.6) and (8.7)). To this end it is useful to interpret Algorithm 12 as a method for iteratively minimizing the

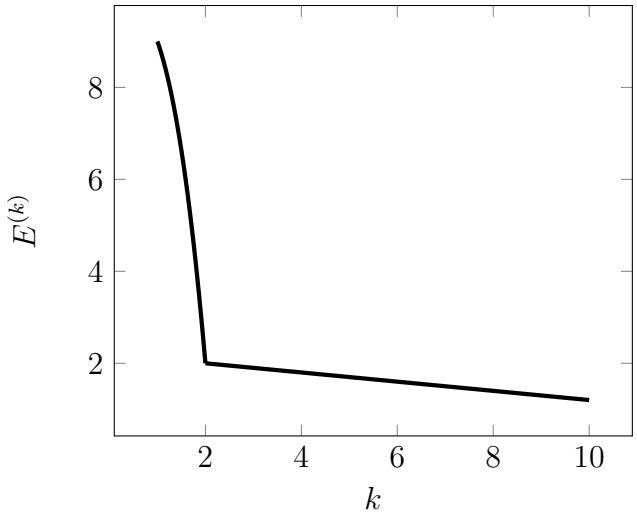


Figure 8.3: The clustering error $E^{(k)}$ achieved by k-means for increasing number k of clusters.

clustering error (8.3). As can be verified easily, the updates (8.6) and (8.7) always modify (update) the cluster means or assignments in such a way that the clustering error (8.3) is never increased. Thus, each new iteration of Algorithm 12 results in cluster means and assignments with a smaller (or the same) clustering error compared to the cluster means and assignments obtained after the previous iteration. Algorithm 12 implements a form of ERM (see Chapter 4) using the clustering error (8.3) as the empirical risk incurred by the predicted cluster assignments $\hat{y}^{(i)}$. Note that after completing a full iteration of Algorithm 12, the cluster means $\{\boldsymbol{\mu}^{(c)}\}_{c=1}^{k}$ are fully determined by the cluster assignments $\{\hat{y}^{(i)}\}_{i=1}^{m}$ via (8.7). It seems natural to terminate Algorithm 12 if the decrease in the clustering error achieved by the most recent iteration is below a prescribed (small) threshold.

Clustering and Classification. There is a strong conceptual link between Algorithm 12 and classification methods (see e.g. Section 3.13). Both methods essentially learn a hypothesis $h(\mathbf{x})$ that maps the feature vector $\mathbf{x}$ to a predicted label $\hat{y} = h(\mathbf{x})$ from a finite set. The practical meaning of the label values is different for Algorithm 12 and classification methods. For classification methods, the meaning of the label values is essentially defined by the training set (of labeled data points) used for ERM (4.3). On the other hand, clustering methods use the predicted label $\hat{y} = h(\mathbf{x})$ as a cluster index.

Another main difference between Algorithm 12 and most classification methods is the choice for the empirical risk used to evaluate the quality or usefulness of a given hypothesis $h(\cdot)$. Classification methods typically use an average loss over labeled data points in a

training set as empirical risk. In contrast, Algorithm 12 uses the clustering error (8.3) as a form of empirical risk. Consider a hypothesis that resembles the cluster assignments $\hat{y}^{(i)}$ obtained after completing an iteration in Algorithm 12, $\hat{y}^{(i)} = h\big(\mathbf{x}^{(i)}\big)$. Then we can rewrite the resulting clustering error achieved after this iteration as

$$\widehat{L}\big(h|\mathcal{D}\big) = (1/m)\sum_{i=1}^{m}\left\|\mathbf{x}^{(i)} - \frac{\sum_{i' \in \mathcal{D}^{(i)}} \mathbf{x}^{(i')}}{\big|\mathcal{D}^{(i)}\big|}\right\|^2 . \text{ with } \mathcal{D}^{(i)} := \big\{i' : h\big(\mathbf{x}^{(i)}\big) = h\big(\mathbf{x}^{(i')}\big)\big\}. \quad (8.8)$$

Note that the i-th summand in (8.8) depends on the entire dataset $\mathcal{D}$ and not only on (the features of) the i-th data point $\mathbf{x}^{(i)}$.

Some Practicalities. For a practical implementation of Algorithm 12 we need to fix three issues.

- Issue 1 ("tie-breaking"): We need to specify what to do if several different cluster indices $c \in \{1, \ldots, k\}$ achieve the minimum value in the cluster assignment update (8.6) during step 2.

- Issue 2 ("empty cluster"): The cluster assignment update (8.6) in step 3 of Algorithm 12 might result in a cluster c with no datapoints associated with it, $|\{i : \hat{y}^{(i)} = c\}| = 0$. For such a cluster c, the update (8.7) is not well-defined.

- Issue 3 ("stopping criterion"): We need to specify a criterion used in step 4 of Algorithm 12 to decide when to stop iterating.

Algorithm 13 is obtained from Algorithm 12 by fixing those three issues [51]. Step 3 of Algorithm 13 solves the first issue mentioned above ("tie breaking"), arising when there are several cluster clusters whose means have minimum distance to a data point $\mathbf{x}^{(i)}$, by assigning $\mathbf{x}^{(i)}$ to the cluster with smallest cluster index (see (8.9)). Step 4 of Algorithm 13 resolves the "empty cluster" issue by computing the variables $b^{(c)} \in \{0, 1\}$ for $c = 1, \ldots, k$. The variable $b^{(c)}$ indicates if the cluster with index c is active ($b^{(c)} = 1$) or the cluster c is inactive ($b^{(c)} = 0$). The cluster c is defined to be inactive if there are no data points assigned to it during the preceding cluster assignment step (8.9). The cluster activity indicators $b^{(c)}$ allows to restrict the cluster mean updates (8.10) only to the clusters c with at least one data point $\mathbf{x}^{(i)}$. To obtain a stopping criterion, step 7 Algorithm 13 monitors the clustering error E_r incurred by the cluster means and assignments obtained after r iterations. Algorithm 13 continues updating cluster assignments (8.9) and cluster means (8.10) as long as the decrease is above a given threshold $\varepsilon \geq 0$.

Algorithm 13 "k-Means II" (slight variation of "Fixed Point Algorithm" in [51])

Input: dataset $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^m$; number k of clusters; tolerance $\varepsilon \geq 0$; initial cluster means $\{\boldsymbol{\mu}^{(c)}\}_{c=1}^k$

1: **Initialize.** set iteration counter $r := 0$; $E_0 := 0$

2: **repeat**

3: for all datapoints $i = 1, \ldots, m$,

$$\hat{y}^{(i)} := \min\{ \operatorname*{argmin}_{c' \in \{1,\ldots,k\}} \|\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(c')}\| \} \quad \text{(update cluster assignments)} \tag{8.9}$$

4: for all clusters $c = 1, \ldots, k$, update the activity indicator

$$b^{(c)} := \begin{cases} 1 & \text{if } |\{i : \hat{y}^{(i)} = c\}| > 0 \\ 0 & \text{else.} \end{cases}$$

5: for all $c = 1, \ldots, k$ with $b^{(c)} = 1$,

$$\boldsymbol{\mu}^{(c)} := \frac{1}{|\{i : \hat{y}^{(i)} = c\}|} \sum_{\{i : \hat{y}^{(i)} = c\}} \mathbf{x}^{(i)} \quad \text{(update cluster means)} \tag{8.10}$$

6: $r := r + 1$ (increment iteration counter)

7: $E_r := \widehat{L}\big(\{\boldsymbol{\mu}^{(c)}\}_{c=1}^k, \{\hat{y}^{(i)}\}_{i=1}^m \mid \mathcal{D}\big)$ (evaluate clustering error (8.3))

8: **until** $r > 1$ and $E_{r-1} - E_r \leq \varepsilon$ (check for sufficient decrease in clustering error)

9: $E^{(k)} := (1/m) \sum_{i=1}^m \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}^{(\hat{y}^{(i)})} \right\|^2$ (compute final clustering error)

Output: cluster assignments $\hat{y}^{(i)} \in \{1, \ldots, k\}$, cluster means $\boldsymbol{\mu}^{(c)}$, clustering error $E^{(k)}$.

For Algorithm 13 to be useful we must ensure that the stopping criterion is met within a finite number of iterations. In other words, we must ensure that the clustering error decrease can be made arbitrarily small within a sufficiently large (but finite) number of iterations. To this end, it is useful to represent Algorithm 13 as a fixed-point iteration

$$\{\hat{y}^{(i)}\}_{i=1}^{m} \mapsto \mathcal{P}\{\hat{y}^{(i)}\}_{m=1}^{m}. \tag{8.11}$$

The operator $\mathcal{P}$, which depends on the dataset $\mathcal{D}$, reads in a list of cluster assignments and delivers an improved list of cluster assignments aiming at reducing the associated clustering error (8.3). Each iteration of Algorithm 13 updates the cluster assignments $\hat{y}^{(i)}$ by applying the operator $\mathcal{P}$. Representing Algorithm 13 as a fixed-point iteration (8.11) allows for an elegant proof of the convergence of Algorithm 13 within a finite number of iterations (even for $\varepsilon = 0$) [51, Thm. 2].

Figure 8.4 depicts the evolution of the cluster assignments and cluster means during the iterations Algorithm 13. Each subplot corresponds to one iteration of Algorithm 13 and depicts the cluster means before that iteration and the clustering assignments (via the marker symbols) after the corresponding iteration. In particular, the upper left subplot depicts the cluster means before the first iteration (which are the initial cluster means) and the cluster assignments obtained after the first iteration of Algorithm 13.

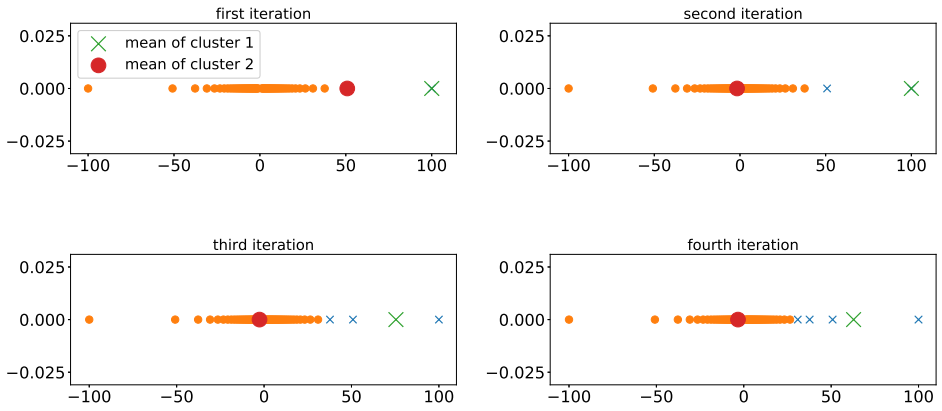


Figure 8.4: The evolution of cluster means (8.7) and cluster assignments (8.6) (depicted as large dot and large cross) during the first four iterations of k-means Algorithm 13.

Consider running Algorithm 13 with tolerance $\varepsilon = 0$ (see step 8) such that the iterations are continued until there is no decrease in the clustering error $E^{(r)}$ (see step 7 of Algorithm

13). As discussed above, Algorithm 13 will terminate after a finite number of iterations. Moreover, for $\varepsilon = 0$, the delivered cluster assignments $\big\{\hat{y}^{(i)}\big\}_{i=1}^{m}$ are fully determined by the delivered clustered means $\big\{\boldsymbol{\mu}^{(c)}\big\}_{c=1}^{k}$,

$$\hat{y}^{(i)} = \min\{ \operatorname*{argmin}_{c' \in \{1,\ldots,k\}} \|\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(c')}\|\}. \tag{8.12}$$

Indeed, if (8.12) does not hold one can show the final iteration r would still decrease the clustering error and the stopping criterion in step 8 would not be met.

If cluster assignments and cluster means satisfy the condition (8.12), we can rewrite the clustering error (8.3) as a function of the cluster means solely,

$$\widehat{L}\big(\big\{\boldsymbol{\mu}^{(c)}\big\}_{c=1}^{k}|\mathcal{D}\big) := (1/m)\sum_{i=1}^{m} \min_{c' \in \{1,\ldots,k\}} \|\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(c')}\|^2. \tag{8.13}$$

Even for cluster assignments and cluster means that do not satisfy (8.12), we can still use (8.13) to lower bound the clustering error (8.3),

$$\widehat{L}\big(\big\{\boldsymbol{\mu}^{(c)}\big\}_{c=1}^{k}|\mathcal{D}\big) \leq \widehat{L}\big(\{\boldsymbol{\mu}^{(c)}\}_{c=1}^{k}, \{\hat{y}^{(i)}\}_{i=1}^{m} \mid \mathcal{D}\big)$$

.

Algorithm 13 iteratively improves the cluster means in order to minimize (8.13). Ideally, we would like Algorithm 13 to deliver cluster means that achieve the global minimum of (8.13) (see Figure 8.5). However, for some combination of dataset $\mathcal{D}$ and initial cluster means, Algorithm 13 delivers cluster means that form only a local optimum of $\widehat{L}\big(\big\{\boldsymbol{\mu}^{(c)}\big\}_{c=1}^{k}|\mathcal{D}\big)$ which is strictly worse (larger) than its global optimum (see Figure 8.5).

The tendency of Algorithm 13 to get trapped around a local minimum of (8.13) depends on the initial choice for cluster means. It is therefore useful to repeat Algorithm 13 several times, with each repetition using a different initial choice for the cluster means. We then pick the cluster assignments $\{\hat{y}^{(i)}\}_{i=1}^{m}$ obtained for the repetition that resulted in the smallest clustering error $E^{(k)}$ (see step 9).

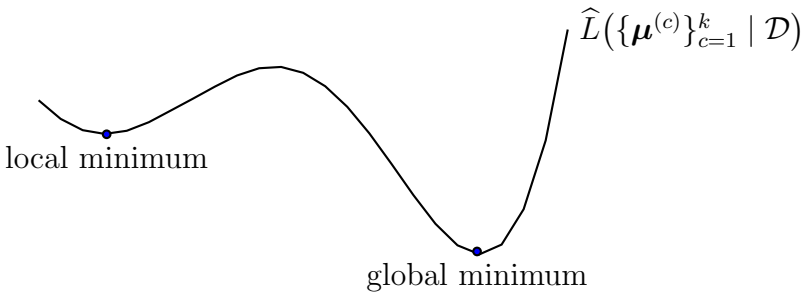


Figure 8.5: The clustering error (8.13) is a non-convex function of the cluster means $\{\boldsymbol{\mu}^{(c)}\}_{c=1}^{k}$. Algorithm 13 iteratively updates cluster means to minimize the clustering error but might get trapped around one of its local minimum.

8.2 Soft Clustering with Gaussian Mixture Models

Consider a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$ that we wish to group into a given number of k different clusters. The hard clustering methods discussed in Section 8.1 deliver cluster assignments $\hat{y}^{(i)}$, for $i = 1, \ldots, m$. The cluster assignment $\hat{y}^{(i)}$ is the index of the cluster to which the ith data point $\mathbf{x}^{(i)}$ is assigned to. These cluster assignments $\hat{y}$ provide rather coarse-grained information. Two data points $\mathbf{x}^{(i)}, \mathbf{x}^{(i')}$ might be assigned to the same cluster c although their distances to the cluster mean $\boldsymbol{\mu}^{(c)}$ might differ significantly. Intuitively, these two data points have a different degree of belonging to the cluster c.

For some clustering applications it is desirable to quantify the degree by which a data point belongs to a cluster. Soft clustering methods use a continues range, such as the closed interval $[0, 1]$, of possible values for the degree of belonging. In contrast, hard clustering methods use only two possible values for the degree of belonging to a specific cluster, either "full belonging" or no "belonging at all". While hard clustering methods assign a given data point to precisely one cluster, soft clustering methods typically assign a data point to several different clusters with non-zero degree of belonging.

This chapter discusses soft clustering methods that compute, for each data point $\mathbf{x}^{(i)}$ in the dataset $\mathcal{D}$, a vector $\widehat{\mathbf{y}}^{(i)} = \big(\hat{y}_1^{(i)}, \ldots, \hat{y}_k^{(i)}\big)^T$. We can interpret the entry $\hat{y}_c^{(i)} \in [0, 1]$ as the degree by which the data point $\mathbf{x}^{(i)}$ belongs to the cluster $\mathcal{C}^{(c)}$. For $\hat{y}_c^{(i)} \approx 1$, we are quite confident in the data point $\mathbf{x}^{(i)}$ belonging to cluster $\mathcal{C}^{(c)}$. In contrast, for $\hat{y}_c^{(i)} \approx 0$, we are quite confident that the data point $\mathbf{x}^{(i)}$ is outside the cluster $\mathcal{C}^{(c)}$.

A widely used soft clustering method uses a probabilistic model for the data points $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^{m}$. Within this model, each cluster $\mathcal{C}^{(c)}$, for $c = 1, \ldots, k$, is represented by a

multivariate normal distribution [9]

$$\mathcal{N}\big(\mathbf{x};\boldsymbol{\mu}^{(c)},\boldsymbol{\Sigma}^{(c)}\big)=\frac{1}{\sqrt{\det\{2\pi\boldsymbol{\Sigma}\}}}\exp\big(-(1/2)\big(\mathbf{x}-\boldsymbol{\mu}^{(c)}\big)^{T}\big(\boldsymbol{\Sigma}^{(c)}\big)^{-1}\big(\mathbf{x}-\boldsymbol{\mu}^{(c)}\big)\big). \tag{8.14}$$

The probability distribution (8.14) is parametrized by a cluster-specific mean vector $\boldsymbol{\mu}^{(c)}$ and an (invertible) cluster-specific covariance matrix $\boldsymbol{\Sigma}^{(c)}$.[1] Let us interpret a specific data point $\mathbf{x}^{(i)}$ as a realization drawn from the probability distribution (8.14) of a specific cluster $c^{(i)}$,

$$\mathbf{x}^{(i)} \sim \mathcal{N}\big(\mathbf{x};\boldsymbol{\mu}^{(c)},\boldsymbol{\Sigma}^{(c)}\big) \text{ with cluster index } c=c^{(i)}. \tag{8.15}$$

We can think of $c^{(i)}$ as the true index of the cluster to which the data point $\mathbf{x}^{(i)}$ belongs to. The variable $c^{(i)}$ selects the cluster distributions (8.14) from which the feature vector $\mathbf{x}^{(i)}$ has been generated (drawn). We will therefore refer to the variable $c^{(i)}$ as the (true) cluster assignment for the ith data point. Similar to the feature vectors $\mathbf{x}^{(i)}$ we also interpret the cluster assignments $c^{(i)}$, for $i=1,\ldots,m$ as realizations of i.i.d. RVs.

In contrast to the feature vectors $\mathbf{x}^{(i)}$, we do not observe (know) the true cluster indices $c^{(i)}$. After all, the goal of soft clustering is to estimate the cluster indices $c^{(i)}$. We obtain a soft clustering method by estimating the cluster indices $c^{(i)}$ based solely on the data points in $\mathcal{D}$. To compute these estimates we assume that the (true) cluster indices $c^{(i)}$ are realizations of i.i.d. RVs with the common probability distribution (or probability mass function)

$$p_c := p(c^{(i)}=c) \text{ for } c=1,\ldots,k. \tag{8.16}$$

The (prior) probabilities p_c, for $c=1,\ldots,k$, are either assumed known or estimated from data [85, 9]. The choice for the probabilities p_c could reflect some prior knowledge about different sizes of the clusters. For example, if cluster $\mathcal{C}^{(1)}$ is known to be larger than cluster $\mathcal{C}^{(2)}$, we might choose the prior probabilities such that $p_1 > p_2$.

The probabilistic model given by (8.15), (8.16) is referred to as a a GMM. As its name suggests, within a GMM the common marginal distribution for the feature vectors $\mathbf{x}^{(i)}$, for $i=1,\ldots,m$, is a (additive) mixture of multivariate normal (Gaussian) distributions,

$$p(\mathbf{x}^{(i)}) = \sum_{c=1}^{k} \underbrace{p(c^{(i)}=c)}_{p_c} \underbrace{p(\mathbf{x}^{(i)}|c^{(i)}=c)}_{\mathcal{N}(\mathbf{x}^{(i)};\boldsymbol{\mu}^{(c)},\boldsymbol{\Sigma}^{(c)})}. \tag{8.17}$$

[1] Note that the expression (8.14) is only valid for an invertible (non-singular) covariance matrix $\boldsymbol{\Sigma}$.

As already mentioned, the cluster assignments $c^{(i)}$ are hidden (unobserved) RVs. We thus have to infer or estimate these variables from the observed data points $\mathbf{x}^{(i)}$ which realizations or i.i.d. RVs with the common distribution (8.17).

The GMM (see (8.15) and (8.16)) lends naturally to a rigorous definition for the degree $y_c^{(i)}$ by which data point $\mathbf{x}^{(i)}$ belongs to cluster c.[2] Let us define the label value $y_c^{(i)}$ as the "a-posteriori" probability of the cluster assignment $c^{(i)}$ being equal to $c \in \{1, \ldots, k\}$:

$$y_c^{(i)} := p(c^{(i)} = c|\mathcal{D}). \tag{8.18}$$

By their very definition (8.18), the degrees of belonging $y_c^{(i)}$ always sum to one,

$$\sum_{c=1}^{k} y_c^{(i)} = 1 \text{ for each } i = 1, \ldots, m. \tag{8.19}$$

We emphasize that we use the conditional cluster probability (8.18), conditioned on the dataset $\mathcal{D}$, for defining the degree of belonging $y_c^{(i)}$. This is reasonable since the degree of belonging $y_c^{(i)}$ depends on the overall (cluster) geometry of the dataset $\mathcal{D}$.

The definition (8.18) for the label values (degree of belongings) $y_c^{(i)}$ involves the GMM parameters $\{\boldsymbol{\mu}^{(c)}, \boldsymbol{\Sigma}^{(c)}, p_c\}_{c=1}^k$ (see (8.17)). Since we do not know these parameters beforehand we cannot evaluate the conditional probability in (8.18). A principled approach to solve this problem is to evaluate (8.18) with the true GMM parameters replaced by some estimates $\{\widehat{\boldsymbol{\mu}}^{(c)}, \widehat{\boldsymbol{\Sigma}}^{(c)}, \hat{p}_c\}_{c=1}^k$. Plugging in the GMM parameter estimates into (8.18) provides us with predictions $\hat{y}_c^{(i)}$ for the degrees of belonging. However, to compute the GMM parameter estimates we would have already needed the degrees of belonging $y_c^{(i)}$. This situation is similar to hard clustering where ultime goals is to jointly optimize cluster means and assignments (see Section 8.1).

Similar to the spirit of Algorithm 12 for hard clustering, we solve the above dilemma of soft clustering by an alternating optimization scheme. This scheme, which is illustrated in Figure 8.6, alternates between updating (optimizing) the predicted degrees of belonging (or soft cluster assignments) $\hat{y}_c^{(i)}$, for $i = 1, \ldots, m$ and $c = 1, \ldots, k$, given the current GMM parameter estimates $\{\widehat{\boldsymbol{\mu}}^{(c)}, \widehat{\boldsymbol{\Sigma}}^{(c)}, \hat{p}_c\}_{c=1}^k$ and then updating (optimizing) these GMM parameter estimates based on the updated predictions $\hat{y}_c^{(i)}$. We summarize the resulting soft clustering method in Algorithm 14. Each iteration of Algorithm 14 consists of an update

[2]Remember that the degree of belonging $y_c^{(i)}$ is considered as the (unknown) label value of a data point. The choice or definition for the labels of data points is a design choice. In particular, we can define the labels of data points using a hypothetical probabilistic model such as the GMM.

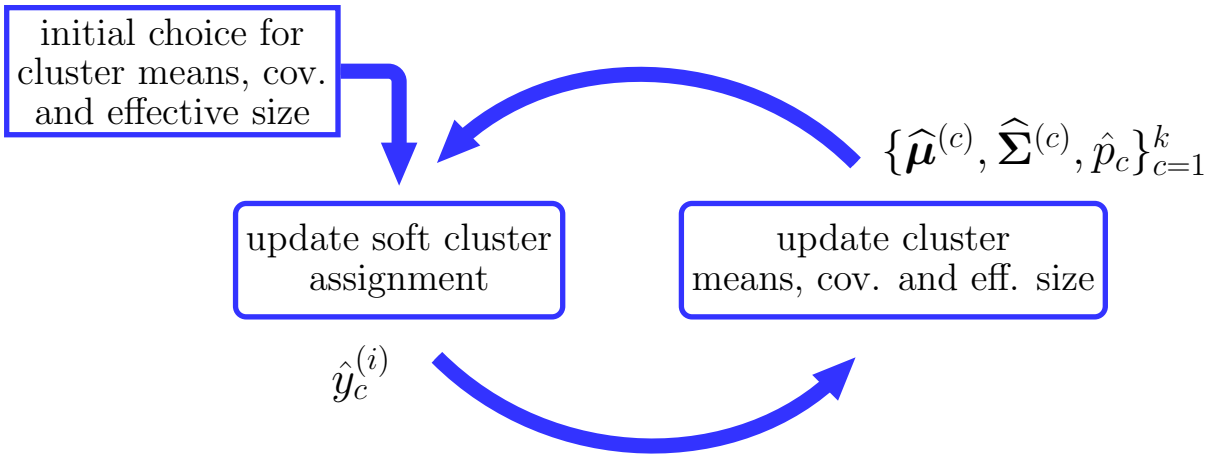


Figure 8.6: The workflow of the soft clustering Algorithm 14. Starting from an initial guess or estimate for the cluster parameters, the soft cluster assignments and cluster parameters are updated (improved) in an alternating fashion.

(8.22) for the degrees of belonging followed by an update (step 3) for the GMM parameters.

To analyze Algorithm 14 it is helpful to interpret (the features of) data points $\mathbf{x}^{(i)}$ as realizations of i.i.d. RVs distributed according to a GMM (8.15)-(8.16). We can then understand Algorithm 14 as a method for estimating the GMM parameters based on observing realizations drawn from the GMM (8.15)-(8.16). We can estimate the parameters of a probability distribution using the ML method (see Section 3.12 and [76, 85]). As its name suggests, ML methods estimate the GMM parameters by maximizing the probability (density)

$$p\big(\mathcal{D}; \{\boldsymbol{\mu}^{(c)}, \boldsymbol{\Sigma}^{(c)}, p_c\}_{c=1}^{k}\big) \tag{8.20}$$

of actually observing the data points in the dataset $\mathcal{D}$.

It can be shown that Algorithm 14 is an instance of a generic approximate maximum likelihood technique referred to as EM (see [58, Chap. 8.5] for more details). In particular, each iteration of Algorithm 14 updates the GMM parameter estimates such that the corresponding probability density (8.20) does not decrease [157]. If we denote the GMM parameter estimate obtained after r iterations of Algorithm 14 by $\boldsymbol{\theta}^{(r)}$ [58, Sec. 8.5.2],

$$p\big(\mathcal{D}; \boldsymbol{\theta}^{(r+1)}\big) \geq p\big(\mathcal{D}; \boldsymbol{\theta}^{(r)}\big) \tag{8.21}$$

As for Algorithm 12, we can also interpret Algorithm 14 as an instance of the ERM prin-

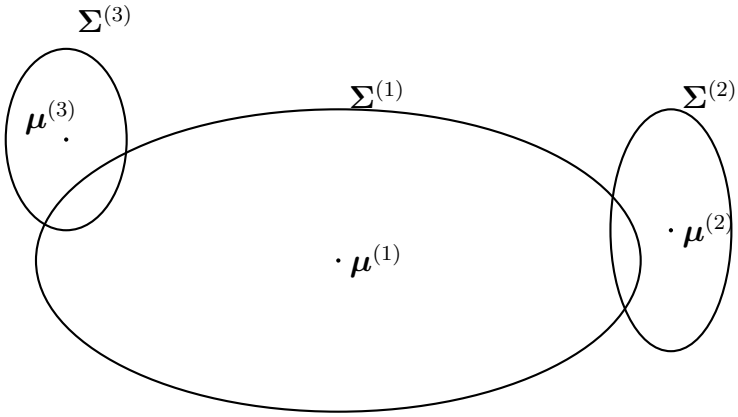


Figure 8.7: The GMM (8.15), (8.16) results in a probability distribution (8.17) for (feature vectors of) data points which is a weighted sum of multivariate normal distributions $\mathcal{N}(\boldsymbol{\mu}^{(c)}, \boldsymbol{\Sigma}^{(c)})$. The weight of the c-th component is the cluster probability $p(c^{(i)} = c)$.

Algorithm 14 "A Soft-Clustering Algorithm" [12]

Input: dataset $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^m$; number k of clusters, initial GMM parameter estimates $\{\widehat{\boldsymbol{\mu}}^{(c)}, \widehat{\boldsymbol{\Sigma}}^{(c)}, \hat{p}_c\}_{c=1}^k$

1: **repeat**

2: for each $i = 1, \ldots, m$ and $c = 1, \ldots, k$, update degrees of belonging

$$\hat{y}_c^{(i)} := \frac{\hat{p}_c \mathcal{N}(\mathbf{x}^{(i)}; \widehat{\boldsymbol{\mu}}^{(c)}, \widehat{\boldsymbol{\Sigma}}^{(c)})}{\sum_{c'=1}^k \hat{p}_{c'} \mathcal{N}(\mathbf{x}^{(i)}; \widehat{\boldsymbol{\mu}}^{(c')}, \widehat{\boldsymbol{\Sigma}}^{(c')})} \tag{8.22}$$

3: for each $c \in \{1, \ldots, k\}$, update GMM parameter estimates:

- $\hat{p}_c := m_c/m$ with effective cluster size $m_c := \sum_{i=1}^m \hat{y}_c^{(i)}$ (cluster probability)
- $\widehat{\boldsymbol{\mu}}^{(c)} := (1/m_c) \sum_{i=1}^m \hat{y}_c^{(i)} \mathbf{x}^{(i)}$ (cluster mean)
- $\widehat{\boldsymbol{\Sigma}}^{(c)} := (1/m_c) \sum_{i=1}^m \hat{y}_c^{(i)} \big(\mathbf{x}^{(i)} - \widehat{\boldsymbol{\mu}}^{(c)}\big)\big(\mathbf{x}^{(i)} - \widehat{\boldsymbol{\mu}}^{(c)}\big)^T$ (cluster covariance matrix)

4: **until** stopping criterion met

Output: predicted degrees of belonging $\widehat{\mathbf{y}}^{(i)} = (\hat{y}_1^{(i)}, \ldots, \hat{y}_k^{(i)})^T$ for $i = 1, \ldots, m$.

ciple discussed in Chapter 4. Indeed, maximizing the probability density (8.20) is equivalent to minimizing the empirical risk

$$\widehat{L}\big(\boldsymbol{\theta} \mid \mathcal{D}\big) := -\log p\big(\mathcal{D}; \boldsymbol{\theta}\big) \text{ with GMM parameters } \boldsymbol{\theta} := \{\boldsymbol{\mu}^{(c)}, \boldsymbol{\Sigma}^{(c)}, p_c\}_{c=1}^{k} \tag{8.23}$$

The empirical risk (8.23) is the negative logarithm of the probability (density) (8.20) of observing the dataset $\mathcal{D}$ as i.i.d. realizations of the GMM (8.17). The monotone increase in the probability density (8.21) achieved by the iterations of Algorithm 14 translate into a monotone decrease of the empirical risk,

$$\widehat{L}\big(\boldsymbol{\theta}^{(r)} \mid \mathcal{D}\big) \leq \widehat{L}\big(\boldsymbol{\theta}^{(r-1)} \mid \mathcal{D}\big) \text{ with iteration counter } r. \tag{8.24}$$

The monotone decrease (8.24) in the empirical risk (8.23) achieved by the iterations of Algorithm 14 naturally lends to a stopping criterion. Let E_r denote the empirical risk (8.23) achieved by the GMM parameter estimates $\boldsymbol{\theta}^{(r)}$ obtained after r iterations in Algorithm 14. Algorithm 14 stops iterating as soon as the decrease $E_{r-1} - E_r$ achieved by the r-th iteration of Algorithm 14 falls below a given (positive) threshold $\varepsilon > 0$.

Similar to Algorithm 12, also Algorithm 14 might get trapped in local minima of the underlying empirical risk. The GMM parameters delivered by Algorithm 14 might only be a local minimum of (8.23) but not the global minimum (see Figure 8.5 for the analogous situation in hard clustering). As for hard clustering Algorithm 12, we typically repeat Algorithm 14 several times. During each repetition of Algorithm 14, we use a different (randomly chosen) initialization for the GMM parameter estimates $\boldsymbol{\theta} = \{\widehat{\boldsymbol{\mu}}^{(c)}, \widehat{\boldsymbol{\Sigma}}^{(c)}, \hat{p}_c\}_{c=1}^{k}$. Each repetition of Algorithm 14 results in a potentially different set of GMM parameter estimates and degrees of belongings $\hat{y}_c^{(i)}$. We then use the results for that repetition that achieves the smallest empirical risk (8.23).

Let us point out an interesting link between soft clustering methods based on GMM (see Algorithm 14) and hard clustering with k-means (see Algorithm 12). Consider the GMM (8.15) with prescribed cluster covariance matrices

$$\boldsymbol{\Sigma}^{(c)} = \sigma^2 \mathbf{I} \text{ for all } c \in \{1, \ldots, k\}, \tag{8.25}$$

with some given variance $\sigma^2 > 0$. We assume the cluster covariance matrices in the GMM to be given by (8.25) and therefore can replace the covariance matrix updates in Algorithm 14 with the assignment $\widehat{\boldsymbol{\Sigma}}^{(c)} := \sigma^2 \mathbf{I}$. It can be verified easily that for sufficiently small variance

σ^2 in (8.25), the update (8.22) tends to enforce $\hat{y}_c^{(i)} \in \{0,1\}$. In other words, each data point $\mathbf{x}^{(i)}$ becomes then effectively associated with exactly one single cluster c whose cluster mean $\widehat{\boldsymbol{\mu}}^{(c)}$ is nearest to $\mathbf{x}^{(i)}$. For $\sigma^2 \to 0$, the soft clustering update (8.22) in Algorithm 14 reduces to the (hard) cluster assignment update (8.6) in k-means Algorithm 12. We can interpret Algorithm 12 as an extreme case of Algorithm 14 that is obtained by fixing the covariance matrices in the GMM to $\sigma^2\mathbf{I}$ with a sufficiently small σ^2.

Combining GMM with linear regression. Let us sketch how Algorithm 14 could be combined with linear regression methods (see Section 3.1). The idea is to first compute the degree of belongings to the clusters for each data point. We then learn separate linear predictors for each cluster using the degree of belongings as weights for the individual loss terms in the training error. To predict the label of a new data point, we first compute the predictions obtained for each cluster-specific linear hypothesis. These cluster-specific predictions are then averaged using the degree of belongings for the new data point as weights.

8.3 Connectivity-based Clustering

The clustering methods discussed in Sections 8.1 and 8.2 can only be applied to data points which are characterized by numeric feature vectors. These methods define the similarity between data points using the Euclidean distance between the feature vectors of these data points. As illustrated in Figure 8.8, these methods can only produce "Euclidean shaped" clusters that are contained either within hyper-spheres (Algorithm 12) or hyper-ellipsoids (Algorithm 14).

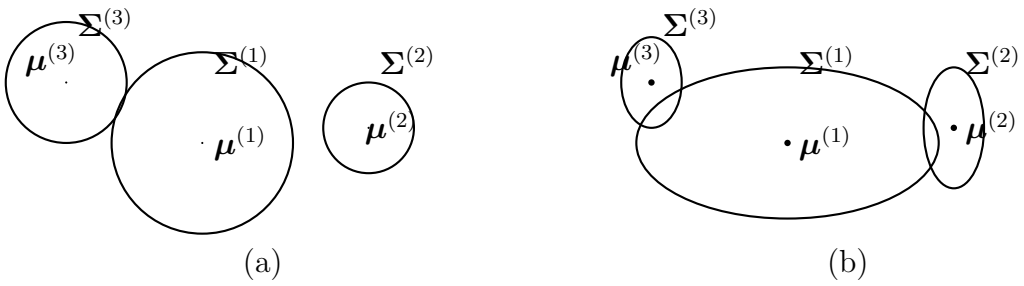


Figure 8.8: (a): Cartoon of typical cluster shapes delivered by k-means Algorithm 13. (b): Cartoon of typical cluster shapes delivered by soft clustering Algorithm 14.

Some applications generate data points for which the construction of useful numeric

features is difficult. Even if we can easily obtain numeric features for data points, the Euclidean distances between them might not reflect the actual similarities between data points. As a case in point, consider data points representing text documents. We could use the histogram of a pre-specified list (dictionary) of words as numeric features for a text document.

In general, a small Euclidean distance between histograms of text documents does not imply that the text documents have similar meanings. Moreover, clusters of similar text documents might have highly complicated shapes in the space of feature vectors that cannot be grouped within hyper-ellipsoids. For datasets with such "non-Euclidean" cluster shapes, k-means or GMM are not suitable as clustering methods. We should then replace the Euclidean distance between feature vectors with another concept to determine or measure the similarity between data points.

Connectivity-based clustering methods do not require any numeric features of data points. These methods cluster data points based on explicitly specifying for any two different data points if they are similar and to what extent. A convenient mathematical tool to represent similarities between the data points of a dataset $\mathcal{D}$ is a weighted undirected graph $\mathcal{G} = \big(\mathcal{V}, \mathcal{E}\big)$. We refer to this graph as the similarity graph of the dataset $\mathcal{D}$ (see Figure 8.9). The nodes $\mathcal{V}$ in this similarity graph $\mathcal{G}$ represent data points in $\mathcal{D}$ and the undirected edges connect nodes that represent similar data points. The extent of similarity is represented by the weights $W_{i,i'}$ for each edge $\{i, i'\} \in \mathcal{E}$.

Given a similarity graph $\mathcal{G}$ of a dataset, connectivity-based clustering methods determine clusters as subsets of nodes that are well connected within the cluster but weakly connected between different clusters. Different concepts for quantifying the connectivity between nodes in a graph yield different clustering methods:

- Spectral clustering methods use eigenvectors of a graph Laplacian matrix to measure the connectivity between nodes [148, 106].

- Flow-based clustering methods measure the connectivity between two nodes via the amount of flow that can be routed between them [73].

We can use connectivity measures between nodes of an empirical graph to construct meaningful numerical feature vectors ("embeddings") for these nodes. These feature vectors can then be fed into the hard-clustering Algorithm 13 or the soft clustering Algorithm 14 (see Figure 8.9).

The algorithm density-based spatial clustering of applications with noise (DBSCAN) considers two data points i, i' as connected if one of them (say i) is a core node and the other node (i') can be reached via a sequence (path) of connected core nodes

$$i^{(1)}, \ldots, i^{(r)} \text{ , with } \{i, i^{(1)}\}, \{i^{(1)}, i^{(2)}\}, \ldots, \{i^{(r)}, i'\} \in \mathcal{E}.$$

DBSCAN considers a node to be a core node if it has a sufficiently large number of neighbours [35]. The minimum number of neighbours required for a node to be considered a core node is a hyper-parameter of DBSCAN. When DBSCAN is applied to data points with numeric feature vectors, it defines two data points as connected if the Euclidean distance between their feature vectors does not exceed a given threshold ε (see Figure 8.10).

In contrast to k-means and GMM, DBSCAN does not require the number of clusters to be specified. The number of clusters is determined automatically by DBSCAN and depends on its hyper-parameters. DBSCAN also performs an implicit outlier detection. The outliers delivered by DBSCAN are those data points which do not belong to the same cluster as any other data point.

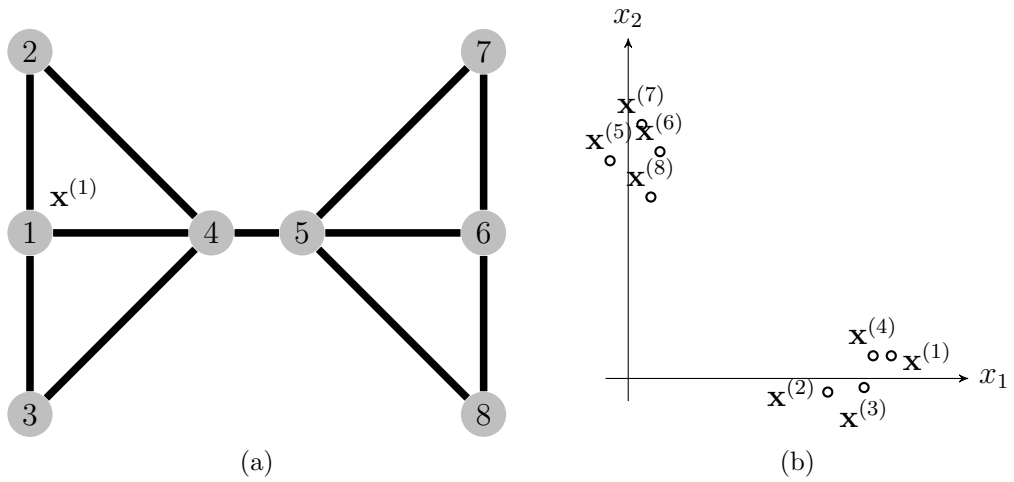


Figure 8.9: Connectivity-based clustering can be obtained by constructing features $\mathbf{x}^{(i)}$ that are (approximately) identical for well-connected data points. (a): A similarity graph for a dataset $\mathcal{D}$ consists of nodes representing individual data points and edges that connect similar data points. (b) Feature vectors of well-connected data points have small Euclidean distance.

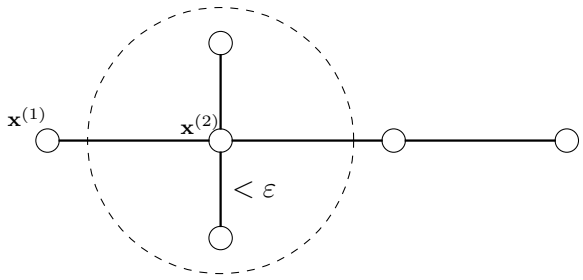


Figure 8.10: DBSCAN assigns two data points to the same cluster if they are reachable. Two data points $\mathbf{x}^{(i)}, \mathbf{x}^{(i')}$ are reachable if there is a path of data points from $\mathbf{x}^{(i')}$ to $\mathbf{x}^{(i)}$. This path consists of a sequence of data points that are within a distance of ε. Moreover, each data point on this path must be a core point which has at least a given number of neighbouring data points within the distance ε.

8.4 Clustering as Preprocessing

In applications it might be beneficial to combine clustering methods with supervised methods such as linear regression. As a point in case, consider a dataset that consists of data points obtained from two different data generation processes (such as two weather observation stations, one located in Helsinki and the other one in Lapland). Let us denote the data points generated by one process by $\mathcal{D}^{(1)}$ and the other one by $\mathcal{D}^{(2)}$. Each data point is characterized by several numeric features and a numeric label. While there would be an accurate linear hypothesis for predicting the label of data points in $\mathcal{D}^{(1)}$ and another linear hypothesis for $\mathcal{D}^{(2)}$ these two are very different. Formally, this approach amounts to using the cluster assignment of a data points as an additional feature. We could also use the degree of belonging delivered by soft clustering methods as additional features.

We could try to use clustering methods to assign any given data point to the corresponding data generation process. If we are lucky, the resulting clusters resemble (approximately) the two data generation processes $\mathcal{D}^{(1)}$ and $\mathcal{D}^{(2)}$. Once we have successfully clustered the data points, we can learn a separate (tailored) hypothesis for each cluster. More generally, we can use the predicted cluster assignments obtained from the methods of Section 8.1 - 8.3 as additional features for each data point.

Let us illustrate the above ideas by combining Algorithm 12 with linear regression. We first group data points into a given number k of clusters and then learn separate linear predictors $h^{(c)}(\mathbf{x}) = \left(\mathbf{w}^{(c)}\right)^{T}\mathbf{x}$ for each cluster $c = 1, \ldots, k$. To predict the label of a new data point with features $\mathbf{x}$, we first assign to the cluster c' with the nearest cluster mean.

We then use the linear predictor $h^{(c')}$ assigned to cluster c' to compute the predicted label $\hat{y} = h^{(c')}(\mathbf{x})$.

8.5 Exercises

Exercise 8.1. Monotonicity of k-means Updates. Show that the updates (8.7) and (8.6) for cluster means and cluster assignments, respectively, never result in an increase of the clustering error (8.3).

Exercise 8.2. How to choose k in k-means? Discuss and experiment with different strategies for choosing the number k of clusters in k-means Algorithm 13.

Exercise 8.3. Local Minima. Apply the hard clustering Algorithm 13 to the dataset $(-10, 1), (10, 1), (-10, -1), (10, -1)$ with initial cluster means $(0, 1), (0, -1)$ and tolerance $\varepsilon = 0$. For this initialization, will Algorithm 13 get trapped in a local minimum of the clustering error (8.13)?

Exercise 8.4. Image Compression with k-means. Apply k-means to image compression. Consider image pixels as data points whose features are RGB intensities. We obtain a simple image compression format by, instead of storing RGB pixel values, storing the cluster means (which are RGB triplets) and the cluster index for each pixel. Try out different values for the number k of clusters and discuss the resulting trade off between achievable reconstruction quality and storage size.

Exercise 8.5. Compression with k-means. Consider $m = 10000$ data points $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}$ which are represented by two numeric features. We apply k-means to cluster the data set into $k = 5$ clusters. How many bits do we need to store the resulting cluster assignments?

Chapter 9

Feature Learning

"Solving Problems By Changing the Viewpoint."

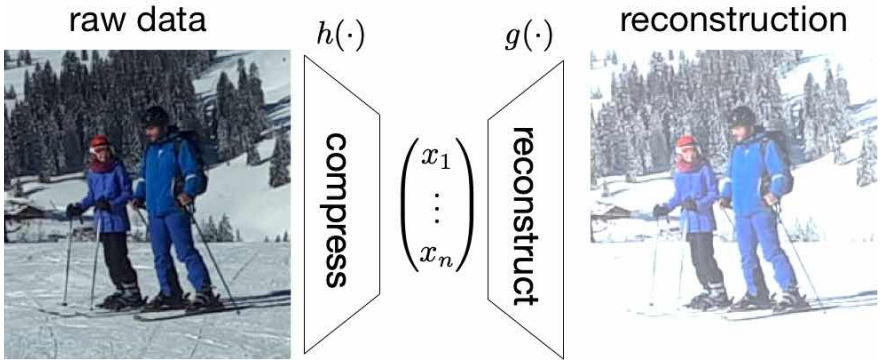


Figure 9.1: Dimensionality reduction methods aim at finding a map h which maximally compresses the raw data while still allowing to accurately reconstruct the original datapoint from a small number of features $x_1, \ldots, x_n$.

Chapter 2 defined features as those properties of a data point that can be measured or computed easily. Sometimes the choice of features follows naturally from the available hard- and software. For example, we might use the numeric measurement $z \in \mathbb{R}$ delivered by a sensing device as a feature. However, we could augment this single feature with new features such as the powers z^2 and z^3 or adding a constant $z+5$. Each of these computations produces a new feature. Which of these additional features are most useful?

Feature learning methods automate the choice of finding good features. These methods learn a hypothesis map that reads in some representation of a data point and transforms it to a set of features. Feature learning methods differ in the precise format of the original

data representation as well as the format of the delivered features. This chapter mainly discusses feature learning methods that require data points being represented by n' numeric raw features and deliver a set of n new numeric features. We will denote the raw features and the learnt new features by $\mathbf{z} = \big(z_1, \ldots, z_{n'}\big)^T \in \mathbb{R}^{n'}$ and $\mathbf{x} = \big(x_1, \ldots, x_n\big)^T \in \mathbb{R}^n$, respectively.

Many ML application domains generate data points for which can access a huge number of raw features. Consider data points being snapshots generated by a smartphone. It seems natural to use the pixel colour intensities as the raw features of the snapshot. Since modern smartphone have "Megapixel cameras", the pixel intensities would provide us with millions of raw features. It might seem a good idea to use as many raw features of a data point as possible since more features should offer more information about a data point and its label y. There are, however, two pitfalls in using an unnecessarily large number of features. The first one is a computational pitfall and the second one is a statistical pitfall.

Computationally, using very long feature vectors $\mathbf{x} \in \mathbb{R}^n$ (with n being billions), might result in prohibitive computational resource requirements (bandwidth, storage, time) of the resulting ML method. Statistically, using a large number of features makes the resulting ML methods more prone to overfitting. For example, linear regression will typically overfit when using feature vectors $\mathbf{x} \in \mathbb{R}^n$ whose length n exceeds the number m of labeled data points used for training (see Chapter 7).

Both from a computational and a statistical perspective, it is beneficial to use only the maximum necessary amount of features. The challenge is to select those features which carry most of the relevant information required for the prediction of the label y. Finding the most relevant features out of a huge number of raw features is the goal of dimensionality reduction methods. Dimensionality reduction methods form an important sub-class of feature learning methods. These methods learn a hypothesis $h(\mathbf{z})$ that maps a long raw feature vector $\mathbf{z} \in \mathbb{R}^{n'}$ to a new (short) feature vector $\mathbf{x} \in \mathbb{R}^n$ with $n \ll n'$.

Beside avoiding overfitting and coping with limited computational resources, dimensionality reduction can also be useful for data visualization. Indeed, if the resulting feature vector has length $n = 2$, we depict data points in the two-dimensional plane in form of a scatterplot.

We will discuss the basic idea underlying dimensionality reduction methods in Section 9.1. Section 9.2 presents one particular example of a dimensionality reduction method that computes relevant features by a linear transformation of the raw feature vector. Section 9.4 discusses a method for dimensionality reduction that exploits the availability of labelled data points. Section 9.6 shows how randomness can be used to obtain computationally cheap

dimensionality reduction .

Most of this chapter discusses dimensionality reduction methods that determine a small number of relevant features from a large set of raw features. However, sometimes it might be useful to go the opposite direction. There are applications where it might be beneficial to construct a large (even infinite) number of new features from a small set of raw features. Section 9.7 will showcase how computing additional features can help to improve the prediction accuracy of ML methods.

9.1 Basic Principle of Dimensionality Reduction

The efficiency of ML methods depends crucially on the choice of features that are used to characterize data points. Ideally we would like to have a small number of highly relevant features to characterize data points. If we use too many features we risk to waste computations on exploring irrelevant features. If we use too few features we might not have enough information to predict the label of a data point. For a given number n of features, dimensionality reduction methods aim at learning an (in a certain sense) optimal map from the data point to a feature vector of length n.

Figure 9.1 illustrates the basic idea of dimensionality reduction methods. Their goal is to learn (or find) a "compression" map $h(\cdot): \mathbb{R}^{n'} \rightarrow \mathbb{R}^{n}$ that transforms a (long) raw feature vector $\mathbf{z} \in \mathbb{R}^{n'}$ to a (short) feature vector $\mathbf{x} = (x_1, \ldots, x_n)^T := h(\mathbf{z})$ (typically $n \ll n'$).

The new feature vector $\mathbf{x} = h(\mathbf{z})$ serves as a compressed representation (or code) for the original feature vector $\mathbf{z}$. We can reconstruct the raw feature vector using a reconstruction map $r(\cdot): \mathbb{R}^{n} \rightarrow \mathbb{R}^{n'}$. The reconstructed raw features $\widehat{\mathbf{z}} := r(\mathbf{x}) = r(h(\mathbf{z}))$ will typically differ from the original raw feature vector $\mathbf{z}$. In general, we obtain a non-zero reconstruction error

$$\underbrace{\widehat{\mathbf{z}}}_{=r(h(\mathbf{z})))} - \mathbf{z}. \tag{9.1}$$

Dimensionality reduction methods learn a compression map $h(\cdot)$ such that the reconstruction error (9.1) is minimized. In particular, for a dataset $\mathcal{D} = \big\{\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(m)}\big\}$, we measure the quality of a pair of compression map h and reconstruction map r by the average reconstruction error

$$\widehat{L}\big(h, r|\mathcal{D}\big) := (1/m) \sum_{i=1}^{m} L\left(\mathbf{z}^{(i)}, r\big(h\big(\mathbf{z}^{(i)}\big)\big)\right). \tag{9.2}$$

Here, $L\left(\mathbf{z}, r\big(h\big(\mathbf{z}^{(i)}\big)\big)\right)$ denotes a loss function that is used to measure the reconstruction error $\underbrace{r\big(h\big(\mathbf{z}^{(i)}\big)\big)}_{\widehat{\mathbf{z}}} - \mathbf{z}$. Different choices for the loss function in (9.2) result in different dimensionality reduction methods. One widely-used choice for the loss is the squared Euclidean norm

$$L\left(\mathbf{z}, g\big(h\big(\mathbf{z}\big)\big)\right) := \big\|\mathbf{z} - g\big(h\big(\mathbf{z}\big)\big)\big\|_2^2. \tag{9.3}$$

Practical dimensionality reduction methods have only finite computational resources. Any practical method must therefore restrict the set of possible compression and reconstruction maps to small subsets $\mathcal{H}$ and $\mathcal{H}^*$, respectively. These subsets are the hypothesis spaces for the compression map $h \in \mathcal{H}$ and the reconstruction map $r \in \mathcal{H}^*$. Feature learning methods differ in their choice for these hypothesis spaces.

Dimensionality reduction methods learn a compression map by solving

$$\begin{aligned} \widehat{h} &= \operatorname*{argmin}_{h \in \mathcal{H}} \min_{r \in \mathcal{H}^*} \widehat{L}\big(h, r|\mathcal{D}\big) \\ &\overset{(9.2)}{=} \operatorname*{argmin}_{h \in \mathcal{H}} \min_{r \in \mathcal{H}^*} (1/m) \sum_{i=1}^{m} L\left(\mathbf{z}^{(i)}, r\big(h\big(\mathbf{z}^{(i)}\big)\big)\right). \end{aligned} \tag{9.4}$$

We can interpret (9.4) as a (typically non-linear) approximation problem. The optimal compression map $\widehat{h}$ is such that the reconstruction$r(\widehat{h}(\mathbf{z}))$, with a suitably chosen reconstruction map r, approximates the original raw feature vector $\mathbf{z}$ with optimal accuracy. Note that we use a single compression map $h(\cdot)$ and a single reconstruction map $r(\cdot)$ for all data points in the dataset $\mathcal{D}$.

We obtain variety of dimensionality methods by using different choices for the hypothesis spaces $\mathcal{H}, \mathcal{H}^*$ and loss function in (9.4). Section 9.2 discusses a method that solves (9.4) for $\mathcal{H}, \mathcal{H}^*$ constituted by linear maps and the loss (9.3). Deep autoencoders are another family of dimensionality reduction methods that solve (9.4) with $\mathcal{H}, \mathcal{H}^*$ constituted by non-linear maps that are represented by deep neural networks [48, Ch. 14].

9.2 Principal Component Analysis

We now consider the special case of dimensionality reduction where the compression and reconstruction map are required to be linear maps. Consider a data point which is characterized by a (typically very long) raw feature vector $\mathbf{z} = \big(z_1, \ldots, z_{n'}\big)^T \in \mathbb{R}^{n'}$ of length n'. The length n' of the raw feature vector might be easily of the order of millions. To obtain a small

set of relevant features $\mathbf{x} = \big(x_1, \ldots, x_n\big)^T \in \mathbb{R}^n$, we apply a linear transformation to the raw feature vector,

$$\mathbf{x} = \mathbf{W}\mathbf{z}. \tag{9.5}$$

Here, the "compression" matrix $\mathbf{W} \in \mathbb{R}^{n \times n'}$ maps (in a linear fashion) the (long) raw feature vector $\mathbf{z} \in \mathbb{R}^{n'}$ to the (shorter) feature vector $\mathbf{x} \in \mathbb{R}^n$.

It is reasonable to choose the compression matrix $\mathbf{W} \in \mathbb{R}^{n \times n'}$ in (9.5) such that the resulting features $\mathbf{x} \in \mathbb{R}^n$ allow to approximate the original data point $\mathbf{z} \in \mathbb{R}^{n'}$ as accurate as possible. We can approximate (or recover) the data point $\mathbf{z} \in \mathbb{R}^{n'}$ back from the features $\mathbf{x}$ by applying a reconstruction operator $\mathbf{R} \in \mathbb{R}^{n' \times n}$, which is chosen such that

$$\mathbf{z} \approx \mathbf{R}\mathbf{x} \overset{(9.5)}{=} \mathbf{R}\mathbf{W}\mathbf{z}. \tag{9.6}$$

The approximation error $\widehat{L}\big(\mathbf{W}, \mathbf{R} \mid \mathcal{D}\big)$ resulting when (9.6) is applied to each data point in a dataset $\mathcal{D} = \{\mathbf{z}^{(i)}\}_{i=1}^m$ is then

$$\widehat{L}\big(\mathbf{W}, \mathbf{R} \mid \mathcal{D}\big) = (1/m) \sum_{i=1}^{m} \|\mathbf{z}^{(i)} - \mathbf{R}\mathbf{W}\mathbf{z}^{(i)}\|^2. \tag{9.7}$$

One can verify that the approximation error $\widehat{L}\big(\mathbf{W}, \mathbf{R} \mid \mathcal{D}\big)$ can only by minimal if the compression matrix $\mathbf{W}$ is of the form

$$\mathbf{W} = \mathbf{W}_{\rm PCA} := \big(\mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(n)}\big)^T \in \mathbb{R}^{n \times n'}, \tag{9.8}$$

with n orthonormal vectors $\mathbf{u}^{(j)}$, for $j = 1, \ldots, n$. The vectors $\mathbf{u}^{(j)}$ are the eigenvectors corresponding to the n largest eigenvalues of the sample covariance matrix

$$\mathbf{Q} := (1/m)\mathbf{Z}^T\mathbf{Z} \in \mathbb{R}^{n' \times n'}. \tag{9.9}$$

Here we used the data matrix $\mathbf{Z} = \big(\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(m)}\big)^T \in \mathbb{R}^{m \times n'}$.[1] It can be verified easily, using the definition (9.9), that the matrix $\mathbf{Q}$ is psd. As a psd matrix, $\mathbf{Q}$ has an eigenvalue

[1] Some authors define the data matrix as $\mathbf{Z} = \big(\widetilde{\mathbf{z}}^{(1)}, \ldots, \widetilde{\mathbf{z}}^{(m)}\big)^T \in \mathbb{R}^{m \times D}$ using "centered" data points $\mathbf{z}^{(i)} - \widehat{\mathbf{m}}$ obtained by subtracting the average $\widehat{\mathbf{m}} = (1/m)\sum_{i=1}^m \mathbf{z}^{(i)}$.

decomposition (EVD) of the form [135]

$$\mathbf{Q} = \big(\mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(n')}\big) \begin{pmatrix} \lambda_1 & \ldots & 0 \\ 0 & \ddots & 0 \\ 0 & \ldots & \lambda_{n'} \end{pmatrix} \big(\mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(n')}\big)^T \tag{9.10}$$

with real-valued eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_{n'} \geq 0$ and orthonormal eigenvectors $\{\mathbf{u}^{(j)}\}_{j=1}^{n'}$.

The feature vectors $\mathbf{x}^{(i)}$ are obtained by applying the compression matrix $\mathbf{W}_{\rm PCA}$ (9.8) to the raw feature vectors $\mathbf{z}^{(i)}$. We refer to the entries of the learnt feature vector $\mathbf{x}^{(i)} = \mathbf{W}_{\rm PCA}\mathbf{z}^{(i)}$ (see (9.8)) as the principal components (PC) of the raw feature vectors $\mathbf{z}^{(i)}$. Algorithm 15 summarizes the overall procedure of determining the compression matrix (9.8) and computing the learnt feature vectors $\mathbf{x}^{(i)}$. This procedure is known as PCA. The length

Algorithm 15 PCA

Input: dataset $\mathcal{D} = \{\mathbf{z}^{(i)} \in \mathbb{R}^{n'}\}_{i=1}^{m}$; number n of PCs.

1: compute the EVD (9.10) to obtain orthonormal eigenvectors $\big(\mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(n')}\big)$ corresponding to (decreasingly ordered) eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_{n'} \geq 0$

2: construct compression matrix $\mathbf{W}_{\rm PCA} := \big(\mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(n)}\big)^T \in \mathbb{R}^{n \times n'}$

3: compute feature vector $\mathbf{x}^{(i)} = \mathbf{W}_{\rm PCA}\mathbf{z}^{(i)}$ whose entries are PC of $\mathbf{z}^{(i)}$

4: compute approximation error $\widehat{L}^{(\rm PCA)} = \sum_{j=n+1}^{n'} \lambda_j$ (see (9.11)).

Output: $\mathbf{x}^{(i)}$, for $i = 1, \ldots, m$, and the approximation error $\widehat{L}^{(\rm PCA)}$.

$n \in \{0, \ldots, n')$ of the delivered feature vectors $\mathbf{x}^{(i)}$, for $i = 1, \ldots, m$, is an input (or hyper-) parameter of Algorithm 15. Two extreme cases are $n = 0$ (maximum compression) and $n = n'$ (no compression). We finally note that the choice for the orthonormal eigenvectors in (9.8) might not be unique. Depending on the sample covariance matrix $\mathbf{Q}$, there might be different sets of orthonormal vectors that correspond to the same eigenvalue of $\mathbf{Q}$. Thus, for a given length n of the new feature vectors, there might be several different matrices $\mathbf{W}$ that achieve the same (optimal) reconstruction error $\widehat{L}^{(\rm PCA)}$.

Computationally, Algorithm 15 essentially amounts to an EVD of the sample covariance matrix $\mathbf{Q}$ (9.9). The EVD of $\mathbf{Q}$ provides not only the optimal compression matrix $\mathbf{W}_{\rm PCA}$ but also the measure $\widehat{L}^{(\rm PCA)}$ for the information loss incurred by replacing the original data points $\mathbf{z}^{(i)} \in \mathbb{R}^{n'}$ with the shorter feature vector $\mathbf{x}^{(i)} \in \mathbb{R}^{n}$. We quantify this information loss by the approximation error obtained when using the compression matrix $\mathbf{W}_{\rm PCA}$ (and

corresponding reconstruction matrix $\mathbf{R}_{\rm opt} = \mathbf{W}_{\rm PCA}^T$),

$$\widehat{L}^{(\rm PCA)} := \widehat{L}\big(\mathbf{W}_{\rm PCA}, \underbrace{\mathbf{R}_{\rm opt}}_{=\mathbf{W}_{\rm PCA}^T} \mid \mathcal{D}\big) = \sum_{j=n+1}^{n'} \lambda_j. \tag{9.11}$$

As depicted in Figure 9.2, the approximation error $\widehat{L}^{(\rm PCA)}$ decreases with increasing number n of PCs used for the new features (9.5). For the extreme case $n=0$, where we completely ignore the raw feature vectors $\mathbf{z}^{(i)}$, the optimal reconstruction error is $\widehat{L}^{(\rm PCA)} = (1/m)\sum_{i=1}^m \big\|\mathbf{z}^{(i)}\big\|^2$. The other extreme case $n = n'$ allows to use the raw features directly as the new features $\mathbf{x}^{(i)} = \mathbf{z}^{(i)}$. This extreme case means no compression at all, and trivially results in a zero reconstruction error $\widehat{L}^{(\rm PCA)}=0$.

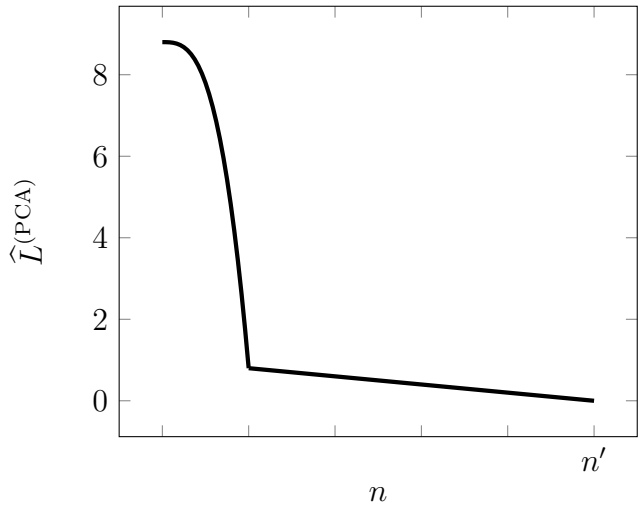


Figure 9.2: Reconstruction error $\widehat{L}^{(\rm PCA)}$ (see (9.11)) of PCA for varying number n of PCs.

9.2.1 Combining PCA with linear regression

One important use case of PCA is as a pre-processing step within an overall ML problem such as linear regression (see Section 3.1). As discussed in Chapter 7, linear regression methods are prone to overfitting whenever the data points are characterized by raw feature vectors $\mathbf{z}$ whose length n' exceeds the number m of labeled data points used in ERM.

One simple but powerful strategy to avoid overfitting is to preprocess the raw features $\mathbf{z}^{(i)} \in \mathbb{R}^{n'}$, for $i = 1, \ldots, m$ by applying PCA. Indeed, PCA Algorithm 15 delivers feature

vectors $\mathbf{x}^{(i)} \in \mathbb{R}^n$ of prescribed length n. Thus, choosing the parameter n such that $n < m$ will typically prevent the follow-up linear regression method from overfitting.

9.2.2 How To Choose Number of PC?

There are several aspects which can guide the choice for the number n of PCs to be used as features.

- To generate data visualizations we might use either $n = 2$ or $n = 3$.
- We should choose n sufficiently small such that the overall ML method fits the available computational resources.
- Consider using PCA as a pre-processing for linear regression (see Section 3.1). In particular, we can use the learnt feature vectors $\mathbf{x}^{(i)}$ delivered by PCA as the feature vectors of data points in plain linear regression methods. To avoid overfitting, we should choose $n < m$ (see Chapter 7).
- Choose n large enough such that the resulting approximation error $\widehat{L}^{(\mathrm{PCA})}$ is reasonably small (see Figure 9.2).

9.2.3 Data Visualisation

If we use PCA with $n = 2$, we obtain feature vectors $\mathbf{x}^{(i)} = \mathbf{W}_{\mathrm{PCA}}\mathbf{z}^{(i)}$ (see (9.5)) which can be depicted as points in a scatterplot (see Section 2.1.3). As an example, consider data points $\mathbf{z}^{(i)}$ obtained from historic recordings of Bitcoin statistics. Each data point $\mathbf{z}^{(i)} \in \mathbb{R}^{n'}$ is a vector of length $n' = 6$. It is difficult to visualise points in an Euclidean space $\mathbb{R}^{n'}$ of dimension $n' > 2$. Therefore, we apply PCA with $n = 2$ which results in feature vectors $\mathbf{x}^{(i)} \in \mathbb{R}^2$. These new feature vectors (of length 2) can be depicted conveniently as the scatterplot in Figure 9.3.

9.2.4 Extensions of PCA

We now briefly discuss variants and extensions of the basic PCA method.

- **Kernel PCA [58, Ch.14.5.4]:** The PCA method is most effective if the raw feature vectors of data points are concentrated around a n-dimensional linear subspace of $\mathbb{R}^{n'}$. Kernel PCA extends PCA to data points that are located near a low-dimensional

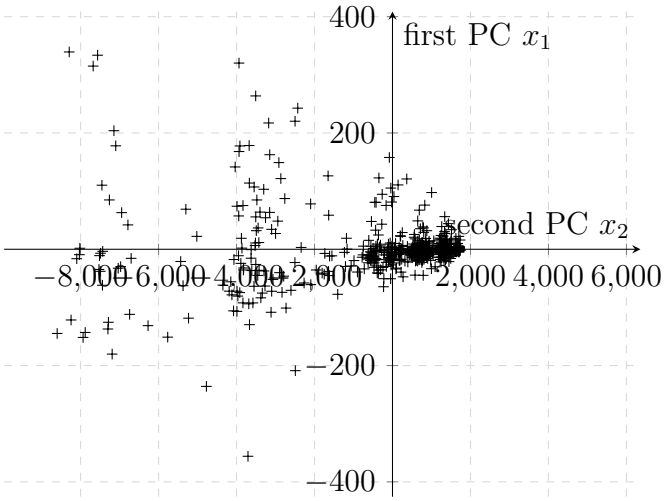


Figure 9.3: A scatterplot of data points with feature vectors $\mathbf{x}^{(i)} = \big(x_1^{(i)}, x_2^{(i)}\big)^T$ whose entries are the first two PCs of the Bitcoin statistics $\mathbf{z}^{(i)}$ of the i-th day.

manifold which might be highly non-linear. This is achieved by applying PCA to transformed feature vectors instead of the original raw feature vectors. Kernel PCA first applies a (typically non-linear) feature map ϕ to the raw feature vectors $\mathbf{z}^{(i)}$ (see Section 3.9) and applies PCA to the transformed feature vectors $\phi\big(\mathbf{z}^{(i)}\big)$, for $i = 1, \ldots, m$.

- **Robust PCA [156]:** The basic PCA Algorithm 15 is sensitive to outliers, i.e., a small number of data points with significantly different statistical properties than the bulk of data points. This sensitivity might be attributed to the properties of the squared Euclidean norm (9.3) which is used in PCA to measure the reconstruction error (9.1). We have seen in Chapter 3 that linear regression (see Section 3.1 and 3.3) can be made robust against outliers by replacing the squared error loss with another loss function. In a similar spirit, robust PCA replaces the squared Euclidean norm with another norm that is less sensitive to having very large reconstruction errors (9.1) for a small number of data points (which are outliers).

- **Sparse PCA [58, Ch.14.5.5]:** The basic PCA method transforms the raw feature vector $\mathbf{z}^{(i)}$ of a data point to a new (shorter) feature vector $\mathbf{x}^{(i)}$. In general each entry $x_j^{(i)}$ of the new feature vector will depend on each entry of the raw feature vector $\mathbf{z}^{(i)}$. More precisely, the new feature $x_j^{(i)}$ depends on all raw features $z_{j'}^{(i)}$ for which the corresponding entry $W_{j,j'}$ of the matrix $\mathbf{W} = \mathbf{W}_{\rm PCA}$ (9.8) is non-zero. For most

datasets, all entries of the matrix $\mathbf{W}_{\rm PCA}$ will typically be non-zero.

In some applications of linear dimensionality reduction we would like to construct new features that depend only on a small subset of raw features. Equivalently we would like to learn a linear compression map $\mathbf{W}$ (9.5) such that each row of $\mathbf{W}$ contains only few non-zero entries. To this end, sparse PCA enforces the rows of the compression matrix $\mathbf{W}$ to contain only a small number of non-zero entries. This enforcement can be implement either using additional constraints on $\mathbf{W}$ or by adding a penalty term to the reconstruction error (9.7).

- **Probabilistic PCA [118, 138]:** We have motivated PCA as a method for learning an optimal linear compression map (matrix) (9.5) such that the compressed feature vectors allows to linearly reconstruct the original raw feature vector with minimum reconstruction error (9.7). Another interpretation of PCA is that of a method that learns a subspace of $\mathbb{R}^{n'}$ that best fits the distribution of the raw feature vectors $\mathbf{z}^{(i)}$, for $i = 1, \ldots, m$. This optimal subspace is precisely the subspace spanned by the rows of $\mathbf{W}_{\rm PCA}$ (9.8).

  Probabilistic principal component analysis (PPCA) interprets the raw feature vectors $\mathbf{z}^{(i)}$ as realizations of i.i.d. RVs. These realizations are modelled as

$$\mathbf{z}^{(i)} = \mathbf{W}^T \mathbf{x}^{(i)} + \boldsymbol{\varepsilon}^{(i)}, \text{ for } i = 1, \ldots, m. \tag{9.12}$$

  Here, $\mathbf{W} \in \mathbb{R}^{n \times n'}$ is some unknown matrix with orthonormal rows. The rows of $\mathbf{W}$ span the subspace around which the raw features are concentrated. The vectors $\mathbf{x}^{(i)}$ in (9.12) are realizations of i.i.d. RVs whose common probability distribution is $\mathcal{N}(\mathbf{0}, \mathbf{I})$. The vectors $\boldsymbol{\varepsilon}^{(i)}$ are realizations of i.i.d. RVs whose common probability distribution is $\mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$ with some fixed but unknown variance σ^2. Note that $\mathbf{W}$ and σ^2 parametrize the joint probability distribution of the feature vectors $\mathbf{z}^{(i)}$ via (9.12). PPCA amounts to maximum likelihood estimation (see Section 3.12) of the parameters $\mathbf{W}$ and σ^2. This maximum likelihood estimation problem can be solved using computationally efficient estimation techniques such as EM [138, Appendix B]. The implementation of PPCA via EM also offers a principled approach to handle missing data. Roughly speaking, the EM method allows to use the probabilistic model (9.12) to estimate missing raw features [138, Sec. 4.1].

9.3 Feature Learning for Non-Numeric Data

We have motivated dimensionality reduction methods as transformations of (very long) raw feature vectors to a new (shorter) feature vector $\mathbf{x}$ such that it allows to reconstruct the raw features $\mathbf{z}$ with minimum reconstruction error (9.1). To make this requirement precise we need to define a measure for the size of the reconstruction error and specify the class of possible reconstruction maps. PCA uses the squared Euclidean norm (9.7) to measure the reconstruction error and only allows for linear reconstruction maps (9.6).

Alternatively, we can view dimensionality reduction as the generation of new feature vectors $\mathbf{x}^{(i)}$ that maintain the intrinsic geometry of the data points with their raw feature vectors $\mathbf{z}^{(i)}$. Different dimensionality reduction methods use different concepts for characterizing the "intrinsic geometry" of data points. PCA defines the intrinsic geometry of data points using the squared Euclidean distances between feature vectors. Indeed, PCA produces feature vectors $\mathbf{x}^{(i)}$ such that for data points whose raw feature vectors have small squared Euclidean distance, also the new feature vectors $\mathbf{x}^{(i)}$ will have small squared Euclidean distance.

Some application domains generate data points for which the Euclidean distances between raw feature vectors does not reflect the intrinsic geometry of data points. As a point in case, consider data points representing scientific articles which can be characterized by the relative frequencies of words from some given set of relevant words (dictionary). A small Euclidean distance between the resulting raw feature vectors typically does not imply that the corresponding text documents are similar. Instead, the similarity between two articles might depend on the number of authors that are contained in author lists of both papers. We can represent the similarities between all articles using a similarity graph whose nodes represent data points which are connected by an edge (link) if they are similar (see Figure 8.9).

Consider a dataset $\mathcal{D} = \big(\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(m)}\big)$ whose intrinsic geometry is characterized by an unweighted similarity graph $\mathcal{G} = \big(\mathcal{V} := \{1, \ldots, m\}, \mathcal{E}\big)$. The node $i \in \mathcal{V}$ represents the i-th data point $\mathbf{z}^{(i)}$. Two nodes are connected by an undirected edge if the corresponding data points are similar.

We would like to find short feature vectors $\mathbf{x}^{(i)}$, for $i = 1, \ldots, m$, such that two data points i, i', whose feature vectors $\mathbf{x}^{(i)}, \mathbf{x}^{(i')}$ have small Euclidean distance, are well-connected to each other. This informal requirement must be made precise by a measure for how well two nodes of an undirected graph are connected. We refer the reader to literature on network theory for an overview and details of various connectivity measures [103].

Let us discuss a simple but powerful technique to map the nodes $i \in \mathcal{V}$ of an undirected graph $\mathcal{G}$ to (short) feature vectors $\mathbf{x}^{(i)} \in \mathbb{R}^n$. This map is such that the Euclidean distances between the feature vectors of two nodes reflect their connectivity within $\mathcal{G}$. This technique uses the Laplacian matrix $\mathbf{L} \in \mathbb{R}^{m \times m}$ which is defined for an undirected graph $\mathcal{G}$ (with node set $\mathcal{V} = \{1, \ldots, m\}$) element-wise

$$L_{i,i'} := \begin{cases} -1 & \text{, if } \{i, i'\} \in \mathcal{E} \\ d^{(i)} & \text{, if } i = i' \\ 0 & \text{otherwise.} \end{cases} . \tag{9.13}$$

Here, $d^{(i)} := \big|\{i' : \{i, i'\} \in \mathcal{E}\}\big|$ denotes the number of neighbours (the degree) of node $i \in \mathcal{V}$. It can be shown that the Laplacian matrix $\mathbf{L}$ is psd [148, Proposition 1]. Therefore we can find a set of orthonormal eigenvectors

$$\mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(m)} \in \mathbb{R}^m \tag{9.14}$$

with corresponding (ordered in a non-decreasing fashion) eigenvalues $\lambda_1 \leq \ldots \leq \lambda_m$ of $\mathbf{L}$.

For a given number n, we construct the feature vector

$$\mathbf{x}^{(i)} := \big(u_i^{(1)}, \ldots, u_i^{(n)}\big)^T$$

for the ith data point. Here, we used the entries of of the first n eigenvectors (9.14). It can be shown that the Euclidean distances between the feature vectors $\mathbf{x}^{(i)}$, for $i = 1, \ldots, m$, reflect the connectivities between data points $i = 1, \ldots, m$ in the similarity graph $\mathcal{G}$. For a more precise statement of this informal claim we refer to the excellent tutorial [148].

To summarize, we can construct numeric feature vectors for (non-numeric) data points via the eigenvectors of the Laplacian matrix of a similarity graph for the data points. Algorithm 16 summarizes this feature learning method which requires as its input a similarity graph for the data points and the desired number n of numeric features. Note that Algorithm 16 does not make any use of the Euclidean distances between raw feature vectors and uses solely the similarity graph $\mathcal{G}$ to determine the intrinsic geometry of $\mathcal{D}$.

Algorithm 16 Feature Learning for Non-Numeric Data

Input: dataset $\mathcal{D} = \{\mathbf{z}^{(i)} \in \mathbb{R}^{n'}\}_{i=1}^{m}$; similarity graph $\mathcal{G}$; number n of features to be constructed for each data point.

1: construct the Laplacian matrix $\mathbf{L}$ of the similarity graph (see (9.13))

2: compute EVD of $\mathbf{L}$ to obtain n orthonormal eigenvectors (9.14) corresponding to the smallest eigenvalues of $\mathbf{L}$

3: for each data point i, construct feature vector

$$\mathbf{x}^{(i)} := \big(u_i^{(1)}, \ldots, u_i^{(n)}\big)^T \in \mathbb{R}^n \tag{9.15}$$

Output: $\mathbf{x}^{(i)}$, for $i = 1, \ldots, m$

9.4 Feature Learning for Labeled Data

We have discussed PCA as a linear dimensionality reduction method. PCA learns a compression matrix that maps raw features $\mathbf{z}^{(i)}$ of data points to new (much shorter) feature vectors $\mathbf{x}^{(i)}$. The feature vectors $\mathbf{x}^{(i)}$ determined by PCA depend solely on the raw feature vectors $\mathbf{z}^{(i)}$ of a given dataset $\mathcal{D}$. In particular, PCA determines the compression matrix such that the new features allow for a linear reconstruction (9.6) with minimum reconstruction error (9.7).

For some application domains we might not only have access to raw feature vectors but also to the label values $y^{(i)}$ of the data points in $\mathcal{D}$. Indeed, dimensionality reduction methods might be used as pre-processing step within a regression or classification problem that involves a labeled training set. However, in its basic form, PCA (see Algorithm 15) does not allow to exploit the information provided by available labels $y^{(i)}$ of data points $\mathbf{z}^{(i)}$. For some datasets, PCA might deliver feature vectors that are not very relevant for the overall task of predicting the label of a data point.

Let us now discuss a modification of PCA that exploits the information provided by available labels of the data points. The idea is to learn a linear construction map (matrix) $\mathbf{W}$ such that the new feature vectors $\mathbf{x}^{(i)} = \mathbf{W}\mathbf{z}^{(i)}$ allow to predict the label $y^{(i)}$ as good as possible. We restrict the prediction to be linear,

$$\hat{y}^{(i)} := \mathbf{r}^T\mathbf{x}^{(i)} = \mathbf{r}^T\mathbf{W}\mathbf{z}^{(i)}, \tag{9.16}$$

with some weight vector $\mathbf{r} \in \mathbb{R}^n$.

While PCA is motivated by minimizing the reconstruction error (9.1), we now aim at minimizing the prediction error $\hat{y}^{(i)} - y^{(i)}$. In particular, we assess the usefulness of a given pair of construction map $\mathbf{W}$ and predictor $\mathbf{r}$ (see (9.16)), using the empirical risk

$$\widehat{L}\big(\mathbf{W}, \mathbf{r} \mid \mathcal{D}\big) := (1/m) \sum_{i=1}^{m} \big(y^{(i)} - \hat{y}^{(i)}\big)^2 \\ \stackrel{(9.16)}{=} (1/m) \sum_{i=1}^{m} \big(y^{(i)} - \mathbf{r}^T \mathbf{W} \mathbf{z}^{(i)}\big)^2. \tag{9.17}$$

to guide the learning of a compressing matrix $\mathbf{W}$ and corresponding linear predictor weights $\mathbf{r}$ ((9.16)).

The optimal matrix $\mathbf{W}$ that minimizes the empirical risk (9.17) can be obtained via the EVD (9.10) of the sample covariance matrix $\mathbf{Q}$ (9.9). Note that we have used the EVD of $\mathbf{Q}$ already for PCA in Section 9.2 (see (9.8)). Remember that PCA uses the n eigenvectors $\mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(n)}$ corresponding to the n largest eigenvalues of $\mathbf{Q}$. In contrast, to minimize (9.17), we need to use a different set of eigenvectors in the rows of $\mathbf{W}$ in general. To find the right set of n eigenvectors, we need the sample cross-correlation vector

$$\mathbf{q} := (1/m) \sum_{i=1}^{m} y^{(i)} \mathbf{z}^{(i)}. \tag{9.18}$$

The entry q_j of the vector $\mathbf{q}$ estimates the correlation between the raw feature $z_j^{(i)}$ and the label $y^{(i)}$. We then define the index set

$$\mathcal{S} := \{j_1, \ldots, j_n\} \text{ such that } \big(q_j\big)^2/\lambda_j \geq \big(q_{j'}\big)^2/\lambda_{j'} \text{ for any } j \in \mathcal{S}, j' \in \{1, \ldots, n'\} \notin \mathcal{S}. \tag{9.19}$$

The set $\mathcal{S}$ is constituted by the indices $j \in \{1, \ldots, n'\}$ of n largest numbers $\big(q_j\big)^2/\lambda_j$. It can then be shown that the rows of the optimal compression matrix $\mathbf{W}$ (minimizing (9.17)) are the eigenvectors $\mathbf{u}^{(j)}$ with $j \in \mathcal{S}$. We summarize the overall feature learning method in Algorithm 17.

The main focus of this section is on regression problems that involve data points with numeric labels (e.g., from the label space $\mathcal{Y} = \mathbb{R}$). Given the raw features and labels of the data points in $\mathcal{D}$, Algorithm 17 determines new feature vectors $\mathbf{x}^{(i)}$ that allow to linearly predict a numeric label with minimum squared error. A similar approach can be used for classification problems involving data points with a finite label space $\mathcal{Y}$. Linear (or Fisher) discriminant analysis aims at constructing a compression matrix $\mathbf{W}$ such that the learnt

Algorithm 17 Linear Feature Learning for Labeled Data

Input: dataset $\big(\mathbf{z}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{z}^{(m)}, y^{(m)}\big)$ with raw features $\mathbf{z}^{(i)} \in \mathbb{R}^{n'}$ and numeric labels $y^{(i)} \in \mathbb{R}$; length n of new feature vectors.

1: compute EVD (9.10) of the sample covariance matrix (9.9) to obtain orthonormal eigenvectors $\big(\mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(n')}\big)$ corresponding to (decreasingly ordered) eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_{n'} \geq 0$

2: compute the sample cross-correlation vector (9.18) and, in turn, the sequence

$$\big(q_1\big)^2/\lambda_1, \ldots, \big(q_{n'}\big)^2/\lambda_{n'} \tag{9.20}$$

3: determine indices $j_1, \ldots, j_n$ of n largest elements in (9.20)

4: construct compression matrix $\mathbf{W} := \big(\mathbf{u}^{(j_1)}, \ldots, \mathbf{u}^{(j_n)}\big)^T \in \mathbb{R}^{n \times n'}$

5: compute feature vector $\mathbf{x}^{(i)} = \mathbf{W}\mathbf{z}^{(i)}$

Output: $\mathbf{x}^{(i)}$, for $i = 1, \ldots, m$, and compression matrix $\mathbf{W}$.

features $\mathbf{x} = \mathbf{W}\mathbf{z}$ of a data point allow to predict its label y as accurately as possible [58].

9.5 Privacy-Preserving Feature Learning

Many important application domains of ML involve sensitive data that is subject to data protection law [149]. Consider a health-care provider (such as a hospital) holding a large database of patient records. From a ML perspective this databases is nothing but a (typically large) set of data points representing individual patients. The data points are characterized by many features including personal identifiers (name, social security number), bio-physical parameters as well as examination results . We could apply ML to learn a predictor for the risk of particular disease given the features of a data point.

Given large patient databases, the ML methods might not be implemented locally at the hospital but using cloud computing. However, data protection requirements might prohibit the transfer of raw patient records that allow to match individuals with bio-physical properties. In this case we might apply feature learning methods to construct new features for each patient such that they allow to learn an accurate hypothesis for predicting a disease but do not allow to identify sensitive properties of the patient such as its name or a social security number.

Let us formalize the above application by characterizing each data point (patient in the

hospital database) using raw feature vector $\mathbf{z}^{(i)} \in \mathbb{R}^{n'}$ and a sensitive numeric property $\pi^{(i)}$. We would like to find a compression map $\mathbf{W}$ such that the resulting features $\mathbf{x}^{(i)} = \mathbf{W}\mathbf{z}^{(i)}$ do not allow to accurately predict the sensitive property $\pi^{(i)}$. The prediction of the sensitive property is restricted to be a linear $\hat{\pi}^{(i)} := \mathbf{r}^T\mathbf{x}^{(i)}$ with some weight vector $\mathbf{r}$.

Similar to Section 9.4 we want to find a compression matrix $\mathbf{W}$ that transforms, in a linear fashion, the raw feature vector $\mathbf{z} \in \mathbb{R}^{n'}$ to a new feature vector $\mathbf{x} \in \mathbb{R}^n$. However the design criterion for the optimal compression matrix $\mathbf{W}$ was different in Section 9.4 where the new feature vectors should allow for an accurate linear prediction of the label. In contrast, here we want to construct feature vectors such that there is no accurate linear predictor of the sensitive property $\pi^{(i)}$.

As in Section 9.4, the optimal compression matrix $\mathbf{W}$ is given row-wise by the eigenvectors of the sample covariance matrix (9.9). However, the choice of which eigenvectors to use is different and based on the entries of the sample cross-correlation vector

$$\mathbf{c} := (1/m)\sum_{i=1}^{m} \pi^{(i)}\mathbf{z}^{(i)}. \tag{9.21}$$

We summarize the construction of the optimal privacy-preserving compression matrix and corresponding new feature vectors in Algorithm 18.

Algorithm 18 Privacy Preserving Feature Learning

Input: dataset $\big(\mathbf{z}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{z}^{(m)}, y^{(m)}\big)$; each data point characterized by raw features $\mathbf{z}^{(i)} \in \mathbb{R}^{n'}$ and (numeric) sensitive property $\pi^{(i)} \in \mathbb{R}$; number n of new features.

1: compute the EVD (9.10) of the sample covariance matrix (9.9) to obtain orthonormal eigenvectors $\big(\mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(n')}\big)$ corresponding to (decreasingly ordered) eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_{n'} \geq 0$

2: compute the sample cross-correlation vector (9.21) and, in turn, the sequence

$$\big(c_1\big)^2/\lambda_1, \ldots, \big(c_{n'}\big)^2/\lambda_{n'} \tag{9.22}$$

3: determine indices $j_1, \ldots, j_n$ of n smallest elements in (9.22)

4: construct compression matrix $\mathbf{W} := \big(\mathbf{u}^{(j_1)}, \ldots, \mathbf{u}^{(j_n)}\big)^T \in \mathbb{R}^{n \times n'}$

5: compute feature vector $\mathbf{x}^{(i)} = \mathbf{W}\mathbf{z}^{(i)}$

Output: feature vectors $\mathbf{x}^{(i)}$, for $i = 1, \ldots, m$, and compression matrix $\mathbf{W}$.

Algorithm 18 learns a map $\mathbf{W}$ to extract privacy-preserving features out of the raw

feature vector of a data point. These new features are privacy-preserving as they do not allow to accurately predict (in a linear fashion) a sensitive property π of the data point. Another formalization for the preservation of privacy can be obtained using information-theoretic concepts. This information-theoretic approach interprets data points, their feature vector and sensitive property, as realizations of RVs. It is then possible to use the mutual information between new features $\mathbf{x}$ and the sensitive (private) property π as an optimization criterion for learning a compression map h (Section 9.1). The resulting feature learning method (referred to as privacy-funnel) differs from Algorithm 18 not only in the optimization criterion for the compression map but also in that it allows it to be non-linear [89, 128].

9.6 Random Projections

Note that PCA uses an EVD of the sample covariance matrix $\mathbf{Q}$ (9.9). The computational complexity (e.g., measured by number of multiplications and additions) for computing this EVD is lower bounded by $n \min\{n'^2, m^2\}$ [46, 139]. This computational complexity can be prohibitive for ML applications with n' and m being of the order of millions or even billions.

There is a computationally cheap alternative to PCA (Algorithm 15) for finding a useful compression matrix $\mathbf{W}$ in (9.5). This alternative is to construct the compression matrix $\mathbf{W}$ entry-wise

$$W_{j,j'} := a_{j,j'} \text{ with } a_{j,j'} \sim p(a). \tag{9.23}$$

The matrix entries (9.23) are realizations $a_{i,j}$ of i.i.d. RVs with some common probability distribution $p(a)$. Different choices for the probability distribution $p(a)$ have been studied in the literature [39]. The Bernoulli distribution is used to obtain a compression matrix with binary entries. Another popular choice for $p(a)$ is the multivariate normal (Gaussian) distribution.

Consider data points whose raw feature vectors $\mathbf{z}$ are located near a s-dimensional subspace of $\mathbb{R}^{n'}$. The feature vectors $\mathbf{x}$ obtained via (9.5) using a random matrix (9.23) allows to reconstruct the raw feature vectors $\mathbf{z}$ with high probability whenever

$$n \geq Cs \log n'. \tag{9.24}$$

The constant C depends on the maximum tolerated reconstruction error η (such that $\|\widehat{\mathbf{z}} - \mathbf{z}\|_2^2 \leq \eta$ for any data point) and the probability that the features $\mathbf{x}$ (see (9.23)) allow for a maximum reconstruction error η [39, Theorem 9.27.].

9.7 Dimensionality Increase

The focus of this chapter is on dimensionality reduction methods that learn a feature map delivering new feature vectors which are (significantly) shorter than the raw feature vectors. However, it might sometimes be beneficial to learn a feature map that delivers new feature vectors which are longer than the raw feature vectors. We have already discussed two examples for such feature learning methods in Sections 3.2 and 3.9. Polynomial regression maps a single raw feature z to a feature vector containing the powers of the raw feature z. This allows to use apply linear predictor maps to the new feature vectors to obtain predictions that depend non-linearly on the raw feature z. Kernel methods might even use a feature map that delivers feature vectors belonging to an infinite-dimensional Hilbert space [125].

Mapping raw feature vectors into higher-dimensional (or even infinite-dimensional) spaces might be useful if the intrinsic geometry of the data points is simpler when looked at in the higher-dimensional space. Consider a binary classification problem where data points are highly inter-winded in the original feature space (see Figure 3.7). Loosely speaking, mapping into higher-dimensional feature space might "flatten-out" a non-linear decision boundary between data points. We can then apply linear classifiers to the higher-dimensional features to achieve accurate predictions.

9.8 Exercises

Exercise 9.1. Computational Burden of Many Features. Discuss the computational complexity of linear regression. How much computation do we need to compute the linear predictor that minimizes the average squared error on a training set?

Exercise 9.2. Power Iteration. The key computational step of PCA amounts to an EVD of the psd matrix (9.9). Consider an arbitrary initial vector $\mathbf{u}^{(r)}$ and the sequence obtained by iterating

$$\mathbf{u}^{(r+1)} := \mathbf{Q}\mathbf{u}^{(r)} / \big\|\mathbf{Q}\mathbf{u}^{(r)}\big\|. \tag{9.25}$$

What (if any) conditions on the initialization $\mathbf{u}^{(r)}$ ensure that the sequence $\mathbf{u}^{(r)}$ converges to the eigenvector $\mathbf{u}^{(1)}$ of $\mathbf{Q}$ that corresponds to its largest eigenvalue λ_1?

Exercise 9.3. Linear Classifiers with High-Dimensional Features. Consider a training set $\mathcal{D}$ consisting of $m = 10^{10}$ labeled data points $\big(\mathbf{z}^{(1)}, y^{(1)}\big), \ldots, \big(\mathbf{z}^{(m)}, y^{(m)}\big)$ with

raw feature vectors $\mathbf{z}^{(i)} \in \mathbb{R}^{4000}$ and binary labels $y^{(i)} \in \{-1, 1\}$. Assume we have used a feature learning method to obtain the new features $\mathbf{x}^{(i)} \in \{0, 1\}^n$ with $n = m$ and such that the only non-zero entry of $\mathbf{x}^{(i)}$ is $x_i^{(i)} = 1$, for $i = 1, \ldots, m$. Can you find a linear classifier that perfectly classifies the training set?

Chapter 10

Transparent and Explainable ML

The successful deployment of ML methods depends on their transparency or explainability. We formalize the notion of an explanation and its effect using a simple probabilistic model in Section 10.1. Roughly speaking, an explanation is any artefact. such as a list of relevant features or a reference data point from a training set, that conveys information about a ML method and its predictions. Put differently, explaining a ML method should reduce the uncertainty (of a human end-user) about its predictions.

XML is umbrella term for techniques that make ML method transparent or explainable. Providing explanations for the predictions of a ML method is particularly important when these predictions inform decision making [23]. It is increasingly becoming a legal requirement to provide explanations for automated decision making systems [53].

Even for applications where predictions are not directly used to inform far-reaching decisions, providing explanations is important. The human end users have an intrinsic desire for explanations that resolve the uncertainty about the prediction. This is known as the "need for closure" in psychology [29, 75]. Beside legal and psychological requirements, providing explanations for predictions might also be useful for validating and verifying ML methods. Indeed, the explanations of ML methods (and its predictions) can point the user (which might be a "domain expert") to incorrect modelling assumptions used by the ML method [37].

XML is challenging since explanations must be tailored (personalized) to human end-users with varying backgrounds and in different contexts [87]. The user background includes the formal education as well as the individual digital literacy. Some users might have received university-level education in ML, while other users might have no relevant formal training (such as an undergraduate course in linear algebra). Linear regression with few features

might be perfectly interpretable for the first group but be considered a "black box" for the latter. To enable tailored explanations we need to model the user background as relevant for understanding the ML predictions.

This chapter discusses XML methods that have access to some user signal or feedback for some data points. Such a user signal might be obtained in various ways, including answers to surveys or bio-physical measurements collected via wearables or medical diagnostics. The user signal is used to determine (to some extent) the end-user background and, in turn, to tailor the delivered explanations for this end-user.

Existing XML methods can be roughly divided into two categories. The first category is referred to as "model-agnostic" [23]). Model-agnostic methods do not require knowledge of the detailed work principles of a ML method. These methods do not require knowledge of the hypothesis space used by a ML method but learn how to explain its predictions by observing them on a training set [22].

A second category of XML methods, sometimes referred to as "white-box" methods [23], uses ML methods that are considered as intrinsically explainable. The intrinsic explainability of a ML method depends crucially on its choice for the hypothesis space (see Section 2.2). This chapter discusses one recent method from each of the two XML categories [72, 69]. The common theme of both methods is the use of information-theoretic concepts to measure the usefulness of explanations [27].

Section 10.1 discusses a recently proposed model-agnostic approach to XML that constructs tailored explanations for the predictions of a given ML method [72]. This approach does not require any details about the internal mechanism of a ML method whose predictions are to be explained. Rather, this approach only requires a (sufficiently large) training set of data points for which the predictions of the ML method are known.

To tailor the explanations to a particular user, we use the values of a user (feedback) signal provided for the data points in the training set. Roughly speaking, the explanations are chosen such that they maximally reduce the "surprise" or uncertainty that the user has about the predictions of the ML method.

Section 10.2 discusses an example for a ML method that uses a hypothesis space that is intrinsically explainable [69]. We construct an explainable hypothesis space by appropriate pruning of a given hypothesis space such as linear maps (see Section 3.1) or non-linear maps represented by either an ANN (see Section 3.11) or decision trees (see Section 3.10). This pruning is implemented via adding a regularization term to ERM (4.3), resulting in an instance of SRM (7.2) which we refer to as explainable empirical risk minimization (EERM).

The regularization term favours hypotheses that are explainable to a user. Similar to the method in Section 10.1, the explainability of a map is quantified by information theoretic quantities. For example, if the original hypothesis space is the set of linear maps using a large number of features, the regularization term might favour maps that depend only on few features that are interpretable. Hence, we can interpret EERM as a feature learning method that aims at learning relevant and interpretable features (see Chapter 9).

10.1 Personalized Explanations for ML Methods

Consider a ML application involving data points with features $\mathbf{x} = \big(x_1, \ldots, x_n\big)^T \in \mathbb{R}^n$ and label $y \in \mathbb{R}$. We use a ML method that reads in some labelled data points

$$\big(\mathbf{x}^{(1)}, y^{(1)}\big), \big(\mathbf{x}^{(2)}, y^{(2)}\big), \ldots, \big(\mathbf{x}^{(m)}, y^{(m)}\big), \tag{10.1}$$

and learns a hypothesis

$$h(\cdot) : \mathbb{R}^n \to \mathbb{R} : \mathbf{x} \mapsto \hat{y} = h(\mathbf{x}). \tag{10.2}$$

The precise working principle of this ML method for how to learn this hypothesis h is not relevant in what follows.

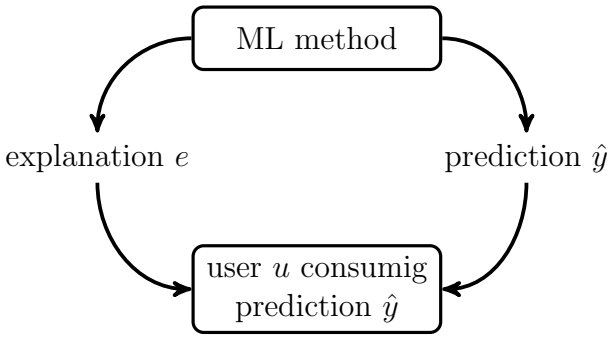


Figure 10.1: An explanation e provides additional information $I(\hat{y}, e|u)$ to a user u about the prediction $\hat{y}$.

The learnt predictor $h(\mathbf{x})$ is applied to the features of a data point to obtain the predicted label $\hat{y} := h(\mathbf{x})$. The prediction $\hat{y}$ is then delivered to a human end-user (see Figure 10.1). Depending on the ML application, this end-user might be a streaming service subscriber [47], a dermatologist [36] or a city planner [158].

Human users of ML methods often have some conception or model for the relation between features $\mathbf{x}$ and label y of a data point. This intrinsic model might vary significantly between users with different (social or educational) background. We will model the user understanding of a data point by a "user summary" $u \in \mathbb{R}$. The summary is obtained by a (possibly stochastic) map from the features $\mathbf{x}$ of a data point. For ease of exposition, we focus on summaries obtained by a deterministic map

$$u(\cdot) : \mathbb{R}^n \to \mathbb{R} : \mathbf{x} \mapsto u := u(\mathbf{x}). \tag{10.3}$$

However, the resulting XML method can be extended to user feedback u modelled as a stochastic maps. In this case, the user feedback u is characterized by a probability distribution $p(u|\mathbf{x})$.

The user feedback u is determined by the features $\mathbf{x}$ of a data point. We might think of the value u for a specific data point as a signal that reflects how the human end-user interprets (or perceives) the data point, given her knowledge (including formal education) and the context of the ML application. We do not assume any knowledge about the details for how the signal value u is formed for a specific data point. In particular, we do not know any properties of the map $u(\cdot) : \mathbf{x} \mapsto u$.

The above approach is quite flexible as it allows for very different forms of user summaries. The user summary could be the prediction obtained from a simplified model, such as linear regression using few features that the user anticipates as being relevant. Another example for a user summary u could be a higher-level feature, such as eye spacing in facial pictures, that the user considers relevant [67].

Note that, since we allow for an arbitrary map in (10.3), the user summary $u(\mathbf{x})$ obtained for a random data point with features $\mathbf{x}$ might be correlated with the prediction $\hat{y} = h(\mathbf{x})$. As an extreme case, consider a very knowledgable user that is able to predict the label of any data point from its features as well as the ML method itself. In this case, the maps (10.2) and (10.3) might be nearly identical. However, in general the predictions delivered by the learnt hypothesis (10.2) will be different from the user summary $u(\mathbf{x})$.

We formalize the act of explaining a prediction $\hat{y} = h(\mathbf{x})$ as presenting some additional quantity e to the user (see Figure 10.1). This explanation e can be any artefact that helps the user to understand the prediction $\hat{y}$, given her understanding u of the data point. Loosely speaking, the aim of providing explanation e is to reduce the uncertainty of the user u about the prediction $\hat{y}$ [75].

For the sake of exposition, we construct explanations e that are obtained via a determin-

istic map

$$e(\cdot) : \mathbb{R}^n \to \mathbb{R} : \mathbf{x} \mapsto e := e(\mathbf{x}), \tag{10.4}$$

from the features $\mathbf{x}$ of a data point. However, the XML methods in this chapter can be generalized without difficulty to handle explanations obtained from a stochastic map. In the end, we only require the specification of the conditional probability distribution $p(e|\mathbf{x})$.

The explanation e (10.4) depends only on the features $\mathbf{x}$ but not explicitly on the prediction $\hat{y}$. However, our method for constructing the map (10.4) takes into account the properties of the predictor map $h(\mathbf{x})$ (10.2). In particular, Algorithm 19 below requires as input the predicted labels $\hat{y}^{(i)}$ for a set of data points (that serve as a training set for our method).

To obtain comprehensible explanations that can be computed efficiently, we must typically restrict the space of possible explanations to a small subset $\mathcal{F}$ of maps (10.4). This is conceptually similar to the restriction of the space of possible predictor functions in a ML method to a small subset of maps which is known as the hypothesis space.

10.1.1 Probabilistic Data Model for XML

In what follows, we model data points as realizations of i.i.d. RVs with common (joint) probability distribution $p(\mathbf{x}, y)$ of features and label (see Section 2.1.4). Modelling the data points as realizations of RVs implies that the user summary u, prediction $\hat{y}$ and explanation e are also realizations of RVs. The joint distribution $p(u, \hat{y}, e, \mathbf{x}, y)$ conforms with the Bayesian network [111] depicted in Figure 10.2. Indeed,

$$p(u, \hat{y}, e, \mathbf{x}, y) = p(u|\mathbf{x}) \cdot p(e|\mathbf{x}) \cdot p(\hat{y}|\mathbf{x}) \cdot p(\mathbf{x}, y). \tag{10.5}$$

We measure the amount of additional information provided by an explanation e for a prediction $\hat{y}$ to some user u via the conditional mutual information (MI) [27, Ch. 2 and 8]

$$I(e; \hat{y}|u) := \mathbb{E}\left\{ \log \frac{p(\hat{y}, e|u)}{p(\hat{y}|u)p(e|u)} \right\}. \tag{10.6}$$

The conditional MI $I(e; \hat{y}|u)$ can also be interpreted as a measure for the amount by which the explanation e reduces the uncertainty about the prediction $\hat{y}$ which is delivered to some user u. Providing the explanation e serves the apparent human need to understand observed phenomena, such as the predictions from a ML method [75].

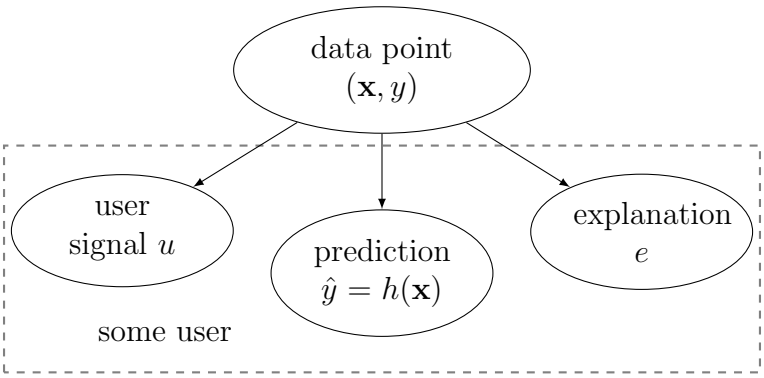


Figure 10.2: A simple probabilistic graphical model (a Bayesian network [84, 79]) for XML. We interpret data points (with features $\mathbf{x}$ and label y) along with the user summary u, e and predicted label $\hat{y}$ as realizations of RVs. These RVs satisfy conditional independence relations encoded by the directed links of the graph [79]. Given the data point, the predicted label $\hat{y}$, the explanation e and the user summary u are conditionally independent. This conditional independence is trivial if all these quantities are obtained from deterministic maps applied to the features $\mathbf{x}$ of the data point.

10.1.2 Computing Optimal Explanations

Capturing the effect of an explanation using the probabilistic model (10.6) offers a principled approach to computing an optimal explanation e. We require the optimal explanation e^* to maximize the conditional MI (10.6) between the explanation e and the prediction $\hat{y}$ conditioned on the user summary u of the data point.

Formally, an optimal explanation e^* solves

$$I(e^*; \hat{y}|u) = \sup_{e \in \mathcal{F}} I(e; \hat{y}|u). \tag{10.7}$$

The choice for the subset $\mathcal{F}$ of valid explanations offers a trade-off between comprehensibility, informativeness and computational cost incurred by an explanation e^* (solving (10.7)).

The maximization problem (10.7) for obtaining optimal explanations is similar to the approach in [22]. However, while [22] uses the unconditional MI between explanation and prediction, (10.7) uses the conditional MI given the user summary u. Therefore, (10.7) delivers personalized explanations that are tailored to the user who is characterized by the summary u.

It is important to note that the construction (10.7) allows for many different forms of

explanations. An explanation could be a subset of features of a data point (see [116] and Section 10.1.2). More generally, explanations could be obtained from simple local statistics (averages) of features that are considered related, such as nearby pixels in an image or consecutive amplitude values of an audio signal. Instead of individual features, carefully chosen data points from a training set can also serve as an explanation [92, 117].

Let us illustrate the concept of optimal explanations (10.7) using linear regression. We model the features $\mathbf{x}$ as a realization of a multivariate normal random vector with zero mean and covariance matrix $\mathbf{C}_x$,

$$\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{C}_x). \tag{10.8}$$

The predictor and the user summary are linear functions

$$\hat{y} := \mathbf{w}^T \mathbf{x}, \text{ and } u := \mathbf{v}^T \mathbf{x}. \tag{10.9}$$

We construct explanations via subsets of individual features x_j that are considered most relevant for a user to understand the prediction $\hat{y}$ (see [98, Definition 2] and [97]). Thus, we consider explanations of the form

$$e := \{x_j\}_{j \in \mathcal{E}} \text{ with some subset } \mathcal{E} \subseteq \{1, \ldots, n\}. \tag{10.10}$$

The complexity of an explanation e is measured by the number $|\mathcal{E}|$ of features that contribute to it. We limit the complexity of explanations by a fixed (small) sparsity level,

$$|\mathcal{E}| \leq s (\ll n). \tag{10.11}$$

Modelling the feature vector $\mathbf{x}$ as Gaussian (10.8) implies that the prediction $\hat{y}$ and user summary u obtained from (10.9) is jointly Gaussian for a given $\mathcal{E}$ (10.4). Basic properties of multivariate normal distributions [27, Ch. 8], allow to develop (10.7) as

$$\begin{aligned} &\max_{\substack{\mathcal{E} \subseteq \{1,\ldots,n\} \\ |\mathcal{E}| \leq s}} I(e; \hat{y}|u) \\ &\qquad = h(\hat{y}|u) - h(\hat{y}|u, \mathcal{E}) \\ &\qquad = (1/2) \log \mathbf{C}_{\hat{y}|u} - (1/2) \log \det \mathbf{C}_{\hat{y}|u, \mathcal{D}^{(\text{train})}} \\ &\qquad = (1/2) \log \sigma^2_{\hat{y}|u} - (1/2) \log \sigma^2_{\hat{y}|u, \mathcal{D}^{(\text{train})}}. \end{aligned} \tag{10.12}$$

Here, $\sigma^2_{\hat{y}|u}$ denotes the conditional variance of the prediction $\hat{y}$, conditioned on the user summary u. Similarly, $\sigma^2_{\hat{y}|u,\mathcal{E}}$ denotes the conditional variance of $\hat{y}$, conditioned on the user summary u and the subset $\{x_j\}_{j\in\mathcal{E}}$ of features. The last step in (10.12) follows from the fact that $\hat{y}$ is a scalar random variable.

The first component of the final expression of (10.12) does not depend on the index set $\mathcal{E}$ used to construct the explanation e (see (10.10)). Therefore, the optimal choice for $\mathcal{E}$ solves

$$\sup_{|\mathcal{E}|\leq s} -(1/2)\log\sigma^2_{\hat{y}|u,\mathcal{E}}. \tag{10.13}$$

The maximization (10.13) is equivalent to

$$\inf_{|\mathcal{E}|\leq s} \sigma^2_{\hat{y}|u,\mathcal{E}}. \tag{10.14}$$

In order to solve (10.14), we relate the conditional variance $\sigma^2_{\hat{y}|u,\mathcal{E}}$ to a particular decomposition

$$\hat{y} = \eta u + \sum_{j\in\mathcal{E}} \beta_j x_j + \varepsilon. \tag{10.15}$$

For an optimal choice of the coefficients η and β_j, the variance of the error term in (10.15) is given by $\sigma^2_{\hat{y}|u,\mathcal{E}}$. Indeed,

$$\min_{\eta,\beta_j\in\mathbb{R}} \mathbb{E}\big\{\big(\hat{y} - \eta u - \sum_{j\in\mathcal{E}} \beta_j x_j\big)^2\big\} = \sigma^2_{\hat{y}|u,e}. \tag{10.16}$$

Inserting (10.29) into (10.14), an optimal choice $\mathcal{E}$ (of feature) for the explanation of prediction $\hat{y}$ to user u is obtained from

$$\inf_{|\mathcal{E}|\leq s} \min_{\eta,\beta_j\in\mathbb{R}} \mathbb{E}\big\{\big(\hat{y} - \eta u - \sum_{j\in\mathcal{E}} \beta_j x_j\big)^2\big\} \tag{10.17}$$

$$= \min_{\|\boldsymbol{\beta}\|_0\leq s} \mathbb{E}\big\{\big(\hat{y} - \eta u - \boldsymbol{\beta}^T\mathbf{x}\big)^2\big\}. \tag{10.18}$$

An optimal subset $\mathcal{E}_{\text{opt}}$ of features defining the explanation e (10.10) is obtained from any solution $\boldsymbol{\beta}_{\text{opt}}$ of (10.18) via

$$\mathcal{E}_{\text{opt}} = \operatorname{supp}\boldsymbol{\beta}_{\text{opt}}. \tag{10.19}$$

Section 10.1.2 uses the probabilistic model (10.8) to construct optimal explanations via the (support of the) solutions $\boldsymbol{\beta}_{\text{opt}}$ of the sparse linear regression problem (10.18). To obtain

a practical algorithm for computing (approximately) optimal explanations (10.19), we approximate the expectation in (10.18) using an average over the training set $\big(\mathbf{x}^{(i)}, \hat{y}^{(i)}, u^{(i)}\big)$, for $i = 1, \ldots, m$. This resulting method for computing personalized explanations is summarized in Algorithm 19.

Algorithm 19 XML Algorithm

Input: explanation complexity s, training set $\big(\mathbf{x}^{(i)}, \hat{y}^{(i)}, u^{(i)}\big)$ for $i = 1, \ldots, m$

1: compute $\widehat{\boldsymbol{\beta}}$ by solving

$$\widehat{\boldsymbol{\beta}} \in \underset{\eta \in \mathbb{R}, \|\boldsymbol{\beta}\|_0 \leq s}{\operatorname{argmin}} \; (1/m) \sum_{i=1}^{m} \big(\hat{y}^{(i)} - \eta u^{(i)} - \boldsymbol{\beta}^T \mathbf{x}^{(i)}\big)^2 \tag{10.20}$$

Output: feature set $\widehat{\mathcal{E}} := \operatorname{supp} \widehat{\boldsymbol{\beta}}$

Algorithm 19 is interactive in the sense that the user has to provide a feedback signal $u^{(i)}$ for the data points with features $\mathbf{x}^{(i)}$. Based on the user feedback $u^{(i)}$, for $i = 1, \ldots, m$, Algorithm 19 learns an optimal subset $\mathcal{E}$ of features (10.10) that are used for the explanation of predictions.

The sparse regression problem (10.20) becomes intractable for large feature length n. However, if the features are weakly correlated with each other and the user summary u, the solutions of (10.20) can be found by efficient convex optimization methods. One popular method to (approximately) solve sparse regression (10.20) is the Lasso (see Section 3.4),

$$\widehat{\boldsymbol{\beta}} \in \underset{\eta \in \mathbb{R}, \boldsymbol{\beta} \in \mathbb{R}^n}{\operatorname{argmin}} (1/m) \sum_{i=1}^{m} \big(\hat{y}^{(i)} - \eta u^{(i)} - \boldsymbol{\beta}^T \mathbf{x}^{(i)}\big)^2 + \lambda \|\boldsymbol{\beta}\|_1. \tag{10.21}$$

There is large body of work that studies the choice of Lasso parameter λ in (10.21) such that solutions (10.21) coincide with the solutions of (10.20) (see [59, 144] and references therein). The proper choice for λ typically requires knowledge of statistical properties of data. If such a probabilistic model is not available, the choice of λ can be guided by simple validation techniques (see Section 6.2).

10.2 Explainable Empirical Risk Minimization

Section 7.1 discussed SRM (7.1) as a method for pruning the hypothesis space $\mathcal{H}$ used in ERM (4.3). This pruning is implemented either via a (hard) constraint as in (7.1) or by

adding a regularization term to the training error as in (7.2). The idea of SRM is to avoid (prune away) hypothesis maps that perform good on the training set but poorly outside, i.e., they do not generalize well. Here, we will use another criterion for steering the pruning and construction of regularization terms. In particular, we use the (intrinsic) explainability of a hypotheses map as a regularization term.

To make the notion of explainability precise we use again the probabilistic model of Section 10.1.1. We interpret data points as realizations of i.i.d. RVs with common (joint) probability distribution $p(\mathbf{x}, y)$ of features $\mathbf{x}$ and label y. A quantitative measure the intrinsic explainability of a hypothesis $h \in \mathcal{H}$ is the conditional (differential) entropy [27, Ch. 2 and 8]

$$H(\hat{y}|u) := -\mathbb{E}\Big\{ \log p(\hat{y}|u) \Big\}. \tag{10.22}$$

The conditional entropy (10.22) indicates the uncertainty about the prediction $\hat{y}$, given the user summary $\hat{u} = u(\mathbf{x})$. Smaller values $H(\hat{y}; u)$ correspond to smaller levels of uncertainty in the predictions $\hat{y}$ that is experienced by user u.

We obtain EERM by requiring a sufficiently small conditional entropy (10.22) of a hypothesis,

$$\hat{h} \in \operatorname*{argmin}_{h \in \mathcal{H}} \widehat{L}\big(h\big) \quad \text{s.t.} \quad H(\hat{y}|\hat{u}) \leq \eta. \tag{10.23}$$

The random variable $\hat{y} = h(\mathbf{x})$ in the constraint of (10.23) is obtained by applying the predictor map $h \in \mathcal{H}$ to the features. The constraint $H(\hat{y}|\hat{u}) \leq \eta$ in (10.23) enforces the learnt hypothesis $\hat{h}$ to be sufficiently explainable in the sense that the conditional entropy $H(\hat{h}|\hat{u}) \leq \eta$ does not exceed a prescribed level η.

Let us now consider the special case of EERM (10.23) for the linear hypothesis space

$$h^{(\mathbf{w})}(\mathbf{x}) := \mathbf{w}^T \mathbf{x} \text{ with some parameter vector } \mathbf{w} \in \mathbb{R}^n. \tag{10.24}$$

Moreover, we assume that the features $\mathbf{x}$ of a data point and its user summary u are jointly Gaussian with mean zero and covariance matrix $\mathbf{C}$,

$$\big(\mathbf{x}^T, \hat{u}\big)^T \sim \mathcal{N}(\mathbf{0}, \mathbf{C}). \tag{10.25}$$

Under the assumptions (10.24) and (10.25) (see [27, Ch. 8]),

$$H(\hat{u}|\hat{y}) = (1/2) \log \sigma^2_{\hat{y}|\hat{u}}. \tag{10.26}$$

Here, we used the conditional variance $\sigma^2_{\hat{y}|\hat{u}}$ of $\hat{y}$ given the random user summary $u = u(\mathbf{x})$. Inserting (10.26) into the generic form of EERM (10.23),

$$\hat{h} \in \operatorname*{argmin}_{h \in \mathcal{H}} \widehat{L}(h) \quad \text{s.t.} \quad \log \sigma^2_{\hat{y}|\hat{u}} \leq \eta. \tag{10.27}$$

By the monotonicity of the logarithm, (10.27) is equivalent to

$$\hat{h} \in \operatorname*{argmin}_{h \in \mathcal{H}} \widehat{L}(h) \quad \text{s.t.} \quad \sigma^2_{\hat{y}|\hat{u}} \leq e^{(\eta)}. \tag{10.28}$$

To further develop (10.29), we use the identity

$$\min_{\eta \in \mathbb{R}} \mathbb{E}\big\{ \big(\hat{y} - \eta u\big)^2 \big\} = \sigma^2_{\hat{y}|\hat{u}}. \tag{10.29}$$

The identity (10.29) relates the conditional variance $\sigma^2_{\hat{y}|\hat{u}}$ to the minimum mean squared error that can be achieved by estimating $\hat{y}$ using a linear estimator $\eta\hat{u}$ with some $\eta \in \mathbb{R}$. Inserting (10.29) and (10.24) into (10.28),

$$\hat{h} \in \operatorname*{argmin}_{\mathbf{w} \in \mathbb{R}^n, \eta \in \mathbb{R}} \widehat{L}(h^{(\mathbf{w})}) \quad \text{s.t.} \quad \mathbb{E}\big\{ \big(\underbrace{\mathbf{w}^T\mathbf{x}}_{\overset{(10.24)}{=}\hat{y}} - \eta\hat{u} \big)^2 \big\} \leq e^{(\eta)}. \tag{10.30}$$

The inequality constraint in (10.30) is convex [14, Ch. 4.2.]. For squared error loss, the objective function $\widehat{L}(h^{(\mathbf{w})})$ is also convex. Thus, for linear least squares regression, we can reformulate (10.30) as an equivalent (dual) unconstrained problem [14, Ch. 5]

$$\hat{h} \in \operatorname*{argmin}_{\mathbf{w} \in \mathbb{R}^n, \eta \in \mathbb{R}} \mathcal{E}(h^{(\mathbf{w})}) + \lambda \mathbb{E}\big\{ \big(\mathbf{w}^T\mathbf{x} - \eta\hat{u}\big)^2 \big\}. \tag{10.31}$$

By convex duality, for a given threshold $e^{(\eta)}$ in (10.30), we can find a value for λ in (10.31) such that (10.30) and (10.31) have the same solutions [14, Ch. 5]. Algorithm 20 below is obtained from (10.31) by approximating the expectation $\mathbb{E}\big\{ \big(\mathbf{w}^T\mathbf{x} - \eta\hat{u}\big)^2 \big\}$ with an average over the data points $\big(\mathbf{x}^{(i)}, \hat{y}^{(i)}, \hat{u}^{(i)}\big)$ for $i = 1, \ldots, m$.

10.3 Exercises

Exercise 10.1. Convexity of Explainable Linear regression Rewrite the optimization problem (10.32) as an equivalent quadratic optimization problem $\min_{\mathbf{v} \in \mathbb{R}^n} \mathbf{v}^T\mathbf{Q}\mathbf{v} + \mathbf{v}^T\mathbf{q}$.

Algorithm 20 Explainable Linear Least Squares Regression

Input: explainability parameter λ, training set $\left(\mathbf{x}^{(i)}, \hat{y}^{(i)}, \hat{u}^{i)}\right)$ for $i = 1, \ldots, m$

1: solve

$$\widehat{\mathbf{w}} \in \underset{\eta \in \mathbb{R}, \mathbf{w} \in \mathbb{R}^n}{\operatorname{argmin}} (1/m) \sum_{i=1}^{m} \underbrace{\left(\hat{y}^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)}\right)^2}_{\text{empirical risk}} + \lambda \underbrace{(\mathbf{w}^T \mathbf{x}^{(i)} - \eta \hat{u}^{(i)})^2}_{\text{explainability}} \tag{10.32}$$

Output: weights $\widehat{\mathbf{w}}$ of explainable linear hypothesis

Identify the matrix $\mathbf{Q} \in \mathbb{R}^{n \times n}$ and the vector $\mathbf{q} \in \mathbb{R}^n$.

Bibliography

[1] K. Abayomi, A. Gelman, and M. A. Levy. Diagnostics for multivariate imputations. *Journal of The Royal Statistical Society Series C-applied Statistics*, 57:273–291, 2008.

[2] M. Abramowitz and I. A. Stegun, editors. *Handbook of Mathematical Functions*. Dover, New York, 1965.

[3] D. Arthur and S. Vassilvitskii. k-means++: the advantages of careful seeding". In *Proc. of the eighteenth annual ACM-SIAM symposium on Discrete algorithms*. Society for Industrial and Applied Mathematics Philadelphia, 2007.

[4] P. Austin, P. Kaski, and K. Kubjas. Tensor network complexity of multilinear maps. *arXiv*, 2018.

[5] R. Baeza-Yates and B. Ribeiro-Neto. *Modern Information Retrieval*. addwes, 1999.

[6] F. Barata, K. Kipfer, M. Weber, P. Tinschert, E. Fleisch, and T. Kowatsch. Towards device-agnostic mobile cough detection with convolutional neural networks. In *2019 IEEE International Conference on Healthcare Informatics (ICHI)*, pages 1–11, 2019.

[7] M. S. Bartlett. An inverse matrix adjustment arising in discriminant analysis. *The Annals of Mathematical Statistics*, 22(1):107 – 111, 1951.

[8] M. Belkin, D. Hsu, S. Ma, and S. Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019.

[9] D. Bertsekas and J. Tsitsiklis. *Introduction to Probability*. Athena Scientific, 2 edition, 2008.

[10] D. P. Bertsekas. *Nonlinear Programming*. Athena Scientific, Belmont, MA, 2nd edition, June 1999.

[11] P. Billingsley. *Probability and Measure*. Wiley, New York, 3 edition, 1995.

[12] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[13] B. Boashash, editor. *Time Frequency Signal Analysis and Processing: A Comprehensive Reference*. Elsevier, Amsterdam, The Netherlands, 2003.

[14] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge Univ. Press, Cambridge, UK, 2004.

[15] P. J. Brockwell and R. A. Davis. *Time Series: Theory and Methods*. Springer New York, 1991.

[16] S. Bubeck. Convex optimization. algorithms and complexity. In *Foundations and Trends in Machine Learning*, volume 8. Now Publishers, 2015.

[17] P. Bühlmann and S. van de Geer. *Statistics for High-Dimensional Data*. Springer, New York, 2011.

[18] S. Carrazza. Machine learning challenges in theoretical HEP. *arXiv*, 2018.

[19] R. Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997.

[20] N. Cesa-Bianchi and G. Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, New York, NY, USA, 2006.

[21] O. Chapelle, B. Schölkopf, and A. Zien, editors. *Semi-Supervised Learning*. The MIT Press, Cambridge, Massachusetts, 2006.

[22] J. Chen, L. Song, M. Wainwright, and M. Jordan. Learning to explain: An information-theoretic perspective on model interpretation. In *Proc. 35th Int. Conf. on Mach. Learning*, Stockholm, Sweden, 2018.

[23] H.-F. Cheng, R. Wang, Z. Zhang, F. O'Connell, T. Gray, F. M. Harper, and H. Zhu. Explaining decision-making algorithms through UI: Strategies to help non-expert stakeholders. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, pages 1–12, New York, NY, USA, 2019. Association for Computing Machinery.

[24] I. Cohen and B. Berdugo. Noise estimation by minima controlled recursive averaging for robust speech enhancement. *IEEE Sig. Proc. Lett.*, 9(1):12–15, Jan. 2002.

[25] D. Cohn, Z. Ghahramani, and M. Jordan. Active learning with statistical models. *J. Artif. Int. Res.*, 4(1):129–145, March 1996.

[26] T. Cover and P. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967.

[27] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley, New Jersey, 2 edition, 2006.

[28] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems 2*, (4):303–314, 1989.

[29] T. K. DeBacker and H. M. Crowson. Influences on cognitive engagement: Epistemological beliefs and need for closure. *British Journal of Educational Psychology*, 76(3):535–551, 2006.

[30] Y. Dodge. *The Oxford Dictionary of Statistical Terms*. Oxford University Press, 2003.

[31] O. Dürr, Y. Pauchard, D. Browarnik, R. Axthelm, and M. Loeser. Deep learning on a raspberry pi for real time face recognition. 01 2015.

[32] B. Efron and R. Tibshirani. Improvements on cross-validation: The 632+ bootstrap method. *Journal of the American Statistical Association*, 92(438):548–560, 1997.

[33] R. Eldan and O. Shamir. The power of depth for feedforward neural networks. *CoRR*, abs/1512.03965, 2015.

[34] Y. C. Eldar, P. Kuppinger, and H. Bölcskei. Block-sparse signals: Uncertainty relations and efficient recovery. *IEEE Trans. Signal Processing*, 58(6):3042–3054, June 2010.

[35] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters a density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, pages 226–231, Portland, Oregon, 1996.

[36] A. Esteva, B. Kuprel, R. A. Novoa, J. Ko, S. M. Swetter, H. M. Blau, and S. Thrun. Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542, 2017.

[37] J. W. et.al. Association between surgical skin markings in dermoscopic images and diagnostic performance of a deep learning convolutional neural network for melanoma recognition. *JAMA Dermatol.*, 155(10):1135–1141, Oct. 2019.

[38] B. Everitt. *Cambridge Dictionary of Statistics*. Cambridge University Press, 2002.

[39] S. Foucart and H. Rauhut. *A Mathematical Introduction to Compressive Sensing*. Springer, New York, 2012.

[40] M. Friendly. A brief history of data visualization. In C. Chen, W. Härdle, and A. Unwin, editors, *Handbook of Computational Statistics: Data Visualization*, volume III. Springer-Verlag, 2006.

[41] R. G. Gallager. *Stochastic Processes: Theory for Applications*. Cambridge University Press, 2013.

[42] A. E. Gamal and Y.-H. Kim. *Network Information Theory*. Cambridge Univ. Press, 2012.

[43] M. Gao, H. Igata, A. Takeuchi, K. Sato, and Y. Ikegaya. Machine learning-based prediction of adverse drug effects: An example of seizure-inducing compounds. *Journal of Pharmacological Sciences*, 133(2):70 – 78, 2017.

[44] W. Gautschi and G. Inglese. Lower bounds for the condition number of vandermonde matrices. *Numer. Math.*, 52:241 – 250, 1988.

[45] G. Golub and C. van Loan. An analysis of the total least squares problem. *SIAM J. Numerical Analysis*, 17(6):883–893, Dec. 1980.

[46] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, 3rd edition, 1996.

[47] C. Gomez-Uribe and N. Hunt. The netflix recommender system: Algorithms, business value, and innovation. *Association for Computing Machinery*, 6(4), January 2016.

[48] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

[49] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *Proc. Neural Inf. Proc. Syst. (NIPS)*, 2014.

[50] R. Gray. *Probability, Random Processes, and Ergodic Properties*. Springer, New York, 2 edition, 2009.

[51] R. Gray, J. Kieffer, and Y. Linde. Locally optimal block quantizer design. *Information and Control*, 45:178 – 198, 1980.

[52] D. Gujarati and D. Porter. *Basic Econometrics*. Mc-Graw Hill, 2009.

[53] P. Hacker, R. Krestel, S. Grundmann, and F. Naumann. Explainable AI under contract and tort law: legal incentives and technical challenges. *Artificial Intelligence and Law*, 2020.

[54] H. Hagras. Toward human-understandable, explainable ai. *Computer*, 51(9):28–36, Sep. 2018.

[55] A. Halevy, P. Norvig, and F. Pereira. The unreasonable effectiveness of data. *IEEE Intelligent Systems*, March/April 2009.

[56] P. Halmos. *Naive set theory*. Springer-Verlag, 1974.

[57] P. R. Halmos. *Measure Theory*. Springer, New York, 1974.

[58] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. Springer, New York, NY, USA, 2001.

[59] T. Hastie, R. Tibshirani, and M. Wainwright. *Statistical Learning with Sparsity. The Lasso and its Generalizations*. CRC Press, 2015.

[60] E. Hazan. *Introduction to Online Convex Optimization*. Now Publishers Inc., 2016.

[61] J. Howard and S. Ruder. Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 328–339, Melbourne, Australia, July 2018. Association for Computational Linguistics.

[62] P. Huber. Approximate models. In C. Huber-Carol, N. Balakrishnan, M. Nikulin, and M. Mesbah, editors, *Goodness-of-Fit Tests and Model Validity. Statistics for Industry and Technology*. Birkhäuser, Boston, MA, 2002.

[63] P. J. Huber. *Robust Statistics*. Wiley, New York, 1981.

[64] L. Hyafil and R. Rivest. Constructing optimal binary decision trees is np-complete. *Information Processing Letters*, 5(1):15–17, 1976.

[65] L. Hyafil and R. L. Rivest. Constructing optimal binary decision trees is np-complete. *Information Processing Letters*, 5(1):15–17, 1976.

[66] G. James, D. Witten, T. Hastie, and R. Tibshirani. *An Introduction to Statistical Learning with Applications in R*. Springer, 2013.

[67] K. Jeong, J. Choi, and G. Jang. Semi-local structure patterns for robust face detection. *IEEE Sig. Proc. Letters*, 22(9), 2015.

[68] A. Jung. A fixed-point of view on gradient methods for big data. *Frontiers in Applied Mathematics and Statistics*, 3, 2017.

[69] A. Jung. Explainable empiricial risk minimization. *submitted to IEEE Sig. Proc. Letters (preprint: https://arxiv.org/pdf/2009.01492.pdf)*, 2020.

[70] A. Jung. Networked exponential families for big data over networks. *IEEE Access*, 8:202897–202909, 2020.

[71] A. Jung, Y. Eldar, and N. Görtz. On the minimax risk of dictionary learning. *IEEE Trans. Inf. Theory*, 62(3):1501 – 1515, Mar. 2016.

[72] A. Jung and P. Nardelli. An information-theoretic approach to personalized explainable machine learning. *IEEE Sig. Proc. Lett.*, 27:825–829, 2020.

[73] A. Jung and Y. SarcheshmehPour. Local graph clustering with network lasso. *IEEE Signal Processing Letters*, 28:106–110, 2021.

[74] A. Jung and N. Tran. Localized linear regression in networked data. *IEEE Sig. Proc. Lett.*, 26(7), Jul. 2019.

[75] J. Kagan. Motives and development. *Journal of Personality and Social Psychology*, 22(1):51–66, 1972.

[76] S. M. Kay. *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, Englewood Cliffs, NJ, 1993.

[77] T. Kibble and F. Berkshire. *Classical Mechanics*. Imperical College Press, 5 edition, 2011.

[78] P. Koehn. Europarl: A parallel corpus for statistical machine translation. In *The 10th Machine Translation Summit, page 79–86., AAMT,*, Phuket, Thailand, 2005.

[79] D. Koller and N. Friedman. *Probabilistic Graphical Models: Principles and Techniques*. Adaptive computation and machine learning. MIT Press, 2009.

[80] A. Krizhevsky, I. Sutskever, and G. Hinton. Imagenet classification with deep convolutional neural networks. In *Neural Information Processing Systems, NIPS*, 2012.

[81] B. Kulis and M. I. Jordan. Revisiting k-means: New algorithms via bayesian nonparametrics. In *Proc. of the 29th International Conference on Machine Learning, ICML 2012, Edinburgh, Scotland, UK, June 26 - July 1, 2012*. icml.cc / Omnipress, 2012.

[82] C. Lampert. Kernel methods in computer vision. *Foundations and Trends in Computer Graphics and Vision*, 2009.

[83] J. Larsen and C. Goutte. On optimal data split for generalization estimation and model selection. In *IEEE Workshop on Neural Networks for Signal Process*, 1999.

[84] S. L. Lauritzen. *Graphical Models*. Clarendon Press, Oxford, UK, 1996.

[85] E. L. Lehmann and G. Casella. *Theory of Point Estimation*. Springer, New York, 2nd edition, 1998.

[86] L. Li, W. Chu, J. Langford, and R. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proc. International World Wide Web Conference*, pages 661–670, Raleigh, North Carolina, USA, April 2010.

[87] Q. V. Liao, D. Gruen, and S. Miller. Questioning the ai: Informing design practices for explainable ai user experiences. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, CHI '20, pages 1–15, New York, NY, USA, 2020. Association for Computing Machinery.

[88] H. Lütkepohl. *New Introduction to Multiple Time Series Analysis*. Springer, New York, 2005.

[89] A. Makhdoumi, S. Salamatian, N. Fawaz, and M. Médard. From the information bottleneck to the privacy funnel. In *2014 IEEE Information Theory Workshop (ITW 2014)*, pages 501–505, 2014.

[90] S. G. Mallat. *A Wavelet Tour of Signal Processing – The Sparse Way*. Academic Press, San Diego, CA, 3 edition, 2009.

[91] K. V. Mardia, J. T. Kent, and J. M. Bibby. *Multivariate Analysis*. Academic Press, 1979.

[92] J. McInerney, B. Lacker, S. Hansen, K. Higley, H. Bouchard, A. Gruson, and R. Mehrotra. Explore, exploit, and explain: personalizing explainable recommendations with bandits. In *Proceedings of the 12th ACM Conference on Recommender Systems*, 2018.

[93] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. Communication-efficient learning of deep networks from decentralized data. In A. Singh and J. Zhu, editors, *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282, Fort Lauderdale, FL, USA, 20–22 Apr 2017. PMLR.

[94] C. Meyer. Generalized inversion of modified matrices. *SIAM J. Appied Mathmetmatics*, 24(3), 1973.

[95] C. Millard, editor. *Cloud Computing Law*. Oxford University Press, 2 edition, May 2021.

[96] T. Mitchell. The need for biases in learning generalizations. Technical Report CBM-TR 5-110,, Rutgers University, New Brunswick, New Jersey, USA, 1980.

[97] C. Molnar. *Interpretable Machine Learning - A Guide for Making Black Box Models Explainable*. [online] Available: https://christophm.github.io/interpretable-ml-book/., 2019.

[98] G. Montavon, W. Samek, and K. Müller. Methods for interpreting and understanding deep neural networks. *Digital Signal Processing*, 73:1–15, 2018.

[99] K. Mortensen and T. Hughes. Comparing amazon's mechanical turk platform to conventional data collection methods in the health and medical research literature. *J. Gen. Intern Med.*, 33(4):533–538, 2018.

[100] R. Muirhead. *Aspects of Multivariate Statistical Theory*. John Wiley & Sons Inc., 1982.

[101] N. Murata. A statistical study on on-line learning. In D. Saad, editor, *On-line Learning in Neural Networks*, pages 63–92. Cambridge University Press, New York, NY, USA, 1998.

[102] Y. Nesterov. *Introductory lectures on convex optimization*, volume 87 of *Applied Optimization*. Kluwer Academic Publishers, Boston, MA, 2004. A basic course.

[103] M. E. J. Newman. *Networks: An Introduction*. Oxford Univ. Press, 2010.

[104] A. Ng. *Shaping and Policy search in Reinforcement Learning*. PhD thesis, University of California, Berkeley, 2003.

[105] A. Y. Ng and M. I. Jordan. On discriminative vs. generative classifiers: A comparison of logistic regression and naive bayes. In T. G. Dietterich, S. Becker, and Z. Ghahramani, editors, *Advances in Neural Information Processing Systems 14*, pages 841–848. MIT Press, 2002.

[106] A. Y. Ng, M. I. Jordan, and Y. Weiss. On spectral clustering: Analysis and an algorithm. In *Adv. Neur. Inf. Proc. Syst.*, 2001.

[107] S. Oymak, B. Recht, and M. Soltanolkotabi. Sharp time–data tradeoffs for linear inverse problems. *IEEE Transactions on Information Theory*, 64(6):4129–4158, June 2018.

[108] S. Pan and Q. Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.

[109] A. Papoulis and S. U. Pillai. *Probability, Random Variables, and Stochastic Processes*. Mc-Graw Hill, New York, 4 edition, 2002.

[110] N. Parikh and S. Boyd. Proximal algorithms. *Foundations and Trends in Optimization*, 1(3):123–231, 2013.

[111] J. Pearl. *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann, 1988.

[112] F. Pedregosa. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011.

[113] L. Pitt and L. G. Valiant. Computational limitations on learning from examples. *J. ACM*, 35(4):965–984, Oct. 1988.

[114] T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda, and Q. Liao. Why and when can deep-but not shallow-networks avoid the curse of dimensionality: A review. *International Journal of Automation and Computing*, 14(5):503–519, 2017.

[115] H. Poor. *An Introduction to Signal Detection and Estimation*. Springer, 2 edition, 1994.

[116] M. Ribeiro, S. Singh, and C. Guestrin. "Why should i trust you?": Explaining the predictions of any classifier. In *Proc. 22nd ACM SIGKDD*, pages 1135–1144, Aug. 2016.

[117] M. Ribeiro, S. Singh, and C. Guestrin. Anchors: High-precision model-agnostic explanations. In *Proc. AAAI Conference on Artificial Intelligence (AAAI)*, 2018.

[118] S. Roweis. EM Algorithms for PCA and SPCA. In *Advances in Neural Information Processing Systems*, pages 626–632. MIT Press, 1998.

[119] W. Rudin. *Principles of Mathematical Analysis*. McGraw-Hill, New York, 3 edition, 1976.

[120] W. Rudin. *Real and Complex Analysis*. McGraw-Hill, New York, 3rd edition, 1987.

[121] S. J. Russel and P. Norvig. *Artificial Intelligence - A Modern Approach*. Prentice Hall, New York, 3 edition, 2010.

[122] N. P. Santhanam and M. J. Wainwright. Information-theoretic limits of selecting binary graphical models in high dimensions. *IEEE Trans. Inf. Theory*, 58(7):4117–4134, Jul. 2012.

[123] Y. SarcheshmehPour, M. Leinonen, and A. Jung. Federated learning from big data over networks. In *Proc. IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP), preprint: https://arxiv.org/pdf/2010.14159.pdf*, 2021.

[124] F. Sattler, K. Müller, and W. Samek. Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints. *IEEE Transactions on Neural Networks and Learning Systems*, 2020.

[125] B. Schölkopf and A. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, Cambridge, MA, USA, Dec. 2002.

[126] S. Shalev-Shwartz and S. Ben-David. *Understanding Machine Learning – from Theory to Algorithms*. Cambridge University Press, 2014.

[127] C. E. Shannon. Communication in the presence of noise. 1948.

[128] Y. Y. Shkel, R. S. Blum, and H. V. Poor. Secrecy by design with applications to privacy and compression. *IEEE Transactions on Information Theory*, 67(2):824–843, 2021.

[129] S.Levine, C. Finn, T. Darrell, and P.Abbeel. End-to-end training of deep visuomotor policies. *J. Mach. Learn. Res.*, 17, 2016.

[130] V. Smith, C.-K. Chiang, M. Sanjabi, and A. Talwalkar. Federated multi-task learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[131] S. Smoliski and K. Radtke. Spatial prediction of demersal fish diversity in the baltic sea: comparison of machine learning and regression-based techniques. *ICES Journal of Marine Science*, 74(1):102–111, 2017.

[132] A. Sorokin and D. Forsyth. Utility data annotation with amazon mechanical turk. In *2008 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, pages 1–8, 2008.

[133] S. Sra, S. Nowozin, and S. J. Wright, editors. *Optimization for Machine Learning*. MIT Press, 2012.

[134] G. Strang. *Linear Algebra and its Applications*. Cengage Learning, 4 edition, 2006.

[135] G. Strang. *Computational Science and Engineering*. Wellesley-Cambridge Press, MA, 2007.

[136] G. Strang. *Introduction to Linear Algebra*. Wellesley-Cambridge Press, MA, 5 edition, 2016.

[137] R. Sutton and A. Barto. *Reinforcement learning: An introduction*. MIT press, Cambridge, MA, 2 edition, 2018.

[138] M. E. Tipping and C. Bishop. Probabilistic principal component analysis. *Journal of the Royal Statistical Society, Series B*, 21/3:611–622, January 1999.

[139] M. E. Tipping and C. M. Bishop. Probabilistic principal component analysis. *J. Roy. Stat. Soc. Ser. B*, 61:611–622, 1999.

[140] N. Tishby and N. Zaslavsky. Deep learning and the information bottleneck principle. In *2015 IEEE Information Theory Workshop (ITW)*, pages 1–5, 2015.

[141] N. Tran, O. Abramenko, and A. Jung. On the sample complexity of graphical model selection from non-stationary samples. *IEEE Transactions on Signal Processing*, 68:17–32, 2020.

[142] N. Tran, H. Ambos, and A. Jung. Classifying partially labeled networked data via logistic network lasso. In *Proc. IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*, pages 3832–3836, Barcelona, Spain, May 2020.

[143] L. G. Valiant. A theory of the learnable. In *Proceedings of the Sixteenth Annual ACM Symposium on Theory of Computing*, STOC '84, pages 436–445, New York, NY, USA, 1984. Association for Computing Machinery.

[144] S. A. van de Geer and P. Bühlmann. On the conditions used to prove oracle results for the Lasso. *Electron. J. Statist.*, 3:1360 – 1392, 2009.

[145] V. N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, 1999.

[146] O. Vasicek. A test for normality based on sample entropy. *Journal of the Royal Statistical Society. Series B (Methodological)*, 38(1):54–59, 1976.

[147] R. Vidal. Subspace clustering. *IEEE Signal Processing Magazine*, 52, March 2011.

[148] U. von Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17(4):395–416, Dec. 2007.

[149] S. Wachter. Data protection in the age of big data. *Nature Electronics*, 2(1):6–7, 2019.

[150] S. Wade and Z. Ghahramani. Bayesian Cluster Analysis: Point Estimation and Credible Balls (with Discussion). *Bayesian Analysis*, 13(2):559 – 626, 2018.

[151] M. Wainwright. *High-Dimensional Statistics: A Non-Asymptotic Viewpoint*. Cambridge: Cambridge University Press, 2019.

[152] M. J. Wainwright. Information-theoretic limits on sparsity recovery in the high-dimensional and noisy setting. *IEEE Trans. Inf. Theory*, 55(12):5728–5741, Dec. 2009.

[153] M. J. Wainwright and M. I. Jordan. *Graphical Models, Exponential Families, and Variational Inference*, volume 1 of *Foundations and Trends in Machine Learning*. Now Publishers, Hanover, MA, 2008.

[154] A. Wang. An industrial-strength audio search algorithm. In *International Symposium on Music Information Retrieval*, Baltimore, MD, 2003.

[155] W. Wang, M. J. Wainwright, and K. Ramchandran. Information-theoretic bounds on model selection for Gaussian Markov random fields. In *Proc. IEEE ISIT-2010*, pages 1373–1377, Austin, TX, Jun. 2010.

[156] J. Wright, Y. Peng, Y. Ma, A. Ganesh, and S. Rao. Robust principal component analysis: Exact recovery of corrupted low-rank matrices by convex optimization. In *Neural Information Processing Systems, NIPS 2009*, 2009.

[157] L. Xu and M. Jordan. On convergence properties of the EM algorithm for Gaussian mixtures. *Neural Computation*, 8(1):129–151, 1996.

[158] X. Yang and Q. Wang. Crowd hybrid model for pedestrian dynamic prediction in a corridor. *IEEE Access*, 7, 2019.

[159] K. Young. Bayesian diagnostics for checking assumptions of normality. *Journal of Statistical Computation and Simulation*, 47(3–4):167 – 180, 1993.